ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

Σχεδίαση και Υλοποίηση Αποκεντρωμένης Εφαρμογής για Tokenization Έργων Τέχνης στο Οικοσύστημα του Sui Blockchain

---

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

του

ΝΙΚΟΛΑΟΥ Δ. ΤΖΙΡΗ

Επιβλέπων: Παναγιώτης Τσανάκας
Καθηγητής Ε.Μ.Π.

Αθήνα, Σεπτέμβριος 2026



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

Σχεδίαση και Υλοποίηση Αποκεντρωμένης Εφαρμογής για Tokenization Έργων Τέχνης στο Οικοσύστημα του Sui Blockchain

---

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

του

ΝΙΚΟΛΑΟΥ Δ. ΤΖΙΡΗ

Επιβλέπων: Παναγιώτης Τσανάκας
Καθηγητής Ε.Μ.Π.

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την 30η Σεπτεμβρίου 2026.

(Υπογραφή)	*(Υπογραφή)*	*(Υπογραφή)*
.....	.....	.....
Παναγιώτης Τσανάκας	Νικόλαος Λεονάρδος	Αριστείδης Παγουρτζής
Καθηγητής Ε.Μ.Π.	Επίκουρος Καθηγητής Ε.Μ.Π.	Καθηγητής Ε.Μ.Π.

Αθήνα, Σεπτέμβριος 2026

Copyright © Νικόλαος Τζιρής, 2026.
Με την επιφύλαξη παντός δικαιώματος. All rights reserved.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου, του τμήματος, του επιβλέποντα ή της επιτροπής που την ενέκρινε.

ΔΗΛΩΣΗ ΜΗ ΛΟΓΟΚΛΟΠΗΣ ΚΑΙ ΑΝΑΛΗΨΗΣ ΠΡΟΣΩΠΙΚΗΣ ΕΥΘΥΝΗΣ

Με πλήρη επίγνωση των συνεπειών του νόμου περί πνευματικών δικαιωμάτων, δηλώνω ενυπογράφως ότι είμαι αποκλειστικός συγγραφέας της παρούσας Διπλωματικής Εργασίας, για την ολοκλήρωση της οποίας κάθε βοήθεια είναι πλήρως αναγνωρισμένη και αναφέρεται λεπτομερώς στην εργασία αυτή. Έχω αναφέρει πλήρως και με σαφείς αναφορές, όλες τις πηγές χρήσης δεδομένων, απόψεων, θέσεων και προτάσεων, ιδεών και λεκτικών αναφορών, είτε κατά κυριολεξία είτε βάσει επιστημονικής παράφρασης. Αναλαμβάνω την προσωπική και ατομική ευθύνη ότι σε περίπτωση αποτυχίας στην υλοποίηση των ανωτέρω δηλωθέντων στοιχείων, είμαι υπόλογος έναντι λογοκλοπής, γεγονός που σημαίνει αποτυχία στην Διπλωματική μου Εργασία και κατά συνέπεια αποτυχία απόκτησης του Τίτλου Σπουδών, πέραν των λοιπών συνεπειών του νόμου περί πνευματικών δικαιωμάτων. Δηλώνω, συνεπώς, ότι αυτή η Διπλωματική Εργασία προετοιμάστηκε και ολοκληρώθηκε από εμένα προσωπικά και αποκλειστικά και ότι, αναλαμβάνω πλήρως όλες τις συνέπειες του νόμου στην περίπτωση κατά την οποία αποδειχθεί, διαχρονικά, ότι η εργασία αυτή ή τμήμα της δεν μου ανήκει διότι είναι προϊόν λογοκλοπής άλλης πνευματικής ιδιοκτησίας.

(Υπογραφή)

.....

Νικόλαος Τζιρής
Διπλωματούχος Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών Ε.Μ.Π.
Αθήνα, Σεπτέμβριος 2026

Στην αδελφή μου Ελευθερία

Περίληψη

Η παραδοσιακή αγορά τέχνης αντιμετωπίζει εδώ και δεκαετίες πέντε θεμελιώδη προβλήματα, συγκεκριμένα την αδιαφάνεια των τιμών, την αβέβαιη πιστοποίηση της γνησιότητας, τη χαμηλή ρευστότητα, τα υψηλά όρια εισόδου για τους μικρούς επενδυτές και τη χαμηλή αμοιβή των καλλιτεχνών από τις μεταπωλήσεις των έργων τέχνης τους.

Η παρούσα διπλωματική εργασία προτείνει μια εφαρμογή tokenization έργων τέχνης, με μηχανισμούς για την αντιμετώπιση των παραπάνω προβλημάτων. Η διαφάνεια υποστηρίζεται με τη δημόσια και αμετάβλητη καταγραφή κάθε συναλλαγής και του ιστορικού ιδιοκτησίας κάθε έργου τέχνης εντός της εφαρμογής. Η αυθεντικοποίηση των έργων εξασφαλίζεται μέσω ψηφιακών πιστοποιητικών γνησιότητας, που προσαρτώνται μόνιμα σε κάθε έργο και δεν μπορούν ούτε να παραποιηθούν ούτε να αποσπαστούν από αυτό. Η κλασματική ιδιοκτησία επιτρέπει την αγορά μεριδίων και τη διαπραγμάτευσή τους σε αποκεντρωμένη αγορά. Η δυνατότητα αυτή μπορεί να διευκολύνει τη συμμετοχή και τη ρευστότητα. Τέλος, σε κάθε μεταπώληση κλασμάτων μέσω της εφαρμογής, τα έξυπνα συμβόλαια καταβάλλουν αυτόματα στον καλλιτέχνη το δικαίωμα παρακολούθησης (royalty).

Στο θεωρητικό μέρος της εργασίας μελετάμε την τεχνολογία blockchain, το tokenization πραγματικών περιουσιακών στοιχείων, τα oracles και τις τεχνολογίες του οικοσυστήματος SUI. Στο πρακτικό μέρος σχεδιάζουμε, υλοποιούμε και αξιολογούμε την εφαρμογή. Τα έξυπνα συμβόλαια είναι γραμμένα στη γλώσσα Move και εκτελούνται στο SUI. Τα αρχεία των έργων τέχνης αποθηκεύονται στο αποκεντρωμένο δίκτυο Walrus, ενώ η διεπαφή χρήστη φιλοξενείται αποκεντρωμένα ως Walrus Site. Το Seal επιτρέπει τη λήψη του κλειδιού αποκρυπτογράφησης του αρχείου υψηλής ανάλυσης κάθε έργου τέχνης στον ιδιοκτήτη και στους κατόχους κλασμάτων του. Η επικοινωνία των έξυπνων συμβολαίων με τον φυσικό κόσμο, δηλαδή οι αναφορές για την κατάσταση των έργων τέχνης, γίνεται μέσα από ένα oracle που σχεδιάσαμε πάνω στο Nautilus. Η εφαρμογή αναπτύχθηκε χωρίς παραδοσιακό backend στο δοκιμαστικό δίκτυο του SUI και αξιολογήθηκε ως προς την ορθότητα και το κόστος των συναλλαγών της. Η υλοποίηση συνδυάζει τα παραπάνω στοιχεία του οικοσυστήματος SUI σε μια ενιαία εφαρμογή για την αγορά τέχνης.

Λέξεις Κλειδιά

Tokenization πραγματικών περιουσιακών στοιχείων, Αποκεντρωμένες εφαρμογές, Αγορά τέχνης, Blockchain, SUI, Move, Έξυπνα συμβόλαια, Πιστοποιητικά γνησιότητας, Κλασματική ιδιοκτησία, Δικαίωμα παρακολούθησης, Αποκεντρωμένη αποθήκευση, Αποκεντρωμένη φιλοξενία, Walrus, Seal, Nautilus, Oracles, Περιβάλλοντα αξιόπιστης εκτέλεσης

Abstract

For decades, the traditional art market has faced five fundamental problems, namely price opacity, uncertain authentication, low liquidity, high entry barriers for small investors, and low earnings for artists from the resale of their artworks.

This thesis proposes an application for artwork tokenization with mechanisms to address these problems. Transparency is supported by the public and immutable recording of every transaction and each artwork's ownership history within the application. Artwork authentication is ensured through digital certificates of authenticity that are permanently attached to each artwork and cannot be forged or detached from it. Fractional ownership allows users to buy shares and trade them on a decentralised market. This capability may facilitate participation and liquidity. Finally, on every resale of fractions through the application, the smart contracts automatically pay the artist a resale royalty.

In the theoretical part of the thesis, we study blockchain technology, the tokenization of real-world assets, oracles and the technologies of the SUI ecosystem. In the practical part, we design, implement and evaluate the application. The smart contracts are written in the Move language and run on SUI. Artwork files are stored on the decentralised Walrus network, while the user interface is hosted in a decentralised manner as a Walrus Site. Seal allows the owner and the holders of an artwork's fractions to obtain the decryption key for its high-resolution file. Communication between the smart contracts and the physical world, namely reports on the condition of the artworks, takes place through an oracle that we designed on top of Nautilus. The application was developed without a traditional backend on the SUI testnet and was evaluated in terms of correctness and transaction costs. The implementation combines these components of the SUI ecosystem into a single application for the art market.

Keywords

Real-world asset tokenization, Decentralised applications, Art market, Blockchain, SUI, Move, Smart contracts, Certificates of authenticity, Fractional ownership, Resale royalty, Decentralised storage, Decentralised hosting, Walrus, Seal, Nautilus, Oracles, Trusted execution environments

Ευχαριστίες

Θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή, κ. Παναγιώτη Τσανάκα, για την εμπιστοσύνη που μου έδειξε, για την υπομονή και την κατανόησή του, καθώς και για την ελευθερία που μου έδωσε στην επιλογή και στη διαμόρφωση του θέματος. Χάρη στην ενθάρρυνσή του γνώρισα έναν πρωτόγνωρο για μένα και ιδιαίτερα ενδιαφέροντα τομέα. Παράλληλα, είχα την ευκαιρία να υλοποιήσω ένα σύστημα που αντιμετωπίζει πραγματικές προκλήσεις της αγοράς τέχνης.

Επιπλέον, ένα ξεχωριστό και βαθύ ευχαριστώ οφείλω στους γιατρούς και το νοσηλευτικό προσωπικό που στάθηκαν δίπλα μου και με φρόντισαν όταν, κατά τη διάρκεια των σπουδών μου, αντιμετώπισα μια πολύ σοβαρή περιπέτεια υγείας. Χάρη στην αφοσίωση, την επιστημονική τους κατάρτιση και την ανθρωπιά τους είμαι σήμερα σε θέση να γράψω αυτές τις γραμμές και να δω αυτό το ταξίδι να ολοκληρώνεται.

Τέλος, εκφράζω την απεριόριστη ευγνωμοσύνη μου στην οικογένειά μου για την αδιάκοπη συμπαράσταση, την κατανόηση και την αγάπη που μου πρόσφερε όλα αυτά τα χρόνια. Ευχαριστώ από καρδιάς τους γονείς μου για τις θυσίες και την ακλόνητη στήριξή τους σε κάθε μου βήμα, καθώς και τους φίλους μου, που με την παρουσία τους έκαναν αυτή τη διαδρομή ομορφότερη.

Αφιερώνω αυτή την εργασία στην αγαπημένη μου αδελφή, Ελευθερία.

Αθήνα, Σεπτέμβριος 2026
Νικόλαος Τζιρής

Περιεχόμενα

Περίληψη **7**

Abstract **9**

Ευχαριστίες **10**

1 Εισαγωγή **17**

1.1 Τα προβλήματα της αγοράς της τέχνης 17

1.2 Προτεινόμενη προσέγγιση 19

1.3 Σκοπός και συνεισφορά της εργασίας 20

1.4 Δομή της εργασίας 22

2 Θεωρητικό υπόβαθρο **23**

2.1 Η τεχνολογία blockchain 23

2.1.1 Αναπαραγωγή μηχανής καταστάσεων και ομοφωνία 23

2.1.2 Αντίσταση σε επιθέσεις Sybil 26

2.1.3 Έξυπνα συμβόλαια 28

2.1.4 Tokens και NFTs 29

2.2 Web3 και αποκεντρωμένες εφαρμογές 30

2.2.1 Αποκεντρωμένες εφαρμογές 30

2.2.2 Διακυβέρνηση και αποκεντρωμένες αυτόνομες οργανώσεις (DAO) 31

2.3 Tokenization πραγματικών περιουσιακών στοιχείων 32

2.3.1 Ορισμός και κύκλος ζωής 32

2.3.2 Ταξινόμηση των RWA 33

2.3.3 Το χάσμα συγχρονισμού και το πρόβλημα των oracles 36

2.3.4 Ταξινόμηση των oracles 37

2.3.5 Oracles επαληθεύσιμου υπολογισμού 41

2.3.6 Τα έργα τέχνης ως πραγματικά περιουσιακά στοιχεία 43

2.3.7 Κλασματική ιδιοκτησία 44

2.3.8 Δικαίωμα παρακολούθησης 44

3 Το οικοσύστημα SUI **45**

3.1 Το blockchain δίκτυο SUI και η γλώσσα Move 45

3.1.1 Το μοντέλο αντικειμένων 45

3.1.2 Το μοντέλο συναλλαγών και τα προγραμματιζόμενα μπλοκ συναλλαγών ... 47

3.1.3 Η γλώσσα Sui Move 47

3.1.4 Ομοφωνία Mysticeti v2 51

3.2 Αποκεντρωμένη αποθήκευση στο Walrus 52

3.2.1 Blobs, shards και η επιτροπή αποθήκευσης 52
3.2.2 Η κωδικοποίηση Red Stuff 53
3.2.3 Εγγραφή και ανάγνωση 55
3.2.4 Εποχές, κόστος και το token WAL 56
3.3 Αποκεντρωμένη φιλοξενία ιστοσελίδων με Walrus Sites 58
3.3.1 Το αντικείμενο Site 58
3.3.2 Ανάγνωση μέσω της πύλης 58
3.4 Αποκεντρωμένος έλεγχος πρόσβασης με Seal 59
3.4.1 Κρυπτογραφία ταυτότητας και κρυπτογραφία κατωφλίου 59
3.4.2 Πολιτικές σε Move 60
3.5 Επαληθεύσιμος υπολογισμός εκτός blockchain με Nautilus 63
3.5.1 Περιβάλλοντα αξιόπιστης εκτέλεσης και AWS Nitro 63
3.5.2 Βεβαίωση και επαλήθευση στο blockchain δίκτυο 64

4 Σχεδίαση και υλοποίηση της εφαρμογής 69

4.1 Ρόλοι χρηστών και σενάρια χρήσης 69
4.2 Λειτουργικές απαιτήσεις 70
4.3 Δικαιώματα ανά ρόλο 71
4.4 Γενική επισκόπηση αρχιτεκτονικής συστήματος 73
4.5 Πειραματική διάταξη 75
4.6 Τεχνολογίες 76
4.7 Ο κύκλος ζωής ενός tokenized έργου τέχνης 77
4.8 Τα αντικείμενα στο SUI 78
4.9 Διοικητική εποπτεία και μητρώα ρόλων 79
4.10 Η διεπαφή χρήστη 81
4.11 Ο κατάλογος και η αναζήτηση 86
4.11.1 Η εγγραφή καταλόγου και τα chunks 87
4.11.2 Οι λίστες ανά διεύθυνση 89
4.11.3 Το index λέξεων 89
4.11.4 Η σελίδα του καταλόγου 90
4.12 Έκδοση έργου τέχνης 93
4.13 Φύλαξη και πιστοποιητικό γνησιότητας 95
4.14 Κλασματοποίηση 97
4.15 Αγορά και δικαίωμα παρακολούθησης 99
4.16 Τα αρχεία στο Walrus και ο έλεγχος πρόσβασης με Seal 102
4.16.1 Ο έλεγχος πρόσβασης με Seal 103
4.17 Σχεδιασμός του oracle με Nautilus 108
4.17.1 Προδιαγραφές πηγών και ο προσομοιωτής 111

4.17.2 Επαλήθευση των αναφορών 113
4.17.3 Συνάθροιση παραθύρου και έκδοση αναφοράς 114
4.17.4 Επαλήθευση και καταγραφή στο έξυπνο συμβόλαιο 115

5 Επίδειξη βασικών περιπτώσεων χρήσης 119

5.1 Σύνδεση wallet 119
5.2 Αρχική ρύθμιση της αγοράς 121
5.3 Έκδοση, φύλαξη και πιστοποίηση 122
5.4 Κλασματοποίηση και ενοποίηση 123
5.5 Αγορά κλασμάτων 124
5.6 Αναφορές κατάστασης και απώλεια 125
5.7 Ειδοποιήσεις ανά ρόλο 126
5.8 Πλοήγηση και αναζήτηση 127
5.9 Δημόσια επαλήθευση και προέλευση 127

6 Αξιολόγηση 131

6.1 Έλεγχος ορθότητας 131
6.2 Κόστος των πράξεων 135
6.3 Ιχνηλάτηση απαιτήσεων 137

7 Επίλογος 139

7.1 Συμπεράσματα 139
7.2 Τεχνικές προκλήσεις 140
7.3 Περιορισμοί και παραδοχές 141
7.4 Μελλοντικές κατευθύνσεις 141

Συντομεύσεις και ακρωνύμια 144

Βιβλιογραφία 146

Κατάλογος Σχημάτων

Σχήμα 2.1 Μια μηχανή καταστάσεων 24
Σχήμα 2.2 Η αρχιτεκτονική ενός κόμβου αναπαραγωγής μηχανής καταστάσεων 25
Σχήμα 2.3 Ο κύκλος ζωής του stake ενός επικυρωτή 28
Σχήμα 2.4 Ο κύκλος ζωής ενός RWA 33
Σχήμα 2.5 Ταξινόμηση των RWA κατά αγαθό αναφοράς 34
Σχήμα 2.6 Το χάσμα συγχρονισμού 36
Σχήμα 2.7 Ένα oracle ανάμεσα στο blockchain δίκτυο και μια εξωτερική πηγή 37
Σχήμα 2.8 Η κατάταξη των oracles 38
Σχήμα 2.9 Εισερχόμενα oracles με μηχανισμό pull και push 39
Σχήμα 2.10 Εξερχόμενα oracles με μηχανισμό pull και push 39
Σχήμα 3.1 Ο κύκλος ζωής μιας συναλλαγής στο SUI 52
Σχήμα 3.2 Η πρωτεύουσα κωδικοποίηση του Red Stuff 54
Σχήμα 3.3 Η δευτερεύουσα κωδικοποίηση και η ανάκτηση του Red Stuff 55
Σχήμα 3.4 Η ροή λειτουργίας του Seal 62
Σχήμα 3.5 Η αρχιτεκτονική ενός Nitro Enclave 64
Σχήμα 3.6 Ο κύκλος ζωής μιας εφαρμογής Nautilus 66
Σχήμα 4.1 Το διάγραμμα συστατικών της εφαρμογής 73
Σχήμα 4.2 Διάγραμμα ανάπτυξης της εγκατάστασης 75
Σχήμα 4.3 Ο κύκλος ζωής ενός tokenized έργου τέχνης 77
Σχήμα 4.4 Το μοντέλο αντικειμένων 78
Σχήμα 4.5 Η σύνδεση του wallet 82
Σχήμα 4.6 Η navbar για τρεις ρόλους 83
Σχήμα 4.7 Παράδειγμα αναδυόμενου μενού ειδοποιήσεων 84
Σχήμα 4.8 Το index και οι μετρητές στο blockchain δίκτυο 86
Σχήμα 4.9 Μια κάρτα της σελίδας του καταλόγου 87
Σχήμα 4.10 Το φίλτρο κατάστασης και η ταξινόμηση της σελίδας του καταλόγου 91
Σχήμα 4.11 Η γραμμή σελιδοποίησης και οι κάρτες του καταλόγου 91
Σχήμα 4.12 Το κλάσμα στο wallet 101
Σχήμα 4.13 Διάγραμμα ακολουθίας έκδοσης έργου τέχνης και αποθήκευσης στο Walrus 106
Σχήμα 4.14 Διαγράμματα ακολουθίας για την ανάγνωση δημόσιας εικόνας και υψηλής ανάλυσης 107
Σχήμα 4.15 Το διάγραμμα συστατικών του oracle 108
Σχήμα 4.16 Διάγραμμα ακολουθίας εγγραφής και ρύθμισης του enclave 109
Σχήμα 4.17 Διάγραμμα ακολουθίας υποβολής αναφορών από εξωτερική πηγή 110
Σχήμα 4.18 Διάγραμμα ακολουθίας συνάθροισης και καταγραφής αναφοράς 111

Σχήμα 4.19 Ο προσομοιωτής εξωτερικών πηγών 113
Σχήμα 5.1 Το παράθυρο σύνδεσης wallet 120
Σχήμα 5.2 Η οθόνη έγκρισης του wallet 120
Σχήμα 5.3 Το μενού του λογαριασμού 120
Σχήμα 5.4 Το μητρώο γκαλερί 121
Σχήμα 5.5 Το δημόσιο ίχνος πράξεων 121
Σχήμα 5.6 Η φόρμα έκδοσης 122
Σχήμα 5.7 Η έκδοση του πιστοποιητικού 122
Σχήμα 5.8 Το πιστοποιημένο έργο τέχνης 123
Σχήμα 5.9 Η κλασματοποίηση 123
Σχήμα 5.10 Η ενοποίηση 124
Σχήμα 5.11 Δημιουργία αγγελίας 124
Σχήμα 5.12 Η ανάλυση πληρωμής 125
Σχήμα 5.13 Η αναφορά κατάστασης 125
Σχήμα 5.14 Το έργο τέχνης μετά τη δήλωση απώλειας 126
Σχήμα 5.15 Οι εκκρεμότητες της γκαλερί 126
Σχήμα 5.16 Αναζήτηση στον κατάλογο 127
Σχήμα 5.17 Η δημόσια επαλήθευση 128
Σχήμα 5.18 Το ιστορικό προέλευσης 128
Σχήμα 5.19 Το αρχείο υψηλής ανάλυσης 129
Σχήμα 7.1 Εμπειρία επαυξημένης πραγματικότητας 142

Κατάλογος Πινάκων

Πίνακας 2.1 Μηχανισμοί αντίστασης σε Sybil και οικογένειες ομοφωνίας 27
Πίνακας 2.2 Οι δύο άξονες κατάταξης των tokens 29
Πίνακας 3.1 Οι μορφές ιδιοκτησίας αντικειμένων στο SUI 46
Πίνακας 4.1 Σενάρια χρήσης 69
Πίνακας 4.2 Λειτουργικές απαιτήσεις 70
Πίνακας 4.3 Δικαιώματα ανά ρόλο στους μηχανισμούς του συστήματος 71
Πίνακας 4.4 Οι τεχνολογίες της εγκατάστασης 76
Πίνακας 4.5 Τα routes της διεπαφής χρήστη 81
Πίνακας 4.6 Οι κατηγορίες ειδοποιήσεων ανά ρόλο 84
Πίνακας 5.1 Οι περιπτώσεις χρήσης του κεφαλαίου 119
Πίνακας 6.1 Έλεγχοι ορθότητας για την αρχική ρύθμιση της αγοράς 131
Πίνακας 6.2 Έλεγχοι ορθότητας για έκδοση, φύλαξη και πιστοποίηση 131
Πίνακας 6.3 Έλεγχοι ορθότητας για την κλασματοποίηση 132
Πίνακας 6.4 Έλεγχοι ορθότητας για την αγορά κλασμάτων 132
Πίνακας 6.5 Έλεγχοι ορθότητας για αναφορές και απώλεια 133
Πίνακας 6.6 Έλεγχοι ορθότητας για την πολιτική Seal 134
Πίνακας 6.7 Έλεγχοι εκτός έξυπνων συμβολαίων 134
Πίνακας 6.8 Δοκιμές μονάδας του oracle 135
Πίνακας 6.9 Το κόστος κάθε πράξης στο δοκιμαστικό δίκτυο 136
Πίνακας 6.10 Ιχνηλάτηση των λειτουργικών απαιτήσεων 137

Κεφάλαιο 1
Εισαγωγή

Όποιος θέλει σήμερα να επενδύσει σε ένα έργο τέχνης θα βρεθεί αντιμέτωπος με μια αγορά όπου οι τιμές δεν είναι δημόσιες, η γνησιότητα δεν είναι εγγυημένη, η πώληση μπορεί να πάρει χρόνια και η είσοδος απαιτεί πολύ μεγάλα χρηματικά ποσά. Στο κεφάλαιο αυτό παρουσιάζουμε τα προβλήματα της αγοράς τέχνης, όπως τα καταγράφει η βιβλιογραφία. Στη συνέχεια, περιγράφουμε την προσέγγιση που προτείνουμε για την αντιμετώπισή τους. Έπειτα, ορίζουμε τον σκοπό και τη συνεισφορά της εργασίας. Το κεφάλαιο κλείνει με τη δομή της υπόλοιπης εργασίας.

1.1 Τα προβλήματα της αγοράς της τέχνης

Η αγορά τέχνης είχε το 2025 πωλήσεις 59,6 δισεκατομμυρίων δολαρίων [1]. Όμως, παρά το μέγεθός της, η βιβλιογραφία καταγράφει σταθερά πέντε προβλήματα, δηλαδή την αδιαφάνεια των τιμών, την αβέβαιη πιστοποίηση της γνησιότητας, τη χαμηλή ρευστότητα, τα υψηλά όρια εισόδου και τη χαμηλή αμοιβή των καλλιτεχνών. Στην ενότητα αυτή εξετάζουμε καθένα από αυτά, με τα μεγέθη που τα τεκμηριώνουν.

Το πρώτο είναι η αδιαφάνεια (opacity). Σε αντίθεση με τις χρηματαγορές, οι τιμές στην αγορά τέχνης δεν είναι δημόσιες [2]. Συγκεκριμένα, οι ελάχιστες τιμές αποδοχής και τα έργα τέχνης που δεν πωλήθηκαν μένουν αδήλωτα [3]. Έτσι, η μόνη ορατή πηγή τιμών για έναν απλό επενδυτή είναι τα αποτελέσματα των δημοπρασιών. Το 2025 όμως, οι δημοπρασίες κάλυψαν μόνο το 42% της αγοράς, ενώ το υπόλοιπο 58% ήταν ιδιωτικές πωλήσεις μέσω εμπόρων και γκαλερί [1]. Ακόμη και τα δημόσια δεδομένα είναι ελλιπή, γιατί η τιμή μιας δημοπρασίας δεν λέει τίποτα για την κατάσταση του έργου τέχνης, την προέλευσή του ή τη γνώμη των ειδικών για αυτό [4]. Αυτή η ασυμμετρία της πληροφορίας ωφελεί όσους βρίσκονται μέσα στην αγορά [5], και οι μεσάζοντες την εκμεταλλεύονται [6].

Το δεύτερο πρόβλημα είναι η πιστοποίηση της γνησιότητας και της προέλευσης (authentication and provenance). Η πλαστογραφία γίνεται σε μεγάλη κλίμακα [7]. Ενδεικτικά, το 2024 η Eurojust κατάσχεσε περίπου 2.000 πλαστά έργα τέχνης, με εκτιμώμενες απώλειες 200 εκατομμυρίων ευρώ [8]. Ακόμη, η πιστοποίηση γνησιότητας των έργων τέχνης βασίζεται σε εκθέσεις πραγματογνωμοσύνης και έγγραφα προέλευσης που συχνά είναι ελλιπή, αλλοιωμένα ή πλαστογραφημένα [9]. Γι' αυτό, η πλαστογράφηση των εγγράφων προέλευσης είναι σήμερα από τους πιο συνηθισμένους τρόπους να νομιμοποιηθεί ένα κλεμμένο ή πλαστό έργο τέχνης [10].

Το τρίτο πρόβλημα είναι η χαμηλή ρευστότητα (illiquidity). Ένα έργο τέχνης δεν πωλείται όποτε θέλει ο κάτοχός του σε μια τρέχουσα τιμή, γιατί δεν υπάρχει οργανωμένη αγορά με συνεχή αποτίμηση. Αντί γι' αυτήν, ο κύριος δημόσιος μηχανισμός είναι η δημοπρασία [11], και τα κόστη της είναι μεγάλα. Συγκεκριμένα, ο αγοραστής πληρώνει προμήθεια αγοραστή (buyer's premium) 10% έως 25% πάνω στην τιμή κατακύρωσης και ο πωλητής προμήθεια γύρω στο 10% [11], [12]. Με τέτοια κόστη, τα έργα τέχνης κρατιούνται εκτός αγοράς για πολλά χρόνια. Πράγματι, από δεδομένα επαναλαμβανόμενων πωλήσεων του 1875 έως το 2000, ο μέσος χρόνος κατοχής υπολογίζεται σε 28 έτη [13].

Το τέταρτο πρόβλημα είναι τα υψηλά όρια εισόδου (high entry barriers). Ένα έργο τέχνης πωλείται ως ενιαία μονάδα, και στα έργα τέχνης καταξιωμένων καλλιτεχνών η τιμή κυμαίνεται από δεκάδες χιλιάδες έως εκατομμύρια δολάρια [14]. Η αγορά ολόκληρου του έργου τέχνης αποκλείει έτσι όσους μπορούν να διαθέσουν μόνο ένα μικρότερο ποσό, γεγονός που παρακινεί τις προσεγγίσεις κλασματικής ιδιοκτησίας [15].

Υψηλά όρια εισόδου υπάρχουν και για τους καλλιτέχνες. Οι Fraiberger κ.ά. [16] παρακολούθησαν τη σταδιοδρομία περίπου 500.000 καλλιτεχνών επί 36 χρόνια και βρήκαν ότι όσοι ξεκίνησαν από ιδρύματα υψηλού κύρους είχαν ποσοστό επαγγελματικής επιβίωσης 39% μετά από δέκα χρόνια, έναντι 14% για τους υπόλοιπους. Επιπλέον, τα έργα τέχνης των πρώτων πωλούνταν σε δημοπρασίες 4,7 φορές πιο συχνά και η μέγιστη τιμή τους ήταν 5,2 φορές υψηλότερη.

Το πέμπτο πρόβλημα είναι η χαμηλή αμοιβή των καλλιτεχνών. Ο καλλιτέχνης πληρώνεται μόνο στην πρώτη πώληση, στην πρωτογενή αγορά. Κάθε ανατίμηση, αντίθετα, στη δευτερογενή αγορά, όπου τα έργα τέχνης μεταπωλούνται, πηγαίνει στους νέους κατόχους [15]. Χαρακτηριστικό παράδειγμα αποτελεί το *Thaw* του Rauschenberg, που πωλήθηκε για 900 δολάρια το 1959 και μεταπωλήθηκε για 85.000 δολάρια το 1973, χωρίς ο δημιουργός να λάβει αμοιβή [15].

Η απάντηση σε αυτό είναι το δικαίωμα παρακολούθησης (droit de suite ή royalty), δηλαδή ένα ποσοστό επί κάθε μεταπώλησης που πηγαίνει στον καλλιτέχνη [17]. Το αναγνωρίζουν πάνω από 80 χώρες, ανάμεσά τους η Ευρωπαϊκή Ένωση, το Ηνωμένο Βασίλειο και η Αυστραλία, ενώ οι Ηνωμένες Πολιτείες και η Κίνα δεν το έχουν [18], [19]. Ακόμη όμως και εκεί που ισχύει, η είσπραξή του βασίζεται στη συμμόρφωση των πωλητών, χωρίς μηχανισμό επιβολής.

Το κύριο ερώτημα της εργασίας είναι επομένως πώς η τεχνολογία blockchain, και συγκεκριμένα το tokenization των περιουσιακών στοιχείων, μπορεί να υποστηρίξει μηχανισμούς αντιμετώπισης αυτών των πέντε προβλημάτων. Εξετάζουμε την τεχνική υλοποίηση και την ορθότητα των μηχανισμών, καθώς και το κόστος των συναλλαγών τους. Η επίδρασή τους στη ρευστότητα και στη συμμετοχή σε μια πραγματική αγορά απαιτεί ξεχωριστή εμπειρική μελέτη.

1.2 Προτεινόμενη προσέγγιση

Tokenization είναι η αναπαράσταση ενός φυσικού περιουσιακού στοιχείου ως token πάνω σε ένα blockchain. Τα στοιχεία που αναπαρίστανται έτσι λέγονται πραγματικά περιουσιακά στοιχεία σε μορφή token *(Real World Assets, RWAs)*. Παρακάτω δείχνουμε πώς απαντά το tokenization σε καθένα από τα πέντε αυτά προβλήματα.

Στην αδιαφάνεια απαντά το ίδιο το blockchain, όπου κάθε μεταβίβαση γράφεται μόνιμα και δεν αλλάζει αναδρομικά [20]. Έτσι, όποιος θέλει μπορεί να βλέπει χωρίς μεσάζοντα τις τιμές, το ιστορικό ιδιοκτησίας και την τεκμηρίωση προέλευσης [21]. Επιπροσθέτως, αυτή η διαφάνεια δυσκολεύει και τη νομιμοποίηση παράνομων εσόδων, στην οποία η αγορά τέχνης είναι εκτεθειμένη [22].

Στην προέλευση και στην τεκμηρίωση της γνησιότητας απαντούν τα μη ανταλλάξιμα tokens *(Non-Fungible Tokens, NFTs)*. Κάθε έργο τέχνης γίνεται ένα μοναδικό αντικείμενο στο blockchain δίκτυο, με δικό του αναγνωριστικό και με μεταδεδομένα που το συνδέουν με το φυσικό έργο τέχνης [23], [24]. Ταυτόχρονα, το ιστορικό ιδιοκτησίας του γράφεται στο blockchain δίκτυο από την έκδοση και μετά [20]. Πάνω στο αντικείμενο αυτό προσαρτάται και το ψηφιακό πιστοποιητικό γνησιότητας, στη θέση των χάρτινων πιστοποιητικών που χάνονται ή πλαστογραφούνται [23]. Η κρίση για τη γνησιότητα μπορεί να παραμένει σε ειδικό, αλλά το πιστοποιητικό της δεν χάνεται και δεν αλλοιώνεται.

Στη ρευστότητα και στα όρια εισόδου απαντά η κλασματική ιδιοκτησία (fractional ownership). Δηλαδή, η ιδιοκτησία του έργου τέχνης γίνεται ένα σύνολο από tokens που μεταβιβάζονται χωριστά [25]. Έτσι, ένας επενδυτής αγοράζει όσα κλάσματα θέλει, με το ποσό που θέλει, και τα πουλά όποτε βρει αγοραστή, χωρίς να πωληθεί ολόκληρο το έργο τέχνης [15]. Επιπλέον, η αγορά των κλασμάτων λειτουργεί συνεχώς και σε όλο τον κόσμο [26].

Στη χαμηλή αμοιβή των καλλιτεχνών απαντούν τα έξυπνα συμβόλαια (smart contracts). Ένα έξυπνο συμβόλαιο είναι κώδικας που εκτελείται μόνος του όταν ισχύσουν οι όροι του [27]. Γι' αυτό, σε κάθε μεταπώληση, το ποσοστό του καλλιτέχνη μπορεί να πηγαίνει στον λογαριασμό του μέσα στην ίδια συναλλαγή, χωρίς να εξαρτάται από τη συμμόρφωση του πωλητή [19], [28].

Για να υλοποιήσουμε την προσέγγιση αυτή χρησιμοποιούμε τη στοίβα SUI (Sui Stack). Η λογική της εφαρμογής τρέχει σε συμβόλαια γραμμένα στη γλώσσα Move [29]. Οι εικόνες και τα αρχεία των έργων τέχνης αποθηκεύονται στο Walrus, το σύστημα αποκεντρωμένης αποθήκευσης του οικοσυστήματος [30]. Το ίδιο το Walrus φιλοξενεί και τη διεπαφή χρήστη της εφαρμογής, ως Walrus Site, στη θέση ενός web server. Το Seal επιτρέπει να ορίσουμε πολιτικές για την αποκρυπτογράφηση των αρχείων υψηλής ανάλυσης [31]. Τέλος, με το Nautilus μπορούμε να φτιάξουμε blockchain oracles και να φέρουμε στο blockchain δίκτυο αναφορές για τη φυσική κατάσταση των έργων τέχνης [32].

1.3 Σκοπός και συνεισφορά της εργασίας

Πλατφόρμες tokenization έργων τέχνης υπάρχουν από το 2018 [33]. Ωστόσο, στις δημοσιευμένες προσεγγίσεις που εξετάσαμε δεν εντοπίσαμε τεκμηριωμένη υλοποίηση που να συνδυάζει το σύνολο των μηχανισμών της παρούσας εργασίας. Συγκεκριμένα, συνδέουμε την αγορά κλασμάτων με δημόσια μητρώα γκαλερί και εξωτερικών πηγών, καταγραφή φύλαξης και πιστοποίησης, αποκεντρωμένη αποθήκευση και φιλοξενία, έλεγχο πρόσβασης με Seal και oracle κατάστασης με Nautilus. Η συνεισφορά βρίσκεται στη σχεδίαση, την ολοκλήρωση και την αξιολόγηση αυτού του συνδυασμού στο δοκιμαστικό δίκτυο. Η εργασία εξετάζει την τεχνική αναπαράσταση και διαχείριση των δικαιωμάτων, χωρίς να θεμελιώνει τη νομική ισχύ τους επί του φυσικού έργου τέχνης. Συγκεκριμένα, οι συνεισφορές της είναι οι εξής:

1. **Θεωρητική μελέτη.** Μελετάμε την τεχνολογία blockchain, το tokenization πραγματικών περιουσιακών στοιχείων και τα oracles, που συνδέουν το blockchain δίκτυο με τον φυσικό κόσμο. Μελετάμε, επίσης, το οικοσύστημα SUI, δηλαδή το blockchain δίκτυο SUI και τη γλώσσα Move, το Walrus, το Seal και το Nautilus, πάνω στα οποία χτίζεται η εφαρμογή.

2. **Αποκεντρωμένη λογική και κατάσταση της εφαρμογής.** Τα έξυπνα συμβόλαια υλοποιούν τους επιχειρησιακούς κανόνες χωρίς παραδοσιακό backend ή ιδιωτική βάση δεδομένων της εφαρμογής. Η αποθήκευση και η φιλοξενία γίνονται στο Walrus, ενώ η πρόσβαση και οι εξωτερικές αναφορές χρησιμοποιούν τις υπηρεσίες της πύλης, των full nodes, του Seal και του oracle.

3. **Κλασματική ιδιοκτησία.** Η ιδιοκτησία κάθε έργου τέχνης σπάει σε κλάσματα, που τα εκδίδει ο ιδιοκτήτης αφού το έργο τέχνης παραληφθεί σε φύλαξη και πιστοποιηθεί. Ένας επενδυτής μπορεί έτσι να αγοράσει με μικρό ποσό και να πουλήσει κλάσματα χωρίς να πωληθεί ολόκληρο το έργο τέχνης, ενώ το ίδιο το έργο τέχνης μένει ένα κοινόχρηστο αντικείμενο που δεν μεταβιβάζεται. Όποιος συγκεντρώσει όλα τα κλάσματα ξαναγίνεται ο μοναδικός ιδιοκτήτης.

4. **Αποκεντρωμένη αποθήκευση αρχείων και φιλοξενία της διεπαφής χρήστη στο Walrus.** Τα αρχεία της εφαρμογής αποθηκεύονται στο Walrus, ένα αποκεντρωμένο δίκτυο αποθήκευσης, ενώ στο blockchain δίκτυο γράφεται μόνο το αναγνωριστικό (hash) κάθε αρχείου, το οποίο προκύπτει από το ίδιο το περιεχόμενό του. Το Walrus φιλοξενεί, επίσης, τα αρχεία της διεπαφής χρήστη, που φτάνουν στον περιηγητή με Walrus Sites.

5. **Αποκεντρωμένος έλεγχος πρόσβασης στα κρυπτογραφημένα αρχεία με Seal.** Η εικόνα υψηλής ανάλυσης κάθε έργου τέχνης αποθηκεύεται κρυπτογραφημένη. Οι κανόνες πρόσβασης σε αυτήν είναι γραμμένοι στα συμβόλαια και η αποκρυπτογράφηση γίνεται με το Seal, χωρίς backend.

6. **Oracle εξωτερικών πηγών με Nautilus.** Η σύνδεση με τον φυσικό κόσμο γίνεται μέσα από ένα oracle που σχεδιάσαμε πάνω στο Nautilus. Μέσα από αυτό, εξωτερικές πηγές (άνθρωπος, συσκευή του διαδικτύου των πραγμάτων *(Internet of Things, IoT)* ή οποιοδήποτε εξωτερικό σύστημα) στέλνουν αναφορές για τη φυσική κατάσταση των έργων τέχνης στα έξυπνα συμβόλαια της εφαρμογής.

7. **Πιστοποιητικό γνησιότητας συνδεδεμένο στο αντικείμενο του έργου τέχνης.** Το πιστοποιητικό είναι συνδεδεμένο στο ίδιο το αντικείμενο του έργου τέχνης. Μένει έτσι πάνω στο έργο τέχνης σε κάθε πώληση, ακόμη και όσο το έργο τέχνης είναι κλασματοποιημένο και δεν έχει έναν μόνο κάτοχο.

8. **Διακυβέρνηση της αγοράς με μητρώα στο blockchain δίκτυο.** Η ρυθμιστική αρχή εγγράφει τις γκαλερί και τις εξωτερικές πηγές σε δύο μητρώα στο blockchain δίκτυο και μπορεί να ανακαλεί τις εγγραφές τους. Πριν από κάθε πράξη του αντίστοιχου ρόλου τα συμβόλαια διαβάζουν τα μητρώα, οπότε η εξουσιοδότηση ελέγχεται εκεί, χωρίς να μεσολαβεί κεντρικός διακομιστής.

9. **Διαφάνεια και λογοδοσία.** Οι τιμές, το ιστορικό ιδιοκτησίας, τα πιστοποιητικά και οι αναφορές των εξωτερικών πηγών για την κατάσταση των έργων τέχνης είναι δημόσια και τα διαβάζει ο καθένας από το blockchain δίκτυο. Με αυτόν τον τρόπο, ο επισκέπτης μπορεί, με το αναγνωριστικό ενός έργου τέχνης, να επαληθεύσει το πιστοποιητικό του, τη γκαλερί που το φυλάσσει και τις αναφορές για την κατάστασή του. Κάθε έργο τέχνης κρατά και το δικό του ιστορικό προέλευσης, με μία εγγραφή για κάθε πράξη πάνω του. Δημόσια είναι επίσης κάθε εγγραφή και κάθε ανάκληση στα μητρώα, όπως και τα μεγέθη της αγοράς, δηλαδή πόσα έργα τέχνης κυκλοφορούν, πόσες πωλήσεις έγιναν και τα ποσά που καταβλήθηκαν ως δικαιώματα παρακολούθησης. Η διεπαφή χρήστη τα δείχνει όλα αυτά δημόσια και σε πραγματικό χρόνο, διαβάζοντάς τα απευθείας από το blockchain δίκτυο.

10. **Αυτόματη επιβολή του δικαιώματος παρακολούθησης.** Ο καλλιτέχνης εκδίδει το έργο τέχνης και τα κλάσματά του, και οι επενδυτές τα αγοράζουν και τα πωλούν μέσα από την αγορά της εφαρμογής. Τα κλάσματα δεν μεταβιβάζονται από πορτοφόλι σε πορτοφόλι, ώστε κάθε αλλαγή κατόχου να περνά υποχρεωτικά από την αγορά. Σε κάθε μεταπώληση το συμβόλαιο καταβάλλει τότε το δικαίωμα παρακολούθησης στον καλλιτέχνη μέσα στην ίδια συναλλαγή. Η επιβολή γίνεται έτσι στο επίπεδο των τύπων της γλώσσας Move και δεν παρακάμπτεται.

1.4 Δομή της εργασίας

Το κεφάλαιο 2 δίνει το θεωρητικό υπόβαθρο. Ξεκινά από την τεχνολογία blockchain, τα έξυπνα συμβόλαια και τα tokens, και συνεχίζει με το Web3, τις αποκεντρωμένες εφαρμογές και τη διακυβέρνησή τους. Έπειτα, φτάνει στο tokenization πραγματικών περιουσιακών στοιχείων, δηλαδή στο αντικείμενο της εργασίας. Μελετά το χάσμα συγχρονισμού ανάμεσα στη φυσική και την ψηφιακή κατάσταση ενός αγαθού και τα oracles που το γεφυρώνουν. Συνεχίζει με τα έργα τέχνης, την κλασματική ιδιοκτησία και το δικαίωμα παρακολούθησης.

Το κεφάλαιο 3 παρουσιάζει το οικοσύστημα SUI, δηλαδή τις τεχνολογίες με τις οποίες χτίζεται η εφαρμογή. Πρόκειται για το blockchain δίκτυο SUI και τη γλώσσα Move, το Walrus και τα Walrus Sites, το Seal και το Nautilus.

Το κεφάλαιο 4 περιέχει τη σχεδίαση και την υλοποίηση της εφαρμογής, με βάση τα διαγράμματα UML της και αποσπάσματα από τον κώδικα. Ξεκινά από τους ρόλους των χρηστών και τα σενάρια χρήσης, τις λειτουργικές απαιτήσεις και τα δικαιώματα κάθε ρόλου. Συνεχίζει με την αρχιτεκτονική και την πειραματική διάταξη στο testnet, τον κύκλο ζωής ενός tokenized έργου τέχνης, τα αντικείμενα στο SUI και τη διεπαφή χρήστη. Έπειτα περνά στους τρεις μηχανισμούς, δηλαδή την κλασματική ιδιοκτησία με την αγορά, τα αρχεία στο Walrus με τον έλεγχο πρόσβασης του Seal, και το oracle με Nautilus.

Το κεφάλαιο 5 δείχνει την εφαρμογή σε λειτουργία μέσα από περιπτώσεις χρήσης, από τη σύνδεση του wallet και την αρχική ρύθμιση της αγοράς μέχρι τη δημόσια επαλήθευση ενός έργου τέχνης. Κάθε περίπτωση παρουσιάζεται βήμα προς βήμα, με στιγμιότυπα από τη διεπαφή χρήστη και τις συναλλαγές στο blockchain δίκτυο.

Το κεφάλαιο 6 αξιολογεί την εφαρμογή. Ελέγχει την ορθότητα των συμβολαίων με αυτοματοποιημένες δοκιμές, μετρά το κόστος κάθε πράξης στο testnet και αντιστοιχεί κάθε απαίτηση του κεφαλαίου 4 στον τρόπο που επαληθεύτηκε.

Το κεφάλαιο 7 είναι ο επίλογος. Συγκεντρώνει τα συμπεράσματα, τις τεχνικές προκλήσεις που συναντήσαμε, τους περιορισμούς και τις παραδοχές της εργασίας, και τις κατευθύνσεις για μελλοντική έρευνα.

Κεφάλαιο 2
Θεωρητικό υπόβαθρο

Στο κεφάλαιο αυτό παρουσιάζουμε το θεωρητικό υπόβαθρο που χρειαζόμαστε για τη σχεδίαση και την υλοποίηση της εφαρμογής μας. Ξεκινάμε από την τεχνολογία blockchain, τα έξυπνα συμβόλαια και τα tokens. Συνεχίζουμε με το Web3 και τις αποκεντρωμένες εφαρμογές. Έπειτα, εξετάζουμε το tokenization πραγματικών περιουσιακών στοιχείων και το πρόβλημα των oracles που το συνοδεύει. Το κεφάλαιο κλείνει με τη θέση του έργου τέχνης ανάμεσά τους, την κλασματική ιδιοκτησία και το δικαίωμα παρακολούθησης.

2.1 Η τεχνολογία blockchain

Η τεχνολογία blockchain έγινε γνωστή το 2008 με το Bitcoin [34]. Οι δυνατότητές της, όμως, είναι ευρύτερες από ένα ψηφιακό νόμισμα. Ο Roughgarden [35] περιγράφει το blockchain ως έναν «υπολογιστή στον ουρανό» *(the computer in the sky)*, δηλαδή έναν υπολογιστή γενικής χρήσης που δεν ανήκει σε κανέναν, αποτελείται από χιλιάδες μηχανές σε όλο τον κόσμο, και τον χρησιμοποιεί όποιος θέλει. Για να δουλεύουν οι μηχανές αυτές σαν μία, χρειάζονται ένα πρωτόκολλο ομοφωνίας *(consensus protocol)*. Το πρόβλημα που λύνει το πρωτόκολλο είναι γνωστό στα κατανεμημένα συστήματα από τη δεκαετία του 1970 ως αναπαραγωγή μηχανής καταστάσεων *(State Machine Replication, SMR)* [36], [37].

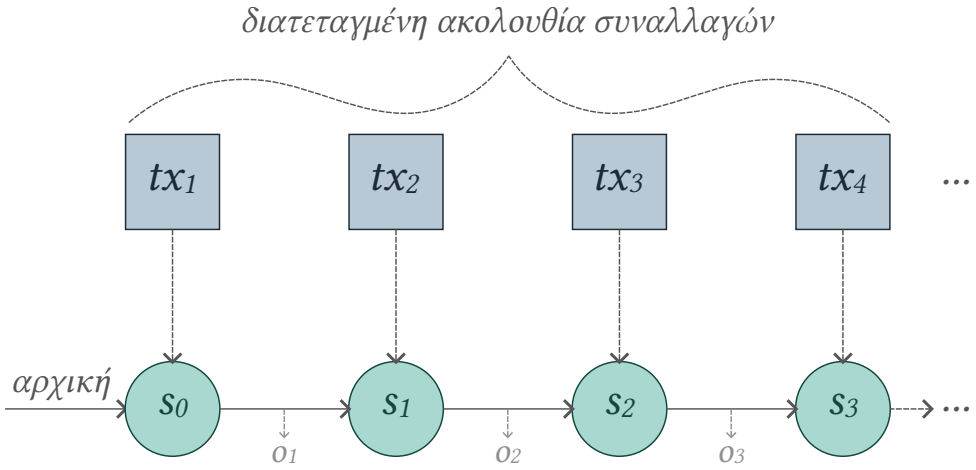
2.1.1 Αναπαραγωγή μηχανής καταστάσεων και ομοφωνία

Μια μηχανή καταστάσεων *(state machine)* είναι μια υπηρεσία που κρατά μια εσωτερική κατάσταση και δέχεται εντολές από πελάτες *(clients)*. Κάθε εντολή, με βάση την τρέχουσα κατάσταση, παράγει μια νέα κατάσταση και ίσως μια έξοδο [37]. Σε ένα blockchain οι εντολές είναι οι συναλλαγές *(transactions, txs)*, οπότε στο εξής μιλάμε για συναλλαγές αντί για εντολές.

Ορισμός 2.1 (Μηχανή καταστάσεων) Μια μηχανή καταστάσεων είναι μια πεντάδα (S, s_0, T, O, δ) [37].

1. S είναι το σύνολο των δυνατών καταστάσεων.
2. $s_0 \in S$ είναι η αρχική κατάσταση.
3. T είναι το σύνολο των συναλλαγών που δέχεται η μηχανή.
4. O είναι το σύνολο των δυνατών εξόδων.
5. $\delta : S \times T \to S \times O$ είναι η συνάρτηση μετάβασης. Αντιστοιχεί την τρέχουσα κατάσταση και μια συναλλαγή σε μια νέα κατάσταση και μια έξοδο.

Η συνάρτηση μετάβασης είναι ντετερμινιστική, δηλαδή η ίδια συναλλαγή στην ίδια κατάσταση δίνει πάντα το ίδιο αποτέλεσμα. Άρα, δύο αντίγραφα της μηχανής που ξεκινούν από την ίδια αρχική κατάσταση και εφαρμόζουν τις ίδιες συναλλαγές με την ίδια σειρά καταλήγουν στην ίδια κατάσταση (**Σχήμα 2.1**).



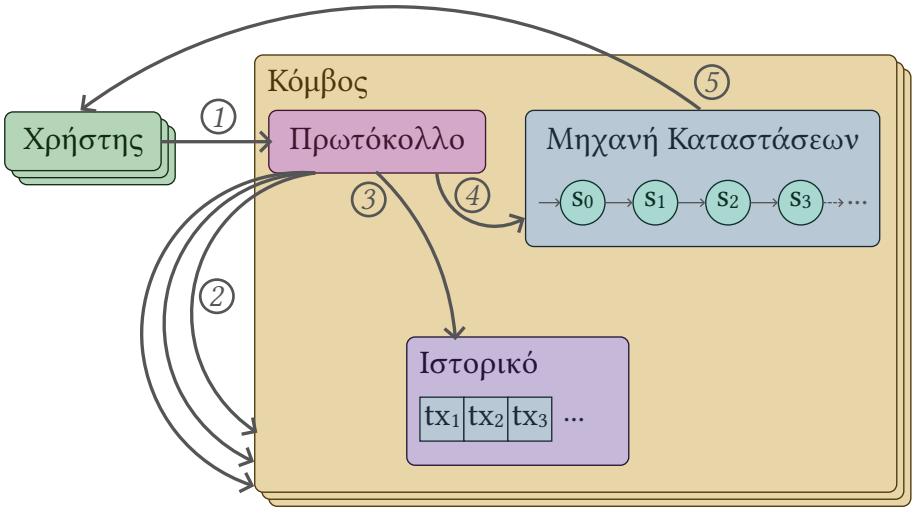
Σχήμα 2.1. Μια μηχανή καταστάσεων. Από την αρχική κατάσταση s_0, κάθε συναλλαγή οδηγεί μέσω της συνάρτησης δ σε νέα κατάσταση. Η ίδια σειρά συναλλαγών καταλήγει στην ίδια κατάσταση.

Ωστόσο, μια μηχανή σε έναν μόνο κόμβο είναι μοναδικό σημείο αστοχίας *(single point of failure)*. Η λύση είναι η αναπαραγωγή *(replication)*, δηλαδή η ίδια μηχανή τρέχει σε πολλούς κόμβους, τα αντίγραφα *(replicas)* ή επικυρωτές *(validators)*, ώστε η υπηρεσία να επιβιώνει όταν κάποιοι αστοχούν [37], [38]. Χάρη στον ντετερμινισμό, το πρόβλημα ανάγεται στο πώς όλα τα έντιμα αντίγραφα θα συμφωνήσουν σε μία κοινή, ολικά διατεταγμένη ακολουθία συναλλαγών. Την ακολουθία αυτή τη λέμε ιστορικό *(history)* ή *log* ή *καθολικό (ledger)*, και είναι ακριβώς το ίδιο το blockchain.

Ένα πρωτόκολλο ομοφωνίας λέμε ότι λύνει το πρόβλημα της αναπαραγωγής μηχανής καταστάσεων *(SMR)* όταν εγγυάται δύο ιδιότητες [37], [38]:

- **Συνέπεια *(consistency)***. Τα ιστορικά δύο έντιμων αντιγράφων δεν έχουν ποτέ διαφορετικές συναλλαγές στην ίδια θέση. Το ένα είναι πάντα πρόθεμα του άλλου.
- **Ζωτικότητα *(liveness)***. Κάθε έγκυρη συναλλαγή που φτάνει σε ένα έντιμο αντίγραφο μπαίνει τελικά στο ιστορικό όλων.

Το **Σχήμα 2.2** δείχνει την αρχιτεκτονική ενός κόμβου που τρέχει ένα τέτοιο πρωτόκολλο.



Σχήμα 2.2. Η αρχιτεκτονική ενός κόμβου. (1) Ένας χρήστης υποβάλλει συναλλαγή. (2) Το πρωτόκολλο συνεννοείται με τους άλλους κόμβους για τη σειρά. (3) Γράφει τη συναλλαγή στο ιστορικό. (4) Την περνά στη μηχανή καταστάσεων. (5) Η έξοδος επιστρέφει στον χρήστη.

Επομένως, η συνέπεια είναι ιδιότητα ασφάλειας *(safety)*, δηλαδή λέει ότι δεν συμβαίνει ποτέ κάτι κακό. Η ζωτικότητα, αντίθετα, λέει ότι κάτι καλό τελικά συμβαίνει. Η δυσκολία είναι να κρατηθούν και οι δύο όταν κάποια αντίγραφα αστοχούν ή δρουν εχθρικά, και όταν το δίκτυο καθυστερεί μηνύματα. Το τι ακριβώς υποθέτουμε για τις αστοχίες, για τη συμμετοχή και για το δίκτυο είναι το μοντέλο του προβλήματος, και έχει τρία μέρη.

1. **Μοντέλο σφαλμάτων *(fault model)***. Ορίζει με ποιους τρόπους επιτρέπεται να αστοχήσει ένα αντίγραφο, μαζί με ένα κατώφλι f για το πλήθος των ελαττωματικών αντιγράφων [39], [40], [41].

   - Στα **σφάλματα κατάρρευσης *(crash)*** το αντίγραφο σταματά οριστικά κάποια στιγμή, αλλά μέχρι τότε ακολουθεί πιστά το πρωτόκολλο.
   - Στα **σφάλματα παράλειψης *(omission)*** παραλείπει μηνύματα που όφειλε να στείλει, αλλά χωρίς να κατασκευάζει ψεύτικα.
   - Στα **βυζαντινά σφάλματα *(Byzantine faults)*** αποκλίνει αυθαίρετα, στέλνει αντικρουόμενα μηνύματα και συνεργάζεται με άλλα ελαττωματικά αντίγραφα εναντίον των έντιμων.

   Η κατάρρευση είναι ειδική περίπτωση της παράλειψης και η παράλειψη ειδική περίπτωση του βυζαντινού σφάλματος, οπότε ένα πρωτόκολλο ανθεκτικό στα βυζαντινά σφάλματα αντέχει και τα άλλα δύο. Για ένα τέτοιο πρωτόκολλο λέμε ότι έχει ανοχή σε βυζαντινά σφάλματα *(Byzantine Fault Tolerance, BFT)* [38].

2. **Μοντέλο συμμετοχής**. Ορίζει ποια και πόσα αντίγραφα τρέχουν το πρωτόκολλο. Σε ένα permissioned σύστημα το σύνολο των αντιγράφων είναι σταθερό και γνωστό από την αρχή, και κάθε αντίγραφο ξέρει ποια είναι τα υπόλοιπα. Σε ένα permissionless σύστημα το δίκτυο είναι ανοιχτό, ο καθένας μπαίνει με όσες ταυτότητες θέλει, και το σύνολο των συμμετεχόντων είναι άγνωστο και μεταβάλλεται διαρκώς [42].

3. **Μοντέλο χρόνου**. Ορίζει τι εγγυάται το δίκτυο για τον χρόνο παράδοσης των μηνυμάτων [43]. Στο σύγχρονο *(synchronous)* μοντέλο υπάρχει γνωστό άνω φράγμα Δ, οπότε ένα μήνυμα που δεν έφτασε στην ώρα του σημαίνει ελαττωματικό αποστολέα. Στο ασύγχρονο *(asynchronous)* μοντέλο δεν υπάρχει κανένα φράγμα, τα μηνύματα φτάνουν τελικά αλλά με αυθαίρετη καθυστέρηση [40]. Στο μερικώς σύγχρονο *(partially synchronous)* μοντέλο το φράγμα υπάρχει, αλλά είτε είναι άγνωστο είτε αρχίζει να ισχύει μετά από μια χρονική στιγμή, τον χρόνο σταθεροποίησης *(Global Stabilization Time, GST)* [43].

Ο συνδυασμός των τριών μοντέλων καθορίζει το τι είναι εφικτό. Συγκεκριμένα, στο ασύγχρονο μοντέλο δεν υπάρχει ντετερμινιστικό πρωτόκολλο ομοφωνίας που να αντέχει έστω και ένα σφάλμα κατάρρευσης *(θεώρημα FLP, 1985)* [40]. Στο μερικώς σύγχρονο, όπου δουλεύουν τα περισσότερα πρακτικά πρωτόκολλα, ένα πρωτόκολλο BFT χρειάζεται $n \geq 3f + 1$ αντίγραφα για να αντέξει f βυζαντινά, δηλαδή τα βυζαντινά πρέπει να είναι λιγότερα από το ένα τρίτο του συνόλου [38], [41], [43]. Ένα αντίγραφο προχωρά σε κάθε βήμα του πρωτοκόλλου όταν ακούσει από τουλάχιστον $2f + 1$ αντίγραφα, και το σύνολο αυτών των αντιγράφων λέγεται απαρτία *(quorum)*.

2.1.2 Αντίσταση σε επιθέσεις Sybil

Τα blockchain όπως το Bitcoin, το Ethereum και το SUI είναι permissionless, οπότε η απαρτία γίνεται δύσκολη. Δηλαδή, ένας κόμβος που δεν ξέρει πόσοι είναι συνολικά οι κόμβοι στο δίκτυο, δεν ξέρει και πόσες ψήφους να περιμένει. Επιπλέον, στο ανοιχτό περιβάλλον οι ταυτότητες είναι ζεύγη κλειδιών και δεν κοστίζουν τίποτα για να δημιουργηθούν, οπότε μία οντότητα μπορεί να εμφανιστεί ως χίλιοι συμμετέχοντες [44]. Η ευπάθεια αυτή των permissionless συστημάτων λέγεται επίθεση Sybil *(Sybil attack)*.

Γι' αυτό, η λύση είναι να δεθεί η επιρροή κάθε συμμετέχοντα με έναν σπάνιο πόρο αντί με τις ταυτότητές του. Στο Proof-of-Work ο πόρος είναι η υπολογιστική ισχύς. Στο Proof-of-Stake ο πόρος είναι το κεφάλαιο που δεσμεύει ο συμμετέχων στο ίδιο το κρυπτονόμισμα του συστήματος [45]. Έτσι, κάθε permissionless blockchain παίρνει δύο ξεχωριστές αποφάσεις [42]:

- **Αντίσταση σε επιθέσεις Sybil**. Ποιος έχει δικαίωμα να προτείνει και να ψηφίσει μπλοκ. Απαντούν μηχανισμοί όπως το Proof-of-Work και το Proof-of-Stake.

- **Πρωτόκολλο ομοφωνίας**. Ποια μπλοκ οριστικοποιούνται και με ποια σειρά. Απαντούν τα πρωτόκολλα ομοφωνίας, όπως της μακρύτερης αλυσίδας (Bitcoin) ή τύπου BFT.

Οι δύο αποφάσεις συνδυάζονται ελεύθερα, όπως δείχνει ο **Πίνακας 2.1**. Στη δεύτερη γραμμή και στη δεύτερη στήλη του πίνακα βρίσκεται το SUI, με Proof-of-Stake και ομοφωνία τύπου BFT.

Πίνακας 2.1. Οι δύο μηχανισμοί αντίστασης σε Sybil και οι δύο οικογένειες ομοφωνίας [42], [46], [47]. Το SUI ανήκει στο κελί Proof-of-Stake με ομοφωνία τύπου BFT [29]. Το Proof-of-Work δεν δίνει σταθερό σύνολο ψηφοφόρων για να σχηματιστεί απαρτία, οπότε στην πράξη συνδυάζεται μόνο με τη μακρύτερη αλυσίδα.

	Μακρύτερης αλυσίδας	**Τύπου BFT**
Proof-of-Work	Bitcoin	δεν χρησιμοποιείται
Proof-of-Stake	Cardano	Algorand, SUI

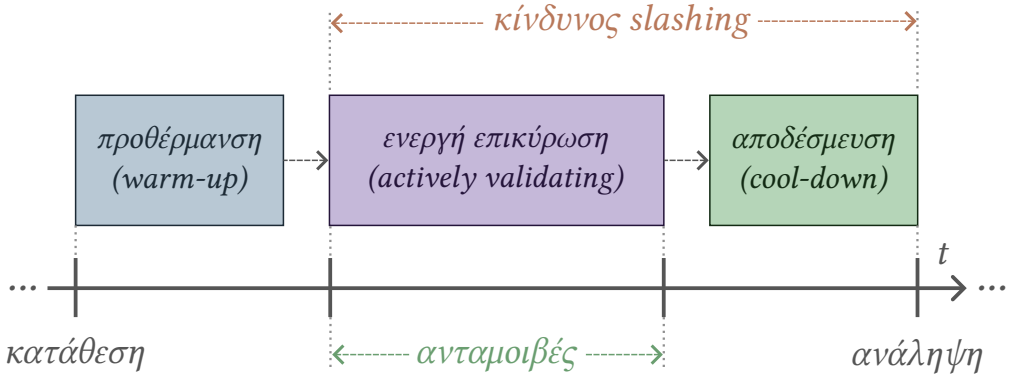
Στο Proof-of-Stake κάθε επικυρωτής *(validator)* δεσμεύει ένα ποσό, το stake του, και η πιθανότητα να επιλεγεί ηγέτης και το βάρος της ψήφου του είναι ανάλογα με αυτό το stake [45]. Αν σπάσει το stake του σε χίλιες ταυτότητες, το άθροισμα μένει ίδιο, οπότε η επίθεση Sybil δεν λειτουργεί. Η απαρτία γίνεται τότε ένα υποσύνολο που συγκεντρώνει πάνω από τα δύο τρίτα του συνολικού stake, και το όριο του ενός τρίτου ισχύει όπως πριν, σε κεφάλαιο αντί για αντίγραφα [46].

Επιπλέον, οι κάτοχοι που δεν θέλουν να λειτουργούν οι ίδιοι ως επικυρωτές μπορούν να αναθέσουν το stake τους σε κάποιον και να μοιράζονται τις ανταμοιβές. Η παραλλαγή αυτή λέγεται Delegated Proof-of-Stake και είναι αυτή που υλοποιεί το SUI [48]. Επίσης, ο χρόνος χωρίζεται σε εποχές *(epochs)* και μέσα σε κάθε εποχή το σύνολο των επικυρωτών και τα ποσά stake τους μένουν σταθερά, οπότε το σύστημα συμπεριφέρεται σαν permissioned [47].

Το Proof-of-Stake έχει τρία πλεονεκτήματα έναντι του Proof-of-Work [45], [46]. Καταναλώνει ελάχιστη ενέργεια, γιατί κανείς δεν χρειάζεται να λύσει υπολογιστικά δύσκολα προβλήματα. Επίσης, επιτρέπει την ομοφωνία τύπου BFT, που δίνει άμεση και ντετερμινιστική οριστικότητα *(finality)*. Δηλαδή, μόλις σχηματιστεί απαρτία το μπλοκ δεν αναιρείται ποτέ. Ακόμα, δίνει και λογοδοσία, γιατί ένας επικυρωτής που ψηφίζει δύο αντικρουόμενα μπλοκ αφήνει δημόσια απόδειξη και μπορεί να τιμωρηθεί. Το Proof-of-Work, αντίθετα, είναι δεμένο με τη μακρύτερη αλυσίδα, όπου η οριστικότητα είναι πάντα πιθανοτική [34].

Η λογοδοσία στηρίζεται στο ότι το stake μένει κλειδωμένο. Συγκεκριμένα, ο επικυρωτής κλειδώνει το κεφάλαιό του με μια κατάθεση και περνά από μια περίοδο προθέρμανσης *(warm-up period)*, ώστε το νέο stake να γίνει γνωστό σε όλους πριν αρχίσει να προσμετράται [49]. Στην ενεργή φάση προτείνει και ψηφίζει μπλοκ και παίρνει ανταμοιβές *(staking rewards)*. Όποιος παραβιάζει τους κανόνες τιμωρείται με slashing, την κατάσχεση μέρους του stake [46]. Όταν αποχωρεί, το stake δεν επιστρέφεται αμέσως. Μένει κλειδωμένο για μια περίοδο αποδέσμευσης

(cool-down), μέσα στην οποία μπορεί ακόμη να κατασχεθεί αν αποκαλυφθεί παλιότερη παραβίαση, και μόνο μετά γίνεται η ανάληψη (**Σχήμα 2.3**).



Σχήμα 2.3. Ο κύκλος ζωής του stake ενός επικυρωτή. Ανταμοιβές κερδίζονται μόνο κατά την ενεργή επικύρωση. Ο κίνδυνος του slashing εκτείνεται και στην περίοδο αποδέσμευσης.

2.1.3 Έξυπνα συμβόλαια

Ο Szabo [27] πρότεινε το 1997 ότι μια σύμβαση μπορεί να ενσωματωθεί σε ένα πρωτόκολλο, ώστε η παραβίασή της να γίνει ακριβή ή αδύνατη. Το παράδειγμά του ήταν ο αυτόματος πωλητής, που εκτελεί μια σύμβαση χωρίς δικαστήριο. Από την πρόταση αυτή έλειπε, όμως, ένας ουδέτερος υπολογιστής που εκτελεί τους κανόνες και δεν τον ελέγχει κανένας συμβαλλόμενος. Ο υπολογιστής αυτός είναι ακριβώς η αναπαραγόμενη μηχανή καταστάσεων της προηγούμενης ενότητας.

Έξυπνο συμβόλαιο *(smart contract, SC)* είναι ένα πρόγραμμα με μόνιμη κατάσταση, αποθηκευμένο μέσα στην κατάσταση του blockchain δικτύου, που εκτελείται από όλους τους κόμβους ως μέρος της συνάρτησης μετάβασης [27], [50]. Το καλούν συναλλαγές και τα αποτελέσματά του γίνονται μέρος της κοινής κατάστασης. Τρεις ιδιότητες του ορισμού έχουν σημασία για την εργασία:

- **Ντετερμινισμός**. Κάθε κόμβος πρέπει να βγάλει το ίδιο αποτέλεσμα, οπότε το συμβόλαιο δεν μπορεί να διαβάσει χρόνο, τυχαιότητα ή εξωτερική πηγή.
- **Δημοσιότητα**. Ο κώδικας και η κατάσταση είναι ορατά σε όλους. Ο καθένας ελέγχει τι κάνει το συμβόλαιο.
- **Αμεταβλητότητα**. Ο κώδικας που δημοσιεύτηκε δεν διορθώνεται εύκολα. Ένα σφάλμα είναι μόνιμο και δημόσιο, δηλαδή εκμεταλλεύσιμο από όποιον το βρει πρώτος.

2.1.4 Tokens και NFTs

Τα έξυπνα συμβόλαια, που είδαμε στην προηγούμενη ενότητα, είναι προγράμματα που τρέχουν πάνω σε ένα δίκτυο blockchain, όμως το ίδιο το δίκτυο αναγνωρίζει μόνο το νόμισμα με το οποίο πληρώνονται οι συναλλαγές του. Μια μετοχή ή η ιδιοκτησία ενός έργου τέχνης δεν υπάρχουν ως έννοιες στο δίκτυο, αν δεν τις ορίσει κάποιο έξυπνο συμβόλαιο. Γι' αυτό, φτιάχτηκαν τα tokens [51].

Token στην τεχνολογία blockchain λέγεται μια εγγραφή στην κατάσταση του blockchain δικτύου που αναπαριστά ένα αγαθό ή ένα δικαίωμα. Το φτιάχνει και το διαχειρίζεται ένα έξυπνο συμβόλαιο, που ορίζει και τους κανόνες μεταβίβασής του [51]. Η βασική διάκριση των tokens είναι οικονομική. Ένα ανταλλάξιμο token *(Fungible Token, FT)* είναι εκείνο του οποίου οποιαδήποτε μονάδα αντικαθιστά οποιαδήποτε άλλη χωρίς να αλλάζει η αξία ή τα δικαιώματα του κατόχου. Σε ένα μη ανταλλάξιμο token *(Non-Fungible Token, NFT)*, αντίθετα, κάθε μονάδα έχει διακριτή ταυτότητα, με δικό της αναγνωριστικό και μεταδεδομένα που τη συνδέουν με ό,τι αναπαριστά [51]. Για παράδειγμα, ένα ευρώ είναι ανταλλάξιμο, ένας πίνακας τέχνης δεν είναι.

Ανάμεσα στα δύο υπάρχουν τα ημι-ανταλλάξιμα tokens *(Semi-Fungible Token, SFT)*. Αυτά είναι tokens με διακριτή ταυτότητα και διαιρετή αξία, όπως ένα ομόλογο συγκεκριμένης έκδοσης [52]. Τα κλάσματα ενός έργου τέχνης βρίσκονται ακριβώς εκεί, γιατί το έργο τέχνης είναι μοναδικό, ενώ αυτά είναι ανταλλάξιμα μεταξύ τους.

Μια δεύτερη διάσταση είναι η μεταβιβασιμότητα. Τα περισσότερα tokens μεταβιβάζονται ελεύθερα, κάποια, όμως, σκόπιμα όχι, όπως ένα token που βεβαιώνει έναν τίτλο σπουδών [53]. Στον **Πίνακας 2.2** παρουσιάζουμε παραδείγματα.

Πίνακας 2.2. Οι δύο άξονες κατάταξης των tokens, ανταλλάξιμο ή μη και μεταβιβάσιμο ή μη.

	Ανταλλάξιμο	**Ημι-ανταλλάξιμο**	**Μη ανταλλάξιμο**
Μεταβιβάσιμο ελεύθερα	• νόμισμα • stablecoin • token χρησιμότητας • token διακυβέρνησης	• ομόλογο • κουπόνι • εισιτήριο ίδιας κατηγορίας θέσεων • αντικείμενο παιχνιδιού ίδιου τύπου	• ψηφιακό έργο • έργο τέχνης • συλλεκτικό αντικείμενο • όνομα τομέα
Μεταβιβάσιμο υπό περιορισμό	• μετοχή σε μητρώο • ρυθμιζόμενο νόμισμα • μερίδιο αμοιβαίου κεφαλαίου • ψηφιακό νόμισμα κεντρικής τράπεζας	• κλάσμα έργου τέχνης • μερίδιο ακινήτου • κλάσμα συλλεκτικού αντικειμένου • ομόλογο με λίστα εγκεκριμένων κατόχων	• ονομαστικό εισιτήριο • άδεια χρήσης • ονομαστική συνδρομή • τίτλος ακινήτου με έγκριση μεταβίβασης
Δεν μεταβιβάζεται ποτέ	• πόντοι φήμης • δικαίωμα ψήφου • πόντοι επιβράβευσης • μονάδες εμπειρίας σε παιχνίδι	-	• διαπιστευτήριο σπουδών • πιστοποιητικό ταυτότητας • άδεια οδήγησης • βεβαίωση συμμετοχής σε εκδήλωση • ιατρική βεβαίωση

2.2 Web3 και αποκεντρωμένες εφαρμογές

Ο ιστός χωρίζεται σε τρεις γενιές, με κριτήριο ποιος παράγει το περιεχόμενο και ποιος ελέγχει τα δεδομένα [54], [55]. Στην πρώτη γενιά (Web1) οι σελίδες ήταν στατικές, δηλαδή λίγοι έγραφαν και πολλοί διάβαζαν δεδομένα. Στη δεύτερη (Web2) το περιεχόμενο το παράγουν οι χρήστες, μέσα σε πλατφόρμες που ανήκουν σε εταιρείες. Ο χρήστης έχει έναν λογαριασμό, ενώ η πλατφόρμα κρατά τα δεδομένα του και μπορεί να αλλάξει τους όρους ή να κλείσει τον λογαριασμό. Στην τρίτη (Web3) ο χρήστης κρατά ένα ζεύγος κλειδιών αντί για λογαριασμό, η ιδιοκτησία γράφεται σε ένα blockchain που δεν ανήκει σε κανέναν και η λογική τρέχει σε έξυπνα συμβόλαια [54].

Τον όρο Web3 τον εισήγαγε το 2014 ο Wood [54], για μια στοίβα πρωτοκόλλων όπου η εμπιστοσύνη σε παρόχους αντικαθίσταται από κρυπτογραφικές εγγυήσεις. Ο όρος δεν έχει σχέση με το «Web 3.0» των Berners-Lee κ.ά. [56], που είναι ο σημασιολογικός ιστός.

2.2.1 Αποκεντρωμένες εφαρμογές

Αποκεντρωμένη εφαρμογή *(decentralized application, DApp)* είναι μια εφαρμογή που τρέχει σε ένα δίκτυο υπολογιστών αντί σε έναν κεντρικό υπολογιστή, οπότε την εκτέλεσή της δεν την ελέγχει μία μόνο οντότητα [57]. Στην περίπτωση του blockchain αυτό σημαίνει ότι η λογική της είναι γραμμένη σε έξυπνα συμβόλαια και η κρίσιμη κατάστασή της ζει στο blockchain δίκτυο, και όχι σε κεντρικό διακομιστή.

Επίσης, οι αποκεντρωμένες εφαρμογές κληρονομούν τις ιδιότητες του δικτύου blockchain στο οποίο τρέχουν. Για αυτόν τον λόγο, μια αποκεντρωμένη εφαρμογή μπορεί να έχει χειρότερες επιδόσεις από μια παραδοσιακή εφαρμογή Web2. Από την άλλη, μπορεί να τη χρησιμοποιήσει οποιοσδήποτε χωρίς να ζητήσει άδεια, ενώ ο κώδικας και η κατάστασή της είναι ορατά σε όλους. Οι ιδιότητες αυτές επιτρέπουν στις αποκεντρωμένες εφαρμογές να αντιμετωπίσουν προβλήματα για τα οποία οι παραδοσιακές εφαρμογές δεν δίνουν αποτελεσματικές λύσεις.

Συγκεκριμένα, σε σχέση με τις παραδοσιακές εφαρμογές, μια αποκεντρωμένη εφαρμογή έχει [58]:

- **Ιδιοκτησία χωρίς μητρώο τρίτου**. Ο χρήστης κατέχει ένα ψηφιακό αντικείμενο επειδή ελέγχει το ιδιωτικό κλειδί που το ξεκλειδώνει, όχι επειδή μια εταιρεία έχει μια εγγραφή στο όνομά του.
- **Συνθεσιμότητα**. Επειδή όλες οι εφαρμογές ζουν στην ίδια μηχανή, ένα συμβόλαιο καλεί ένα άλλο μέσα στην ίδια συναλλαγή, χωρίς συμφωνία με τους δημιουργούς του. Η συναλλαγή είτε πετυχαίνει ολόκληρη είτε αποτυγχάνει ολόκληρη. Οι Werner κ.ά. [58] παρομοιάζουν τα συμβόλαια με τουβλάκια Lego που κουμπώνουν μεταξύ τους.

- **Επαληθευσιμότητα**. Όποιος θέλει μπορεί να διαβάσει τον κώδικα του συμβολαίου και όλη την ιστορία της κατάστασής του, και να ελέγξει μόνος του ότι η εφαρμογή κάνει αυτό που λέει. Αντίθετα, σε μια πλατφόρμα Web2 ο κώδικας και τα δεδομένα μένουν κλειστά, οπότε ο χρήστης πρέπει να εμπιστευτεί τον πάροχο.

Μερικά χαρακτηριστικά παραδείγματα αποκεντρωμένων εφαρμογών είναι:

- **Uniswap**. Ανταλλακτήριο tokens χωρίς βιβλίο εντολών [59]. Ένα συμβόλαιο κρατά αποθέματα δύο tokens και ορίζει την τιμή από την αναλογία τους. Ο χρήστης στέλνει το ένα token στο συμβόλαιο και παίρνει από τα αποθεματά του το άλλο, χωρίς να χρειάζεται να βρεθεί κάποιος άλλος χρήστης που θέλει την αντίστροφη ανταλλαγή [58].

- **MakerDAO**. Εκδίδει το stablecoin DAI [60]. Ο χρήστης κλειδώνει εγγύηση σε ένα συμβόλαιο και δανείζεται DAI. Η ισοτιμία με το δολάριο κρατιέται από τους κανόνες του συμβολαίου και από ψηφοφορία των κατόχων του token διακυβέρνησης [58].

- **OpenSea**. Αγορά για μη ανταλλάξιμα tokens *(NFT)* [61]. Η διεπαφή χρήστη είναι συμβατικός ιστότοπος, αλλά η ιδιοκτησία και η μεταβίβαση γίνονται στο συμβόλαιο του κάθε token [62].

- **Farcaster**. Αποκεντρωμένο κοινωνικό δίκτυο πάνω στο Ethereum, εκτός του χώρου της χρηματοοικονομικής [63]. Η ταυτότητα του χρήστη και οι σχέσεις του με τους υπόλοιπους ανήκουν στον ίδιο και όχι στην εφαρμογή.

2.2.2 Διακυβέρνηση και αποκεντρωμένες αυτόνομες οργανώσεις (DAO)

Οι εφαρμογές της προηγούμενης ενότητας έχουν παραμέτρους που κάποιος πρέπει να ορίζει. Στο MakerDAO, για παράδειγμα, κάποιος αποφασίζει ποια tokens γίνονται δεκτά ως εγγύηση [58]. Ένα συμβόλαιο με σφάλμα πρέπει επίσης να αντικατασταθεί. Ο κώδικας, όμως, δεν αλλάζει από μόνος του. Κάποιος πρέπει να έχει το δικαίωμα να τον αλλάξει.

Στην πράξη, στις εφαρμογές αποκεντρωμένων χρηματοοικονομικών *(Decentralized Finance, DeFi)* το δικαίωμα αυτό το έχει στην αρχή η ομάδα που έφτιαξε την εφαρμογή, ή μια μικρή επιτροπή [58]. Η ομάδα υπόσχεται ότι αργότερα θα δώσει τον έλεγχο στους χρήστες.

Η απάντηση σε αυτό το πρόβλημα είναι η αποκεντρωμένη αυτόνομη οργάνωση *(Decentralized Autonomous Organization, DAO)*. Η DAO είναι μια οργάνωση της οποίας οι κανόνες λειτουργίας και λήψης αποφάσεων είναι γραμμένοι σε έξυπνα συμβόλαια σε δημόσιο blockchain και εκτελούνται αυτόματα, χωρίς κεντρικό έλεγχο [64]. Στις εφαρμογές DeFi η DAO έχει ένα token διακυβέρνησης [58]. Όποιος κατέχει το token ψηφίζει για τις αλλαγές της εφαρμογής, ανάλογα με το πλήθος των tokens του. Για να υποβάλει πρόταση, χρειάζεται ένα ελάχιστο πλήθος tokens, δικών του ή εξουσιοδοτημένων από άλλους κατόχους. Η πρόταση είναι εκτελέσιμος κώδικας και, αν συγκεντρώσει το ελάχιστο πλήθος ψήφων, την εκτελεί το ίδιο το συμβόλαιο [58].

2.3 Tokenization πραγματικών περιουσιακών στοιχείων

Ό,τι είδαμε ως εδώ αφορούσε αγαθά που γεννιούνται ψηφιακά. Ένα token υπάρχει επειδή το λέει το blockchain δίκτυο, και αυτό είναι η μόνη αρχή που αποφασίζει ποιος το κατέχει. Ένας πίνακας, όμως, δεν λειτουργεί έτσι. Υπάρχει ανεξάρτητα από κάθε τέτοιο δίκτυο, μπορεί να καταστραφεί ή να κλαπεί, και την ιδιοκτησία του την κρίνει τελικά ένα δικαστήριο. Το tokenization πραγματικών περιουσιακών στοιχείων *(Real World Assets, RWAs)* είναι η προσπάθεια να γεφυρωθεί αυτό το χάσμα [65], [66].

2.3.1 Ορισμός και κύκλος ζωής

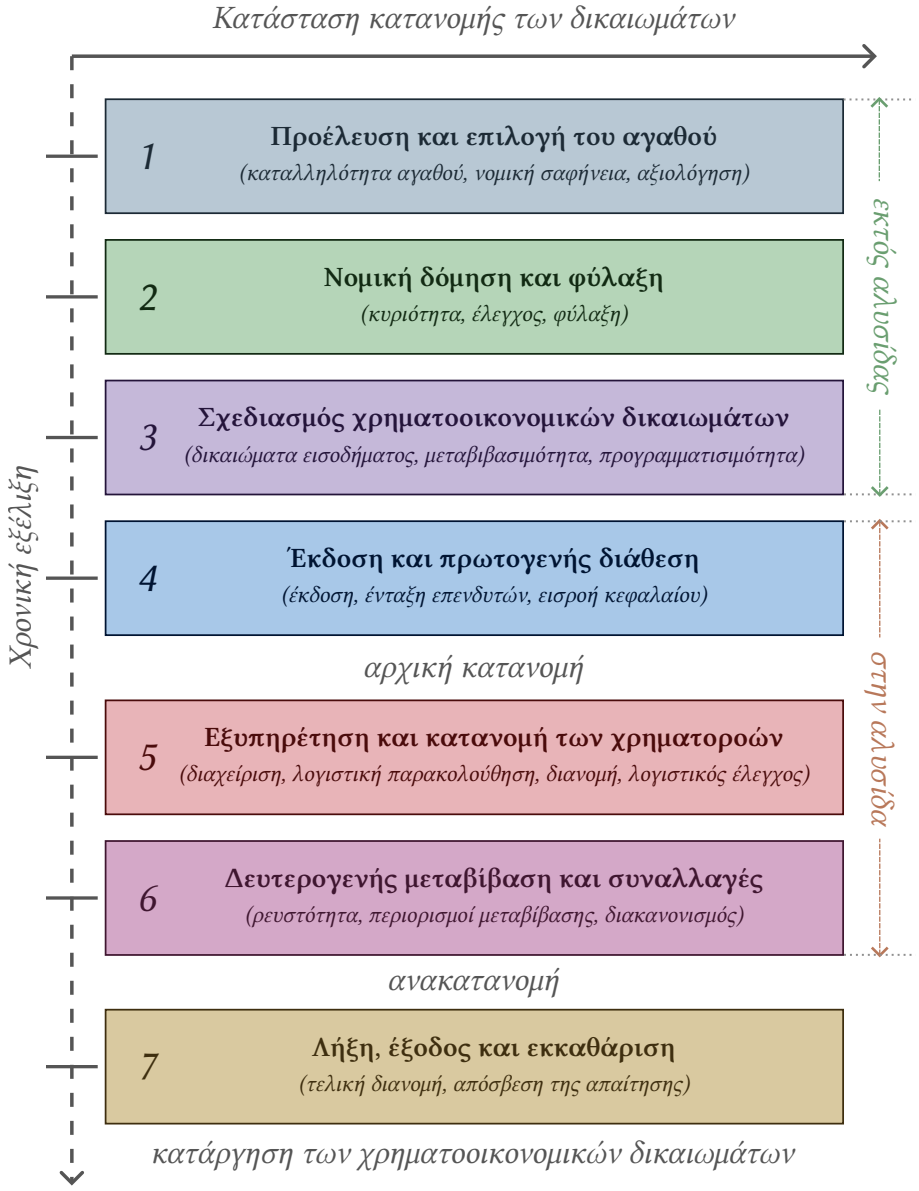
Το tokenization ενός πραγματικού περιουσιακού στοιχείου είναι η αναπαράσταση οικονομικών δικαιωμάτων επί ενός περιουσιακού στοιχείου εκτός blockchain, μέσω ενός ή περισσότερων tokens σε ένα blockchain. [65]. Τα περιουσιακά στοιχεία που αναπαρίστανται με αυτόν τον τρόπο ονομάζονται πραγματικά περιουσιακά στοιχεία σε μορφή token *(Real World Assets, RWAs)*. Στόχος της διαδικασίας είναι η μείωση του κόστους συναλλαγής και επαλήθευσης, καθώς και του ελάχιστου απαιτούμενου ποσού επένδυσης [67].

Κάθε RWA έχει δύο καταστάσεις, μία εκτός blockchain δικτύου και μία μέσα στο blockchain δίκτυο. Η πρώτη περιλαμβάνει τα νομικά δικαιώματα επί του αγαθού και, όταν πρόκειται για φυσικό αντικείμενο, και τη φυσική του κατάσταση. Η δεύτερη καταγράφει την κατοχή των tokens και τα δεδομένα που διαχειρίζονται τα έξυπνα συμβόλαια. Η ορθή λειτουργία του συστήματος προϋποθέτει τη συνεπή αντιστοίχιση των δύο καταστάσεων, η οποία δεν εξασφαλίζεται από το blockchain μόνο του [65].

Το tokenization υπόσχεται τρία πράγματα [65].

- **Συγχρονισμός της κατάστασης**. Οι μεταβολές του φυσικού αγαθού αποτυπώνονται στα μεταδεδομένα του token.
- **Προγραμματισιμότητα**. Το συμβόλαιο εκτελεί μόνο του τα γεγονότα του κύκλου ζωής του αγαθού, όπως την καταβολή ενός δικαιώματος σε κάθε μεταπώληση.
- **Ατομικός διακανονισμός**. Γνωστός και ως παράδοση έναντι πληρωμής. Η μεταβίβαση και η πληρωμή είτε ολοκληρώνονται και οι δύο είτε καμία, γιατί είναι μία συναλλαγή [68].

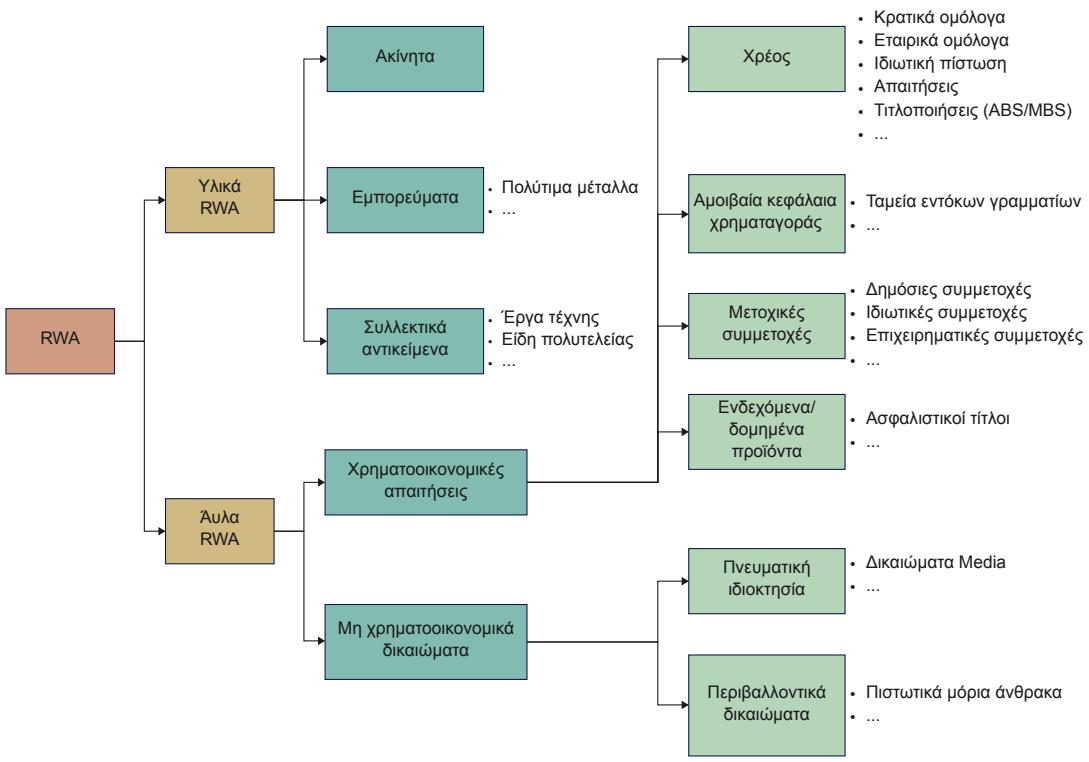
Ο κύκλος ζωής ενός RWA έχει επτά στάδια [65] (**Σχήμα 2.4**). Τα στάδια είναι η προέλευση και επιλογή του αγαθού, η νομική δόμηση και φύλαξη, ο σχεδιασμός των δικαιωμάτων που αγοράζει ο κάτοχος του token, η έκδοση και πρωτογενής διάθεση, η εξυπηρέτηση, η δευτερογενής μεταβίβαση, και η λήξη με πώληση, εξαργύρωση ή εκκαθάριση.



Σχήμα 2.4. Ο κύκλος ζωής ενός RWA σε επτά στάδια, κατά [65]. Τα τρία πρώτα στάδια τρέχουν εξ ολοκλήρου εκτός blockchain δικτύου, τα τρία επόμενα τα αυτοματοποιεί το συμβόλαιο.

2.3.2 Ταξινόμηση των RWA

Οι Luo κ.ά. [65] ταξινομούν τα RWA με βάση τη φύση του αγαθού αναφοράς και τα δικαιώματα που συνδέονται με αυτό (**Σχήμα 2.5**). Η πρώτη διαίρεση χωρίζει τα υλικά *(tangible)* από τα άυλα *(intangible)* αγαθά, ανάλογα με το αν το αγαθό έχει φυσική μορφή. Τα άυλα χωρίζονται ξανά σε δύο ομάδες. Από τη μία, οι χρηματοοικονομικές απαιτήσεις *(financial claims)* δίνουν συμβατικό δικαίωμα σε μελλοντικές χρηματοροές που ορίζονται εκ των προτέρων. Από την άλλη, τα μη χρηματοοικονομικά δικαιώματα *(non-financial rights)* δίνουν δικαίωμα χρήσης, άδειας ή συμμετοχής σε έσοδα, χωρίς προκαθορισμένες πληρωμές.



Σχήμα 2.5. Ταξινόμηση των RWA κατά αγαθό αναφοράς, κατά [65]. Το έργο τέχνης της εφαρμογής ανήκει στα συλλεκτικά αντικείμενα, στο υλικό σκέλος του δέντρου.

Τα υλικά *(tangible)* RWA έχουν τρεις κατηγορίες [65].

- **Ακίνητα *(real estate)***. Το token είναι συνήθως μερίδιο σε ακίνητο που παράγει εισόδημα. Για παράδειγμα, η Lofty κρατά ενοικιαζόμενα ακίνητα στις ΗΠΑ μέσω εταιρειών ειδικού σκοπού *(Special Purpose Vehicle, SPV)* και μοιράζει τα ενοίκια μέσα στο blockchain δίκτυο [65], [69].

- **Εμπορεύματα *(commodities)***. Η δραστηριότητα συγκεντρώνεται στα πολύτιμα μέταλλα, γιατί αποθηκεύονται και ελέγχονται εύκολα. Για παράδειγμα, το PAX Gold και το Tether Gold καλύπτονται από ράβδους χρυσού σε φύλαξη και εξαργυρώνονται μέσω επαγγελματία θεματοφύλακα [65], [70].

- **Συλλεκτικά αντικείμενα *(collectibles)***. Έργα τέχνης και είδη πολυτελείας, το μικρότερο τμήμα της αγοράς. Η δευτερογενής αγορά μένει ρηχή και η αποτίμηση εξαρτάται από εκτιμητές [65]. Σε αυτή την κατηγορία ανήκει η εφαρμογή μας.

Οι χρηματοοικονομικές απαιτήσεις είναι η μεγαλύτερη ομάδα και έχουν τέσσερις κατηγορίες [65].

- **Χρέος *(debt)***. Κρατικά ομόλογα και έντοκα γραμμάτια, εταιρικά ομόλογα, ιδιωτική πίστωση, απαιτήσεις και τιτλοποιήσεις. Για παράδειγμα, η Société Générale Forge εκδίδει εταιρικά ομόλογα σε ψηφιακή μορφή σε ρυθμιζόμενη πλατφόρμα [65], [71]. Άλλα παραδείγματα αποτελούν τα Maple και Goldfinch, που δανείζουν μικρές επιχειρήσεις μέσα στο blockchain δίκτυο [70].

- **Αμοιβαία κεφάλαια χρηματαγοράς *(money-market funds)***. Ο κάτοχος έχει μερίδιο σε ένα συλλογικό όχημα που κρατά βραχυπρόθεσμο χρέος, όχι το ίδιο το ομόλογο. Για παράδειγμα, το ταμείο BUIDL της BlackRock και η πλατφόρμα Benji της Franklin Templeton είναι τα γνωστότερα παραδείγματα [65], [71].

- **Μετοχικές συμμετοχές *(equity)***. Δημόσιες, ιδιωτικές και επιχειρηματικές συμμετοχές, όπως οι εκδόσεις της Securitize [65], [71]. Σε αυτή την κατηγορία, η ανάπτυξη είναι αργή, γιατί το δίκαιο κινητών αξιών περιορίζει ποιος επενδύει και πού γίνεται η διαπραγμάτευση.

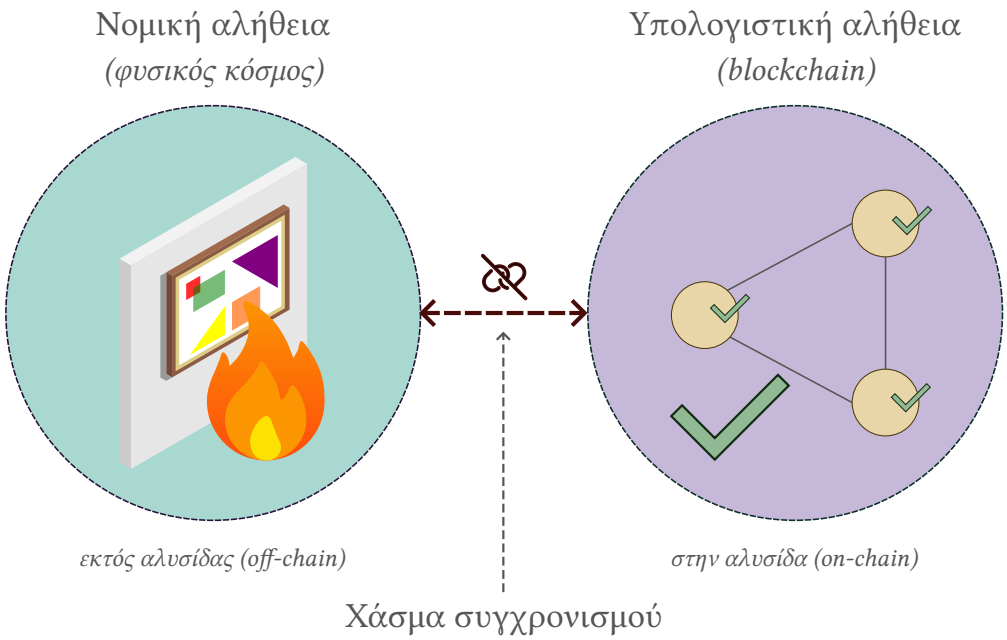
- **Ενδεχόμενα και δομημένα προϊόντα *(contingent and structured)***. Η απόδοση εξαρτάται από ένα γεγονός, όπως στους ασφαλιστικούς τίτλους για φυσικές καταστροφές [65]. Εδώ όμως, η αγορά είναι ακόμη πειραματική.

Παράλληλα, τα μη χρηματοοικονομικά δικαιώματα έχουν δύο κατηγορίες [65]. Η πνευματική ιδιοκτησία *(IP and royalties)* περιλαμβάνει tokens που δίνουν μερίδιο στα δικαιώματα μουσικών ή κινηματογραφικών έργων [65], ενώ τα περιβαλλοντικά δικαιώματα *(environmental and resource rights)* έχουν γνωστότερο παράδειγμα τα πιστωτικά μόρια άνθρακα *(carbon credits)* του Toucan [71].

Το δικαίωμα παρακολούθησης που συνοδεύει το έργο τέχνης δεν ανήκει ωστόσο στο υλικό σκέλος του δέντρου. Είναι δικαίωμα πνευματικής ιδιοκτησίας, δηλαδή άυλο αγαθό.

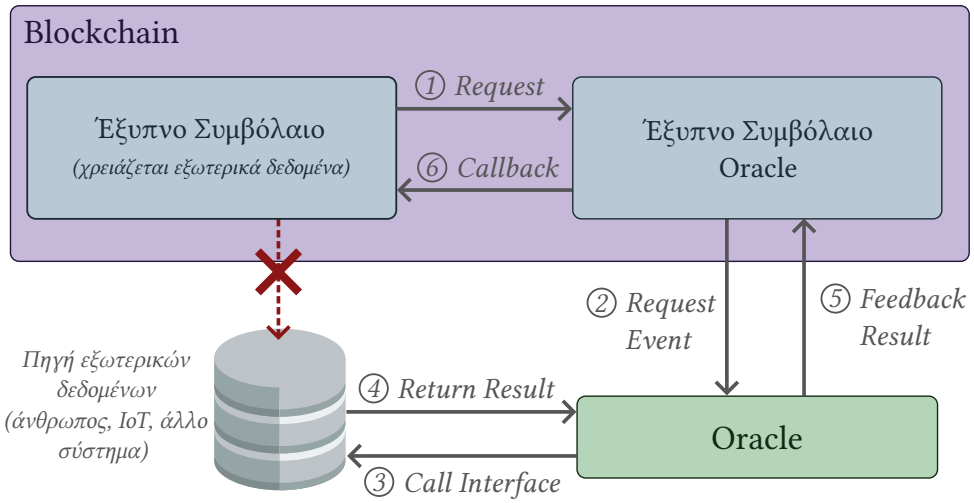
2.3.3 Το χάσμα συγχρονισμού και το πρόβλημα των oracles

Στην ενότητα 2.3.1 είδαμε ότι κάθε RWA έχει δύο καταστάσεις, τη νομική εκτός blockchain δικτύου και την υπολογιστική μέσα σε αυτό, και ότι το σύστημα δουλεύει αποτελεσματικά μόνο όσο οι δύο συμφωνούν. Τη συμφωνία αυτή δεν την εξασφαλίζει το blockchain δίκτυο. Αν μια φωτιά κάψει τον πίνακα ή κάποιος τον κλέψει από τον χώρο φύλαξης, το δίκτυο δεν το μαθαίνει. Τα αντίγραφα συνεχίζουν να συμφωνούν μεταξύ τους, αλλά αυτό στο οποίο συμφωνούν δεν ισχύει πια στον φυσικό κόσμο. Το πρόβλημα αυτό το ονομάζουμε χάσμα συγχρονισμού (**Σχήμα 2.6**). Για να κλείσει, κάποιος πρέπει να πει στο blockchain δίκτυο τι έγινε.



Σχήμα 2.6. Το χάσμα συγχρονισμού. Η ομοφωνία εγγυάται τη συνέπεια μόνο της δεξιάς πλευράς. Όταν το φυσικό αγαθό καταστραφεί, τα αντίγραφα συνεχίζουν να συμφωνούν σε μια κατάσταση που δεν ισχύει πια.

Το blockchain δίκτυο είναι κλειστό σύστημα. Ένα συμβόλαιο διαβάζει μόνο δεδομένα που βρίσκονται ήδη στο blockchain [72]. Ο λόγος είναι ο ντετερμινισμός της ενότητας 2.1. Αν ένα έξυπνο συμβόλαιο διάβαζε μόνο του μια τιμή από τον έξω κόσμο, κάθε επικυρωτής θα τη διάβαζε από δικό του κανάλι σε δική του στιγμή, και δύο έντιμοι επικυρωτές θα υπολόγιζαν διαφορετικές καταστάσεις από τις ίδιες συναλλαγές. Τα εξωτερικά δεδομένα φτάνουν, λοιπόν, στο blockchain δίκτυο με έναν μόνο τρόπο, γραμμένα μέσα σε μια συναλλαγή που υπογράφει κάποιος. Ο μηχανισμός αυτός λέγεται oracle. Είναι ένας εξωτερικός μηχανισμός που παρατηρεί γεγονότα του πραγματικού κόσμου και τα αναφέρει στο blockchain δίκτυο, για να τα χρησιμοποιήσουν τα συμβόλαια [73] (**Σχήμα 2.7**).

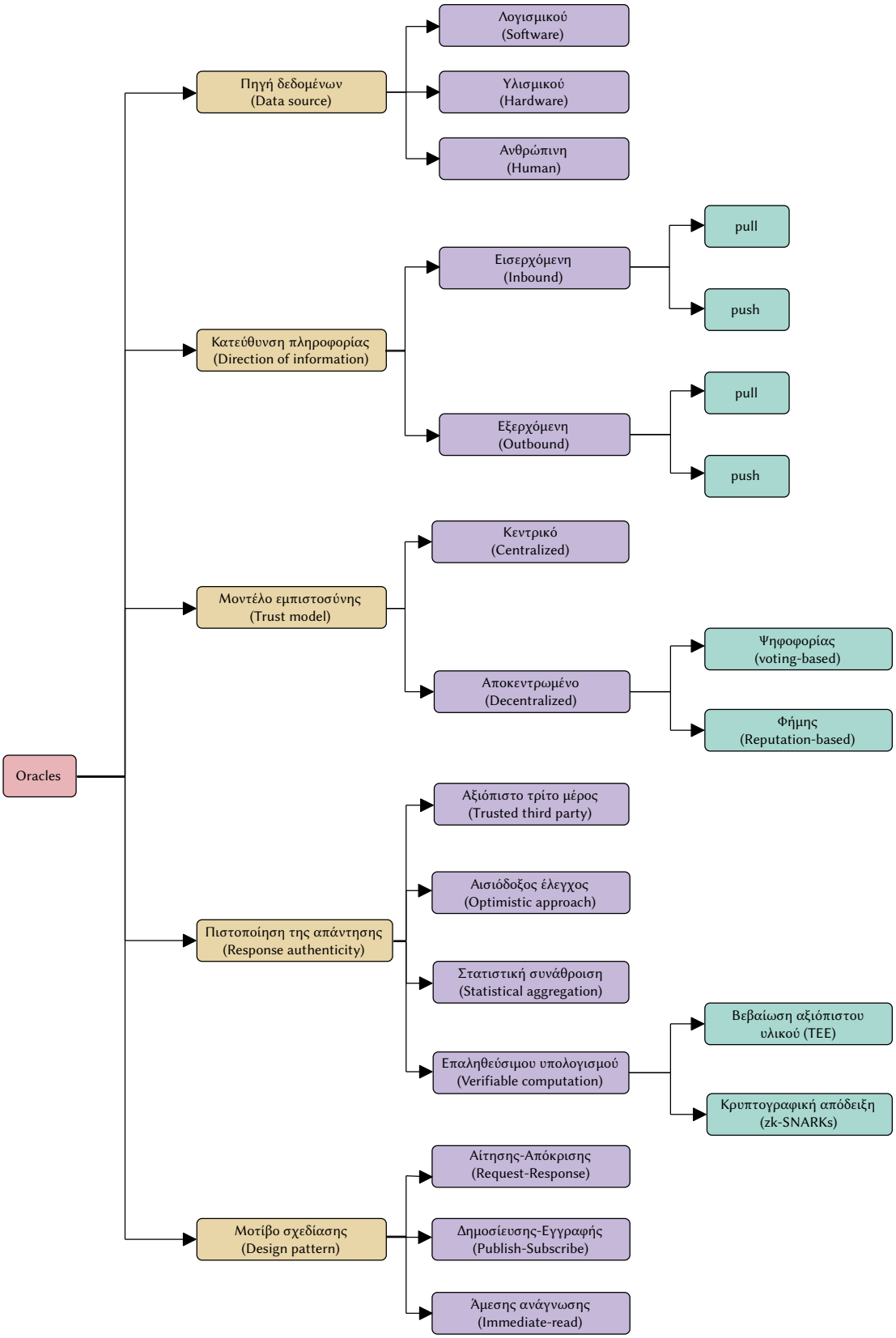


Σχήμα 2.7. Ένα oracle ανάμεσα στο blockchain δίκτυο και μια εξωτερική πηγή δεδομένων. Το συμβόλαιο δεν μπορεί να διαβάσει μόνο του την πηγή. (1) Ζητά μια τιμή από το συμβόλαιο του oracle. (2) Το oracle λαμβάνει το αίτημα ως γεγονός του blockchain δικτύου. (3) Καλεί τη διεπαφή της πηγής. (4) Παίρνει το αποτέλεσμα. (5) Το γράφει στο blockchain δίκτυο με νέα συναλλαγή. (6) Το συμβόλαιο ενημερώνεται.

Το blockchain δίκτυο εγγυάται ότι η τιμή που γράφτηκε δεν θα αλλοιωθεί μετά. Δεν εγγυάται, όμως, ότι ήταν σωστή όταν γράφτηκε. Έτσι, η εμπιστοσύνη που το δίκτυο αφαίρεσε από τους μεσάζοντες ξαναμπαίνει στο πρόσωπο του oracle. Αυτό είναι το πρόβλημα των oracles *(The Oracle Problem)* [74]. Για παράδειγμα, στα πρωτόκολλα αποκεντρωμένων χρηματοοικονομικών *(DeFi)* ο πιο συνηθισμένος τρόπος επίθεσης σήμερα είναι η χειραγώγηση του oracle που δίνει τις τιμές [75]. Τα συμβόλαια στις επιθέσεις αυτές εκτελούν πιστά τη λογική τους πάνω σε τιμές που δεν έπρεπε ποτέ να θεωρηθούν αληθινές.

2.3.4 Ταξινόμηση των oracles

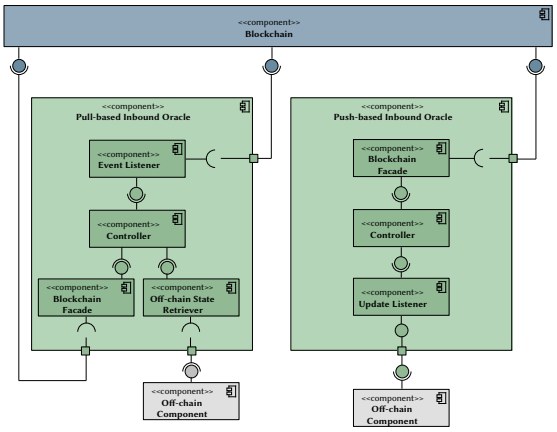
Οι βιβλιογραφικές μελέτες των oracles δεν συμφωνούν σε μία απόλυτη ταξινόμηση των oracles [72], [73], [75], [76], [77], [78], [79]. Ο Beniiche [76] κατατάσσει τα oracles με τρία κριτήρια, την πηγή των δεδομένων, την κατεύθυνση της πληροφορίας και την εμπιστοσύνη. Οι Al-Breiki κ.ά. [73] προσθέτουν το μοτίβο σχεδίασης. Οι Mühlberger κ.ά. [72], αντίθετα, κρατούν μόνο την κατεύθυνση και το ποιος ξεκινά τη ροή πληροφορίας. Οι Pasdar κ.ά. [77] χωρίζουν τις υλοποιήσεις σε ψηφοφορίας *(voting-based)* και φήμης *(reputation-based)*. Οι Eskandari κ.ά. [75] αναλύουν τη λειτουργία ενός oracle σε διαδοχικές μονάδες, από τις πηγές μέχρι τη συνάθροιση και τη διευθέτηση διαφωνιών. Οι Bartholic κ.ά. [78] ταξινομούν με βάση το ποιοι μπορούν να τα απαντήσουν. Τέλος, οι Finkbeiner και Almashaqbeh [79] χωρίζουν την εμπιστοσύνη σε δύο ερωτήσεις, ποιος τροφοδοτεί το blockchain δίκτυο και πώς βεβαιώνεται η απάντηση. Διαβάσαμε τις μελέτες αυτές και συνθέσαμε μια δική μας ταξινόμηση με πέντε κριτήρια, την πηγή των δεδομένων, την κατεύθυνση της ροής, το μοντέλο εμπιστοσύνης, την πιστοποίηση της απάντησης και το μοτίβο σχεδίασης (**Σχήμα 2.8**).



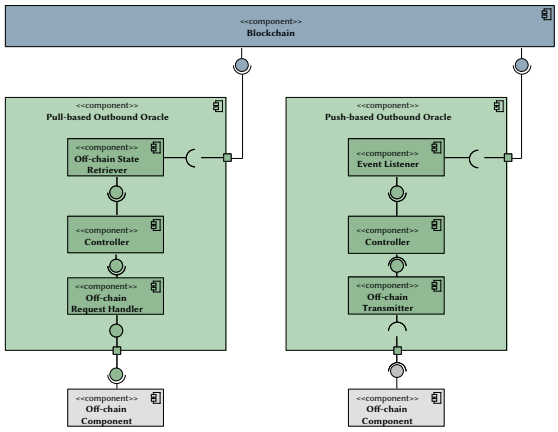
Σχήμα 2.8. Η κατάταξη των oracles που συνθέσαμε από τις βιβλιογραφικές μελέτες, με βάση την πηγή των δεδομένων, την κατεύθυνση της ροής, το μοντέλο εμπιστοσύνης, την πιστοποίηση της απάντησης και το μοτίβο σχεδίασης.

Πηγή δεδομένων. Τα oracles λογισμικού *(software oracles)* διαβάζουν διαδικτυακές πηγές, δηλαδή βάσεις δεδομένων, διακομιστές, ιστοσελίδες και διεπαφές προγραμματισμού εφαρμογών *(Application Programming Interfaces, APIs)*. Τα oracles υλισμικού *(hardware oracles)*, αντίθετα, διαβάζουν τον φυσικό κόσμο μέσα από αισθητήρες, σαρωτές και γενικά κάθε συσκευή IoT. Τέλος, τα ανθρώπινα oracles *(human oracles)* είναι άνθρωποι με ειδική γνώση, που απαντούν σε ερωτήματα που δύσκολα απαντά μια μηχανή [73], [76].

Κατεύθυνση της ροής πληροφορίας. Τα εισερχόμενα oracles *(inbound oracles)* μεταφέρουν πληροφορία από τον έξω κόσμο στο blockchain δίκτυο. Τα εξερχόμενα oracles *(outbound oracles)* κάνουν το αντίστροφο, μεταφέρουν πληροφορία από αυτό προς τα έξω. Οι Mühlberger κ.ά. [72] χωρίζουν επιπλέον κάθε κατεύθυνση με βάση το ποιος ξεκινά τη ροή. Στο μοτίβο pull τη ροή την ξεκινά η πλευρά που ζητά τα δεδομένα. Έτσι, ένα εισερχόμενο oracle pull περιμένει αίτηση από συμβόλαιο του blockchain δικτύου, φέρνει την κατάσταση από την εξωτερική πηγή και τη γράφει με συναλλαγή. Στο μοτίβο push, αντίθετα, τη ροή την ξεκινά η πλευρά που έχει τα δεδομένα. Ένα εισερχόμενο oracle push παρακολουθεί, επομένως, την πηγή και, όταν αλλάξει η κατάστασή της, την προωθεί στο blockchain δίκτυο, αφού τη φιλτράρει ή την ελέγξει αν χρειάζεται (**Σχήμα 2.9**). Κατά τον ίδιο τρόπο, στα εξερχόμενα oracles το pull απαντά σε ερωτήματα εξωτερικών συστημάτων για την κατάσταση του blockchain δικτύου, ενώ το push ακούει γεγονότα του blockchain δικτύου και ενεργεί προς τα έξω (**Σχήμα 2.10**). Παράδειγμα του δεύτερου είναι η έξυπνη κλειδαριά που ανοίγει μόλις κατατεθεί η πληρωμή [73].



Σχήμα 2.9. Εισερχόμενα oracles με μηχανισμό pull (αριστερά) και push (δεξιά). Στο pull η αίτηση ξεκινά από το blockchain δίκτυο και το oracle φέρνει την κατάσταση της πηγής. Στο push το oracle παρακολουθεί την πηγή και προωθεί κάθε αλλαγή. Ο ελεγκτής μπορεί να φιλτράρει ή να ελέγξει τα δεδομένα πριν γραφτούν.



Σχήμα 2.10. Εξερχόμενα oracles με μηχανισμό pull (αριστερά) και push (δεξιά). Στο pull ένα εξωτερικό σύστημα ζητά την κατάσταση του blockchain δικτύου. Στο push το oracle ακούει γεγονότα του blockchain δικτύου και ενημερώνει το εξωτερικό σύστημα.

Μοντέλο εμπιστοσύνης *(trust model).* Οι βιβλιογραφικές μελέτες δίνουν στο κριτήριο αυτό δύο τιμές. Τα κεντρικά oracles *(centralized oracles)* παίρνουν τα δεδομένα από μία πηγή ή έναν κόμβο. Απαντούν γρήγορα, αλλά είναι μοναδικό σημείο αστοχίας [73]. Αντίθετα, τα αποκεντρωμένα oracles *(decentralized oracles)* ρωτούν πολλούς κόμβους ή πολλές πηγές και συνθέτουν μία απάντηση. Η σύνθεση γίνεται με ψηφοφορία, όπου το βάρος κάθε ψήφου συνήθως εξαρτάται από το stake του κόμβου, ή με επιλογή των κόμβων ανάλογα με τη φήμη τους [77]. Η διάκριση αυτή, όμως, απαντά σε μία μόνο ερώτηση, πόσοι τροφοδοτούν το blockchain δίκτυο. Δεν λέει πώς ελέγχεται ότι η απάντηση είναι αξιόπιστη. Τις δύο ερωτήσεις τις χωρίζουν ρητά οι Finkbeiner και Almashaqbeh, σε μια συστηματοποίηση 24 ακαδημαϊκών και εμπορικών oracles [79]. Γι' αυτό, κρατάμε στο μοντέλο εμπιστοσύνης μόνο την πρώτη ερώτηση, δηλαδή ο τρόπος επιλογής των πηγών κάνει ένα oracle κεντρικό ή αποκεντρωμένο. Ο έλεγχος της απάντησης είναι το επόμενο κριτήριο.

Πιστοποίηση της απάντησης *(response authenticity).* Είναι ο τρόπος με τον οποίο ένα oracle αποδεικνύει ότι η απάντησή του είναι αξιόπιστη. Ο τρόπος αυτός εξαρτάται από τον μηχανισμό που κρατά τις πηγές έντιμες, ο οποίος ορίζει και την υπόθεση εμπιστοσύνης *(trust assumption).* Η υπόθεση είναι κρυπτογραφική *(cryptographic)*, αξιόπιστο υλικό *(trusted hardware)*, αξιόπιστο τρίτο μέρος *(trusted third party)* ή οικονομική ασφάλεια *(economic security).* Με βάση τον μηχανισμό αυτόν, οι Finkbeiner και Almashaqbeh βρίσκουν τέσσερις τρόπους πιστοποίησης [79]:

- **Αξιόπιστο τρίτο μέρος *(trusted third party).*** Η πηγή είναι έμπιστη, οπότε η απάντηση γίνεται δεκτή όπως έρχεται, χωρίς καμία πιστοποίηση.
- **Αισιόδοξος έλεγχος *(optimistic approach).*** Η απάντηση γίνεται οριστική αφού περάσει μια περίοδος αμφισβήτησης χωρίς κάποιος να τη διαψεύσει. Η πηγή έχει καταθέσει stake, που το χάνει αν η αμφισβήτηση πετύχει.
- **Στατιστική συνάθροιση *(statistical aggregation).*** Πολλές πηγές απαντούν και η απάντηση είναι συνήθως η διάμεσος, γιατί αντέχει σε μία ακραία τιμή.
- **Επαληθεύσιμος υπολογισμός *(verifiable computation).*** Μία πηγή στέλνει την απάντηση μαζί με ρητό τεκμήριο γνησιότητας *(explicit authenticity proof)*, δηλαδή απόδειξη ότι δεν την άλλαξε. Είναι η ισχυρότερη εγγύηση, αλλά κοστίζει σε πολυπλοκότητα και κρυπτογραφία ή απαιτεί ισχυρές υποθέσεις, όπως το αξιόπιστο υλικό.

Τα δύο κριτήρια είναι ανεξάρτητα, με μία εξαίρεση. Η στατιστική συνάθροιση προϋποθέτει πολλές πηγές, άρα αποκεντρωμένο oracle. Οι άλλοι τρεις τρόποι, αντίθετα, συνδυάζονται και με τις δύο τιμές του μοντέλου εμπιστοσύνης. Τον τέταρτο τρόπο τον ονομάζουμε με τον όρο της κρυπτογραφίας για την ανάθεση υπολογισμού σε μη έμπιστο μέρος, και τον εξετάζουμε χωριστά στην ενότητα 2.3.5, γιατί σε αυτόν ανήκει το oracle της εφαρμογής μας.

Μοτίβο σχεδίασης. Αφορά την αρχιτεκτονική του oracle και τα μοτίβα είναι τρία [73], [76]. Στο μοτίβο αίτησης και απόκρισης *(request-response)* το συμβόλαιο καταγράφει το αίτημά του στο blockchain δίκτυο. Το oracle το παρατηρεί ως γεγονός ή ως αλλαγή κατάστασης, φέρνει τα δεδομένα, τα υπογράφει και τα επιστρέφει με νέα συναλλαγή. Το μοτίβο αυτό ταιριάζει όταν το σύνολο των δεδομένων είναι πολύ μεγάλο για να αποθηκευτεί σε συμβόλαιο και κάθε φορά χρειάζεται ένα μικρό μέρος του. Στο μοτίβο δημοσίευσης και εγγραφής *(publish-subscribe)*, αντίθετα, το oracle ενημερώνει το δικό του συμβόλαιο κάθε φορά που αλλάζουν τα δεδομένα, όπως οι τιμές ή ο καιρός. Τα εγγεγραμμένα συμβόλαια τα διαβάζουν από εκεί ή ακούν την ενημέρωση. Τέλος, στο μοτίβο άμεσης ανάγνωσης *(immediate-read)* τα δεδομένα γράφονται μία φορά σε ένα συμβόλαιο. Όποιος τα χρειάζεται τα διαβάζει από εκεί τη στιγμή που παίρνει μια απόφαση, ίσως και ποτέ ξανά. Παραδείγματα είναι τα ακαδημαϊκά πιστοποιητικά και οι ταυτότητες.

Με τα πέντε κριτήρια στη θέση τους, ένα oracle περιγράφεται με μία τιμή σε καθένα. Για παράδειγμα, ένα oracle που διαβάζει την ιστοσελίδα μιας εταιρείας όταν του το ζητήσει ένα συμβόλαιο είναι λογισμικού, εισερχόμενο και κεντρικό [76]. Αν το συμβόλαιο δέχεται ό,τι διαβάζει χωρίς απόδειξη, η πιστοποίησή του στηρίζεται σε αξιόπιστο τρίτο μέρος, και το μοτίβο του είναι αίτησης και απόκρισης.

2.3.5 Oracles επαληθεύσιμου υπολογισμού

Η ταξινόμηση της προηγούμενης ενότητας αφήνει ανοιχτό το ερώτημα από πού πηγάζει η εμπιστοσύνη σε ένα oracle. Οι Bartholic κ.ά. [78] το συνδέουν με το πλήθος όσων μπορούν να δώσουν την απάντηση. Όταν μπορούν πολλοί, όπως για την ισοτιμία ενός νομίσματος, αρκεί η εμπιστοσύνη στον μηχανισμό που συνθέτει τις ανεξάρτητες απαντήσεις. Αντίθετα, όταν την ξέρει ένας ή λίγοι με ειδική εξουσία, όπως το κράτος για την ιθαγένεια ενός πολίτη, δεν υπάρχει πληθυσμός για σύγκριση. Τότε το oracle μπορεί μόνο να εγγυηθεί τη γνησιότητα και την ακεραιότητα αυτού που μεταφέρει, και πρέπει να δεχτεί ως αληθές ό,τι του λέει η πηγή [78]. Οι πολλές πηγές δεν προσθέτουν στην περίπτωση αυτή έλεγχο, ενώ για ευαίσθητα δεδομένα αυξάνουν και τον κίνδυνο διαρροής [74].

Ερώτημα του δεύτερου είδους είναι και η κατάσταση ενός έργου τέχνης, γιατί την απαντά αυτός που το φυλάσσει ή το επιθεωρεί. Σε τέτοια ερωτήματα δεν έχει σημασία πόσες πηγές έχει το oracle, αλλά πώς βεβαιώνεται ότι μετέφερε πιστά ό,τι είπε η πηγή. Αυτό κάνουν τα oracles επαληθεύσιμου υπολογισμού, που απαντούν με απόδειξη αντί για συμφωνία πολλών κόμβων. Τον όρο τον δανειζόμαστε από την κρυπτογραφία, όπου τον όρισαν οι Gennaro κ.ά. [80] το 2010. Ένας υπολογιστικά αδύναμος πελάτης αναθέτει τον υπολογισμό μιας συνάρτησης σε εργάτες που δεν εμπιστεύεται, και οι εργάτες επιστρέφουν το αποτέλεσμα μαζί με απόδειξη ότι ο υπολογισμός έγινε σωστά.

Το oracle είναι ακριβώς ένας τέτοιος εργάτης, δηλαδή ένας υπολογισμός εκτός blockchain δικτύου που το συμβόλαιο δεν εμπιστεύεται. Οι τέσσερις τρόποι πιστοποίησης της ενότητας 2.3.4 είναι, από τη σκοπιά αυτή, τέσσερις μηχανισμοί ελέγχου του. Μόνο το ρητό τεκμήριο γνησιότητας, ωστόσο, δεν χρειάζεται καμία υπόθεση για το πώς αποτυγχάνει ο εργάτης [81].

Ονομάζουμε, επομένως, oracle επαληθεύσιμου υπολογισμού ένα oracle που παράγει το αποτέλεσμα εκτός blockchain δικτύου και το στέλνει μαζί με ένα τεκμήριο, το οποίο το συμβόλαιο ελέγχει μέσα στο blockchain δίκτυο πριν να το αποδεχθεί. Ένα τέτοιο oracle μπορεί να είναι ένας μόνο κόμβος, χωρίς εμπιστοσύνη στον χειριστή του, γιατί αν ο χειριστής αλλάξει το αποτέλεσμα, το τεκμήριο δεν ελέγχεται [82]. Η βιβλιογραφία περιγράφει τέτοια συστήματα από το 2016, όπως το Town Crier πάνω σε Intel SGX [82] και τα τεκμήρια γνησιότητας του Provable [73], αλλά συνήθως τα κατατάσσει στα κεντρικά oracles επειδή έχουν έναν κόμβο [83], [84]. Η δική μας ταξινόμηση, αντίθετα, τα ξεχωρίζει από το πλήθος των κόμβων.

Το τεκμήριο το παράγουν δύο τεχνολογίες, που δεν δίνουν την ίδια εγγύηση.

- **Βεβαίωση αξιόπιστου υλικού**. Ένα περιβάλλον αξιόπιστης εκτέλεσης *(Trusted Execution Environment, TEE)* τρέχει τον κώδικα σε μνήμη που δεν διαβάζει ούτε το λειτουργικό σύστημα, και υπογράφει το αποτέλεσμα μαζί με σύνοψη του κώδικα, με κλειδί που ξέρει μόνο το υλικό [85]. Το συμβόλαιο ελέγχει την υπογραφή με το δημόσιο κλειδί του oracle που εκτελείται στο έμπιστο περιβάλλον [79]. Η εγγύηση ισχύει όσο ισχύει η εμπιστοσύνη στον κατασκευαστή του υλικού, και δεν καλύπτει τη διαθεσιμότητα, γιατί ο διαχειριστής της μηχανής μπορεί να τερματίσει τη λειτουργία του oracle όποτε θέλει [85]. Το Town Crier και το Nautilus ανήκουν εδώ.

- **Κρυπτογραφική απόδειξη**. Μια απόδειξη μηδενικής γνώσης, όπως τα zk-SNARK, βεβαιώνει ότι ο υπολογισμός έγινε σωστά χωρίς εμπιστοσύνη σε κατασκευαστή ή χειριστή [86]. Η παραγωγή της, όμως, είναι τάξεις μεγέθους ακριβότερη από τον ίδιο τον υπολογισμό.

Το όριο της κατηγορίας είναι κοινό και για τις δύο τεχνολογίες. Το τεκμήριο βεβαιώνει τι έγινε με την είσοδο, δεν λέει όμως αν η είσοδος ήταν αληθινή. Το Town Crier το δηλώνει ρητά, αφού υποθέτει ότι οι πελάτες εμπιστεύονται τις πηγές τους [82]. Για τα ερωτήματα που απαντά ένας ή λίγοι, όπως η κατάσταση ενός έργου τέχνης, αυτό είναι ό,τι καλύτερο μπορεί να κάνει ένα oracle. Το oracle της εφαρμογής μας ανήκει, επομένως, στην κατηγορία αυτή, με βεβαίωση αξιόπιστου υλικού που ελέγχεται μέσα στο blockchain δίκτυο.

2.3.6 Τα έργα τέχνης ως πραγματικά περιουσιακά στοιχεία

Με τα oracles κλείνει το γενικό μέρος για τα RWA. Μένει να δούμε πού βρίσκεται ανάμεσά τους το έργο τέχνης και γιατί είναι η δύσκολη περίπτωση. Από την ανάλυση των Luo κ.ά. [65] ξεχωρίζουμε τρία χαρακτηριστικά ενός αγαθού που κάνουν το tokenization του δύσκολο.

- **Ανταλλαξιμότητα**. Μια ουγγιά χρυσού δεν χρειάζεται εξατομικευμένη ταυτότητα, ένας πίνακας χρειάζεται δικό του token [65].
- **Επαληθευσιμότητα της κατάστασης**. Τα αντικειμενικά δεδομένα αποδεικνύονται κρυπτογραφικά, ενώ η υποκειμενική κατάσταση του αγαθού, όπως μια ζημιά, χρειάζεται ανθρώπινη κρίση [65].
- **Φύλαξη**. Το αγαθό μπορεί να κλαπεί ή να καταστραφεί και η ασφάλεια του token περιορίζεται από τον ασθενέστερο κρίκο εκτός blockchain δικτύου [65].

Η αγορά επιβεβαιώνει την εικόνα αυτή. Τα tokenized αγαθά, χωρίς τα stablecoins, είναι σήμερα 38,7 δισεκατομμύρια δολάρια. Πάνω από τα τρία τέταρτα είναι κρατικό χρέος και τα ταμεία που το κρατούν (44%), μορφές πίστωσης (20%) και εμπορεύματα, κυρίως πολύτιμα μέταλλα (13%) [71], [87]. Τα ομόλογα και οι ράβδοι χρυσού είναι ανταλλάξιμα, έχουν τυποποιημένη τεκμηρίωση και αδιαμφισβήτητη αποτίμηση, δηλαδή στενό χάσμα συγχρονισμού. Το tokenization προχώρησε, επομένως, πρώτα εκεί όπου το πρόβλημα ήταν εύκολο.

Αντίθετα, τα έργα τέχνης βρίσκονται στο άλλο άκρο. Ένας πίνακας είναι μοναδικός. Η γνησιότητα και η κατάστασή του κρίνονται από ανθρώπους. Χρειάζεται και φυσική φύλαξη σε όλη τη ζωή του token. Συγκεντρώνει δηλαδή και τις τρεις δυσκολίες, γι' αυτό τα έργα τέχνης μένουν μικρό τμήμα της αγοράς [88]. Το δείχνουν και οι κατηγορίες που καταγράφονται. Τα ακίνητα, η μόνη κατηγορία με μοναδικό φυσικό αγαθό, είναι το 0,6% της αγοράς, και τα έργα τέχνης δεν εμφανίζονται καν ως χωριστή κατηγορία [87].

Τα ίδια τρία χαρακτηριστικά δείχνουν και τι αλλάζει στους μεσάζοντες. Εξαφανίζονται όσοι τηρούσαν το μητρώο και εκτελούσαν τη μεταβίβαση, γιατί αυτό το αναλαμβάνει το blockchain δίκτυο με καλύτερες εγγυήσεις. Παραμένουν, αντίθετα, όσοι φυλάσσουν και όσοι κρίνουν. Κάποιος πρέπει να φυλάει τον πίνακα και κάποιος να βεβαιώσει ότι είναι γνήσιος. Καμία από τις δύο δουλειές δεν είναι υπολογισμός πάνω σε δεδομένα του blockchain δικτύου, οπότε καμία δεν αντικαθίσταται από συμβόλαιο [65], [89]. Σε ένα σύστημα με tokens η γκαλερί κρατά επομένως τη φύλαξη και την πιστοποίηση, χάνει το μητρώο και τον έλεγχο της μεταβίβασης, και κάθε πράξη της είναι δημόσια. Η θέση της γκαλερί στην εφαρμογή μας, όπως την είδαμε στο κεφάλαιο 1, προκύπτει από αυτόν τον διαχωρισμό.

2.3.7 Κλασματική ιδιοκτησία

Οι πλατφόρμες κλασματοποίησης συγκλίνουν σε ένα μοτίβο [25], [90]. Σε αυτό το NFT μεταβιβάζεται σε ένα συμβόλαιο, το θησαυροφυλάκιο *(vault)*. Το συμβόλαιο το κλειδώνει και εκδίδει ανταλλάξιμα tokens, καθένα σταθερό ποσοστό της ιδιοκτησίας. Το μοτίβο στηρίζεται σε μια αναλλοίωτη συνθήκη *(invariant)*, ότι όσο τα κλάσματα κυκλοφορούν το NFT μένει κλειδωμένο. Αν η συνθήκη παραβιαστεί και το NFT αποδεσμευτεί, τα κλάσματα συνεχίζουν να κυκλοφορούν χωρίς να αντιστοιχούν πλέον στο περιουσιακό στοιχείο.

Το δύσκολο σημείο είναι το αντίστροφο, η επανενοποίηση. Η βιβλιογραφία καταγράφει τρεις μηχανισμούς [25]. Η πλήρης συγκέντρωση απαιτεί να αποκτήσει ένας αγοραστής όλα τα κλάσματα, και ο τελευταίος κάτοχος μπορεί να ζητήσει υπερβολικό τίμημα ή να αρνηθεί. Η δημοπρασία εξαγοράς αντιμετωπίζει το πρόβλημα, καθώς όποιος προσφέρει τίμημα πάνω από ένα κατώφλι παίρνει το έργο τέχνης και το τίμημα μοιράζεται. Η ψηφοφορία θέτει κάθε πρόταση εξαγοράς σε έγκριση της πλειοψηφίας. Η εφαρμογή μας υλοποιεί την πλήρη συγκέντρωση, δηλαδή ο κάτοχος όλων των κλασμάτων ενοποιεί το έργο τέχνης. Επιλέξαμε τον μηχανισμό αυτόν επειδή οι άλλοι δύο θέλουν αποτίμηση του έργου τέχνης ή διακυβέρνηση των κατόχων.

2.3.8 Δικαίωμα παρακολούθησης

Το δικαίωμα παρακολούθησης *(droit de suite ή royalty)* είναι το ποσοστό επί κάθε μεταπώλησης ενός έργου τέχνης που αποδίδεται στον δημιουργό του. Στην Ελλάδα ορίζεται από το άρθρο 5 του Ν. 2121/1993 [91] και από τον Ν. 3524/2007 [92]. Σε κάθε μεταπώληση πρωτοτύπου έργου τέχνης στην οποία συμμετέχει επαγγελματίας της αγοράς, όπως οίκος δημοπρασιών ή γκαλερί, ο δημιουργός και οι κληρονόμοι του δικαιούνται ποσοστό επί της τιμής πώλησης. Το ποσό το καταβάλλει ο πωλητής, και όταν μεσολαβεί επαγγελματίας της αγοράς ευθύνεται και αυτός εις ολόκληρον. Το πρόβλημα είναι ότι, όπως είδαμε στο κεφάλαιο 1, η είσπραξή του βασίζεται στη συμμόρφωση των πωλητών [18], [19].

Κατά τον ίδιο τρόπο, σε ένα blockchain δίκτυο το δικαίωμα επιβάλλεται μόνο αν οι ίδιοι οι κανόνες μεταβίβασης το απαιτούν. Αλλιώς δύο μέρη συμφωνούν εκτός blockchain δικτύου και μεταβιβάζουν χωρίς πληρωμή, και το δικαίωμα δεν είναι δεσμευτικό. Οι Whitaker και Kräussl [15] προτείνουν έναν δεύτερο τρόπο. Ο καλλιτέχνης κρατά μερίδιο στο δικό του έργο τέχνης και κερδίζει από την ανατίμησή του. Η εφαρμογή μας υλοποιεί και τα δύο. Το δικαίωμα καταβάλλεται μέσα στη συναλλαγή της αγοράς, και ο καλλιτέχνης μπορεί να κρατήσει όσα κλάσματα θέλει.

Κεφάλαιο 3
Το οικοσύστημα SUI

Στο κεφάλαιο αυτό παρουσιάζουμε τις τέσσερις τεχνολογίες με τις οποίες χτίζεται η εφαρμογή. Ξεκινάμε από το SUI και τη γλώσσα Move. Συνεχίζουμε με το Walrus για την αποθήκευση των αρχείων και με τα Walrus Sites, που χτίζονται πάνω του, για τη φιλοξενία της διεπαφής χρήστη. Έπειτα, εξετάζουμε το Seal για τον έλεγχο πρόσβασης. Τέλος, παρουσιάζουμε το Nautilus για τη σύνδεση με τον φυσικό κόσμο.

3.1 Το blockchain δίκτυο SUI και η γλώσσα Move

Το SUI είναι ένα blockchain δίκτυο πρώτου επιπέδου *(Layer-1)*. Σχεδιάστηκε για χαμηλή καθυστέρηση *(latency)* και υψηλό ρυθμό συναλλαγών *(throughput)* [29]. Το ανέπτυξε η Mysten Labs, μια εταιρεία που ίδρυσαν το 2021 στελέχη του προγράμματος Diem της Meta. Το κύριο δίκτυό του *(mainnet)* ξεκίνησε τον Μάιο του 2023. Από το Diem κληρονόμησε και τη γλώσσα Move [93].

3.1.1 Το μοντέλο αντικειμένων

Τα περισσότερα blockchain δίκτυα, όπως το Ethereum, έχουν μοντέλο λογαριασμών *(account model)*. Σε αυτό το μοντέλο η κατάσταση είναι μία ενιαία καθολική δομή, και τα ψηφιακά αγαθά είναι εγγραφές μέσα στη μνήμη κάποιου έξυπνου συμβολαίου. Ένα token ERC-20, για παράδειγμα, είναι μια γραμμή σε έναν πίνακα του συμβολαίου που το εξέδωσε. Κάθε συναλλαγή που το αφορά περνά, επομένως, από το ίδιο συμβόλαιο. Επειδή όλοι οι χρήστες αλλάζουν την ίδια κοινή κατάσταση, το δίκτυο εκτελεί όλες τις συναλλαγές σειριακά [94].

Το SUI αντιστρέφει τη σχεδίαση αυτή με ένα μοντέλο αντικειμένων *(object model)*. Εδώ η βασική μονάδα της κατάστασης είναι το αντικείμενο *(object)*. Κάθε αντικείμενο είναι μια αυτοτελής οντότητα με δικό της αναγνωριστικό, αριθμό έκδοσης, ιδιοκτήτη και δεδομένα, και ο τύπος των δεδομένων ορίζεται σε Move [95]. Το αναγνωριστικό δεν αλλάζει ποτέ, ενώ η έκδοση αυξάνεται σε κάθε τροποποίηση. Επιπλέον, μόνο μία συναλλαγή μπορεί να τροποποιήσει ένα αντικείμενο σε δεδομένη έκδοση. Γι' αυτό, κάθε αντικείμενο έχει γραμμικό ιστορικό [96].

Ο ιδιοκτήτης καθορίζει ποιος χρησιμοποιεί το αντικείμενο και ποιοι έλεγχοι πρόσβασης χρειάζονται. Ο **Πίνακας 3.1** δίνει τις μορφές ιδιοκτησίας και τη σημασία τους για τη σχεδίαση των συμβολαίων [97]. Η ιδιοκτησία δεν αποτελεί από μόνη της μέτρηση της καθυστέρησης μιας συναλλαγής, η οποία εξαρτάται και από τα υπόλοιπα αντικείμενα εισόδου και την επεξεργασία της.

Πίνακας 3.1. Οι μορφές ιδιοκτησίας αντικειμένων στο SUI, ποιος χρησιμοποιεί το αντικείμενο και ποιοι κανόνες πρόσβασης επηρεάζουν τη σχεδίαση των συμβολαίων [97].

Μορφή ιδιοκτησίας	Ποιος το χρησιμοποιεί	Σημασία για το συμβόλαιο
Ιδιοκτησία από διεύθυνση *(address-owned)*	Μόνο η διεύθυνση που το κατέχει.	Η κατοχή ελέγχεται από το δίκτυο κατά τη χρήση του ως εισόδου.
Ιδιοκτησία από αντικείμενο *(object-owned)*	Μέσω του γονικού αντικειμένου και των σχετικών συναρτήσεων Move.	Η πρόσβαση εξαρτάται από τους κανόνες του γονέα.
Κοινόχρηστο *(shared)*	Οποιαδήποτε διεύθυνση, με τους ελέγχους που γράφει ρητά ο κώδικας Move.	Το συμβόλαιο ελέγχει ρητά την εξουσιοδότηση κάθε μεταβολής.
Αμετάβλητο *(immutable)*	Οποιαδήποτε διεύθυνση το διαβάζει, καμία δεν το τροποποιεί. Τα δημοσιευμένα πακέτα Move είναι αμετάβλητα.	Μπορεί να διαβάζεται από πολλές συναλλαγές χωρίς σύγκρουση εγγραφών.
Τυλιγμένο *(wrapped)*	Μόνο μέσα από το αντικείμενο που το περιέχει *(wrapper).*	Η πρόσβαση γίνεται μέσω του αντικειμένου που το περιέχει.
Ιδιόκτητο με ομοφωνία *(party)*	Μόνο η διεύθυνση που το κατέχει, αλλά η συναλλαγή διατάσσεται από την ομοφωνία.	Συνδυάζει περιορισμένη πρόσβαση με διαχείριση εκδόσεων μέσω ομοφωνίας.

Η διάκριση αυτή καθορίζει τη σχεδίαση της εφαρμογής. Για παράδειγμα, το αντικείμενο που αναπαριστά ένα έργο τέχνης πρέπει να το αλλάζουν πολλοί, όπως ο καλλιτέχνης, η γκαλερί και η αγορά. Γι' αυτό, δεν μπορεί να ανήκει σε μία διεύθυνση, και είναι κοινόχρηστο αντικείμενο. Ποιος είναι ο ιδιοκτήτης του έργου τέχνης το λέει τότε ένα πεδίο μέσα στο αντικείμενο. Το πεδίο αυτό το αλλάζει μόνο ο κώδικας του module που ορίζει το έργο τέχνης, όταν το έργο τέχνης πωληθεί. Ένα κλάσμα, αντίθετα, ανήκει σε μία διεύθυνση, αυτή του κατόχου του.

Επιπλέον, το SUI έχει δυναμικά πεδία *(dynamic fields).* Είναι πεδία που προστίθενται σε ένα αντικείμενο κατά την εκτέλεση, χωρίς ο τύπος του να τα προβλέπει [98]. Στην εφαρμογή μας, με αυτά δένονται στο έργο τέχνης το πιστοποιητικό γνησιότητας και η καταχώριση φύλαξης, αφού το έργο τέχνης εκδοθεί και χωρίς να αλλάξει ο τύπος του.

Τέλος, το μοντέλο αντικειμένων επιτρέπει την παράλληλη εκτέλεση. Κάθε συναλλαγή δηλώνει ρητά τα αντικείμενα εισόδου της, και η δήλωση αυτή υπογράφεται μαζί με τη συναλλαγή. Ο επικυρωτής ξέρει, επομένως, τι διαβάζει και τι γράφει η συναλλαγή πριν από την εκτέλεσή της. Αν δύο συναλλαγές δεν έχουν κοινό αντικείμενο εισόδου, τότε το αποτέλεσμα είναι το ίδιο είτε τρέξουν η μία μετά την άλλη είτε ταυτόχρονα. Γι' αυτό, οι επικυρωτές τις εκτελούν παράλληλα [29], [99].

3.1.2 Το μοντέλο συναλλαγών και τα προγραμματιζόμενα μπλοκ συναλλαγών

Οι συναλλαγές στο SUI είναι προγραμματιζόμενα μπλοκ συναλλαγών *(Programmable Transaction Blocks, PTB)*, δηλαδή ακολουθίες έως 1.024 εντολών [100]. Κάθε εντολή καλεί μια συνάρτηση Move. Το αποτέλεσμά της τροφοδοτεί την επόμενη εντολή. Ολόκληρο το μπλοκ εκτελείται ως μία μονάδα, οπότε είτε πετυχαίνουν όλες οι εντολές είτε καμία. Στη Solidity, για παράδειγμα, ο συνδυασμός κλήσεων σε διαφορετικά συμβόλαια με την ίδια εγγύηση θέλει ένα ακόμη συμβόλαιο που τις τυλίγει *(wrapper contract)*. Στο SUI, αντίθετα, τον συνδυασμό τον κάνει ο πελάτης *(client)*, χωρίς να δημοσιεύσει κώδικα [94]. Στην εφαρμογή μας η αγορά κλασμάτων είναι ένα τέτοιο μπλοκ. Η πρώτη εντολή διασπά το τίμημα από το νόμισμα που πληρώνει το αέριο *(gas)*. Η δεύτερη καλεί τη συνάρτηση αγοράς, η οποία πληρώνει τον πωλητή, καταβάλλει το δικαίωμα παρακολούθησης στον καλλιτέχνη και παραδίδει το κλάσμα.

Κάθε συναλλαγή πληρώνει τέλη σε SUI, για τον υπολογισμό και για την αποθήκευση [101]. Αρχικά, ο χρήστης υπογράφει το μπλοκ και το στέλνει σε έναν πλήρη κόμβο *(full node)*. Ο πλήρης κόμβος το προωθεί σε έναν επικυρωτή *(validator)*, ο οποίος ελέγχει την υπογραφή, τα αντικείμενα εισόδου και το αέριο. Έπειτα, η ομοφωνία διατάσσει τη συναλλαγή, και κάθε επικυρωτής την εκτελεί ντετερμινιστικά. Τέλος, ο πλήρης κόμβος παίρνει τις αλλαγές που προκάλεσε η συναλλαγή *(effects)* από τον επικυρωτή στον οποίο την έστειλε, και από απαρτία επικυρωτών μόνο επιβεβαιώσεις ότι οριστικοποιήθηκε. Από εκείνη τη στιγμή η συναλλαγή είναι οριστική [102].

3.1.3 Η γλώσσα Sui Move

Σε παραδοσιακές γλώσσες έξυπνων συμβολαίων, όπως η Solidity, τα ψηφιακά περιουσιακά στοιχεία αναπαρίστανται ως αριθμητικές τιμές σε καθολικούς πίνακες. Ένας αριθμός, όμως, αντιγράφεται με μία ανάθεση και μηδενίζεται με μία άλλη. Ένα φυσικό αγαθό, αντίθετα, δεν διπλασιάζεται ούτε εξαφανίζεται, αλλά αλλάζει κάτοχο. Η Move μεταφέρει αυτή τη συμπεριφορά στα ψηφιακά αγαθά μέσω ενός συστήματος γραμμικών τύπων *(linear types)* [103]. Στη γλώσσα αυτή κάθε τιμή εξ ορισμού μπορεί μόνο να δημιουργηθεί, να μετακινηθεί και να γίνει unpack στα πεδία της. Στη Sui Move η αρχή αυτή ενσωματώνεται στο μοντέλο αντικειμένων, όπου κάθε αυτόνομος πόρος διαθέτει καθολικά μοναδικό αναγνωριστικό (`id: UID`) και ανεξάρτητο κύκλο ζωής [104], [105].

Ο κώδικας οργανώνεται σε modules, τα οποία ομαδοποιούνται σε πακέτα *(packages)*. Κάθε module ορίζει τύπους δεδομένων *(structs)* και συναρτήσεις που ενεργούν πάνω σε αυτούς. Η γλώσσα επιβάλλει αυστηρή ενθυλάκωση *(encapsulation)*. Τα πεδία μιας δομής δεδομένων είναι ιδιωτικά και προσβάσιμα μόνο από το module που την ορίζει. Κανένας εξωτερικός κώδικας δεν μπορεί να διαβάσει, να τροποποιήσει ή να κατασκευάσει απευθείας μια δομή δεδομένων, παρά μόνο καλώντας τις δημόσιες συναρτήσεις του module [105].

Κάθε πρόσθετη συμπεριφορά πέρα από τη μετακίνηση αποτελεί προνόμιο που δηλώνεται ρητά στον ορισμό της δομής δεδομένων. Το σύστημα ακολουθεί την αρχή της απαγόρευσης εξ ορισμού *(deny by default)* και βασίζεται σε τέσσερις ιδιότητες *(abilities)* [106], [107]:

- `key`. Επιτρέπει στον τύπο να αποθηκευτεί ως αυτόνομο αντικείμενο στο SUI (απαιτεί ως πρώτο πεδίο ένα αναγνωριστικό `id: UID`).
- `store`. Επιτρέπει στον τύπο να αποθηκευτεί μέσα σε άλλα αντικείμενα ή να μεταβιβαστεί ελεύθερα με την εντολή `transfer::public_transfer`.
- `copy`. Επιτρέπει την αντιγραφή της τιμής (δημιουργία πιστού αντιγράφου στη μνήμη).
- `drop`. Επιτρέπει την απόρριψη της τιμής όταν αυτή βγαίνει εκτός εμβέλειας *(scope)*.

Ο συνδυασμός των παραπάνω ιδιοτήτων επιτρέπει τη διαμόρφωση συγκεκριμένων αρχιτεκτονικών μοτίβων [105]. Ένας τύπος με `key` αλλά χωρίς `drop` και χωρίς `copy` ονομάζεται πόρος *(resource)*. Η γλώσσα επιβάλλει δύο κανόνες στους πόρους:

1. Ένας πόρος δεν αντιγράφεται ούτε διαγράφεται αυτόματα. Κάθε τιμή του πρέπει υποχρεωτικά είτε να μεταβιβαστεί σε νέα θέση είτε να γίνει ρητά unpack στα συστατικά της πεδία.
2. Τον πόρο τον δημιουργεί και του κάνει unpack αποκλειστικά ο κώδικας του module που τον ορίζει.

Χάρη στους δύο αυτούς κανόνες, το πλήθος των στιγμιοτύπων ενός πόρου μεταβάλλεται μόνο από το δικό του module. Κανένας εξωτερικός κώδικας δεν μπορεί να πλαστογραφήσει ή να καταστρέψει έναν πόρο. Στην εφαρμογή μας το έργο τέχνης ορίζεται ως τέτοιος πόρος (**Κώδικας 3.1**).

```sui move
public struct Artwork has key {
    id: UID,
    title: String,
    artist: address,
    owner: Option<address>,
}
```

Κώδικας 3.1. Απλοποιημένη έκδοση του ορισμού του έργου τέχνης ως πόρου στη Sui Move, με τέσσερα από τα πεδία του.

Εκτός από τα στατικά πεδία μιας δομής δεδομένων, η Sui Move επιτρέπει την προσθήκη πεδίων κατά τον χρόνο εκτέλεσης μέσω των δυναμικών πεδίων *(dynamic fields)* [98]. Ένα δυναμικό πεδίο συνδέεται στο αναγνωριστικό `&mut UID` του γονικού αντικειμένου με ένα ζεύγος ονόματος και τιμής. Το όνομα μπορεί να είναι οποιαδήποτε τιμή με `copy + drop + store`, ενώ η τιμή αρκεί να διαθέτει την ιδιότητα `store` [105].

Η γλώσσα διακρίνει τα απλά δυναμικά πεδία από τα δυναμικά πεδία αντικειμένων *(dynamic object fields)*. Στα απλά δυναμικά πεδία η τιμή αποτελεί εσωτερικό δεδομένο του γονέα. Αντίθετα, στα δυναμικά πεδία αντικειμένων, η προσαρτημένη τιμή είναι ένα αυτόνομο αντικείμενο SUI με ιδιότητα `key`. Το αντικείμενο αυτό αποκτά μορφή ιδιοκτησίας από αντικείμενο *(object-owned)*, διατηρεί το δικό του αναγνωριστικό και παραμένει αναζητήσιμο σε εξωτερικούς εξερευνητές *(explorers)*.

Τα δυναμικά πεδία προσφέρουν δύο ουσιαστικά πλεονεκτήματα:

1. **Κλιμάκωση αποθήκευσης με σταθερό κόστος.** Τα δυναμικά πεδία αποθηκεύονται ως ξεχωριστά αντικείμενα. Όταν μια συναλλαγή διαβάζει το γονικό αντικείμενο, ο επικυρωτής δεν φορτώνει στη μνήμη τα δυναμικά πεδία του, παρά μόνο εκείνο που ζητείται ρητά. Έτσι, το κόστος αερίου παραμένει σταθερό ακόμη και αν προσαρτηθούν χιλιάδες εγγραφές.

2. **Δυναμική επέκταση αντικειμένων.** Επιτρέπουν σε διαφορετικά modules να επεκτείνουν ένα αντικείμενο εκ των υστέρων, χωρίς να τροποποιηθεί ο τύπος του. Στην εφαρμογή μας το πιστοποιητικό γνησιότητας προσαρτάται στο έργο τέχνης ως δυναμικό πεδίο αντικειμένου, όπως αναλύεται στην ενότητα 4.13 (**Κώδικας 4.14**), ενώ οι καταχωρίσεις φύλαξης και το ιστορικό προέλευσης προσαρτώνται ως απλά δυναμικά πεδία.

Επιπλέον, η Move επιλύει όλες τις κλήσεις συναρτήσεων στατικά. Στη γλώσσα δεν υπάρχει δυναμική δρομολόγηση κλήσεων *(dynamic dispatch)* ούτε συναρτήσεις ανάκτησης ελέγχου *(fallback functions)* [94]. Επομένως, ο γράφος κλήσεων μέσα σε μία συναλλαγή είναι άκυκλος. Η ιδιότητα αυτή εξαλείφει τις επιθέσεις επανεισόδου *(reentrancy attacks)*, οι οποίες αποτελούν μία από τις συχνότερες πηγές κενών ασφαλείας στα παραδοσιακά έξυπνα συμβόλαια.

Σε επίπεδο συναρτήσεων, η Sui Move διακρίνει ειδικές κατηγορίες λειτουργίας. Οι συναρτήσεις με τον προσδιοριστή `entry` μπορούν να κληθούν απευθείας από προγραμματιζόμενα μπλοκ συναλλαγών (PTBs). Αντίθετα, οι απλές δημόσιες συναρτήσεις συντίθενται σε μεγαλύτερες ροές εκτέλεσης. Επιπλέον, κάθε module μπορεί να ορίσει μία ειδική συνάρτηση αρχικοποίησης `init`. Η συνάρτηση αυτή εκτελείται ακριβώς μία φορά, αυτόματα, κατά τη δημοσίευση του πακέτου στο δίκτυο.

Τέλος, το σύστημα ιδιοτήτων και η σχεδίαση της γλώσσας επιτρέπουν την υλοποίηση δύο κρίσιμων μοτίβων ασφάλειας που χρησιμοποιεί η εφαρμογή μας:

- **Το μοτίβο δυνατοτήτων (capability pattern).** Ένα capability είναι ένα αντικείμενο που λειτουργεί ως ψηφιακό διαπιστευτήριο πρόσβασης [105]. Στο σύστημά μας η δυνατότητα διαχείρισης εκφράζεται με το αντικείμενο `AdminCap`, το οποίο εκδίδεται μία φορά κατά την αρχικοποίηση του πακέτου από τη συνάρτηση `init`. Κάθε συνάρτηση διαχείρισης ζητά μια αναφορά στο `AdminCap` ως όρισμα εισόδου. Έτσι, ο έλεγχος δικαιωμάτων γίνεται απευθείας από το σύστημα τύπων κατά την εκτέλεση, χωρίς να χρειάζονται έλεγχοι της διεύθυνσης του αποστολέα στον κώδικα.

- **Η περιορισμένη μεταβίβαση μέσω απουσίας του `store`.** Ένα αντικείμενο με `key` αλλά χωρίς `store` δεν μπορεί να μεταφερθεί με την καθολική εντολή `transfer::public_transfer` [105]. Μπορεί να μετακινηθεί μόνο από συναρτήσεις του ίδιου του module του, μέσω της εντολής `transfer::transfer`. Στην εφαρμογή μας το κλάσμα (`Fraction`) στερείται σκόπιμα την ιδιότητα `store` (**Κώδικας 3.2**). Με αυτόν τον τρόπο κάθε μεταβίβαση κλάσματος περνά υποχρεωτικά από τη συνάρτηση αγοράς του συμβολαίου, αποτρέποντας την παράκαμψη του δικαιώματος παρακολούθησης του καλλιτέχνη.

```sui move
public struct Fraction has key {
    id: UID,
    artwork_id: ID,
    amount: u64,
}

// Μεταβίβαση αποκλειστικά από συναρτήσεις του πακέτου του
public(package) fun deliver(fraction: Fraction, to: address) {
    transfer::transfer(fraction, to);
}
```

Κώδικας 3.2. Απλοποιημένη έκδοση του κλάσματος (`Fraction`) στη Sui Move. Η απουσία της ιδιότητας `store` περιορίζει τη μεταβίβασή του αποκλειστικά σε συναρτήσεις του module του.

3.1.4 Ομοφωνία Mysticeti v2

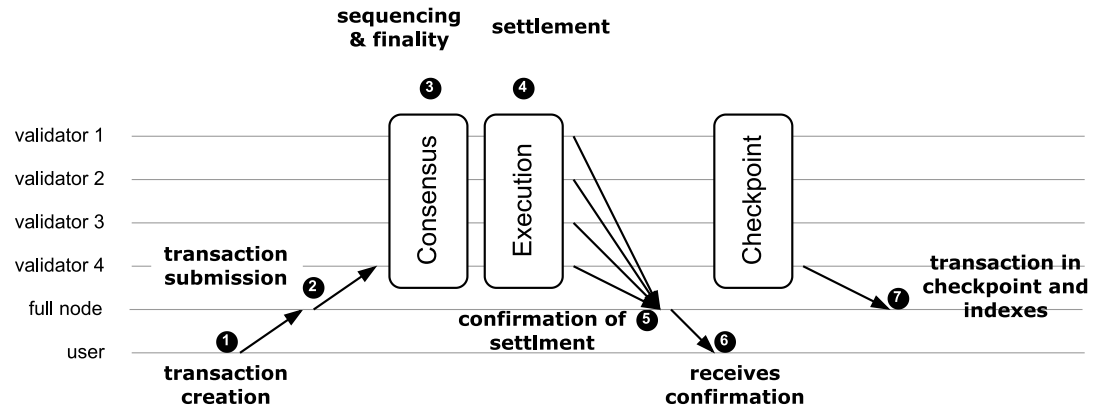
Το μοντέλο αντικειμένων καθορίζει ποιος μπορεί να χρησιμοποιήσει τα δεδομένα μιας συναλλαγής. Η σειρά επεξεργασίας των συναλλαγών καθορίζεται από το πρωτόκολλο ομοφωνίας Mysticeti v2 [108]. Σύμφωνα με την ταξινόμηση της ενότητας 2.1, το πρωτόκολλο είναι τύπου BFT και δίνει ντετερμινιστική οριστικότητα. Το SUI χρησιμοποιεί Delegated Proof-of-Stake για την επιλογή των επικυρωτών και τον καθορισμό της βαρύτητας της ψήφου τους [99].

Η ομοφωνία βασίζεται σε κατευθυνόμενο άκυκλο γράφο *(Directed Acyclic Graph, DAG)* [108]. Οι επικυρωτές προτείνουν παράλληλα μπλοκ με συναλλαγές και αναφορές σε προηγούμενα μπλοκ. Μέσα στον ίδιο γράφο ελέγχουν την εγκυρότητα των συναλλαγών και συμφωνούν στη σειρά τους. Οι ψήφοι αποδοχής προκύπτουν από τις συνδέσεις μεταξύ των μπλοκ, ενώ οι ψήφοι απόρριψης δηλώνονται ρητά. Έτσι, η επικύρωση των συναλλαγών γίνεται μέσα στην ομοφωνία, χωρίς χωριστή συλλογή υπογραφών για κάθε συναλλαγή πριν από την υποβολή της στον γράφο.

Αρχικά, ο χρήστης υπογράφει τη συναλλαγή και τη στέλνει σε έναν πλήρη κόμβο [102]. Ο Transaction Driver είναι ο μηχανισμός του πλήρους κόμβου που αναλαμβάνει την υποβολή και την παραλαβή του αποτελέσματος. Επιλέγει έναν επικυρωτή, ο οποίος ελέγχει την υπογραφή, τα αντικείμενα εισόδου και το διαθέσιμο αέριο. Αν οι έλεγχοι πετύχουν, ο επικυρωτής περιλαμβάνει τη συναλλαγή σε μπλοκ που προτείνει στην ομοφωνία. Οι υπόλοιποι επικυρωτές επαναλαμβάνουν τους ελέγχους πριν αποδεχθούν τη συναλλαγή.

Στη συνέχεια, οι επικυρωτές εκτελούν τις συναλλαγές που αποδέχθηκε και διέταξε η ομοφωνία [99]. Όλες οι συναλλαγές περνούν από το Mysticeti v2, ανεξάρτητα από το αν χρησιμοποιούν κοινόχρηστα αντικείμενα ή αντικείμενα ιδιοκτησίας από διεύθυνση. Ωστόσο, συναλλαγές χωρίς συγκρουόμενες προσβάσεις στα αντικείμενα εισόδου μπορούν να εκτελεστούν παράλληλα. Η κοινή σειρά που ορίζει η ομοφωνία δεν απαιτεί, επομένως, σειριακή εκτέλεση όλων των συναλλαγών.

Μετά την εκτέλεση, ο πλήρης κόμβος παίρνει τα αποτελέσματα από έναν επικυρωτή και επιβεβαιώσεις των ίδιων αποτελεσμάτων από απαρτία επικυρωτών [102]. Αν δεν συγκεντρώσει αυτές τις επιβεβαιώσεις, περιμένει την ένταξη της συναλλαγής σε πιστοποιημένο σημείο ελέγχου (checkpoint). Και στις δύο περιπτώσεις αποκτά απόδειξη οριστικότητας και επιστρέφει το αποτέλεσμα στον χρήστη. Τα σημεία ελέγχου κρατούν το μόνιμο ιστορικό που χρησιμοποιούν οι κόμβοι για τον συγχρονισμό τους.



Σχήμα 3.1. Ο κύκλος ζωής μιας συναλλαγής στο SUI [102].

Στο **Σχήμα 3.1** η ομοφωνία προηγείται της εκτέλεσης, στα βήματα 3 και 4 αντίστοιχα. Στο κάτω μέρος, ο χρήστης παίρνει την επιβεβαίωση στο βήμα 6, ενώ δεξιά φαίνεται η καταγραφή σε σημείο ελέγχου στο βήμα 7. Η επιβεβαίωση μπορεί να προηγηθεί αυτής της καταγραφής, εφόσον ο πλήρης κόμβος έχει ήδη πιστοποιήσει τα αποτελέσματα μέσω απαρτίας επικυρωτών.

3.2 Αποκεντρωμένη αποθήκευση στο Walrus

Το SUI αποθηκεύει αρκετά αποδοτικά μικρά αντικείμενα, όχι όμως μεγάλα αρχεία. Κάθε byte που γράφεται στο SUI αντιγράφεται σε όλους τους επικυρωτές. Έτσι, ο συντελεστής πλεονασμού *(replication factor)*, δηλαδή ο λόγος του δεσμευμένου χώρου προς το μέγεθος του αρχείου, ξεπερνά το 100 [30]. Για ένα μεγάλο αρχείο το κόστος αυτό είναι υψηλό. Ένας κεντρικός εξυπηρετητής θα έλυνε το πρόβλημα, αλλά τότε μέρος της κατάστασης της εφαρμογής θα ζούσε σε μηχάνημα που ελέγχει μία μόνο οντότητα. Δηλαδή, η εφαρμογή δεν θα ήταν πια πλήρως αποκεντρωμένη.

Το Walrus είναι η απάντηση του οικοσυστήματος SUI σε αυτό το πρόβλημα [30]. Το Walrus είναι ένα αποκεντρωμένο δίκτυο αποθήκευσης, που χρησιμοποιεί το SUI για τον συντονισμό του. Δηλαδή, τα ίδια τα αρχεία μένουν στους κόμβους του Walrus, αλλά τα μεταδεδομένα τους, οι πληρωμές και τα πιστοποιητικά διαθεσιμότητας τους γράφονται στο SUI [109], [110].

3.2.1 Blobs, shards και η επιτροπή αποθήκευσης

Η μονάδα αποθήκευσης στο Walrus είναι το blob, δηλαδή μια ακολουθία bytes χωρίς εσωτερική δομή. Κάθε blob έχει ένα αναγνωριστικό *(blob ID)*. Το αναγνωριστικό είναι η σύνοψη μιας δέσμευσης Merkle *(Merkle commitment)* πάνω στα κωδικοποιημένα κομμάτια του blob [30]. Επομένως, δύο ίδια αρχεία έχουν το ίδιο αναγνωριστικό, ενώ κάθε αλλοίωση κάποιου αρχείου δίνει διαφορετικό αναγνωριστικό. Επιπλέον, όποιος ξέρει το blob ID μπορεί να ζητήσει το blob και να επαληθεύσει ότι πήρε το σωστό αρχείο.

Τα blobs τα κρατά μια επιτροπή αποθήκευσης *(storage committee)*. Η επιτροπή έχει $n = 3f + 1$ shards, από τα οποία το πολύ f ελέγχονται από βυζαντινούς κόμβους, όπως και στο SUI [30]. Το shard είναι η μονάδα με την οποία μοιράζεται ο χώρος του δικτύου Walrus. Σε κάθε εποχή το συμβόλαιο του Walrus στο SUI αναθέτει σε κάθε κόμβο ένα ή περισσότερα shards, ανάλογα με το stake του. Έτσι, ο κάθε κόμβος κρατά όλα τα δεδομένα των shards που του έχουν ανατεθεί [109]. Στη συνέχεια, γράφουμε «κόμβος» με τη σύμβαση ένα shard ανά κόμβο, όπως και το paper όταν περιγράφει το πρωτόκολλο.

3.2.2 Η κωδικοποίηση Red Stuff

Το ερώτημα που μένει ανοιχτό από την προηγούμενη υποενότητα είναι πώς μοιράζεται ένα blob στους n κόμβους. Η διανομή πρέπει να κοστίζει λίγο σε χώρο και να αντέχει την απώλεια κόμβων. Η απλούστερη λύση είναι να κρατά κάθε κόμβος ένα πλήρες αντίγραφο. Τότε, όμως, ο συντελεστής πλεονασμού είναι n, δηλαδή η ίδια πλήρης αντιγραφή που κάνει και το blockchain δίκτυο, και το πρόβλημα απλώς μεταφέρεται.

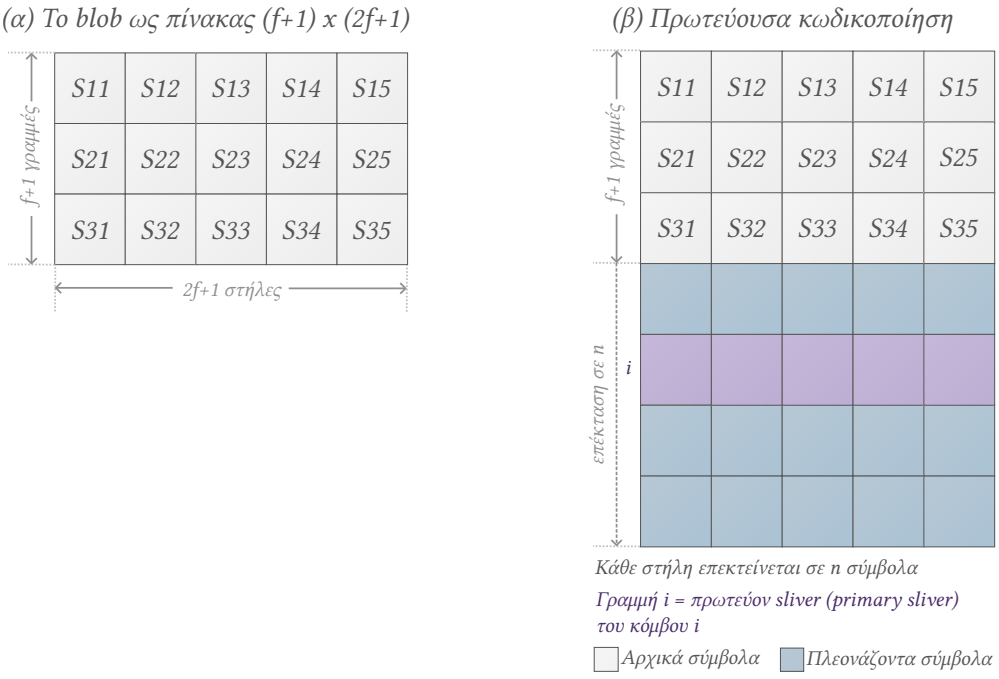
Η λύση που χρησιμοποιεί το Walrus είναι η κωδικοποίηση erasure *(erasure coding)*, με βάση τους κώδικες Reed-Solomon [111]. Συγκεκριμένα, το blob κόβεται σε κομμάτια ίσου μεγέθους, που τα λέμε σύμβολα *(symbols)*. Στη συνέχεια προστίθενται λίγα ακόμη σύμβολα με έναν μαθηματικό κανόνα *(polynomial interpolation)*. Ο κανόνας παράγει τα επιπλέον σύμβολα έτσι ώστε το blob να μπορεί να ανακατασκευαστεί από οποιοδήποτε σύνολο συμβόλων με πλήθος ίσο με εκείνο των αρχικών κομματιών.

Με αυτόν τον τρόπο μειώνουμε τον συντελεστή πλεονασμού. Συγκεκριμένα, με πλήρη αντιγραφή κάθε κόμβος θα κρατούσε ολόκληρο το blob, οπότε ο χώρος που δεσμεύεται θα ήταν n φορές το blob. Όμως, με κωδικοποίηση erasure κάθε κόμβος κρατά ένα μόνο σύμβολο. Ο συνολικός χώρος είναι τότε το άθροισμα όλων των συμβόλων, δηλαδή τα αρχικά κομμάτια συν τα επιπλέον σύμβολα. Επομένως, ο συντελεστής πλεονασμού γίνεται ο λόγος του πλήθους των συμβόλων προς το πλήθος των αρχικών κομματιών, δηλαδή ένας μικρός αριθμός που δεν εξαρτάται από το n. Τα επιπλέον σύμβολα είναι το τίμημα της αντοχής, γιατί το blob χάνεται μόνο αν χαθούν περισσότερα σύμβολα από όσα προσθέσαμε. Όσο περισσότερα προσθέσουμε, τόσο περισσότερους κόμβους αντέχει να χάσει το δίκτυο, αλλά τόσο μεγαλώνει και ο συντελεστής.

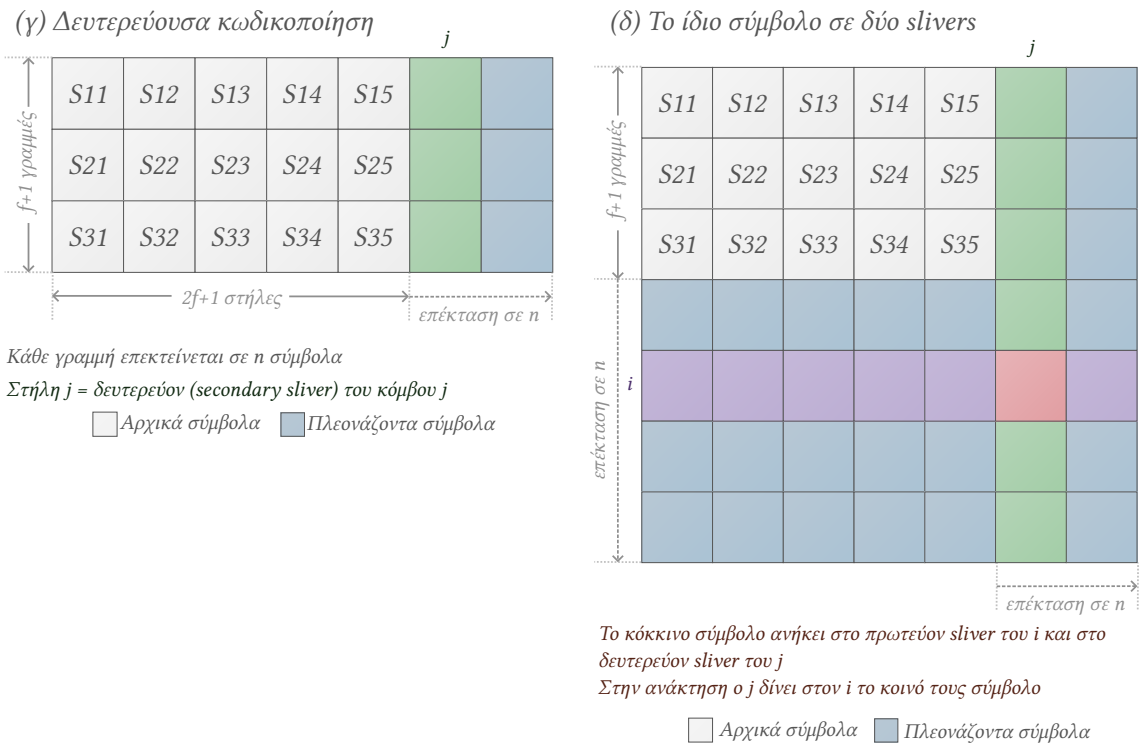
Ωστόσο, η κωδικοποίηση Reed-Solomon σε μία μόνο διάσταση έχει ένα σημαντικό πρόβλημα, την ανάκτηση. Όταν ένας κόμβος χάσει το shard του, πρέπει για κάθε blob να κατεβάσει από τους άλλους κόμβους δεδομένα ίσα με ολόκληρο το blob, μόνο για να ξαναφτιάξει το δικό του μικρό μερίδιο [30]. Σε ένα permissionless δίκτυο η σύνθεση των κόμβων αλλάζει σε κάθε εποχή, οπότε οι ανακτήσεις είναι συχνές. Στην περίπτωση αυτή, το κόστος γίνεται αρκετά μεγάλο.

Το Walrus λύνει αυτό το πρόβλημα με την κωδικοποίηση Red Stuff, που εφαρμόζει τον ίδιο κώδικα Reed-Solomon σε δύο διαστάσεις αντί για μία [30]. Αρχικά, το blob κόβεται σε $(f+1)(2f+1)$ σύμβολα, που σχηματίζουν έναν πίνακα με $f+1$ γραμμές και $2f+1$ στήλες. Έπειτα, κάθε στήλη επεκτείνεται με επιπλέον σύμβολα σε μήκος n, οπότε ο πίνακας αποκτά n γραμμές. Η γραμμή i είναι το πρωτεύον sliver *(primary sliver)* του κόμβου i. Το **Σχήμα 3.2** δείχνει το βήμα αυτό στο παράδειγμα $f = 2$, όπου ο πίνακας έχει 3 γραμμές και 5 στήλες και επεκτείνεται σε 7 γραμμές.

Η δεύτερη διάσταση προκύπτει με τον ίδιο τρόπο, από τις γραμμές. Κάθε γραμμή του αρχικού πίνακα επεκτείνεται σε μήκος n, οπότε προκύπτουν n στήλες. Η στήλη i είναι το δευτερεύον sliver *(secondary sliver)* του κόμβου i. Κάθε κόμβος παίρνει έτσι ένα ζεύγος slivers, μια γραμμή και μια στήλη, που τέμνονται σε ένα κοινό σύμβολο. Το **Σχήμα 3.3** δείχνει τη δεύτερη επέκταση και τον ρόλο του κοινού συμβόλου στην ανάκτηση. Η δεύτερη διάσταση ανεβάζει τον συντελεστή πλεονασμού από το 3 στο 4,5, πάντα δύο τάξεις μεγέθους κάτω από την αποθήκευση στο SUI, αλλά κάνει την ανάκτηση φθηνή [30].



Σχήμα 3.2. Η πρωτεύουσα κωδικοποίηση του Red Stuff, στο παράδειγμα $f = 2$ και $n = 7$. (α) Το blob ως πίνακας. (β) Κάθε στήλη επεκτείνεται σε n σύμβολα και η γραμμή i είναι το πρωτεύον sliver του κόμβου i.



Σχήμα 3.3. Η δευτερεύουσα κωδικοποίηση και η ανάκτηση του Red Stuff, στο ίδιο παράδειγμα. (γ) Κάθε γραμμή επεκτείνεται σε n σύμβολα και η στήλη j είναι το δευτερεύον sliver του κόμβου j. (δ) Το κοινό σύμβολο ανήκει και στα δύο slivers, οπότε στην ανάκτηση ο j το δίνει στον i.

Η διπλή κωδικοποίηση δίνει δύο ιδιότητες [30]:

1. Το blob ανακατασκευάζεται από οποιαδήποτε $f + 1$ πρωτεύοντα slivers. Το δίκτυο αντέχει έτσι την απώλεια σχεδόν των δύο τρίτων των κόμβων.

2. Ένας κόμβος που έχασε το ζεύγος του το ανακτά με ένα μόνο σύμβολο από κάθε άλλο κόμβο, το κοινό τους σύμβολο. Μεταφέρει έτσι $O\left(\frac{|B|}{n}\right)$ αντί για $O(|B|)$ bytes, όπου $|B|$ είναι το μέγεθος του blob, και δεν χρειάζεται να αποκωδικοποιήσει ολόκληρο το blob.

3.2.3 Εγγραφή και ανάγνωση

Πάνω στην κωδικοποίηση αυτή ορίζονται τα δύο πρωτόκολλα του δικτύου, η εγγραφή και η ανάγνωση [30], [110]. Και τα δύο τα εκτελεί ο πελάτης του Walrus, δηλαδή το εκτελέσιμο `walrus` ή το SDK μέσα σε μια εφαρμογή. Ο πελάτης μιλά απευθείας με τους κόμβους αποθήκευσης και με το SUI. Υπάρχουν όμως και προαιρετικές υπηρεσίες HTTP που το κάνουν για λογαριασμό άλλων. Συγκεκριμένα, υπάρχει δημόσιος aggregator για την ανάγνωση και publisher για την εγγραφή [109].

Εγγραφή. Η εγγραφή ενός blob έχει πέντε βήματα:

1. Ο πελάτης που ανεβάζει το blob το κωδικοποιεί και υπολογίζει το αναγνωριστικό του.
2. Καταχωρεί το blob στο SUI και δεσμεύει χώρο για συγκεκριμένο πλήθος εποχών.
3. Στέλνει σε κάθε κόμβο το ζεύγος slivers του μαζί με τις δεσμεύσεις. Ο κόμβος ελέγχει το ζεύγος πάνω στις δεσμεύσεις, το αποθηκεύει και απαντά με υπογραφή.
4. Όταν ο πελάτης μαζέψει $2f + 1$ υπογραφές, τις ενώνει σε ένα πιστοποιητικό διαθεσιμότητας *(availability certificate)*.
5. Δημοσιεύει το πιστοποιητικό στο SUI.

Η στιγμή της δημοσίευσης λέγεται σημείο διαθεσιμότητας *(Point of Availability, PoA)*. Από εκεί και πέρα την ευθύνη να μείνει το blob διαθέσιμο δεν την έχει πια ο πελάτης που το ανέβασε, αλλά οι κόμβοι αποθήκευσης του Walrus. Όποιος ξέρει το αναγνωριστικό μπορεί πλέον να ανακατασκευάσει το blob, για όσες εποχές έχει πληρωθεί ο χώρος του.

Ανάγνωση. Η ανάγνωση είναι η αντίστροφη διαδρομή, με τρία βήματα:

1. Ο πελάτης που θέλει το blob ζητά από τους κόμβους τις δεσμεύσεις και τα slivers του, με το αναγνωριστικό του.
2. Μόλις συγκεντρώσει $f + 1$ πρωτεύοντα slivers, ανακατασκευάζει το blob.
3. Ξανακωδικοποιεί το αποτέλεσμα και συγκρίνει τη δέσμευση με το αναγνωριστικό. Αν δεν ταιριάζουν, απορρίπτει το blob.

Δύο πελάτες που ζητούν το ίδιο αναγνωριστικό παίρνουν επομένως πάντα το ίδιο blob ή την ίδια αποτυχία [30]. Το ίδιο εξηγεί γιατί ο aggregator και ο publisher δεν χρειάζονται εμπιστοσύνη. Όποιος διαβάζει από έναν aggregator επαληθεύει το blob με το αναγνωριστικό. Όποιος γράφει μέσω ενός publisher, αντίστοιχα, ελέγχει στο SUI ότι το blob έφτασε στο σημείο διαθεσιμότητας [109].

3.2.4 Εποχές, κόστος και το token WAL

Το δίκτυο λειτουργεί σε εποχές αποθήκευσης *(storage epochs)*. Μια εποχή διαρκεί δύο εβδομάδες στο κύριο δίκτυο και μία ημέρα στο δοκιμαστικό [109], [112]. Στο τέλος κάθε εποχής η επιτροπή μπορεί να αλλάξει. Τότε οι νέοι κόμβοι ανακτούν τα slivers τους με το πρωτόκολλο ανάκτησης του Red Stuff. Το Walrus έχει δικό του token, το WAL, με Delegated Proof-of-Stake όμοιο με εκείνο του SUI. Με αυτό πληρώνεται η αποθήκευση για συγκεκριμένο πλήθος εποχών.

Μετά τη λήξη της περιόδου αυτής, η αποθήκευση πρέπει να ανανεωθεί. Ο πόρος αποθήκευσης και το ίδιο το blob είναι κανονικά αντικείμενα του SUI, οπότε ένα συμβόλαιο Move μπορεί να τα διαχειρίζεται [109].

Το κόστος υπολογίζεται πάνω στο κωδικοποιημένο μέγεθος, όχι στο μέγεθος του αρχείου. Το κωδικοποιημένο μέγεθος είναι περίπου 4,5 φορές το αρχικό, συν 64 MB μεταδεδομένων ανά blob, ανεξάρτητα από το μέγεθός του [113]. Για ένα blob του 1 MB πληρώνεται επομένως χώρος περίπου 68 MB, ενώ για ένα blob του 1 GB περίπου 4,56 GB. Τα μικρά αρχεία είναι, δηλαδή, ακριβά σε σχέση με το μέγεθός τους. Για πολλά μικρά αρχεία το Walrus προσφέρει το quilt, ένα blob που κωδικοποιεί πολλά αρχεία μαζί και πληρώνει τα μεταδεδομένα μία φορά, ενώ κάθε αρχείο του ανακτάται ξεχωριστά [114]. Η τιμή του χώρου ορίζεται σε δολάρια, 0,023 ανά GB τον μήνα, και το ποσό σε WAL προσαρμόζεται με την τιμή του token. Πέρα από τον χώρο, κάθε καταχώριση πληρώνει μια εφάπαξ χρέωση εγγραφής σε WAL, και οι έως τρεις συναλλαγές στο SUI, δηλαδή δέσμευση χώρου, καταχώριση και πιστοποίηση, πληρώνουν gas σε SUI [113].

3.3 Αποκεντρωμένη φιλοξενία ιστοσελίδων με Walrus Sites

Το Walrus μπορεί να αποθηκεύσει οποιοδήποτε αρχείο, άρα θα μπορούσε και τα αρχεία μιας διεπαφής χρήστη. Πάνω σε αυτή την ιδέα στηρίζονται τα Walrus Sites, δηλαδή η υπηρεσία αποκεντρωμένης φιλοξενίας ιστοσελίδων του Walrus [115]. Όμως, τα αρχεία δεν αρκούν από μόνα τους. Για να γίνουν ιστοσελίδα, χρειάζεται και κάτι που να λέει ποιο αρχείο αντιστοιχεί σε κάθε διαδρομή. Γι' αυτό, ένα Walrus Site αποτελείται από δύο μέρη. Το πρώτο είναι τα αρχεία του, που αποθηκεύονται ως blobs στο Walrus. Το δεύτερο είναι το ευρετήριό του, δηλαδή ένα αντικείμενο `Site` στο SUI.

3.3.1 Το αντικείμενο Site

Το ευρετήριο καταγράφει πού βρίσκεται το αρχείο κάθε διαδρομής. Συγκεκριμένα, για κάθε διαδρομή URL του site, π.χ. `/index.html`, το αντικείμενο `Site` έχει ένα δυναμικό πεδίο με όνομα τη διαδρομή. Η τιμή του πεδίου είναι το αναγνωριστικό του blob που περιέχει το αρχείο, μαζί με τις επικεφαλίδες HTTP με τις οποίες θα σταλεί, π.χ. τον τύπο περιεχομένου `text/html` [112]. Το ότι το ευρετήριο είναι αντικείμενο του SUI έχει δύο συνέπειες. Πρώτον, μόνο το πορτοφόλι που δημοσίευσε το site μπορεί να το αλλάξει, γιατί σε αυτό ανήκει το αντικείμενο. Δεύτερον, κάθε αλλαγή του είναι συναλλαγή, οπότε το ιστορικό των εκδόσεων του site είναι δημόσιο.

3.3.2 Ανάγνωση μέσω της πύλης

Για να φτάσει ο περιηγητής στο αντικείμενο αυτό, ξεκινά από τη διεύθυνση που πληκτρολογεί ο χρήστης. Το πρώτο τμήμα της διεύθυνσης, το subdomain, ονομάζει το site. Είναι είτε ένα όνομα SuiNS [116], είτε το ίδιο το αναγνωριστικό του αντικειμένου.

Η διεύθυνση, όμως, δεν αρκεί, γιατί ο περιηγητής δεν μπορεί να διαβάσει απευθείας από το SUI και το Walrus. Γι' αυτό, ανάμεσά τους υπάρχει η πύλη *(portal)*. Η πύλη είναι και το δεύτερο τμήμα της διεύθυνσης, δηλαδή το domain μετά το subdomain. Στη διεύθυνση `docs.wal.app`, π.χ., το `docs` είναι το όνομα SuiNS και το `wal.app` η πύλη. Όταν η πύλη λάβει μια αίτηση, επιλύει πρώτα το subdomain σε αναγνωριστικό αντικειμένου. Έπειτα, διαβάζει από το SUI το δυναμικό πεδίο της διαδρομής που ζήτησε ο περιηγητής. Τέλος, ανακτά το blob από έναν aggregator του Walrus και το στέλνει στον περιηγητή με τις επικεφαλίδες του πεδίου [115].

Η πύλη δεν φιλοξενεί τίποτα. Μεταφράζει μόνο κάθε αίτηση του περιηγητή σε αναγνώσεις από το SUI και το Walrus, και επιστρέφει ό,τι διάβασε. Γι' αυτό, αν μια πύλη κλείσει ή συμπεριφερθεί κακόβουλα, ο χρήστης πηγαίνει σε οποιαδήποτε άλλη. Το ίδιο το site μένει ανέπαφο, γιατί το περιεχόμενο κάθε διαδρομής καθορίζεται μοναδικά από το SUI και το Walrus. Όποιος πάλι δεν θέλει να εμπιστεύεται καμία πύλη, τρέχει δική του. Στην πράξη, η δημόσια πύλη `wal.app` της Mysten Labs εξυπηρετεί σήμερα μόνο ονόματα SuiNS στο κύριο δίκτυο [117]. Στο δοκιμαστικό δίκτυο δεν υπάρχει δημόσια πύλη, γι' αυτό η εφαρμογή μας τρέχει δική της.

3.4 Αποκεντρωμένος έλεγχος πρόσβασης με Seal

Κάθε blob στο Walrus είναι δημόσιο, γιατί ένα permissionless δίκτυο αποθήκευσης δεν διαθέτει κεντρική αρχή που να κρίνει ποιος δικαιούται να διαβάσει ένα αρχείο. Το blockchain δίκτυο, από την πλευρά του, λύνει το πρόβλημα της ταυτοποίησης, δηλαδή ποιος υπογράφει μια συναλλαγή. Δεν μπορεί, όμως, να αποθηκεύσει μυστικά δεδομένα στην κατάστασή του, αφού κάθε πληροφορία στο ιστορικό (ή log ή καθολικό) είναι πλήρως δημόσια. Δεν λύνει, επομένως, το πρόβλημα της εξουσιοδότησης, δηλαδή ποιος αποκρυπτογραφεί και πότε [118], [119]. Σε ένα τέτοιο περιβάλλον, τα δεδομένα αποθηκεύονται μία φορά, ενώ τα δικαιώματα πρόσβασης και η ιδιοκτησία των αντικειμένων αλλάζουν δυναμικά πάνω στο blockchain. Αν κάθε αλλαγή κατόχου απαιτούσε εκ νέου κρυπτογράφηση για το κλειδί του νέου παραλήπτη, τότε η κλασική κρυπτογραφία δημόσιου κλειδιού *(public-key cryptography)* δεν θα μπορούσε να λειτουργήσει σε ένα αμετάβλητο δίκτυο αποθήκευσης.

Το κενό αυτό καλύπτει το Seal, η υπηρεσία αποκεντρωμένης διαχείρισης μυστικών του οικοσυστήματος SUI [120]. Το σύστημα διαχωρίζει την κρυπτογράφηση από τον παραλήπτη και τη συνδέει με έναν κανόνα. Λύνει το πρόβλημα συνδυάζοντας δύο μηχανισμούς. Ο πρώτος είναι η κρυπτογραφία ταυτότητας *(identity-based encryption)*, στην οποία δημόσιο κλειδί μπορεί να είναι οποιαδήποτε ακολουθία bytes. Ο δεύτερος είναι μια πολιτική πρόσβασης *(policy)* γραμμένη σε Move, η οποία αποφασίζει ποιος δικαιούται να λάβει το ιδιωτικό κλειδί που αντιστοιχεί σε αυτή την ταυτότητα [31]. Ο κάτοχος κρυπτογραφεί για τον κανόνα μία φορά, τη στιγμή της αποθήκευσης. Αργότερα, όποιος χρήστης ικανοποιεί τον κανόνα τη στιγμή του αιτήματος αποκτά το κλειδί και αποκρυπτογραφεί τα δεδομένα.

3.4.1 Κρυπτογραφία ταυτότητας και κρυπτογραφία κατωφλίου

Η κρυπτογραφία ταυτότητας λύνει το πρόβλημα της διαχείρισης πιστοποιητικών. Στην κλασική ασύμμετρη κρυπτογραφία ο αποστολέας χρειάζεται το πιστοποιημένο δημόσιο κλειδί του παραλήπτη, άρα μια σύνθετη υποδομή δημόσιου κλειδιού. Για να αποφευχθεί η απαίτηση αυτή, ο Shamir [121] πρότεινε το 1984 να χρησιμοποιείται ως δημόσιο κλειδί η ίδια η ταυτότητα του παραλήπτη. Την πρώτη πρακτική κατασκευή έδωσαν αργότερα οι Boneh και Franklin [122]. Μια γεννήτρια ιδιωτικών κλειδιών *(private key generator)* παράγει ένα κύριο μυστικό και ένα κύριο δημόσιο κλειδί, και από το κύριο μυστικό παράγει το ιδιωτικό κλειδί κάθε ταυτότητας. Ο αποστολέας κρυπτογραφεί με το κύριο δημόσιο κλειδί και την ταυτότητα, χωρίς προηγούμενη επικοινωνία με τον παραλήπτη. Τέλος, ο κάτοχος του ιδιωτικού κλειδιού της ταυτότητας αποκρυπτογραφεί.

Το Seal μεταφέρει την ιδέα αυτή σε ένα εντελώς νέο πλαίσιο. Στην κλασική χρήση, η ταυτότητα αντιπροσωπεύει ένα φυσικό πρόσωπο, όπως μια διεύθυνση ηλεκτρονικού ταχυδρομείου. Στο Seal, αντίθετα, ως ταυτότητα δεν ορίζεται ένας χρήστης, αλλά το αναγνωριστικό του ίδιου του προστατευόμενου πόρου *(resource identity)*, δηλαδή μια ακολουθία bytes κάτω από τον

χώρο ονομάτων ενός πακέτου Move. Στην εφαρμογή μας, για παράδειγμα, η ταυτότητα είναι το αναγνωριστικό του αρχείου εικόνας μαζί με ένα τυχαίο αλάτι *(salt)*. Ο αποστολέας κρυπτογραφεί τα δεδομένα προς την ταυτότητα του πόρου μία φορά. Το αντίστοιχο ιδιωτικό κλειδί δεν παραδίδεται εκ των προτέρων σε κανέναν, αλλά παράγεται δυναμικά μόνο όταν ένας αιτών αποδείξει ότι πληροί τις προϋποθέσεις της πολιτικής στο blockchain δίκτυο.

Επειδή η ασύμμετρη κρυπτογραφία είναι υπολογιστικά ακριβή για μεγάλα αρχεία, το Seal ακολουθεί υβριδική σχεδίαση *(hybrid encryption)*. Η κρυπτογραφία ταυτότητας υλοποιείται πάνω στην ελλειπτική καμπύλη BLS12-381 [31] και προστατεύει αποκλειστικά ένα τυχαίο συμμετρικό κλειδί. Τα ίδια τα δεδομένα κρυπτογραφούνται τοπικά με αυτό το συμμετρικό κλειδί μέσω AES-256-GCM [123].

Στην αρχική της μορφή, η κρυπτογραφία ταυτότητας έχει το μειονέκτημα της κεντρικής φύλαξης κλειδιών. Η γεννήτρια κατέχει το κύριο μυστικό και μπορεί να αποκρυπτογραφήσει όλα τα δεδομένα. Η λύση είναι η κρυπτογραφία κατωφλίου *(threshold cryptography)*, όπου το μυστικό μοιράζεται σε πολλούς ανεξάρτητους φορείς. Η βάση της είναι ο διαμοιρασμός μυστικού *(secret sharing)* του Shamir [124]. Ένα μυστικό διασπάται σε n μερίδια, ώστε να ανακτάται από οποιαδήποτε t από αυτά, ενώ λιγότερα από t μερίδια δεν αποκαλύπτουν καμία πληροφορία. Το ζεύγος (t, n) ονομάζεται κατώφλι *(threshold)*, γι' αυτό και ονομάζεται κρυπτογραφία κατωφλίου.

Το Seal εφαρμόζει την αρχή αυτή αποκεντρώνοντας τις γεννήτριες σε n ανεξάρτητους διακομιστές κλειδιών *(key servers)*. Κάθε διακομιστής διαθέτει δικό του κύριο μυστικό και κύριο δημόσιο κλειδί. Το μυστικό που διαμοιράζεται είναι το συμμετρικό κλειδί των δεδομένων. Διασπάται σε n μερίδια, και κάθε μερίδιο κρυπτογραφείται προς την ταυτότητα του πόρου με το κύριο δημόσιο κλειδί διαφορετικού διακομιστή. Για την αποκρυπτογράφηση απαιτούνται τουλάχιστον t μερίδια [119]. Η εμπιστευτικότητα διατηρείται όσο έχουν παραβιαστεί λιγότεροι από t διακομιστές, ενώ η διαθεσιμότητα εξασφαλίζεται όσο τουλάχιστον t διακομιστές παραμένουν σε λειτουργία.

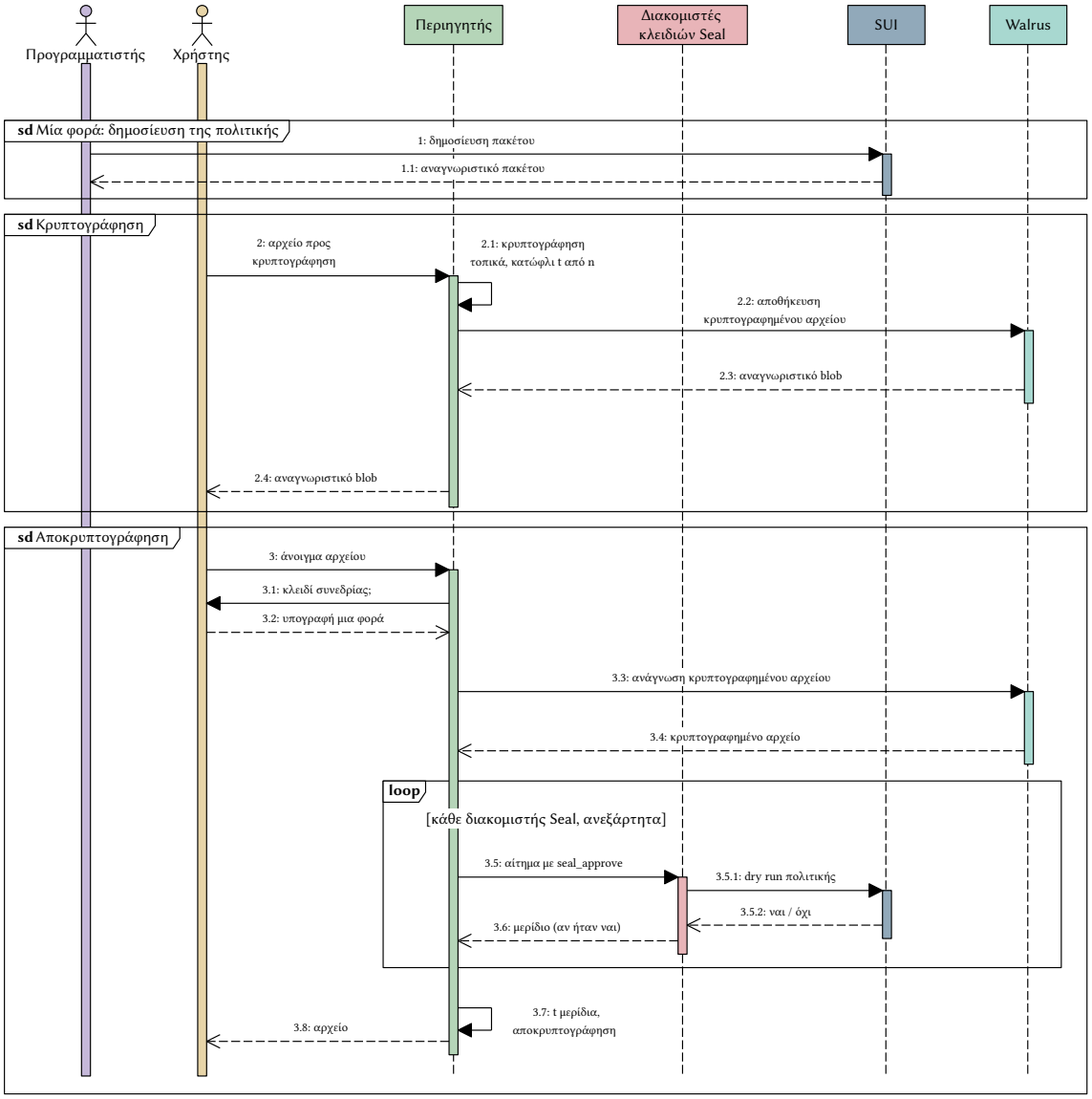
3.4.2 Πολιτικές σε Move

Μένει ο δεύτερος μηχανισμός, η πολιτική. Ο πυρήνας του Seal είναι η αντιστοίχιση ταυτοτήτων και πακέτων Move. Ένα πακέτο με διεύθυνση PkgId ελέγχει όλες τις ταυτότητες που ξεκινούν με το πρόθεμα PkgId, δηλαδή τον χώρο ονομάτων *(namespace)* του [31]. Μέσα στο πακέτο, η εξουσιοδότηση εκφράζεται με συναρτήσεις που ξεκινούν με `seal_approve`. Η πρώτη παράμετρός τους είναι η ζητούμενη ταυτότητα χωρίς το πρόθεμα. Αν η πρόσβαση δεν εγκρίνεται, η συνάρτηση τερματίζει με σφάλμα. Επιπλέον, δεν επιτρέπεται να αλλάξει την κατάσταση του blockchain δικτύου [125]. Ο λόγος είναι ο τρόπος που καλείται. Οι διακομιστές κλειδιών δεν υποβάλλουν συναλλαγή για να την καλέσουν. Αντί γι' αυτό, την τρέχουν σε δοκιμαστική εκτέλεση *(dry run)* σε έναν πλήρη κόμβο, πάνω στην τρέχουσα κατάσταση, χωρίς να γράψουν

τίποτα [125], [126]. Η πολιτική διαβάζει, επομένως, τη ζωντανή κατάσταση του blockchain δικτύου, όπως ιδιοκτησία αντικειμένων ή μητρώα δικαιωμάτων, τη στιγμή του αιτήματος.

Ο διακομιστής κλειδιών είναι σκόπιμα ελαφρύς. Κρατά μόνο το δικό του ζεύγος κύριων κλειδιών και παράγει κλειδιά ταυτοτήτων κατά παραγγελία. Δεν αποθηκεύει κλειδιά εφαρμογών ή χρηστών, και δεν βλέπει ποτέ καθαρά δεδομένα [118], [126]. Ό,τι χρειάζεται για την απόφαση το φέρνει το ίδιο το αίτημα. Κάθε αίτημα πρέπει να είναι υπογεγραμμένο από τον χρήστη και να περιέχει το μπλοκ συναλλαγών που καλεί τη `seal_approve`. Πρέπει, επίσης, να δίνει ένα εφήμερο κλειδί μιας χρήσης, ώστε η απάντηση να διαβάζεται μόνο από τον αιτούντα [31].

Η συνολική ροή έχει πέντε βήματα, τα οποία δείχνει το **Σχήμα 3.4**. Αρχικά, ο προγραμματιστής δημοσιεύει το πακέτο με τις πολιτικές. Έπειτα, η εφαρμογή κρυπτογραφεί τοπικά, με ταυτότητα, κατώφλι και διακομιστές της επιλογής της. Το κρυπτοκείμενο το αποθηκεύει όπου θέλει, στο Walrus για την εφαρμογή μας. Στη συνέχεια, ο χρήστης εγκρίνει στο πορτοφόλι του ένα κλειδί συνεδρίας *(session key)* για συγκεκριμένο πακέτο και χρονικό παράθυρο, ώστε να μην υπογράφει σε κάθε αίτημα. Η εφαρμογή στέλνει τότε το αίτημα στους διακομιστές, και καθένας αξιολογεί ανεξάρτητα την πολιτική και, αν ικανοποιείται, επιστρέφει το μερίδιό του. Τέλος, με t μερίδια η εφαρμογή ανακατασκευάζει το κλειδί και αποκρυπτογραφεί στον περιηγητή.



Σχήμα 3.4. Η ροή λειτουργίας του Seal. Η κρυπτογράφηση δεν χρειάζεται επικοινωνία με τους διακομιστές. Η αποκρυπτογράφηση χρειάζεται t από n διακομιστές, που αξιολογούν ανεξάρτητα την ίδια πολιτική.

Δύο περιορισμοί του Seal επηρεάζουν τη σχεδίαση της εφαρμογής μας [31], [127]. Πρώτον, το σύνολο των διακομιστών ορίζεται στην κρυπτογράφηση και δεν αλλάζει για τα υπάρχοντα κρυπτοκείμενα. Δεύτερον, ο διακομιστής παράγει ένα σταθερό κλειδί ανά ταυτότητα. Όποιος το πήρε μία φορά το κρατά, οπότε η ανάκληση της πρόσβασης εμποδίζει μόνο τα μελλοντικά αιτήματα.

3.5 Επαληθεύσιμος υπολογισμός εκτός blockchain με Nautilus

Τα έξυπνα συμβόλαια στο SUI εκτελούνται ντετερμινιστικά και δεν μπορούν να αντλήσουν απευθείας πληροφορία από τον έξω κόσμο, ούτε να εκτελέσουν υπολογισμούς που απαιτούν ιδιωτικά δεδομένα. Στην ενότητα 2.3.5 είδαμε ότι τη λύση σε αυτόν τον περιορισμό δίνουν τα oracles επαληθεύσιμου υπολογισμού, όπου η εκτέλεση ανατίθεται σε ασφαλές υλικό. Το Nautilus είναι το εργαλείο του οικοσυστήματος SUI για τον σκοπό αυτό [128]. Ο υπολογισμός τρέχει μέσα σε έναν θάλαμο *(enclave)*, δηλαδή σε ένα TEE, και ο θάλαμος υπογράφει το αποτέλεσμα. Έπειτα, ένα συμβόλαιο Move επαληθεύει στο blockchain δίκτυο τόσο την υπογραφή όσο και τη βεβαίωση του υλικού για τον κώδικα που εκτελείται. Το Nautilus δίνει και τα δύο μέρη, ένα πακέτο Move για τον έλεγχο και ένα πρότυπο *(template)* εξυπηρετητή για τον κώδικα του θαλάμου [129].

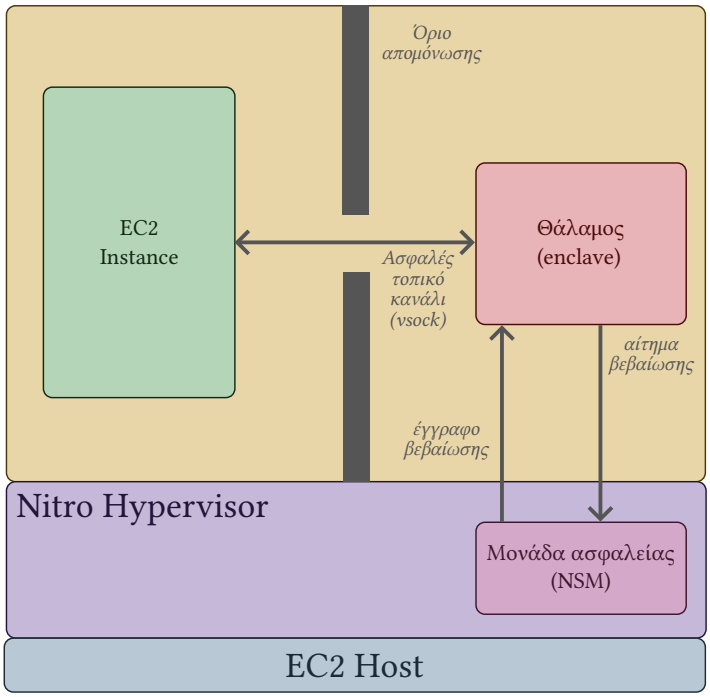
3.5.1 Περιβάλλοντα αξιόπιστης εκτέλεσης και AWS Nitro

Ο θάλαμος είναι ένα TEE, οπότε ξεκινάμε από αυτό. Ένα TEE παρέχει τρεις ιδιότητες [130], [131]:

- **Απομόνωση *(isolation).*** Ο κώδικας και τα δεδομένα στο εσωτερικό του δεν διαβάζονται ούτε αλλάζουν από λογισμικό απέξω, ούτε από το λειτουργικό σύστημα ούτε από τον διαχειριστή της μηχανής.
- **Μετρήσιμη εκκίνηση *(measured boot).*** Κατά την εκκίνηση το υλικό υπολογίζει μια σύνοψη του κώδικα και της διαμόρφωσής του.
- **Βεβαίωση *(attestation).*** Το περιβάλλον παράγει, με τη βοήθεια του υλικού, μια υπογεγραμμένη δήλωση της μέτρησής του, την οποία μπορούν να ελέγξουν απομακρυσμένα μέρη.

Οι ιδιότητες αυτές ορίζονται απέναντι σε έναν συγκεκριμένο αντίπαλο. Στο μοντέλο απειλής ο αντίπαλος ελέγχει όλο το λογισμικό της μηχανής και το δίκτυο. Στο Nautilus ο αντίπαλος αυτός είναι ο ίδιος ο διαχειριστής της εφαρμογής, αφού ο θάλαμος τρέχει στη δική του υποδομή [32]. Εκτός μοντέλου μένουν, αντίθετα, ο κατασκευαστής του υλικού και οι επιθέσεις πλευρικών καναλιών *(side-channel attacks)*. Το TEE, δηλαδή, θεωρεί έμπιστο τον κατασκευαστή και δεν προστατεύει από τις επιθέσεις αυτές [131].

Το Nautilus χρησιμοποιεί ως TEE τα Amazon Web Services, AWS Nitro Enclaves. Ό,τι τρέχει μέσα στον θάλαμο πακετάρεται σε ένα αρχείο εικόνας θαλάμου *(Enclave Image File, EIF)*. Τα Nitro Enclaves υλοποιούν την απομόνωση σε επίπεδο εικονικής μηχανής, όπως δείχνει το **Σχήμα 3.5**. Ο θάλαμος παίρνει δικούς του πυρήνες και δική του φυσική μνήμη από τη γονική μηχανή *(parent instance)*, που στο σχήμα είναι το EC2 Instance, και η γονική μηχανή χάνει κάθε πρόσβαση σε αυτά [132].



Σχήμα 3.5. Η αρχιτεκτονική ενός Nitro Enclave. Ο θάλαμος και η γονική μηχανή χωρίζονται από το όριο απομόνωσης που επιβάλλει ο επόπτης Nitro, και επικοινωνούν μόνο μέσα από το ασφαλές τοπικό κανάλι. Η μονάδα ασφαλείας είναι η μόνη που υπογράφει έγγραφα βεβαίωσης. Σχεδιασμένο με βάση την τεκμηρίωση της AWS [132].

Ο θάλαμος δεν έχει δίσκο ούτε διεπαφή δικτύου, και κανένας χειριστής δεν μπορεί να συνδεθεί σε αυτόν. Το μόνο κανάλι του προς τον έξω κόσμο είναι μια τοπική υποδοχή *(vsock)* προς τη γονική μηχανή, το ασφαλές τοπικό κανάλι του σχήματος, και η γονική μηχανή προωθεί την κίνηση για λογαριασμό του. Τέλος, μέσα στον επόπτη τρέχει η μονάδα ασφαλείας *(Nitro Security Module, NSM)*, το μόνο στοιχείο που μπορεί να υπογράψει βεβαιώσεις. Τα δύο κάθετα βέλη στα δεξιά του σχήματος δείχνουν το αίτημα του θαλάμου προς τη μονάδα και το υπογεγραμμένο έγγραφο βεβαίωσης που παίρνει πίσω.

Η μετρήσιμη εκκίνηση δίνει στον θάλαμο την ταυτότητά του. Η ταυτότητα είναι ένα σύνολο συνόψεων SHA-384 του κώδικά του, τα μητρώα διαμόρφωσης πλατφόρμας *(Platform Configuration Registers, PCR)*. Το PCR0 μετρά το αρχείο εικόνας, το PCR1 τον πυρήνα και τις παραμέτρους εκκίνησης, και το PCR2 την εφαρμογή [132]. Τα γράφει ο επόπτης *(hypervisor)* με βάση αυτό που πραγματικά φορτώθηκε. Αν αλλάξει, επομένως, ένα byte του κώδικα, η μέτρηση αλλάζει ριζικά.

3.5.2 Βεβαίωση και επαλήθευση στο blockchain δίκτυο

Η τρίτη ιδιότητα, η βεβαίωση, παίρνει τη μορφή ενός εγγράφου. Το έγγραφο βεβαίωσης *(attestation document)* είναι μια δομή υπογεγραμμένη από το TEE. Περιέχει τις τιμές των PCR, ένα δημόσιο κλειδί που επιλέγει η εφαρμογή του θαλάμου, μια χρονοσφραγίδα *(timestamp)*, και

την αλυσίδα πιστοποιητικών *(certificate chain)* μέχρι τη ρίζα εμπιστοσύνης του υλικού [132]. Όποιος παραλαμβάνει το έγγραφο κάνει τέσσερα βήματα για να το ελέγξει:

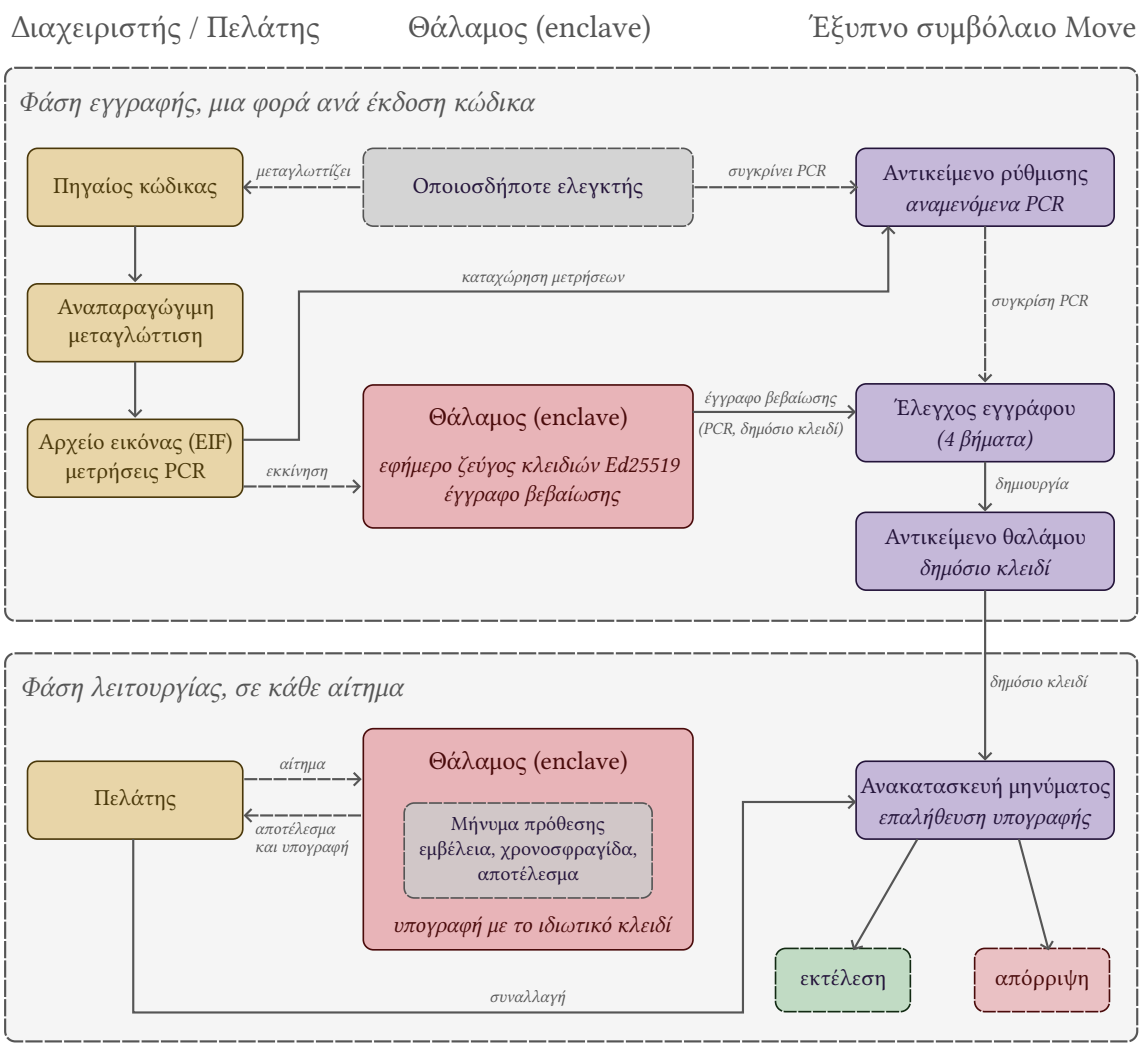
1. Αναλύει τη δομή του εγγράφου.
2. Επαληθεύει την αλυσίδα πιστοποιητικών μέχρι τη ρίζα εμπιστοσύνης.
3. Επαληθεύει την υπογραφή του TEE.
4. Συγκρίνει τις τιμές των PCR με τις αναμενόμενες.

Στο Nautilus και τα τέσσερα βήματα τρέχουν στο blockchain δίκτυο. Τα τρία πρώτα τα κάνει μια συνάρτηση του Sui Framework, και το τέταρτο το πακέτο `enclave` του Nautilus [32], [129]. Αν και τα τέσσερα περάσουν, το συμβόλαιο ξέρει ότι το δημόσιο κλειδί του εγγράφου προήλθε από θάλαμο που τρέχει ακριβώς την εικόνα με τις καταχωρημένες μετρήσεις.

Μένει, ωστόσο, ένα κενό. Η μέτρηση λέει ποια εικόνα τρέχει, όχι τι κάνει ο κώδικάς της. Το κενό αυτό το κλείνουν οι αναπαραγώγιμες μεταγλωττίσεις *(reproducible builds)*, στις οποίες από τον ίδιο πηγαίο κώδικα προκύπτει πάντα η ίδια εικόνα. Επομένως, όποιος θέλει μπορεί να διαβάσει τον δημοσιευμένο κώδικα, να τον μεταγλωττίσει και να συγκρίνει τα PCR που παίρνει με τα καταχωρημένα στο blockchain δίκτυο [129]. Αν οι τιμές συμπίπτουν, ο κώδικας που διάβασε είναι ο κώδικας που τρέχει. Η εμπιστοσύνη στον διαχειριστή γίνεται έτσι δημόσιος έλεγχος του κώδικα.

Με τα δύο αυτά εργαλεία, τη βεβαίωση και την αναπαραγώγιμη μεταγλώττιση, στήνεται μια εφαρμογή Nautilus. Η εφαρμογή έχει δύο μέρη και δύο φάσεις, τα οποία δείχνει το **Σχήμα 3.6**. Το πρώτο μέρος είναι ο εξυπηρετητής *(server)* που τρέχει μέσα στον θάλαμο. Ο εξυπηρετητής παράγει στην εκκίνηση ένα εφήμερο ζεύγος κλειδιών υπογραφής Ed25519. Το ιδιωτικό κλειδί μένει στη μνήμη του θαλάμου, που δεν έχει δίσκο, και χάνεται σε κάθε επανεκκίνηση. Το δημόσιο κλειδί είναι αυτό που μπαίνει στο έγγραφο βεβαίωσης, όπως είδαμε πιο πάνω.

Το δεύτερο μέρος είναι το συμβόλαιο. Το συμβόλαιο αποθηκεύει τις αναμενόμενες τιμές των PCR σε ένα κοινόχρηστο αντικείμενο ρύθμισης (`EnclaveConfig`) και ένα εγγεγραμμένο δημόσιο κλειδί σε κάθε αντικείμενο θαλάμου (`Enclave`). Και τα δύο αντικείμενα έχουν μια παράμετρο τύπου *(type parameter)*, δηλαδή ένα όνομα τύπου που ορίζει η εφαρμογή, ώστε ο θάλαμος μιας εφαρμογής να μη χρησιμοποιείται από το συμβόλαιο άλλης [129].



Σχήμα 3.6. Ο κύκλος ζωής μιας εφαρμογής Nautilus. Πάνω η φάση εγγραφής, μία φορά ανά έκδοση κώδικα. Κάτω η φάση λειτουργίας, σε κάθε αίτημα. Το δημόσιο κλειδί που καταχωρείται στην πρώτη φάση επαληθεύει τις υπογραφές της δεύτερης.

Η φάση εγγραφής, στην πάνω ζώνη του σχήματος, γίνεται μία φορά ανά έκδοση του κώδικα. Ο διαχειριστής δημοσιεύει τον κώδικα, καταχωρεί τις μετρήσεις στο αντικείμενο ρύθμισης, εκκινεί τον θάλαμο και υποβάλλει το έγγραφο βεβαίωσης στο συμβόλαιο. Το συμβόλαιο ελέγχει το έγγραφο με τα τέσσερα βήματα και δημιουργεί τότε το αντικείμενο θαλάμου με το δημόσιο κλειδί. Στην ίδια ζώνη φαίνεται και ο εξωτερικός ελεγκτής, που μεταγλωττίζει τον ίδιο κώδικα και συγκρίνει τα PCR με τα καταχωρημένα.

Η φάση λειτουργίας, στην κάτω ζώνη, γίνεται σε κάθε αίτημα. Ο θάλαμος υπολογίζει το αποτέλεσμα και το βάζει σε ένα μήνυμα πρόθεσης *(intent message)*, μαζί με μια εμβέλεια *(scope)* και μια χρονοσφραγίδα. Έπειτα, σειριοποιεί το μήνυμα με την κανονική δυαδική σειριοποίηση *(Binary Canonical Serialization, BCS)* και το υπογράφει με το ιδιωτικό του κλειδί. Το συμβόλαιο ανακατασκευάζει το ίδιο μήνυμα από τα ορίσματα της κλήσης και επαληθεύει την υπογραφή με το δημόσιο κλειδί του αντικειμένου θαλάμου, πριν εκτελέσει οποιαδήποτε λογική [32], [129].

Το κλειδί αυτό είναι το κάθετο βέλος που ενώνει τις δύο ζώνες του σχήματος. Η εμβέλεια δένει την υπογραφή σε έναν τύπο μηνύματος, ώστε να μην ισχύει για κανέναν άλλον, και η χρονοσφραγίδα επιτρέπει στο συμβόλαιο να απορρίπτει παλιά μηνύματα.

Ο χωρισμός σε δύο φάσεις έχει και οικονομικό λόγο. Η ακριβή πράξη, ο έλεγχος του εγγράφου βεβαίωσης, πληρώνεται μία φορά ανά έκδοση. Η φθηνή πράξη, μία επαλήθευση υπογραφής Ed25519, πληρώνεται σε κάθε αίτημα.

Συνολικά, η εμπιστοσύνη στηρίζεται σε δύο στοιχεία. Το ένα είναι το ασφαλές υλικό του TEE με τη ρίζα εμπιστοσύνης του. Το άλλο είναι ο δημοσιευμένος κώδικας, τον οποίο ελέγχει δημόσια όποιος θέλει [32]. Το Nautilus εγγυάται, επομένως, ότι ό,τι φτάνει στο blockchain δίκτυο προήλθε από τον κώδικα που μετρήθηκε. Τα δύο όρια που είδαμε στην ενότητα 2.3.5, η διαθεσιμότητα και η αλήθεια της εισόδου, ισχύουν όμως και εδώ. Ο διαχειριστής μπορεί να σβήσει τον θάλαμο, και μια λάθος ανάγνωση της πηγής υπογράφεται εξίσου πιστά.

Κεφάλαιο 4

Σχεδίαση και υλοποίηση της εφαρμογής

Στο κεφάλαιο αυτό παρουσιάζουμε τη σχεδίαση της εφαρμογής και την υλοποίησή της πάνω στη στοίβα SUI, από τους ρόλους και τις απαιτήσεις μέχρι το oracle. Κάθε ενότητα δείχνει την απόφαση, το σχήμα ή τον πίνακα που την περιγράφει, και τον κώδικα στο σημείο όπου ενώνονται δύο συστήματα.

4.1 Ρόλοι χρηστών και σενάρια χρήσης

Στην εφαρμογή μας εμπλέκονται έξι ρόλοι. Από τις ανάγκες τους, διατυπώνουμε τα σενάρια χρήσης του **Πίνακας 4.1**, με τη μορφή «θέλω ενέργεια, ώστε όφελος».

Πίνακας 4.1. Τα σενάρια χρήσης ανά ρόλο, με τη μορφή «θέλω ενέργεια, ώστε όφελος».

Ρόλος	Σενάριο χρήσης
Καλλιτέχνης	Θέλω να εκδώσω το έργο τέχνης μου, ώστε να αποκτήσει μοναδική και δημόσια επαληθεύσιμη ψηφιακή ταυτότητα
Καλλιτέχνης	Θέλω να κλασματοποιήσω το έργο τέχνης μου, ώστε πολλοί επενδυτές να αποκτήσουν μερίδιο και εγώ να κρατήσω όσα κλάσματα θέλω
Καλλιτέχνης	Θέλω να εισπράττω αυτόματα ποσοστό σε κάθε μεταπώληση, ώστε να μη στηρίζομαι στην εθελοντική συμμόρφωση κανενός
Επενδυτής	Θέλω να αγοράζω και να πουλάω κλάσματα οποιαδήποτε στιγμή, ώστε να επενδύω με το ποσό που αντέχω και να ρευστοποιώ όταν το χρειάζομαι
Επενδυτής	Θέλω να βλέπω το πιστοποιητικό, το ιστορικό και την κατάσταση φύλαξης πριν αγοράσω, και να ειδοποιούμαι όταν κάτι αλλάζει, ώστε να μην αγοράζω στα τυφλά
Επενδυτής	Θέλω να πουλάω μέρος μόνο της θέσης μου, ώστε να ρευστοποιώ όσο χρειάζομαι χωρίς να πουλήσω το σύνολο
Γκαλερί	Θέλω να καταχωρώ την παραλαβή έργων τέχνης σε φύλαξη και να εκδίδω τα πιστοποιητικά τους, ώστε η επαγγελματική μου πράξη να έχει δημόσιο και μόνιμο τεκμήριο
Γκαλερί	Θέλω να βλέπω τις αναφορές κατάστασης των έργων τέχνης που φυλάσσω και να δηλώνω απώλεια όταν συμβεί, ώστε το blockchain δίκτυο να μην περιγράφει κάτι που δεν ισχύει
Ρυθμιστική αρχή	Θέλω να εγγράφω και να ανακαλώ γκαλερί και πηγές με άμεση ισχύ, ώστε να εποπτεύω την αγορά χωρίς να επεμβαίνω σε ιδιοκτησία ή συναλλαγές
Ρυθμιστική αρχή	Θέλω να ρυθμίζω τις παραμέτρους της αγοράς, δηλαδή το ποσοστό του δικαιώματος παρακολούθησης και το παράθυρο συνάθροισης του oracle
Εξωτερική πηγή	Θέλω να υποβάλλω περιοδικές αναγνώσεις για την παρουσία και την κατάσταση των έργων τέχνης, ώστε οι κάτοχοι να έχουν πρόσφατη εικόνα του φυσικού κόσμου
Επισκέπτης	Θέλω να βλέπω ποιος πιστοποίησε ένα έργο τέχνης και το ιστορικό του με μόνο το αναγνωριστικό του, ώστε να ελέγχω τις καταγεγραμμένες πράξεις
Επισκέπτης	Θέλω να βλέπω ποιος εποπτεύει την αγορά και τι έχει πράξει, ποιες γκαλερί και πηγές αναγνωρίζονται, και τι κατέγραψε το oracle, ώστε η εμπιστοσύνη στην αρχή να είναι ελέγξιμη και όχι δεδομένη

4.2 Λειτουργικές απαιτήσεις

Τα σενάρια της ενότητας 4.1 δίνουν τις λειτουργικές απαιτήσεις της εφαρμογής. Ο **Πίνακας 4.2** τις απαριθμεί με τον δρώντα καθεμιάς.

Πίνακας 4.2. Λειτουργικές απαιτήσεις, ομαδοποιημένες κατά στάδιο του κύκλου ζωής.

Κωδ.	Απαίτηση	Δρων
Πρόσβαση και πληροφόρηση		
ΛΑ-01	Σύνδεση με πορτοφόλι Web3 και επιβεβαίωση του ρόλου από τα μητρώα	Όλοι
ΛΑ-02	Πλοήγηση και αναζήτηση στον κατάλογο των έργων τέχνης	Οποιοσδήποτε
ΛΑ-03	Δημόσια επαλήθευση προέλευσης, γνησιότητας και φύλαξης	Οποιοσδήποτε
ΛΑ-04	Δημόσια διαφάνεια για την αγορά, τα μητρώα, τις πράξεις της ρυθμιστικής αρχής και το oracle	Οποιοσδήποτε
ΛΑ-05	Ειδοποιήσεις ανά ρόλο για ό,τι εκκρεμεί ή χρειάζεται προσοχή	Κάθε συνδεδεμένος
Έκδοση και φύλαξη		
ΛΑ-06	Έκδοση έργου τέχνης με μεταδεδομένα και αρχεία	Καλλιτέχνης/Επενδυτής
ΛΑ-07	Κρυπτογράφηση της εικόνας υψηλής ανάλυσης του έργου τέχνης, με πολιτική ιδιοκτησίας	Καλλιτέχνης/Επενδυτής
ΛΑ-08	Ακύρωση της έκδοσης πριν από την παραλαβή του έργου τέχνης από τη γκαλερί	Καλλιτέχνης/Επενδυτής
ΛΑ-09	Καταχώριση παραλαβής σε φύλαξη από εγγεγραμμένη γκαλερί	Γκαλερί
ΛΑ-10	Έκδοση πιστοποιητικού γνησιότητας από τη γκαλερί που φυλάσσει το έργο τέχνης	Γκαλερί
ΛΑ-11	Δήλωση απώλειας και μετάβαση του έργου τέχνης στην κατάσταση Lost	Γκαλερί
Κλασματοποίηση και αγορά		
ΛΑ-12	Κλασματοποίηση του έργου τέχνης, με το έργο τέχνης να παραμένει δημόσια προσπελάσιμο	Καλλιτέχνης/Επενδυτής
ΛΑ-13	Αγορά και πώληση κλασμάτων με αυτόματο δικαίωμα παρακολούθησης	Καλλιτέχνης/Επενδυτής
ΛΑ-14	Ενοποίηση όλων των κλασμάτων	Κάτοχος όλων των κλασμάτων
Oracle (Σύνδεση με δεδομένα εξωτερικών πηγών)		
ΛΑ-15	Αποστολή περιοδικών αναγνώσεων από εγγεγραμμένη εξωτερική πηγή προς το oracle, που τις συναθροίζει σε μία υπογεγραμμένη αναφορά ανά παράθυρο	Εξωτερική πηγή
ΛΑ-16	Δήλωση των μετρήσεων του oracle από την αρχή, εγγραφή του με επαλήθευση της βεβαίωσης στο blockchain δίκτυο και αποδοχή του από την αρχή	Ρυθμιστική αρχή, oracle
Διακυβέρνηση		
ΛΑ-17	Εγγραφή και ανάκληση οντοτήτων στα δύο μητρώα, με άμεση ισχύ και δημόσιο ίχνος	Ρυθμιστική αρχή
ΛΑ-18	Ανάθεση νέας γκαλερί φύλαξης για έργο τέχνης ανακληθείσας γκαλερί	Ρυθμιστική αρχή
ΛΑ-19	Ορισμός των εποπτικών παραμέτρων	Ρυθμιστική αρχή
Φυσική εξαργύρωση		
ΛΑ-20	Αίτημα και ανάκληση αιτήματος εξαργύρωσης από τον ιδιοκτήτη μη κλασματοποιημένου έργου τέχνης, με επιβεβαίωση παράδοσης από τη γκαλερί φύλαξης	Ιδιοκτήτης, γκαλερί

4.3 Δικαιώματα ανά ρόλο

Κάθε απαίτηση στον πίνακα **4.2** έχει έναν δρώντα. Στον πίνακα **4.3** ξεχωρίζουμε τα δικαιώματα ενός ρόλου από τις πράξεις που απαιτούν σχέση με συγκεκριμένο έργο τέχνης. Οι έξι πρώτες γραμμές είναι δημόσιες αναγνώσεις και δεν απαιτούν wallet. Η στήλη της εξωτερικής πηγής αφορά το πρόγραμμα που στέλνει υπογεγραμμένες αναγνώσεις στο oracle, χωρίς να υποβάλλει συναλλαγές. Δεν περιορίζει την πρόσβαση του χειριστή του στις δημόσιες σελίδες.

Πίνακας 4.3. Δικαιώματα ανά ρόλο στους μηχανισμούς του συστήματος.

Μηχανισμός	Εξωτερική πηγή	Επισκέπτης	Καλλιτέχνης / Επενδυτής	Γκαλερί	Ρυθμιστική αρχή
Αρχική σελίδα με τα μεγέθη της αγοράς	×	✓	✓	✓	✓
Πλοήγηση και αναζήτηση στον κατάλογο	×	✓	✓	✓	✓
Σελίδα έργου τέχνης	×	✓	✓	✓	✓
Δημόσια επαλήθευση με αναγνωριστικό	×	✓	✓	✓	✓
Δημόσια διαφάνεια	×	✓	✓	✓	✓
Ενημέρωση για γκαλερί	×	✓	✓	✓	✓
Σύνδεση wallet και ρόλος	×	×	✓	✓	✓
Ειδοποιήσεις ανά ρόλο	×	×	✓	✓	✓
Χαρτοφυλάκιο	×	×	✓	✓	✓
Έκδοση έργου τέχνης	×	×	✓	✓	✓
Ακύρωση έκδοσης	×	×	✓	✓	✓
Πρόσβαση στην εικόνα υψηλής ανάλυσης	×	×	✓	✓	✓
Κλασματοποίηση	×	×	✓	✓	✓
Δημιουργία και ακύρωση αγγελίας	×	×	✓	✓	✓
Αγορά κλασμάτων	×	×	✓	✓	✓
Ενοποίηση κλασμάτων	×	×	✓	✓	✓
Αίτημα και ανάκληση αιτήματος εξαργύρωσης	×	×	✓	✓	✓
Επιβεβαίωση φυσικής εξαργύρωσης	×	×	×	✓	×
Καταχώριση παραλαβής σε φύλαξη	×	×	×	✓	×
Έκδοση πιστοποιητικού γνησιότητας	×	×	×	✓	×
Δήλωση απώλειας	×	×	×	✓	×
Αποστολή ανάγνωσης στο oracle	✓	×	×	×	×
Εγγραφή και ανάκληση στα μητρώα	×	×	×	×	✓
Ανάθεση νέας γκαλερί φύλαξης	×	×	×	×	✓
Ορισμός εποπτικών παραμέτρων	×	×	×	×	✓
Δήλωση μετρήσεων και αποδοχή oracle	×	×	×	×	✓

✓ επιτρέπεται × δεν επιτρέπεται ✓ υπό προϋπόθεση

Οι ενδείξεις υπό προϋπόθεση παραπέμπουν στους ελέγχους της αντίστοιχης πράξης. Κατά την έκδοση ορίζεται για την παραλαβή μια γκαλερί με ενεργή εγγραφή στο μητρώο. Η ακύρωση

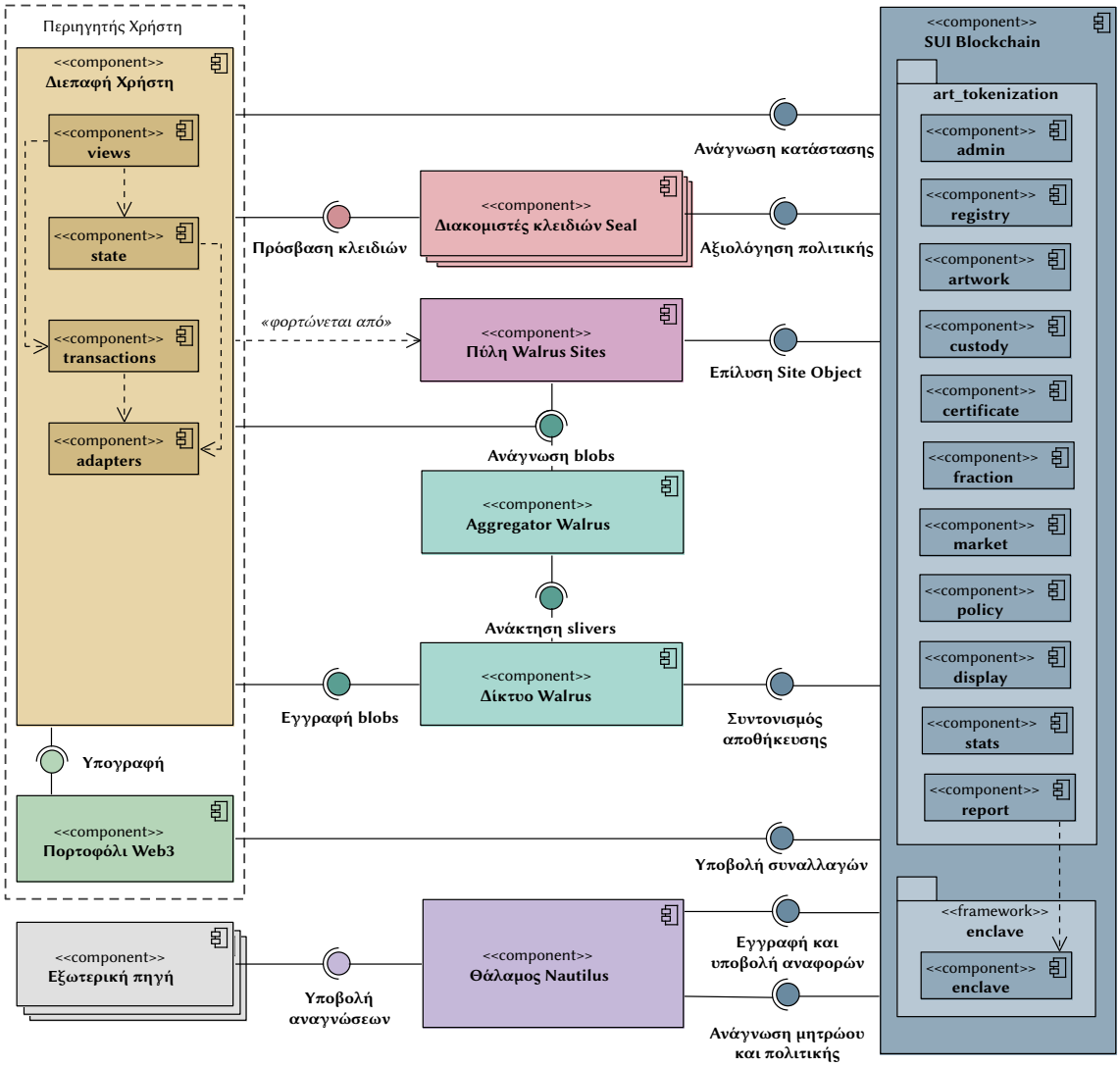
επιτρέπεται στον εκδότη πριν από τη φύλαξη. Η υψηλή ανάλυση είναι διαθέσιμη στον ιδιοκτήτη ή σε κάτοχο κλάσματος του συγκεκριμένου έργου τέχνης. Η κλασματοποίηση και το αίτημα εξαργύρωσης απαιτούν ιδιοκτησία, μαζί με τους ελέγχους κατάστασης που παρουσιάζουμε στις αντίστοιχες ενότητες. Η ενοποίηση απαιτεί όλα τα κλάσματα, ενώ η αγορά και η διαχείριση αγγελιών ελέγχουν την ποσότητα, την πληρωμή και τον πωλητή.

Αντίστοιχα, η γκαλερί πρέπει να έχει ενεργή εγγραφή στο μητρώο. Για την παραλαβή πρέπει να είναι η γκαλερί που έχει οριστεί για το συγκεκριμένο έργο τέχνης. Για την πιστοποίηση, τη δήλωση απώλειας και την επιβεβαίωση εξαργύρωσης πρέπει να το φυλάσσει ήδη. Το `AdminCap` δεν παρακάμπτει αυτούς τους ελέγχους.

Η εξωτερική πηγή χρειάζεται ενεργή εγγραφή και έγκυρη υπογραφή. Αν η ίδια διεύθυνση έχει περισσότερους ρόλους, εφαρμόζονται οι προϋποθέσεις του ρόλου με τον οποίο εκτελεί κάθε πράξη. Η ανάκληση αφαιρεί τα δικαιώματα γκαλερί ή πηγής, όχι τα δικαιώματα που προκύπτουν από ιδιοκτησία ή τις δημόσιες αναγνώσεις.

4.4 Γενική επισκόπηση αρχιτεκτονικής συστήματος

Τις απαιτήσεις και τον κύκλο ζωής τα εξυπηρετεί ένα σύστημα από επτά συστατικά και ένα πακέτο Move. Το **Σχήμα 4.1** τα δείχνει ως διάγραμμα συστατικών.



Σχήμα 4.1. Το διάγραμμα συστατικών της εφαρμογής.

Ο χρήστης φτάνει στο σύστημα από τον browser του, όπου τρέχουν η διεπαφή χρήστη και το πορτοφόλι Web3. Η διεπαφή χρήστη είναι εφαρμογή React με TypeScript, που διαβάζει την κατάσταση από έναν full node του SUI και συνθέτει τις συναλλαγές. Δεν τις υπογράφει όμως η ίδια, αλλά τις δίνει στο wallet, γιατί μόνο το wallet κρατά κλειδιά και υποβάλλει συναλλαγές στο SUI.

Οι συναλλαγές αυτές καταλήγουν στο πακέτο Move `art_tokenization`, που είναι τα έξυπνα συμβόλαια της εφαρμογής. Εκεί ζει όλη η λογική, δηλαδή η έκδοση και ο κύκλος ζωής του έργου τέχνης, τα μητρώα και τα δικαιώματα κάθε ρόλου, η κλασματοποίηση και η αγορά με το δικαίωμα παρακολούθησης, οι πολιτικές πρόσβασης του Seal και η καταγραφή των αναφορών

του oracle. Το πακέτο έχει έντεκα modules, ένα ανά ευθύνη. Η μόνη εξωτερική του εξάρτηση είναι το πακέτο `enclave` του Nautilus, δηλαδή το πακέτο Move με το οποίο το Nautilus διαχειρίζεται το oracle στο blockchain δίκτυο. Το module `report` το χρησιμοποιεί. Το πακέτο αυτό ενσωματώνεται όπως έχει δημοσιευτεί από τη Mysten Labs. Με αυτόν τον τρόπο, το Nautilus αναλαμβάνει την υλοποίηση των βασικών μηχανισμών για το oracle μας.

Εκτός από το πακέτο `enclave`, που είναι εξάρτηση κώδικα, το `art_tokenization` και η διεπαφή χρήστη στηρίζονται και σε πέντε συστατικά υποδομής, δηλαδή σε υπηρεσίες του οικοσυστήματος του SUI. Συγκεκριμένα:

- **SUI Blockchain**. Εκτελεί τις συναλλαγές του πακέτου μας και κρατά την κατάσταση όλων των αντικειμένων. Η διεπαφή χρήστη διαβάζει την κατάσταση από αυτό, και το wallet υποβάλλει σε αυτό τις συναλλαγές.

- **Δίκτυο Walrus**. Αποθηκεύει τα αρχεία των έργων τέχνης, δηλαδή τις εικόνες και την τεκμηρίωση των πιστοποιητικών, και τα αρχεία της ίδιας της διεπαφής χρήστη. Ο browser ανεβάζει τα αρχεία σε αυτό απευθείας, στέλνοντας τα slivers στους nodes του.

- **Aggregator Walrus**. Είναι ένας δημόσιος server του οικοσυστήματος, που τρέχει τον client του Walrus. Δέχεται αιτήματα ανάγνωσης, ζητά τα slivers από τους nodes και ανασυνθέτει το blob. Από αυτόν διαβάζουν τα αρχεία η διεπαφή χρήστη και η πύλη.

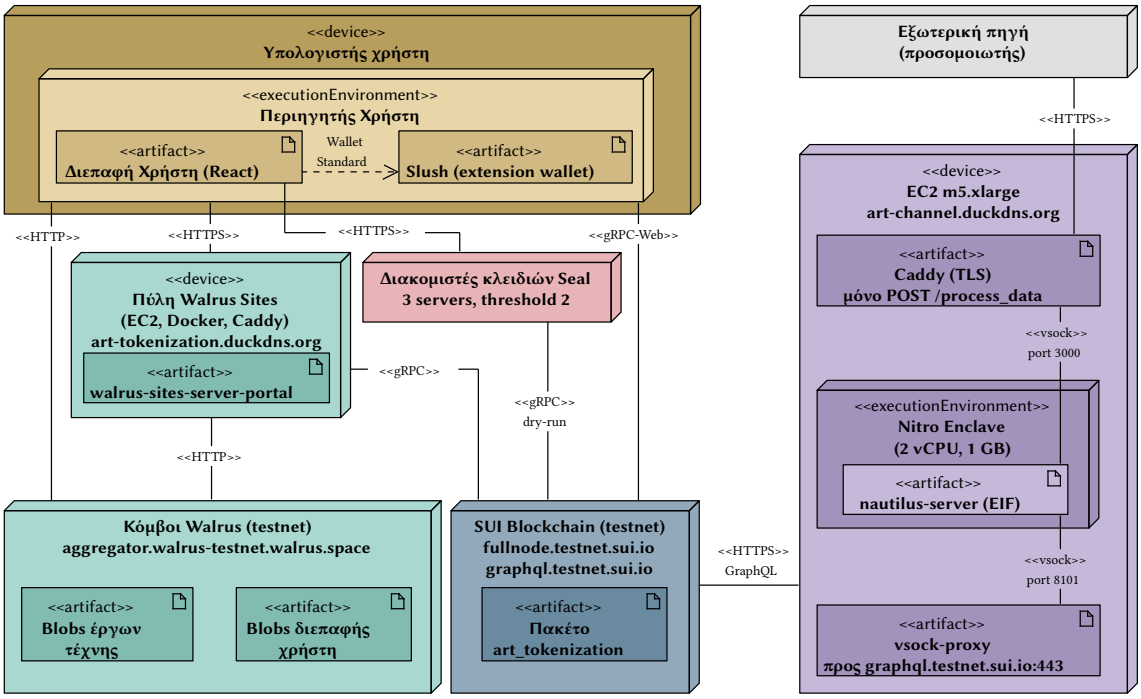
- **Διακομιστές κλειδιών Seal**. Στη δοκιμαστική διάταξη (testnet) χρησιμοποιούνται τρεις, με threshold δύο, κατά την έννοια της ενότητας 3.4.1. Γενικά, το σχήμα υποστηρίζει οποιοδήποτε κατώφλι t από n ανεξάρτητους διακομιστές. Όταν ένας χρήστης ζητά να διαβάσει το κρυπτογραφημένο αρχείο ενός έργου τέχνης, κάθε server τρέχει την πολιτική πρόσβασης του πακέτου πάνω στο blockchain δίκτυο και δίνει το μερίδιό του μόνο αν η πολιτική το επιτρέπει.

- **Πύλη Walrus Sites**. Μεταφράζει το URL της εφαρμογής σε ένα αντικείμενο ιστότοπου στο SUI και στα blobs του Walrus που περιέχουν τα αρχεία της διεπαφής χρήστη, και τα παραδίδει στον browser. Η μόνη σχέση της με τη διεπαφή χρήστη είναι ότι φορτώνει τη διεπαφή χρήστη.

Ο φυσικός κόσμος, από την άλλη, μπαίνει στο σύστημα από το oracle του Nautilus, έναν server Rust που εκτελείται με την τεχνολογία AWS Nitro Enclaves. Το oracle δέχεται τις υπογεγραμμένες αναγνώσεις της εξωτερικής πηγής και ελέγχει στο μητρώο ότι η πηγή είναι ενεργή. Δεν στέλνει όμως κάθε ανάγνωση στο blockchain δίκτυο. Τις συγκεντρώνει σε παράθυρα συνάθροισης, με διάρκεια που ορίζει η ρυθμιστική αρχή. Όταν κλείσει ένα παράθυρο, υποβάλλει μόνο του μία υπογεγραμμένη αναφορά με την επικρατέστερη κατάσταση των αναγνώσεων που συγκέντρωσε.

4.5 Πειραματική διάταξη

Τα συστατικά της προηγούμενης ενότητας τρέχουν στο δοκιμαστικό δίκτυο (testnet) του SUI [133]. Το **Σχήμα 4.2** δείχνει τη διάταξη ως διάγραμμα ανάπτυξης. Οι υπηρεσίες του οικοσυστήματος είναι οι δημόσιες του testnet, με δύο εξαιρέσεις που στήνουμε εμείς, τη γονική μηχανή που φιλοξενεί το oracle και την πύλη Walrus Sites.



Σχήμα 4.2. Διάγραμμα ανάπτυξης της εγκατάστασης στο SUI testnet.

- **SUI**. Ο δημόσιος full node `fullnode.testnet.sui.io`, μέσω gRPC για τον browser και μέσω της υπηρεσίας GraphQL για το oracle.
- **Walrus**. Ο δημόσιος aggregator `aggregator.walrus-testnet.walrus.space`.
- **Seal**. Ο διακομιστής κλειδιών της Mysten Labs και δύο ανεξάρτητοι ανοιχτοί διακομιστές του testnet, με threshold δύο.
- **Oracle**. Μία instance Elastic Compute Cloud, EC2 τύπου `m5.xlarge` στη Φρανκφούρτη, με ενεργή την τεχνολογία Nitro Enclaves. Το oracle παίρνει 2 vCPU και 1 GB. Στη γονική μηχανή τρέχουν ο reverse proxy Caddy, που τερματίζει το TLS στο `art-channel.duckdns.org` και προωθεί στο oracle μόνο την υποβολή ανάγνωσης, και ο vsock-proxy, που είναι η μοναδική έξοδος του oracle προς την υπηρεσία GraphQL.
- **Πύλη Walrus Sites**. Το testnet δεν έχει δημόσια πύλη, οπότε τρέχουμε δική μας με το Docker image `mysten/walrus-sites-server-portal`, τοπικά για την ανάπτυξη και σε EC2 πίσω από Caddy στο `art-tokenization.duckdns.org`.

4.6 Τεχνολογίες

Ο **Πίνακας 4.4** συγκεντρώνει τις τεχνολογίες της εγκατάστασης ανά μέρος του συστήματος.

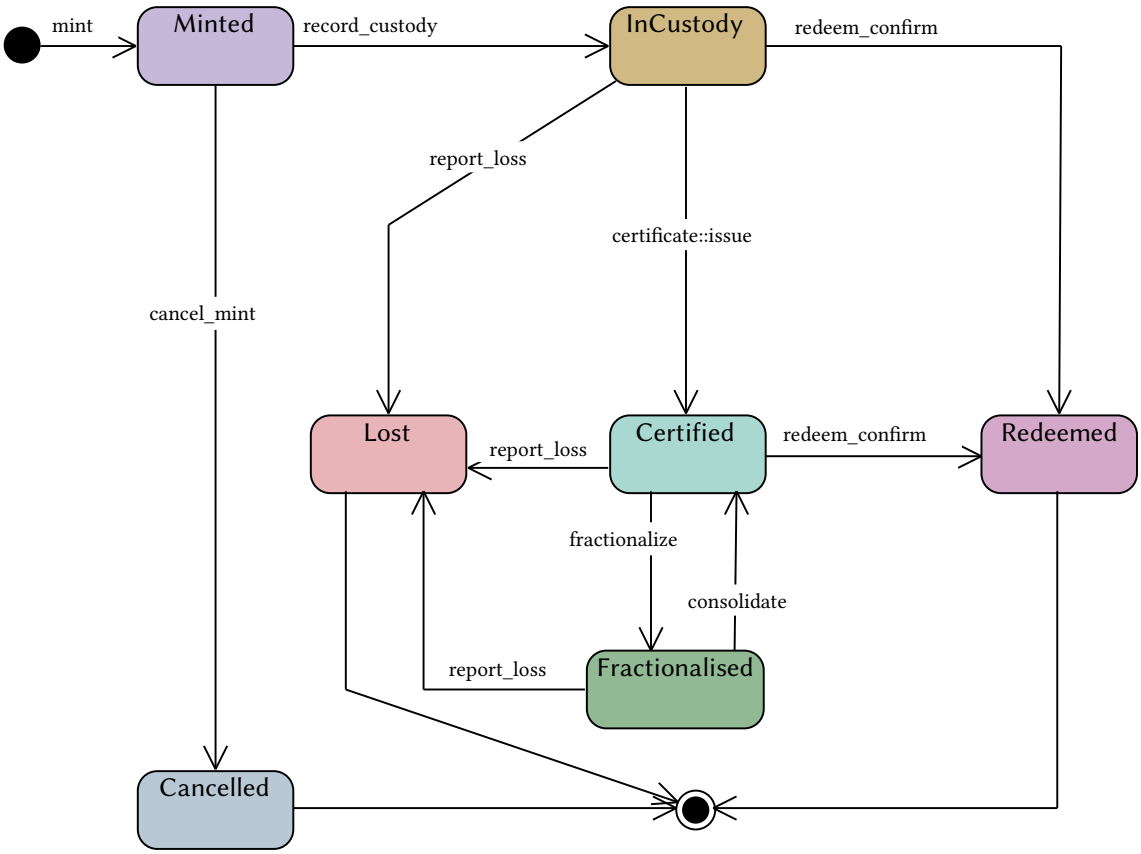
Πίνακας 4.4. Οι τεχνολογίες της εγκατάστασης, ανά μέρος του συστήματος.

Μέρος	Τεχνολογία	Ρόλος
Συμβόλαια	Move, έκδοση 2024, πακέτο `enclave` του Nautilus	Το πακέτο `art_tokenization` με τα 11 modules και τις δοκιμές του
Blockchain δίκτυο	SUI testnet, full node gRPC, υπηρεσία GraphQL	Ο browser μιλά με τον full node, το oracle με το GraphQL
Διεπαφή χρήστη	React 19, TypeScript 6, Vite 8, Tailwind 4, react-router 8	Views, κατασκευή σε στατικά αρχεία, μορφοποίηση, έντεκα routes
Σύνδεση με το δίκτυο	`@mysten/sui` 2.26, gRPC-Web	Ανάγνωση κατάστασης, σύνθεση PTB, προσομοίωση πριν από την υπογραφή
Wallet	`@mysten/dapp-kit-react` 2.1, `dapp-kit-core` 1.6, Slush	Εύρεση wallets μέσω Wallet Standard, σύνδεση, υπογραφή, το Slush είναι το wallet της επίδειξης
Αποθήκευση	Walrus testnet, `@mysten/walrus` 1.2, `walrus-wasm`	Κωδικοποίηση Red Stuff στον browser, upload και λήψη blobs
Έλεγχος πρόσβασης	Seal, `@mysten/seal` 1.4, τρεις διακομιστές κλειδιών	Κρυπτογράφηση, κλειδί συνεδρίας, μερίδια κλειδιού, αποκρυπτογράφηση
Φιλοξενία	Walrus Sites, `site-builder` 2.12, πύλη σε Docker, SuiNS	Δημοσίευση της διεπαφής χρήστη ως quilt, όνομα `art-tokenization.sui`
Oracle	AWS Nitro Enclaves, Nautilus, Rust 1.96, axum 0.7, fastcrypto, `sui-sdk-types`	Το oracle, αναπαραγώγιμη εικόνα με StageX και `nitro-cli`
Γονική μηχανή	Amazon Linux 2023, Caddy, vsock-proxy, systemd	Τερματισμός TLS, ένα route προς το oracle, η μοναδική έξοδός του
Συσκευή-πηγή	Python 3, `cryptography`	Ζεύγη κλειδιών Ed25519, υπογραφή αναγνώσεων, προσομοιωτής εξωτερικών πηγών

Οι τεχνολογίες του πίνακα που ανήκουν στο οικοσύστημα SUI προκύπτουν από την επιλογή της στοίβας, που έγινε για τέσσερις λόγους. Το μοντέλο αντικειμένων αναπαριστά ένα μοναδικό έργο τέχνης ως αυτοτελές αντικείμενο με dynamic fields για ό,τι προσαρτάται αργότερα. Το σύστημα τύπων της Move εγγυάται ότι το έργο τέχνης δεν αντιγράφεται και δεν χάνεται, και το `key` χωρίς `store` κάνει δομική την επιβολή του δικαιώματος παρακολούθησης. Το Mysticeti δίνει finality κάτω του δευτερολέπτου με χαμηλό κόστος και για τα shared αντικείμενα, που είναι σχεδόν όλες οι συναλλαγές μας. Και το οικοσύστημα δίνει, με τα ίδια εργαλεία και τον ίδιο κώδικα Move, την αποθήκευση, τον έλεγχο πρόσβασης και τη σύνδεση με τον φυσικό κόσμο [134]. Οι υπόλοιπες είναι δικές μας επιλογές και ακολουθούν ό,τι ήδη προσφέρει το οικοσύστημα. React και TypeScript, επειδή έτσι είναι γραμμένο το DApp Kit. Vite, γιατί παράγει στατικά αρχεία, τη μόνη μορφή που δέχεται ένα Walrus Site. gRPC-Web στον browser, ως το πρωτόκολλο του νέου client, και GraphQL στο oracle, γιατί περνά από έναν μόνο host HTTPS, ό,τι ακριβώς επιτρέπει ο vsock-proxy. Rust με axum και fastcrypto, επειδή είναι το template του Nautilus και η κρυπτογραφική βιβλιοθήκη του ίδιου του SUI, και StageX, γιατί οι μετρήσεις PCR θέλουν αναπαραγώγιμη μεταγλώττιση. Python για την προσομοίωση της εξωτερικής συσκευής, για πιο γρήγορη και εύκολη υλοποίηση.

4.7 Ο κύκλος ζωής ενός tokenized έργου τέχνης

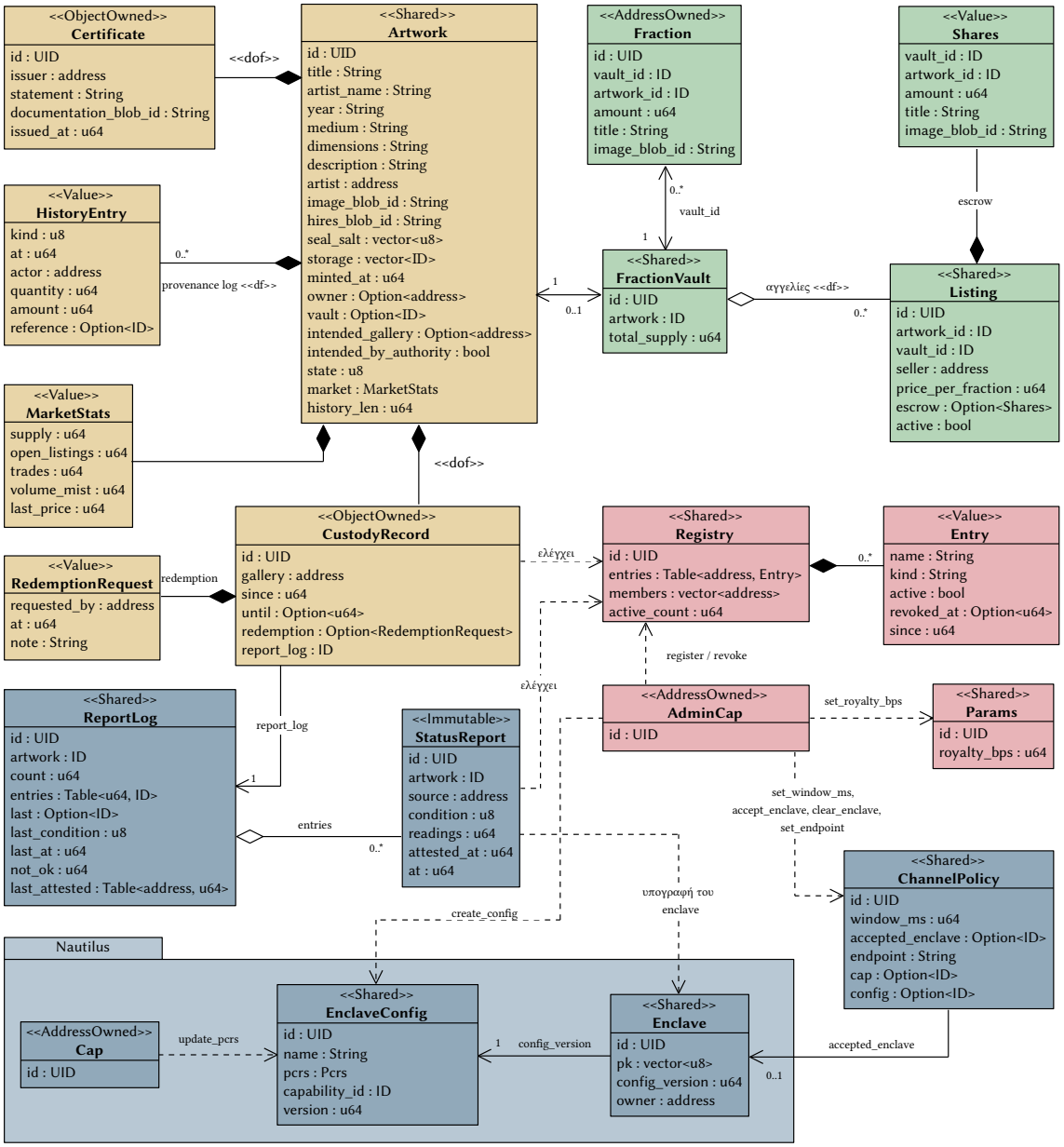
Το έργο τέχνης που διαχειρίζεται η εφαρμογή μας περνά από επτά καταστάσεις. Οι απαιτήσεις της έκδοσης, της φύλαξης και της κλασματοποίησης είναι στην ουσία οι μεταβάσεις ανάμεσά τους, γι' αυτό το **Σχήμα 4.3** τις δείχνει ως διάγραμμα καταστάσεων. Οι καταστάσεις είναι οι επτά τιμές του πεδίου `state`, και κάθε βέλος φέρει τη συνάρτηση που το εκτελεί. Η κύρια διαδρομή έχει τέσσερα βήματα. Το έργο τέχνης εκδίδεται, παραλαμβάνεται σε φύλαξη από τη γκαλερί που όρισε ο καλλιτέχνης, πιστοποιείται από την ίδια γκαλερί και κλασματοποιείται από τον ιδιοκτήτη του. Από εκεί και πέρα τρέχει η αγορά. Η μόνη επιστροφή από την κλασματοποιημένη κατάσταση είναι η ενοποίηση, που γίνεται όταν μία διεύθυνση συγκεντρώσει όλα τα κλάσματα. Πριν από την παραλαβή, ωστόσο, ο ιδιοκτήτης μπορεί ακόμη να ακυρώσει την έκδοση, γιατί τίποτα δεν έχει συμβεί στο φυσικό έργο τέχνης. Εκτός από την ακύρωση, τερματικές καταστάσεις είναι η απώλεια, που τη δηλώνει η γκαλερί, και η εξαργύρωση, δηλαδή η φυσική παράδοση που κλείνει τη φύλαξη.



Σχήμα 4.3. Ο κύκλος ζωής ενός tokenized έργου τέχνης.

4.8 Τα αντικείμενα στο SUI

Κρατάμε την ιδιοκτησία, τη φύλαξη και τα στοιχεία της αγοράς σε αντικείμενα στο SUI. Το enclave διατηρεί χωριστά τα προσωρινά παράθυρα αναγνώσεων στη μνήμη του. Το **Σχήμα 4.4** παρουσιάζει το διάγραμμα κλάσεων του συστήματος, αποτυπώνοντας τα αντικείμενα και τις μεταξύ τους σχέσεις με βάση τα είδη ιδιοκτησίας που αναλύθηκαν στην ενότητα 3.1.1.



Σχήμα 4.4. Το διάγραμμα κλάσεων του μοντέλου αντικειμένων. Ο γεμάτος ρόμβος δηλώνει σύνθεση (απευθείας ενσωμάτωση τιμής όπως το `MarketStats`, dynamic field `<<df>>` για σειριακές εγγραφές όπως το `HistoryEntry`, ή dynamic object field `<<dof>>` για αυτόνομα αντικείμενα όπως το `Certificate` και το `CustodyRecord`), ο άδειος ρόμβος συνάθροιση μέσω αναγνωριστικών (`ID`) προς ανεξάρτητα αντικείμενα, και οι διακεκομμένες γραμμές λειτουργικές εξαρτήσεις ελέγχου και διαχείρισης.

Ακολουθεί συνοπτική αντιστοίχιση των βασικών αντικειμένων του μοντέλου:

- **`Artwork`** (Shared Object). Το ψηφιακό δίδυμο του φυσικού έργου τέχνης, με τα μεταδεδομένα, τα αναγνωριστικά των blobs στο Walrus και την τρέχουσα κατάσταση (4.12).
- **`MarketStats`** (Value). Συγκεντρωτικά στατιστικά της αγοράς και της κλασματοποίησης (προσφορά, ενεργές αγγελίες, συναλλαγές, όγκος), ενσωματωμένα απευθείας κατά τιμή στο `Artwork`.
- **`HistoryEntry`** (Value). Εγγραφή ιστορικού προέλευσης (provenance log), προσαρτημένη στο έργο τέχνης ως δυναμικό πεδίο (`df`) με αύξοντα δείκτη.
- **`Certificate`** (Object-Owned). Το πιστοποιητικό γνησιότητας, προσαρτημένο μόνιμα στο `Artwork` ως dynamic object field (`dof`) από τη θεματοφύλακα γκαλερί (4.13).
- **`CustodyRecord`** (Object-Owned). Η καταχώριση φυσικής φύλαξης, προσαρτημένη ως dynamic object field στο `Artwork` (4.13).
- **`FractionVault`** (Shared Object). Το θησαυροφυλάκιο της κλασματοποίησης, που τηρεί τη συνολική προσφορά και καταχωρίζει τις ανοιχτές αγγελίες στο index (4.14).
- **`Fraction`** (Address-Owned). Το κλάσμα συνιδιοκτησίας, με `key` χωρίς `store`, ώστε κάθε μεταβίβαση να εκτελείται αποκλειστικά μέσω της αγοράς (4.14).
- **`Listing`** (Shared Object). Αγγελία πώλησης μεριδίων, με escrow (`Shares`) μέχρι την εκτέλεση ή την ακύρωση (4.15).
- **`ReportLog`** (Shared Object). Το αρχείο περιβαλλοντικών αναφορών ενός έργου τέχνης υπό φύλαξη (4.13, 4.17).
- **`StatusReport`** (Immutable). Αμετάβλητη αναφορά κατάστασης που εκδίδει το oracle μετά από συνάθροιση μετρήσεων (4.17).
- **`ChannelPolicy`** (Shared Object). Η πολιτική του καναλιού αναφορών (παράθυρο συνάθροισης, αποδεκτό oracle) (4.17).
- **`EnclaveConfig`, `Enclave`, `Cap`** (Nautilus). Διαχείριση μετρήσεων εικόνας, δημόσιου κλειδιού και capabilities του oracle (4.17).
- **`Registry<T>`** (Shared Object). Τα μητρώα ρόλων για γκαλερί και εξωτερικές πηγές (4.9).
- **`AdminCap`** (Address-Owned). Το capability διοικητικής εποπτείας της ρυθμιστικής αρχής (4.9).

4.9 Διοικητική εποπτεία και μητρώα ρόλων

Το μοντέλο αντικειμένων της προηγούμενης ενότητας καθορίζει τη δομή των δεδομένων, αλλά η λειτουργία της αγοράς προϋποθέτει έγκυρους ρόλους συμμετεχόντων. Η ρυθμιστική αρχή ασκεί την εποπτεία αυτή και επιβάλλει τους ελέγχους ρόλων της ενότητας 4.3 μέσω του `AdminCap` και δύο κοινόχρηστων μητρώων `Registry<T>`. Τα μητρώα αυτά είναι το `Registry<Gallery>` για τις εγκεκριμένες γκαλερί και το `Registry<Source>` για τις εξωτερικές πηγές του oracle. Η κατοχή του `AdminCap` αποτελεί τον μοναδικό έλεγχο εξουσιοδότησης, χωρίς σύνδεση με κωδικό και χωρίς συνεδρία.

Το capability αυτό συμπυκνώνει την εξουσία της αρχής, καθώς αυτή δεν αποτελεί ειδικό λογαριασμό αλλά όποιον κατέχει το `AdminCap`. Το αντικείμενο παράγεται στη δημοσίευση από την `init` του module `admin` και μεταβιβάζεται στον εκδότη. Στο module `registry` (**Κώδικας 4.1**), η παράμετρος `T` του `Registry<phantom T>` αξιοποιεί phantom types μέσω των κενών δομών `Gallery` και `Source` (`has drop`). Το `phantom` δηλώνει ότι το `T` δεν καταλαμβάνει χώρο αποθήκευσης, επιτυγχάνοντας ασφάλεια τύπων στη Move με μηδενικό κόστος. Έτσι, το δίκτυο απορρίπτει στη μεταγλώττιση τυχόν χρήση μητρώου πηγών σε συναρτήσεις γκαλερί. Για την εγγραφή μιας οντότητας, η αρχή καλεί τη `register<T>` παρέχοντας το `AdminCap` στην παράμετρο `_`. Για το address-owned `AdminCap`, το SUI ελέγχει ότι ανήκει στον αποστολέα της συναλλαγής. Ο περιορισμός αυτός δεν αφορά τα shared ή immutable αντικείμενα που χρησιμοποιούνται ως άλλα ορίσματα.

```move
// admin.move
public struct AdminCap has key, store { id: UID }
public struct Params has key { id: UID, royalty_bps: u64 }

// registry.move
public struct Gallery has drop {}
public struct Source has drop {}
public struct Registry<phantom T> has key {
    id: UID,
    entries: Table<address, Entry>,
    members: vector<address>,
    active_count: u64,
}

public fun register<T>(
    reg: &mut Registry<T>, _: &AdminCap, who: address, name: String, kind: String, clock: &Clock, ctx: &TxContext,
) {
    assert!(!name.is_empty(), EEmptyName);
    let at = clock.timestamp_ms();
    reg.entries.add(who, Entry { name, kind, since: at, active: true, revoked_at: option::none() });
    reg.members.push_back(who);
    reg.active_count = reg.active_count + 1;
    event::emit(Registered { registry: reg.id.to_inner(), who, name, kind, at, by: ctx.sender() });
}

public fun is_active<T>(reg: &Registry<T>, who: address): bool {
    reg.entries.contains(who) && reg.entries[who].active
}
```

Κώδικας 4.1. Αποσπάσματα των modules `admin` και `registry`, με απλοποιημένη τη διαδρομή πρώτης εγγραφής. Δεν παρουσιάζεται η επανενεργοποίηση ανακληθείσας εγγραφής. Το `AdminCap` παρέχει διοικητική εξουσία και τα `Registry<phantom T>` ξεχωρίζουν μέσω των τύπων `Gallery` και `Source`.

Χάρη στην ικανότητα `store`, το `AdminCap` μπορεί να μεταβιβαστεί σε λογαριασμό πολλαπλών υπογραφών ή σε ένα DAO. Αντίθετα, οι παράμετροι `Params` αποτελούν shared αντικείμενο, ώστε να τις αναγιγνώσκει η αγορά σε κάθε πώληση.

Οι πράξεις γκαλερί και η on-chain καταγραφή αναφοράς ελέγχουν την ενεργή εγγραφή με την `is_active<T>`. Μετά την ανάκληση, οι επόμενοι έλεγχοι στο συμβόλαιο απορρίπτουν τις πράξεις του αντίστοιχου ρόλου. Το enclave κρατά προσωρινά το αποτέλεσμα του ελέγχου πηγής, αλλά το συμβόλαιο το επανελέγχει πριν καταγράψει την τελική αναφορά.

4.10 Η διεπαφή χρήστη

Η διεπαφή χρήστη έχει τέσσερα μέρη, που φαίνονται στο **Σχήμα 4.1**. Τα views είναι οι σελίδες του **Πίνακας 4.5**. Το state μεταφράζει τα αντικείμενα και τα events του blockchain δικτύου σε μορφή που ταιριάζει στα views, όπως θα έκανε το API ενός backend. Τα transactions χτίζουν τις συναλλαγές, μία ανά πράξη των έξυπνων συμβολαίων. Οι adapters συνδέουν τη διεπαφή χρήστη με το SUI, το wallet, το Walrus και το Seal. Η εφαρμογή φορτώνεται από την πύλη Walrus Sites, με τη ροή της ενότητας 3.3, και από εκεί και πέρα ο browser διαβάζει και υποβάλλει απευθείας στο SUI.

Πίνακας 4.5. Τα routes της διεπαφής χρήστη και το view που εξυπηρετεί την καθεμία. Ποιες βλέπει κάθε ρόλος το δίνει ο **Πίνακας 4.3**.

Route	View	Περιεχόμενο
`/`	`LandingView`	Αρχική σελίδα, μετρητές της πλατφόρμας, τα νεότερα έργα τέχνης
`/catalogue`	`CatalogueView`	Κατάλογος με φίλτρα, ταξινόμηση και αναζήτηση
`/artwork/:id`	`ArtworkView`	Η σελίδα του έργου τέχνης, με εικόνα, πιστοποιητικό, φύλαξη, αναφορές, ιστορικό και αγορά
`/verify`, `/verify/:id`	`VerifyView`	Επαλήθευση έργου τέχνης με μόνο το αναγνωριστικό του
`/transparency`	`TransparencyView`	Μητρώα, ίχνος των πράξεων της ρυθμιστικής αρχής, το oracle και οι αναφορές κατάστασης
`/galleries`	`GalleryInfoView`	Ενημέρωση για γκαλερί που θέλουν να ενταχθούν
`/portfolio`	`PortfolioView`	Τα έργα τέχνης και τα κλάσματα της συνδεδεμένης διεύθυνσης, αγγελίες, κλασματοποίηση
`/mint`	`MintView`	Η φόρμα έκδοσης έργου τέχνης
`/gallery`	`GalleryDashboardView`	Ο πίνακας της γκαλερί, με παραλαβές, πιστοποιητικά, αναφορές και απώλεια
`/admin`	`AdminView`	Η σελίδα διοίκησης της ρυθμιστικής αρχής, με μητρώα, παραμέτρους και oracle

Η δρομολόγηση. Οι διαδρομές του **Πίνακας 4.5** είναι διευθύνσεις του browser, όπως `/artwork/0x...`, και τις αναλύει το react-router μέσα στον browser. Επειδή η πύλη παρέχει στατικά αρχεία, το `ws-resources.json` του Walrus Site (**Κώδικας 4.2**) δηλώνει τον κανόνα `"/*": "/index.html"`, ώστε κάθε διαδρομή που δεν είναι αρχείο να παραδίδεται ως η αρχική σελίδα και να αναλαμβάνει το react-router. Έτσι ένας σύνδεσμος ή ο κωδικός QR του πιστοποιητικού ανοίγει απευθείας τη σελίδα του έργου τέχνης.

```json
{
  "routes": {
    "/*": "/index.html"
  },
  "site_name": "art-tokenization",
  "object_id": "0xfa64...cc89b"
}
```

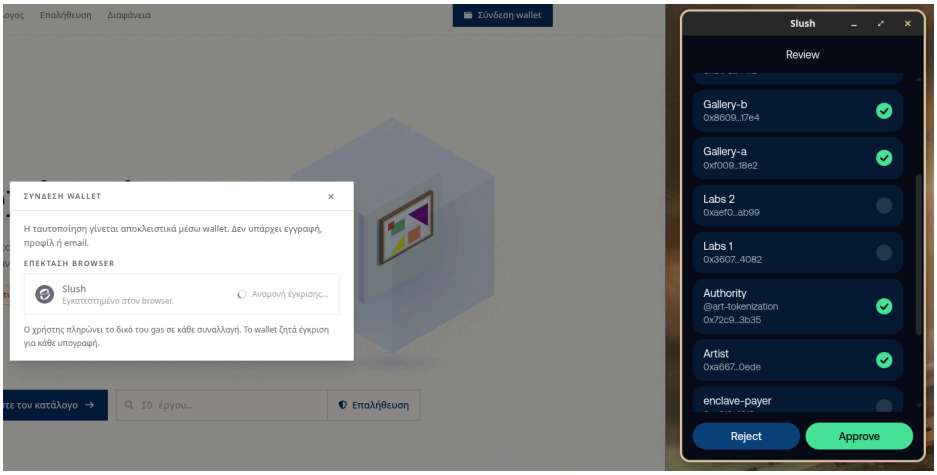
Κώδικας 4.2. Το `ws-resources.json` του Walrus Site.

Η σύνδεση. Ο μόνος τρόπος ταυτοποίησης είναι το wallet, γιατί η διεύθυνσή του είναι η ταυτότητα με την οποία βλέπουν τον χρήστη τα συμβόλαια. Δεν υπάρχει λογαριασμός, κωδικός ή συνεδρία. Ο **Κώδικας 4.3** δείχνει τα δύο αντικείμενα από τα οποία περνά κάθε επικοινωνία. Το πρώτο είναι ο client gRPC, που συνδέεται με τον full node. Πάνω του φορτώνεται το πρόσθετο του Walrus, που δίνει στον browser τον κώδικα WebAssembly για την κωδικοποίηση των blobs. Το δεύτερο είναι το DApp Kit, που χρησιμοποιεί τον ίδιο client. Αυτό βρίσκει τα wallets του browser μέσω του Wallet Standard και αναλαμβάνει τη σύνδεση και την υπογραφή. Η σύνδεση ξεκινά από το παράθυρο «Σύνδεση wallet» της εφαρμογής και ολοκληρώνεται στην οθόνη έγκρισης του wallet (**Σχήμα 4.5**). Από εκείνη τη στιγμή η διεύθυνση είναι διαθέσιμη σε κάθε view (**Σχήμα 4.6**).

```ts
export const suiClient = new SuiGrpcClient({
  network: 'testnet', baseUrl: config.grpcUrl,
}).$extend(walrus({ wasmUrl: walrusWasmUrl }));

export const dAppKit = createDAppKit({
  networks: ['testnet'],
  defaultNetwork: 'testnet',
  createClient: () => suiClient,
});
```

Κώδικας 4.3. Ο client του SUI και το DApp Kit της διεπαφής χρήστη. Ένας client gRPC με το πρόσθετο του Walrus εξυπηρετεί και τις αναγνώσεις και το wallet.



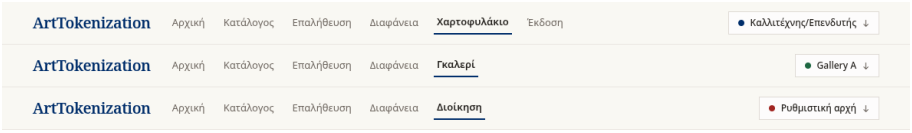
Σχήμα 4.5. Η σύνδεση του wallet. Αριστερά το παράθυρο «Σύνδεση wallet» της εφαρμογής, που περιμένει την έγκριση. Δεξιά η οθόνη του Slush, όπου ο χρήστης επιλέγει ποιους λογαριασμούς εκθέτει στην εφαρμογή. Η δεύτερη οθόνη ανήκει στο extension και όχι στη σελίδα.

Η υπογραφή. Κάθε συναλλαγή υποβάλλεται από ένα μόνο σημείο, τη συνάρτηση του **Κώδικας 4.4**. Τα transactions την καλούν με ένα έτοιμο PTB, και αυτή ορίζει τον αποστολέα, προσομοιώνει το PTB στον full node, το δίνει στο wallet για υπογραφή και εκτέλεση, και περιμένει το αποτέλεσμα. Η προσομοίωση προηγείται, ώστε ένα σφάλμα Move να εμφανιστεί με το μήνυμά του προτού το wallet ζητήσει έγκριση που θα απαιτούσε χρέωση gas.

```typescript
setSigner(async (tx) => {
  const sender = dAppKit.stores.$connection.get().account?.address;
  if (!sender) throw new Error('Δεν υπάρχει συνδεδεμένο wallet.');
  tx.setSenderIfNotSet(sender);
  const sim = await suiClient.simulateTransaction({ transaction: tx });
  if (sim.$kind === 'FailedTransaction')
    throw new Error(explain(sim.FailedTransaction.status));
  const res = await dAppKit.signAndExecuteTransaction({ transaction: tx });
  const executed =
    res.$kind === 'Transaction' ? res.Transaction : res.FailedTransaction;
  if (!executed.status.success) throw new Error(explain(executed.status));
  const final = await suiClient.waitForTransaction({
    digest: executed.digest, include: { effects: true, objectTypes: true },
  });
  notifyChainChanged();
  return { digest: executed.digest, created: createdObjects(final) };
});
```

Κώδικας 4.4. Η υπογραφή μιας συναλλαγής.

Ο ρόλος. Σε κάθε ανανέωση το hook του **Κώδικας 4.5** αποφασίζει τον ρόλο της συνδεδεμένης διεύθυνσης. Γκαλερί είναι όποιος έχει ενεργή εγγραφή στο μητρώο γκαλερί, αρχή ο κάτοχος του `AdminCap`, και καλλιτέχνης ή επενδυτής κάθε άλλη διεύθυνση. Τα views κρύβουν ή δείχνουν σελίδες με βάση αυτό, ενώ ο έλεγχος που μετρά γίνεται στο έξυπνο συμβόλαιο (**Σχήμα 4.6**).



Σχήμα 4.6. Η navbar για τρεις συνδεδεμένες διευθύνσεις.

```typescript
export function useRole() {
  const { address } = useWallet();
  const entry = useRead(address ? `gallery:${address}` : null, () =>
    chain.galleryOf(address!),
  );
  const params = useParams();
  const gallery = entry.data && entry.data.active ? entry.data : null;
  const authority = (params.data ?? NO_PARAMS).authority ?? config.authority;
  return {
    address, connected: !!address, isArtist: !!address,
    gallery, isGallery: !!gallery,
    isAuthority: !!address && address === authority,
  };
}
```

Κώδικας 4.5. Ο ρόλος της συνδεδεμένης διεύθυνσης.

Η παραγωγή ειδοποιήσεων. Η πλατφόρμα δεν βασίζεται σε κεντρικό εξυπηρετητή μηνυμάτων, υπηρεσία push ειδοποιήσεων ή εξωτερικό indexer. Αντίθετα, ικανοποιεί την απαίτηση των ειδοποιήσεων ανά ρόλο (ΛΑ-05) με αποκεντρωμένο τρόπο. Συγκεκριμένα, η διεπαφή χρήστη εξετάζει τα δημόσια αντικείμενα και τα γεγονότα που ανακτώνται απευθείας από τον κόμβο του SUI μέσω gRPC, συνθέτοντας τοπικά τις εκκρεμότητες και τις προειδοποιήσεις που αφορούν τη συνδεδεμένη διεύθυνση (**Πίνακας 4.6**).

Πίνακας 4.6. Οι κατηγορίες ειδοποιήσεων ανά ρόλο, οι συνθήκες παραγωγής τους από την κατάσταση της αλυσίδας και οι ενέργειες που προτείνονται στον χρήστη (ΛΑ-05).

Είδος ειδοποίησης	Συνθήκη παραγωγής στην αλυσίδα	Ενέργεια στη διεπαφή
Γκαλερί φύλαξης		
Εκκρεμής παραλαβή (`intake`)	Έργο σε κατάσταση `minted` με δηλωμένη τη διεύθυνση της γκαλερί	Καταχώριση παραλαβής στο `/gallery#attention`
Αίτημα εξαργύρωσης (`redemption`)	Ενεργό αίτημα παράδοσης φυσικού έργου τέχνης από τον ιδιοκτήτη	Επιβεβαίωση παράδοσης στο `/gallery#attention`
Αναφορά προβλήματος (`report`)	Καταγραφή φθοράς ή απώλειας (`condition` $\neq$ `OK`) στο log του oracle	Έλεγχος φυσικής κατάστασης ή δήλωση απώλειας
Ρυθμιστική αρχή		
Ορφανό έργο τέχνης υπό φύλαξη (`orphan`)	Έργο σε ενεργή φύλαξη με γκαλερί ανακληθείσα από το μητρώο	Ανάθεση νέας γκαλερί στο `/admin#custody`
Καλλιτέχνες και επενδυτές		
Αναφορά προβλήματος σε κλάσματα	Καταγραφή φθοράς ή απώλειας στο oracle για έργο τέχνης με ενεργά κλάσματα	Ενημέρωση στο `/portfolio#notifications`
Αποκλεισμένη έκδοση (`mint-blocked`)	Έκδοση όπου η δηλωθείσα γκαλερί ανακλήθηκε προ της παραλαβής	Ακύρωση έκδοσης στο `/portfolio#works`
Ανάκληση γκαλερί ιδιόκτητου έργου τέχνης	Έργο υπό φύλαξη του οποίου η γκαλερί ανακλήθηκε από το μητρώο	Ενημέρωση ιδιοκτήτη εν αναμονή νέας ανάθεσης

Όταν ο χρήστης συνδέει το πορτοφόλι του, η διεπαφή φιλτράρει τα αντικείμενα που σχετίζονται με τη διεύθυνσή του. Αν διαπιστωθεί ανάγκη ενέργειας, εμφανίζεται ένδειξη ειδοποίησης με αριθμητικό δείκτη εκκρεμοτήτων στη γραμμή πλοήγησης (**Σχήμα 4.7**). Πατώντας στο εικονίδιο, αναπτύσσεται αναδυόμενο μενού με τις άμεσες εκκρεμότητες, από το οποίο ο χρήστης μεταβαίνει απευθείας στην αντίστοιχη καρτέλα με προκατασκευασμένη τη συναλλαγή επίλυσης.



Σχήμα 4.7. Παράδειγμα αναδυόμενου μενού ειδοποιήσεων στη διεπαφή χρήστη με εκκρεμείς ενέργειες παραλαβής.

Η συνάρτηση `deriveNotifications` του επιπέδου κατάστασης (`state/derive.ts`) εκτελείται τοπικά σε κάθε ανανέωση δεδομένων της αλυσίδας (**Κώδικας 4.6**). Εξετάζει τα αντικείμενα που αντιστοιχούν στον ρόλο της συνδεδεμένης διεύθυνσης (`intended`, `custody`, `issued`, `owned`, `held`, `orphans`) και παράγει τις αντίστοιχες ειδοποιήσεις.

```typescript
export function deriveNotifications(input: DeriveInput): Notice[] {
  const { address, isGallery, isAuthority } = input;
  if (!address) return [];
  const out: Notice[] = [];
  const active = new Set(input.galleries.filter((g) => g.active).map((g) => g.address));

  if (isGallery) {
    for (const a of pendingIntakes(input.intended))
      out.push({ id: `intake-${a.id}`, kind: 'intake', title: a.title,
                 artworkId: a.id, to: '/gallery#attention', at: a.mintedAt });
    for (const a of pendingRedemptions(input.custody))
      out.push({ id: `redemption-${a.id}`, kind: 'redemption', title: a.title,
                 artworkId: a.id, to: '/gallery#attention', at: a.redemption!.at });
    for (const a of input.custody) {
      const r = needsAttention(a);
      if (r) out.push({ id: `report-${r.id}`, kind: 'report', title: a.title,
                        artworkId: a.id, to: '/gallery#attention', at: r.at });
    }
  } else if (isAuthority) {
    for (const a of orphanedCustody(input.orphans, input.galleries))
      out.push({ id: `orphan-${a.id}`, kind: 'orphan', title: a.title,
                 artworkId: a.id, to: '/admin#custody', at: a.mintedAt });
  } else {
    for (const a of input.held) {
      const r = needsAttention(a);
      if (r) out.push({ id: `report-${r.id}`, kind: 'report', title: a.title,
                        artworkId: a.id, to: '/portfolio#notifications', at: r.at });
    }
  }
  return out.sort((x, y) => y.at.localeCompare(x.at));
}
```

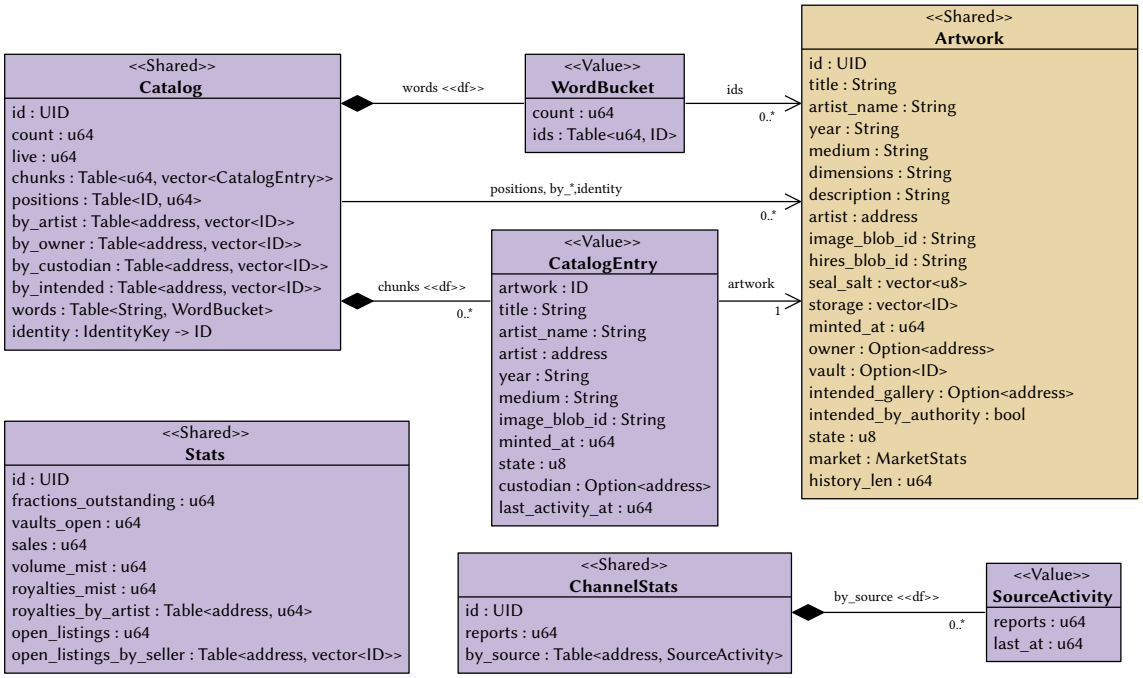
Κώδικας 4.6. Παραγωγή ειδοποιήσεων ανά ρόλο βάσει της κατάστασης της αλυσίδας (`state/derive.ts`).

Όπως αποτυπώνεται στον **Κώδικας 4.6**, η συνάρτηση διατρέχει σειριακά τις συλλογές αντικειμένων και συνθέτει αντικείμενα τύπου `Notice`. Για κάθε ειδοποίηση καταγράφεται μοναδικό αναγνωριστικό (`id`), το είδος συμβάντος (`kind`), ο τίτλος του έργου τέχνης, το επεξηγηματικό μήνυμα και ο σύνδεσμος άμεσης μετάβασης (`to`). Πριν επιστραφεί το αποτέλεσμα, οι διπλές ειδοποιήσεις αφαιρούνται μέσω ενός συνόλου `Set` και οι υπόλοιπες ταξινομούνται σε φθίνουσα χρονολογική σειρά (`localeCompare`), διασφαλίζοντας ότι οι πλέον πρόσφατες εκκρεμότητες εμφανίζονται στην κορυφή του μενού χωρίς καμία ανάγκη ενδιάμεσου εξυπηρετητή.

4.11 Ο κατάλογος και η αναζήτηση

Τα αντικείμενα της ενότητας 4.8 μπορούν να δώσουν απαντήσεις μόνο αν η σελίδα μπορεί πρώτα να τα εντοπίσει. Οι κλήσεις που χρησιμοποιεί ο adapter ανακτούν αντικείμενα με γνωστό ID, αντικείμενα μιας διεύθυνσης, συναλλαγές και events. Δεν παρέχουν την αναζήτηση λέξεων στα metadata που χρειάζεται ο κατάλογός μας. Μια λύση θα ήταν ένας εξωτερικός indexer που διαβάζει τα events και ενημερώνει μια βάση δεδομένων.

Στη δική μας εφαρμογή επιλέξαμε να κρατήσουμε το index στο πακέτο `art_tokenization`, στο shared αντικείμενο `Catalog`. Έτσι, ο κατάλογος και η αναζήτηση δεν χρειάζονται backend server της εφαρμογής. Το oracle παραμένει ξεχωριστός server για τις εξωτερικές αναφορές. Το **Σχήμα 4.8** δείχνει τα μέρη του `Catalog` και ποιες συναρτήσεις γράφουν σε καθένα.



Σχήμα 4.8. Το index και οι μετρητές στο blockchain δίκτυο.

Τον κατάλογο (`Catalog`) ενημερώνουν τρεις ομάδες συναρτήσεων, δηλαδή η έκδοση, οι συναρτήσεις που αλλάζουν την κατάσταση ενός έργου τέχνης και η ανάθεση από τη ρυθμιστική αρχή. Όλες τον ενημερώνουν στην ίδια συναλλαγή με την πράξη τους, ώστε το index να μη μένει ποτέ πίσω από τα αντικείμενα. Τα μεγέθη της αγοράς και του καναλιού αναφορών, αντίθετα, δεν είναι μέσα στον κατάλογο. Τα κρατούν δύο χωριστοί μετρητές, το `Stats` και το `ChannelStats`, για δύο λόγους. Πρώτον, οι συναλλαγές που γράφουν στο ίδιο shared αντικείμενο εκτελούνται η μία μετά την άλλη, οπότε μια πώληση δεν πρέπει να περιμένει στην ίδια ουρά με τις εκδόσεις. Δεύτερον, μια αναφορά κατάστασης δεν πρέπει να γράφει ποτέ σε αντικείμενο της ρυθμιστικής αρχής.

4.11.1 Η εγγραφή καταλόγου και τα chunks

Κάθε κάρτα της σελίδας του καταλόγου, όπως αυτή του **Σχήμα 4.9**, σχεδιάζεται από μία εγγραφή καταλόγου, το struct `CatalogEntry` του **Κώδικας 4.7**. Η εγγραφή έχει έντεκα πεδία, αντίγραφο όσων χρειάζεται η κάρτα από το έργο τέχνης, δηλαδή ποιο είναι, τον τίτλο, τον καλλιτέχνη, το έτος και το μέσο, τη δημόσια εικόνα, πότε εκδόθηκε, σε ποια κατάσταση είναι, ποια γκαλερί το φυλάσσει και πότε άλλαξε τελευταία φορά. Δεν έχει περιγραφή, διαστάσεις, εικόνα υψηλής ανάλυσης ή ιστορικό. Αυτά αποθηκεύονται μόνο στο `Artwork` και διαβάζονται όταν ανοίξει η σελίδα του έργου τέχνης. Η `mint` γράφει την εγγραφή καταλόγου (`CatalogEntry`) μία φορά, και η `sync` ενημερώνει έπειτα μόνο τρία πεδία στις μεταβολές κατάστασης ή ανάθεσης φύλαξης, την κατάσταση, τη γκαλερί φύλαξης και τον χρόνο της τελευταίας ενημέρωσης. Το `last_activity_at` αφορά αυτές τις μεταβολές και όχι κάθε συναλλαγή του έργου τέχνης. Οι αγορές και οι αγγελίες δεν καλούν τη `sync`. Τα υπόλοιπα πεδία δεν αλλάζουν, γιατί δεν αλλάζουν ούτε στο έργο τέχνης.



Σχήμα 4.9. Μια κάρτα της σελίδας του καταλόγου. Ό,τι δείχνει προέρχεται από την εγγραφή καταλόγου (`CatalogEntry`) του έργου τέχνης, χωρίς ανάγνωση του ίδιου του έργου τέχνης.

```
public struct CatalogEntry has store, copy, drop {
    artwork: ID,
    title: String,
    artist_name: String,
    artist: address,
    year: String,
    medium: String,
    image_blob_id: String,
    minted_at: u64,
    state: u8,
    custodian: Option<address>,
    last_activity_at: u64,
}
```

Κώδικας 4.7. Η εγγραφή καταλόγου ενός έργου τέχνης, από το module `artwork`. Τα τρία τελευταία πεδία ενημερώνονται από τη `sync`, ενώ τα υπόλοιπα γράφονται μία φορά στην έκδοση.

Οι εγγραφές καταλόγου (`CatalogEntry`) αποθηκεύονται σε ομάδες των εκατό, τα chunks. Ο ομώνυμος πίνακας του καταλόγου (`Catalog`) έχει τύπο `Table<u64, vector<CatalogEntry>>`, δηλαδή κλειδί έναν ακέραιο, τον αριθμό του chunk, και τιμή τη λίστα των εγγραφών του, και κάθε ζεύγος είναι δικό του dynamic field. Επιλέξαμε τα chunks ώστε μια ενημέρωση του καταλόγου να μη χρειάζεται να ξαναγράψει όλες τις εγγραφές σε ένα αντικείμενο. Με ένα dynamic field ανά έργο τέχνης, αντίθετα, ο browser θα έπρεπε να ανακτήσει ένα αντικείμενο για κάθε εγγραφή. Τα chunks ομαδοποιούν τις ανακτήσεις και περιορίζουν το τμήμα του καταλόγου που ξαναγράφει η `sync`. Το μέγεθος κάθε chunk εξαρτάται από τα metadata των εγγραφών του και δεν είναι σταθερό.

Για να βρίσκει η `sync` την εγγραφή καταλόγου (`CatalogEntry`), ο κατάλογος (`Catalog`) κρατά στον πίνακα `positions` τη θέση κάθε έργου τέχνης, και από αυτήν προκύπτει το chunk με διαίρεση και η θέση μέσα του με υπόλοιπο. Για παράδειγμα, το 250ό έργο τέχνης που εκδόθηκε έχει θέση 249, οπότε ζει στο chunk 2, στη θέση 49. Γενικά, το έργο τέχνης με θέση p στον κατάλογο βρίσκεται στο chunk c και στη θέση i μέσα στο chunk, σύμφωνα με τις σχέσεις που ακολουθούν.

$$c = \left\lfloor \frac{p}{100} \right\rfloor, \quad i = p \bmod 100. \tag{4.1}$$

Ο **Κώδικας 4.8** δείχνει πώς φαίνεται ένα chunk όπως το επιστρέφει ο full node. Το `name` είναι το κλειδί του dynamic field, δηλαδή ο αριθμός του chunk, και το `value` η λίστα των εγγραφών καταλόγου (`CatalogEntry`).

```json
{
  "id": "0x7c5f...c2fa",
  "name": "0",
  "value": [
    {
      "artwork": "0x2559...315c",
      "title": "Irises",
      "artist_name": "Vincent van Gogh",
      "artist": "0xa667...0ede",
      "year": "1889",
      "medium": "Ελαιογραφία σε καμβά",
      "image_blob_id": "oviOm9YlNdeodo2o3qEjen_KcsyNA7AXEvGMox8wBnM",
      "minted_at": "1788227239999",
      "state": 2,
      "custodian": "0xf009...18e2",
      "last_activity_at": "1788227753921"
    },
    ...
  ]
}
```

Κώδικας 4.8. Το πρώτο chunk του καταλόγου, όπως το επιστρέφει ο full node του testnet. Ένα dynamic field με κλειδί το 0 στο `name` και τιμή τη λίστα των εγγραφών, από τις οποίες φαίνεται μία.

4.11.2 Οι λίστες ανά διεύθυνση

Η ερώτηση «ποια αντικείμενα ανήκουν σε αυτή τη διεύθυνση;» του full node δεν βρίσκει ποτέ ένα έργο τέχνης, γιατί το έργο τέχνης είναι shared και δεν ανήκει σε καμία διεύθυνση. Το χαρτοφυλάκιο ενός χρήστη και ο πίνακας μιας γκαλερί χρειάζονται επομένως δικές τους λίστες. Ο κατάλογος (`Catalog`) κρατά τέσσερις, καθεμία ένας πίνακας από διεύθυνση σε λίστα αναγνωριστικών (`Table<address, vector<ID>>`). Η `by_artist` λέει ποια έργα τέχνης εξέδωσε μια διεύθυνση, η `by_owner` ποια κατέχει, η `by_custodian` ποια φυλάσσει και η `by_intended` ποια περιμένει να παραλάβει. Οι λίστες περιέχουν μόνο αναγνωριστικά έργων τέχνης (`ID`). Η σελίδα διαβάζει τη λίστα της διεύθυνσης και έπειτα τα ίδια τα έργα τέχνης, σε μία παρτίδα.

Κάθε πράξη που αλλάζει μια σχέση ενημερώνει τη λίστα της. Η έκδοση βάζει το έργο τέχνης στην `by_artist` και στην `by_owner` του καλλιτέχνη και στην `by_intended` της γκαλερί προορισμού. Η παραλαβή το μετακινεί από την `by_intended` στην `by_custodian` της γκαλερί. Η κλασματοποίηση το βγάζει από την `by_owner`, γιατί το έργο τέχνης δεν έχει πια έναν ιδιοκτήτη, και η ενοποίηση το ξαναβάζει στον νέο. Η ανάθεση από την αρχή το βάζει στην `by_intended` της νέας γκαλερί, ενώ η εξαργύρωση και η απώλεια το βγάζουν από την `by_custodian`. Η `by_artist` δεν αλλάζει ποτέ. Ο **Κώδικας 4.9** δίνει ένα παράδειγμα με έναν καλλιτέχνη και μία γκαλερί. Ο καλλιτέχνης έχει εκδώσει τρία έργα τέχνης και κατέχει τα δύο, γιατί το τρίτο είναι κλασματοποιημένο. Η γκαλερί φυλάσσει το πρώτο και το τρίτο και περιμένει να παραλάβει το δεύτερο.

```json
{
  "by_artist":    { "0xa1...": ["0x11...", "0x22...", "0x33..."] },
  "by_owner":     { "0xa1...": ["0x11...", "0x22..."] },
  "by_custodian": { "0xf0...": ["0x11...", "0x33..."] },
  "by_intended":  { "0xf0...": ["0x22..."] }
}
```

Κώδικας 4.9. Οι τέσσερις λίστες ανά διεύθυνση, σε ένα παράδειγμα με έναν καλλιτέχνη (`0xa1…`) και μία γκαλερί (`0xf0…`). Κάθε λίστα είναι πίνακας από διεύθυνση σε αναγνωριστικά έργων τέχνης.

4.11.3 Το index λέξεων

Η αναζήτηση χρειάζεται την αντίστροφη ερώτηση, από μια λέξη στα έργα τέχνης που την έχουν. Ο πίνακας `words` την απαντά. Έχει κλειδί μια λέξη και τιμή ένα bucket (`WordBucket`), δηλαδή τη λίστα των έργων τέχνης που έχουν τη λέξη αυτή. Κάθε bucket κρατά το πλήθος τους και έναν δεύτερο πίνακα από θέση σε αναγνωριστικό (`ID`) έργου τέχνης (`Artwork`), όπως στον **Κώδικας 4.10**. Τις λέξεις τις βγάζει η διεπαφή χρήστη κατά την έκδοση. Παίρνει τον τίτλο, το όνομα του καλλιτέχνη, το έτος και το μέσο, και τα κόβει σε λέξεις, έως δώδεκα. Έπειτα κανονικοποιεί κάθε λέξη, δηλαδή τη γράφει με πεζά, χωρίς τόνους και χωρίς τελικό σίγμα, ώστε η ίδια λέξη να γράφεται πάντα με τον ίδιο τρόπο. Από το «Ελαιογραφία σε καμβά» προκύπτουν έτσι οι λέξεις `ελαιογραφια`, `σε` και `καμβα`, και από τον «Vincent van Gogh» οι `vincent`, `van` και

gogh. Το έξυπνο συμβόλαιο ελέγχει μόνο το πλήθος και το μήκος τους, και προσθέτει το έργο τέχνης στο bucket (`WordBucket`) κάθε λέξης, στη θέση `count`, αφού δημιουργήσει το bucket αν δεν υπάρχει.

Ο δεύτερος πίνακας μέσα στο bucket (`WordBucket`) είναι `Table` και όχι `vector` για τον ίδιο λόγο με τα chunks. Μια λέξη όπως `ελαιογραφια` θα την έχουν τα περισσότερα έργα τέχνης. Με `vector` κάθε έκδοση θα ξανάγραφε ολόκληρη τη λίστα της, ενώ με `Table` προσθέτει ένα dynamic field χωρίς να αλλάξει τα προηγούμενα. Από το `words` δεν αφαιρείται ποτέ τίποτα. Ένα ακυρωμένο ή χαμένο έργο τέχνης μένει στις λέξεις του, και η σελίδα το κρύβει με το φίλτρο κατάστασης πάνω στην εγγραφή καταλόγου (`CatalogEntry`) του.

```json
{
  "words": {
    "vermeer":     { "count": 2, "ids": { "0": "0x11...", "1": "0x22..." } },
    "ελαιογραφια": { "count": 3, "ids": { "0": "0x11...", "1": "0x22...", "2": "0x33..." } },
    "1665":        { "count": 1, "ids": { "0": "0x11..." } }
  }
}
```

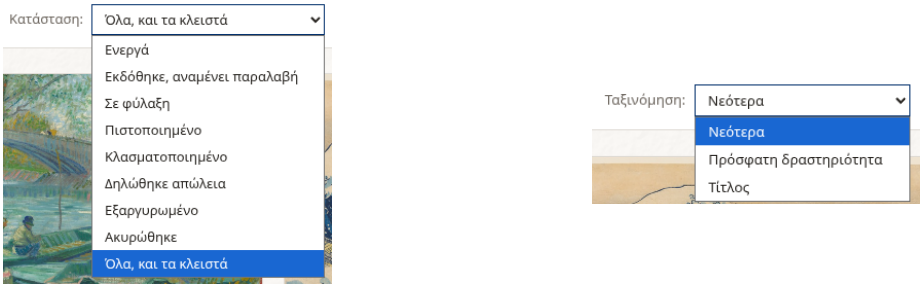
Κώδικας 4.10. Το index λέξεων, σε ένα παράδειγμα με τρεις λέξεις, όπως θα το έδειχνε ένας explorer με τους πίνακες ξεδιπλωμένους. Το `count` είναι το πλήθος των έργων τέχνης της λέξης και τα κλειδιά του `ids` οι θέσεις τους.

4.11.4 Η σελίδα του καταλόγου

Όταν ο χρήστης ανοίγει τη σελίδα του καταλόγου, το view ζητά το index από το state, και ο adapter του SUI το διαβάζει σε τρία βήματα (**Κώδικας 4.11**). Πρώτα διαβάζει τον κατάλογο (`Catalog`), για να πάρει το πλήθος των εγγραφών καταλόγου (`CatalogEntry`) και το αναγνωριστικό του πίνακα `chunks`. Έπειτα υπολογίζει τοπικά τα αναγνωριστικά των chunks που υπάρχουν, γιατί το αναγνωριστικό κάθε dynamic field προκύπτει από τον γονέα και το κλειδί [98], οπότε δεν χρειάζεται να ρωτήσει κανέναν. Τέλος ζητά τα chunks σε μία παρτίδα και μετατρέπει κάθε εγγραφή καταλόγου (`CatalogEntry`) σε μορφή για το view, με την κατάσταση ως όνομα και την εικόνα ως διεύθυνση στον aggregator του Walrus. Για έως πέντε χιλιάδες έργα τέχνης αυτό απαιτεί δύο κλήσεις στον full node. Το αποτέλεσμα μένει στην cache του state, που αδειάζει μετά από κάθε συναλλαγή της εφαρμογής.

Από εκεί και πέρα όλα γίνονται στη μνήμη του browser, χωρίς άλλη κλήση. Το **Σχήμα 4.10** δείχνει τα δύο μενού της σελίδας. Το φίλτρο κατάστασης δουλεύει πάνω στο πεδίο `state` της εγγραφής καταλόγου. Η προεπιλογή «Ενεργά» δείχνει τις τέσσερις μη τερματικές καταστάσεις, το «Σε φύλαξη» περιλαμβάνει και τα πιστοποιημένα και τα κλασματοποιημένα, γιατί παραμένουν σε φύλαξη, και το «Όλα, και τα κλειστά» δείχνει και τα εξαργυρωμένα, τα χαμένα και τα ακυρωμένα. Η ταξινόμηση χρησιμοποιεί τρία πεδία, τη θέση στον κατάλογο για τα νεότερα, το `last_activity_at` για την πρόσφατη δραστηριότητα, και τον τίτλο.

Η σελιδοποίηση χωρίζει τα φιλτραρισμένα αποτελέσματα σε σελίδες των έξι καρτών, σε διάταξη δύο γραμμών των τριών στηλών. Το **Σχήμα 4.11** δείχνει τη γραμμή πλοήγησης στο κάτω μέρος των καρτών. Το συνολικό πλήθος σελίδων υπολογίζεται από το μέγεθος της φιλτραρισμένης λίστας με $M = \max(1, \lceil \frac{N}{6} \rceil)$. Ο χρήστης μετακινείται με τα κουμπιά «Προηγούμενα» και «Επόμενα», τα οποία απενεργοποιούνται αυτόματα στα δύο άκρα. Κάθε αλλαγή στην αναζήτηση ή στα φίλτρα επαναφέρει την πλοήγηση στην πρώτη σελίδα. Παράλληλα, η σελιδοποίηση υλοποιεί καθυστερημένη φόρτωση (lazy loading). Ενώ ο browser διατηρεί στη μνήμη τις ελαφριές εγγραφές για όλα τα έργα τέχνης, τα πλήρη αντικείμενα και οι εικόνες από το Walrus ανακτώνται μόνο για τις έξι κάρτες της ενεργής σελίδας, ώστε να μην επιβαρύνεται το δίκτυο.



Σχήμα 4.10. Το φίλτρο κατάστασης και η ταξινόμηση της σελίδας του καταλόγου. Και τα δύο δουλεύουν πάνω στις εγγραφές καταλόγου (`CatalogEntry`) που έχει ήδη ο browser, χωρίς κλήση στο δίκτυο.



Σχήμα 4.11. Η γραμμή σελιδοποίησης και οι κάρτες του καταλόγου, με έξι κάρτες ανά σελίδα και τα πλήκτρα πλοήγησης στο κάτω μέρος.

Η αναζήτηση ξεκινά όταν ο χρήστης πληκτρολογήσει μια φράση. Η διεπαφή χρήστη την κανονικοποιεί με τον ίδιο κώδικα που έτρεξε στην έκδοση, οπότε το «Van Gogh 1889» γίνεται `van`, `gogh` και `1889`. Για κάθε λέξη ο adapter υπολογίζει το αναγνωριστικό του bucket (`WordBucket`) της, το διαβάζει, και έπειτα διαβάζει τα `count` αναγνωριστικά του σε παρτίδες έως 50 αντικειμένων. Για bucket με ένα έως 50 αναγνωριστικά χρειάζονται δύο κλήσεις, αφού έχει ανακτηθεί το `Catalog`. Για μεγαλύτερο bucket προστίθενται αντίστοιχες παρτίδες. Κρατά έπειτα την τομή των λιστών, δηλαδή τα έργα τέχνης που έχουν όλες τις λέξεις, και το view φιλτράρει με αυτήν τις εγγραφές καταλόγου (`CatalogEntry`) που έχει ήδη στη μνήμη. Η αναζήτηση βρίσκει

επομένως μόνο ολόκληρες λέξεις. Το gog δεν βρίσκει το gogh, γιατί δεν υπάρχει bucket με αυτό το κλειδί.

```typescript
const u64Entry = (table: ObjectId, index: number) =>
  deriveDynamicFieldID(table, 'u64', bcs.u64().serialize(index).toBytes());
const stringEntry = (table: ObjectId, key: string) =>
  deriveDynamicFieldID(table, '0x1::string::String', bcs.string().serialize(key).toBytes());

async function catalogIndex(): Promise<CatalogSummary[]> {
  const catalog = await catalogObject();
  const chunkIds = Array.from({ length: Math.ceil(num(catalog.count) / CHUNK) },
    (_, k) => u64Entry(idOf(catalog.chunks), k));
  const chunks = await multiGet(chunkIds);
  return chunks.flatMap((chunk, k) =>
    (fieldsOf(chunk)?.value ?? []).map((e, i) => summaryOf(inner(e), k * CHUNK + i)));
}

async function searchWord(word: string): Promise<ObjectId[]> {
  const catalog = await catalogObject();
  const bucket = inner(fieldsOf(await getOne(stringEntry(idOf(catalog.words), word)))?.value);
  if (!bucket) return [];
  const entries = await multiGet(
    Array.from({ length: num(bucket.count) }, (_, i) => u64Entry(idOf(bucket.ids), i)));
  return entries.map((e) => String(fieldsOf(e)?.value));
}
```

Κώδικας 4.11. Η ανάγνωση του καταλόγου και η αναζήτηση λέξης από τον client, απλοποιημένες. Τα αναγνωριστικά των dynamic fields υπολογίζονται τοπικά και τα αντικείμενα ζητούνται σε μία παρτίδα.

Πάνω σε αυτό το μοντέλο ανάγνωσης στηρίζεται η δημόσια επαλήθευση (ΛΑ-03). Ένας επισκέπτης χωρίς wallet δίνει μόνο το αναγνωριστικό ενός έργου τέχνης, πληκτρολογώντας το ή ανοίγοντας τη διεύθυνση `/verify/` με το αναγνωριστικό, που είναι και ο προορισμός του κωδικού QR στο έγγραφο του πιστοποιητικού. Η σελίδα διαβάζει το έργο τέχνης, το πιστοποιητικό και την καταχώριση φύλαξης από τα σταθερά κλειδιά τους, το log αναφορών, τις εγγραφές του provenance log, και το μητρώο γκαλερί για την επωνυμία και την κατάσταση της γκαλερί που φυλάσσει το έργο τέχνης. Δείχνει ποιος το εξέδωσε, ποια γκαλερί το πιστοποίησε και πότε, ποια το φυλάσσει σήμερα, τις εξωτερικές αναφορές, και κάθε πράξη με σύνδεσμο προς τη συναλλαγή της. Τίποτα δεν περνά από server της εφαρμογής, οπότε η επαλήθευση γίνεται και από έναν SUI explorer, όπως ο SuiScan.

4.12 Έκδοση έργου τέχνης

Η έκδοση ενός έργου τέχνης υλοποιείται στο module `artwork`. Το ψηφιακό δίδυμο του φυσικού έργου τέχνης αναπαρίσταται από τη δομή δεδομένων `Artwork` (**Κώδικας 4.12**). Η δομή αυτή περιλαμβάνει τα βασικά μεταδεδομένα (τίτλος, καλλιτέχνης, έτος, υλικό, διαστάσεις, περιγραφή), τη διεύθυνση του δημιουργού, καθώς και τα ids των δύο αρχείων στο Walrus, δηλαδή του δημόσιου αρχείου εικόνας (`image_blob_id`) και του κρυπτογραφημένου αρχείου υψηλής ανάλυσης (`hires_blob_id`).

```sui move
public struct Artwork has key {
    id: UID,
    title: String,
    artist: address,
    image_blob_id: String,
    hires_blob_id: String,
    seal_salt: vector<u8>,
    storage: vector<ID>,
    minted_at: u64,
    owner: Option<address>,
    vault: Option<ID>,
    intended_gallery: Option<address>,
    state: u8,
    // ...
}
```

Κώδικας 4.12. Η δομή δεδομένων `Artwork` στο module `artwork`. Πεδία μεταδεδομένων, αναγνωριστικά αρχείων Walrus, κρυπτογραφικό salt ταυτότητας Seal και κατάσταση κύκλου ζωής.

Για την προστασία της ιδιωτικότητας και τον μελλοντικό έλεγχο πρόσβασης με το Seal, κάθε έργο τέχνης δεσμεύει ένα κρυπτογραφικό salt (`seal_salt`). Στη συνάρτηση `mint` (**Κώδικας 4.13**), η συνένωση των bytes του `image_blob_id` και του `seal_salt` σχηματίζει το κλειδί `IdentityKey`. Το κλειδί αυτό καταχωρίζεται ως δυναμικό πεδίο στον κατάλογο (`Catalog`). Αν επιχειρηθεί δεύτερη έκδοση για το ίδιο αρχείο εικόνας με το ίδιο salt, η συνάρτηση διακόπτει άμεσα την εκτέλεση με το σφάλμα `EIdentityAlreadyBound`.

Η συνάρτηση `mint` ελέγχει επίσης ότι τα υποχρεωτικά πεδία δεν είναι κενά και ότι η δηλωμένη γκαλερί (`intended_gallery`) είναι ενεργή στο μητρώο γκαλερί (`Registry<Gallery>`). Με τη δημιουργία του, το `Artwork` αποκτά την αρχική κατάσταση `STATE_MINTED` (τιμή `0`), και ως ιδιοκτήτης (`owner`) ορίζεται η διεύθυνση του καλλιτέχνη. Ταυτόχρονα, αρχικοποιείται το ημερολόγιο ιστορικού προέλευσης (provenance log) με την εγγραφή `HIST_MINTED`, και εκπέμπεται το event `ArtworkMinted`.

```sui move
public fun mint(
    catalog: &mut Catalog, galleries: &Registry<Gallery>,
    title: String, image_blob_id: String, hires_blob_id: String,
    seal_salt: vector<u8>, intended_gallery: address,
    keywords: vector<String>, clock: &Clock, ctx: &mut TxContext,
) {
    assert!(!title.is_empty(), EEmptyTitle);
    assert!(!image_blob_id.is_empty(), EEmptyImage);
    assert!(!seal_salt.is_empty(), EEmptySalt);
    assert!(galleries.is_active(intended_gallery), EGalleryNotActive);
    let key = IdentityKey { id: identity_bytes(&image_blob_id, &seal_salt) };
    assert!(!df::exists(&catalog.id, key), EIdentityAlreadyBound);

    let at = clock.timestamp_ms();
    let artist = ctx.sender();
    let mut artwork = Artwork {
        id: object::new(ctx),
        title, artist, image_blob_id, hires_blob_id, seal_salt,
        minted_at: at, owner: option::some(artist), vault: option::none(),
        intended_gallery: option::some(intended_gallery),
        state: STATE_MINTED,
        // ...
    };
    let id = artwork.id.to_inner();
    df::add(&mut catalog.id, key, id);
    // ... καταχώριση στον κατάλογο και στο index λέξεων ...
    log(&mut artwork, HIST_MINTED, at, artist, 0, 0, option::none());
    event::emit(ArtworkMinted { artwork: id, artist, title: artwork.title, intended_gallery, at });
    transfer::share_object(artwork);
}
```

Κώδικας 4.13. Η συνάρτηση `mint` στο module `artwork`. Έλεγχος ενεργού γκαλερί, δέσμευση ταυτότητας Seal στον κατάλογο και δημοσίευση ως shared αντικείμενο.

Επιλέξαμε το `Artwork` να δημοσιεύεται ως shared αντικείμενο μέσω της `transfer::share_object`. Έτσι, η γκαλερί μπορεί να καταχωρίζει την παραλαβή και να προσθέτει το πιστοποιητικό χωρίς συνυπογραφή του ιδιοκτήτη. Οι συναρτήσεις του συμβολαίου ελέγχουν ποιος επιτρέπεται να κάνει κάθε μεταβολή, ενώ το πεδίο `owner: Option<address>` κρατά τη λογική ιδιοκτησία. Το oracle διαβάζει το `Artwork`, αλλά καταγράφει τις αναφορές στο ξεχωριστό shared `ReportLog` και ενημερώνει το `ChannelStats`. Δεν προσαρτά νέα πεδία στο `Artwork` σε κάθε αναφορά.

Όσο το έργο τέχνης παραμένει στην κατάσταση `STATE_MINTED`, η φυσική παραλαβή δεν έχει ακόμη ξεκινήσει. Ο ιδιοκτήτης διατηρεί επομένως το δικαίωμα ακύρωσης μέσω της συνάρτησης `cancel_mint`. Η συνάρτηση επαληθεύει ότι ο αποστολέας είναι ο τρέχων ιδιοκτήτης, μεταφέρει την κατάσταση σε `STATE_CANCELLED` (τιμή 6), αφαιρεί το έργο τέχνης από τη λίστα αναμονής της γκαλερί, και καταγράφει την ακύρωση στο provenance log.

4.13 Φύλαξη και πιστοποιητικό γνησιότητας

Η φυσική υπόσταση ενός tokenized έργου τέχνης υποστηρίζεται από το module `custody` για την παρακολούθηση της φυσικής φύλαξης και από το module `certificate` για την έκδοση του ψηφιακού πιστοποιητικού γνησιότητας (**Κώδικας 4.14**).

Η παραλαβή σε φύλαξη. Όταν το φυσικό έργο τέχνης παραδοθεί στη γκαλερί, η γκαλερί καλεί τη συνάρτηση `record_custody`. Η συνάρτηση ελέγχει ότι η γκαλερί έχει ενεργή εγγραφή στο μητρώο (`galleries.is_active`) και ότι είναι η γκαλερί που έχει οριστεί για την παραλαβή (`intended_gallery`). Στην πρώτη παραλαβή κατασκευάζει το `CustodyRecord` και το συνδέει στο `Artwork` ως dynamic object field. Παράλληλα, δημιουργεί το shared `ReportLog`, το οποίο κρατά τα IDs των αναφορών των εξωτερικών πηγών και τα συνοπτικά στοιχεία τους. Η κατάσταση του έργου τέχνης μεταβαίνει σε `STATE_IN_CUSTODY` (τιμή `1`), και εκπέμπεται το event `CustodyRecorded`.

Η έκδοση πιστοποιητικού γνησιότητας. Μετά την παραλαβή, οι ειδικοί της γκαλερί επιθεωρούν το έργο τέχνης και συντάσσουν την τεχνική τεκμηρίωση. Η έκδοση του πιστοποιητικού γίνεται μέσω της συνάρτησης `certificate::issue`. Η συνάρτηση επαληθεύει ότι ο αποστολέας είναι ενεργή γκαλερί και φυλάσσει το έργο τέχνης (`custody::assert_custodian`), ότι αυτό δεν φέρει ήδη πιστοποιητικό, και ότι η δήλωση γνησιότητας δεν είναι κενή. Το αντικείμενο `Certificate` περιλαμβάνει τη διεύθυνση του εκδότη, τη χρονοσφραγίδα, τη δήλωση (`statement`), και το αναγνωριστικό του εγγράφου τεκμηρίωσης στο Walrus (`documentation_blob_id`). Το πιστοποιητικό προσαρτάται πάνω στο `Artwork` με την εντολή `dof::add`, και η κατάσταση μεταβαίνει σε `STATE_CERTIFIED` (τιμή `2`).

Η δημόσια επαλήθευση δείχνει ποια γκαλερί υπέγραψε τη δήλωση και τι καταγράφηκε στην αλυσίδα. Το συμβόλαιο δεν εξετάζει το φυσικό έργο τέχνης ούτε αποδεικνύει μόνο του τη γνησιότητά του. Γι' αυτήν εξακολουθούμε να στηριζόμαστε στην επιθεώρηση και την ευθύνη της γκαλερί.

Η επιλογή των dynamic object fields. Επιλέξαμε το `dof::add`, ώστε το `Certificate` και το `CustodyRecord` να διατηρούν το δικό τους `UID`. Και τα δύο έχουν τα abilities `key` και `store` και γίνονται object-owned μετά την προσάρτηση. Στη δική μας υλοποίηση ισχύουν τα εξής:

1. Τα αντικείμενα αυτά διατηρούν το καθολικό τους ID στο SUI, οπότε είναι άμεσα αναζητήσιμα και ορατά σε εξωτερικούς explorers κάτω από το γονικό έργο τέχνης.
2. Κανένας χρήστης δεν μπορεί να τα αποσπάσει ή να τα μεταβιβάσει σε άλλο έργο τέχνης. Δεν υπάρχει δημόσια συνάρτηση εξαγωγής τους. Το πιστοποιητικό και η καταχώριση φύλαξης παραμένουν μόνιμα και αμετάκλητα ενσωματωμένα στο ψηφιακό δίδυμο.

Ο κύκλος ζωής της φύλαξης. Το module `custody` καλύπτει επίσης τις δυνατές μεταβολές στη φυσική φύλαξη:

- **Επανατοποθέτηση (`reassign`):** Αν η ρυθμιστική αρχή ανακαλέσει την άδεια μιας γκαλερί, μπορεί να χρησιμοποιήσει το `AdminCap` για να ορίσει νέα γκαλερί για την παραλαβή του έργου τέχνης (ΛΑ-18).
- **Φυσική εξαργύρωση (`redeem_request`, `redeem_withdraw`, `redeem_confirm`, ΛΑ-20):** Ο ιδιοκτήτης μπορεί να ζητήσει την επιστροφή του φυσικού αντικειμένου, υπό την προϋπόθεση ότι το έργο τέχνης δεν είναι κλασματοποιημένο και βρίσκεται σε ανοιχτή φύλαξη. Ο αιτών μπορεί να ανακαλέσει το εκκρεμές αίτημα. Η ενεργή γκαλερί φύλαξης επιβεβαιώνει την παράδοση, η φύλαξη κλείνει (`until: Some(at)`), και το έργο τέχνης περνά στην τερματική κατάσταση `STATE_REDEEMED` (τιμή 4).
- **Δήλωση απώλειας (`report_loss`):** Σε περίπτωση καταστροφής ή κλοπής, η γκαλερί δηλώνει επίσημα την απώλεια. Το έργο τέχνης μεταβαίνει στην τερματική κατάσταση `STATE_LOST` (τιμή 5).
- **Συσχέτιση αποθήκευσης (`attach_storage`):** Ο ιδιοκτήτης ή η ενεργή γκαλερί που φυλάσσει το έργο τέχνης μπορεί να προσθέσει IDs στο πεδίο `storage: vector<ID>`. Η διεπαφή χρησιμοποιεί τα IDs των `SharedBlob` που δημιουργήθηκαν στη ροή αποθήκευσης. Η συνάρτηση καταχωρίζει αυτά τα IDs, χωρίς να ελέγχει η ίδια τον τύπο των αντικειμένων ή αν αντιστοιχούν στα αρχεία του έργου τέχνης.

```move
public struct CertificateKey has copy, drop, store {}
public struct Certificate has key, store {
    id: UID, issuer: address, issued_at: u64, statement: String, documentation_blob_id: String,
}
public fun issue(
    artwork: &mut Artwork, galleries: &Registry<Gallery>, catalog: &mut Catalog, statement: String,
    documentation_blob_id: String, clock: &Clock, ctx: &mut TxContext,
) {
    let issuer = ctx.sender();
    assert!(galleries.is_active(issuer), ENotActive);
    artwork.assert_not_terminal(); custody::assert_custodian(artwork, issuer);
    assert!(!exists(artwork), EAlreadyCertified); assert!(!statement.is_empty(), EEmptyStatement);
    let at = clock.timestamp_ms();
    let cert = Certificate { id: object::new(ctx), issuer, issued_at: at, statement, documentation_blob_id };
    let certificate = object::id(&cert);
    dof::add(artwork.uid_mut(), CertificateKey {}, cert);
    artwork.set_certified();
    catalog.sync(artwork, option::some(issuer), at);
    artwork.log(artwork::hist_certified(), at, issuer, 0, 0, option::some(certificate));
    event::emit(CertificateIssued { artwork: object::id(artwork), certificate, issuer, at });
}
```

Κώδικας 4.14. Έκδοση πιστοποιητικού γνησιότητας στο module `certificate`. Προσάρτηση του αντικειμένου `Certificate` στο έργο τέχνης ως dynamic object field (`dof::add`).

4.14 Κλασματοποίηση

Η κλασματοποίηση ενός πιστοποιημένου έργου τέχνης υλοποιείται στο module `fraction` (**Κώδικας 4.15**). Μέσω της διαδικασίας αυτής, η ενιαία ιδιοκτησία αντικαθίσταται από έναν προκαθορισμένο αριθμό διαιρετών μεριδίων, επιτρέποντας τη συμμετοχική συνιδιοκτησία.

Προϋποθέσεις κλασματοποίησης. Η συνάρτηση `fractionalize` επιτρέπει τη διάσπαση μόνο όταν ικανοποιούνται ταυτόχρονα επτά αυστηροί έλεγχοι:

1. Το έργο τέχνης δεν βρίσκεται σε τερματική κατάσταση (`artwork.assert_not_terminal`).
2. Δεν είναι ήδη κλασματοποιημένο (`!artwork.is_fractionalised()`).
3. Ο καλών είναι ο νόμιμος ιδιοκτήτης (`artwork.assert_owner(by)`).
4. Το έργο τέχνης βρίσκεται σε ενεργό φυσική φύλαξη (`custody::is_open(artwork)`).
5. Φέρει εκδοθέν πιστοποιητικό γνησιότητας (`certificate::exists(artwork)`).
6. Δεν εκκρεμεί αίτημα φυσικής εξαργύρωσης (`!custody::redemption_pending(artwork)`).
7. Η συνολική προσφορά κλασμάτων είναι θετικός αριθμός (`total_supply > 0`).

Η μετάβαση ιδιοκτησίας και το θησαυροφυλάκιο. Όταν ικανοποιούνται οι παραπάνω έλεγχοι, η συνάρτηση αφαιρεί την ατομική ιδιοκτησία από το `Artwork` (`owner: None`). Στη συνέχεια δημιουργεί ένα shared αντικείμενο θησαυροφυλακίου (`FractionVault`), το οποίο συνδέεται αμφίδρομα με το έργο τέχνης (`vault: Some(vault_id)`). Το θησαυροφυλάκιο κρατά τη συνολική προσφορά (`total_supply`) και είναι η ρίζα κάτω από την οποία καταχωρίζονται οι ανοιχτές αγγελίες στο index.

Ταυτόχρονα, δημιουργείται το πρώτο αντικείμενο `Fraction`, με ποσότητα ίση με το `total_supply`, και μεταβιβάζεται στον ιδιοκτήτη. Το `Artwork` αποκτά την κατάσταση `STATE_FRACTIONALISED` (τιμή `3`), και εκπέμπεται το event `Fractionalised`.

Η απαγόρευση εξωχρηματιστηριακής μεταβίβασης. Όπως είδαμε στο μοντέλο αντικειμένων, η δομή `Fraction` διαθέτει την ιδιότητα `key`, αλλά στερείται της ιδιότητας `store`. Στη Sui Move, ένα αντικείμενο χωρίς `store` δεν μπορεί να μεταβιβαστεί μέσω της γενικής εντολής `transfer::public_transfer`. Κανένα wallet δεν μπορεί να αποστείλει το κλάσμα απευθείας σε άλλη διεύθυνση. Η μόνη επιτρεπτή διαδρομή αλλαγής κατόχου είναι η εσωτερική συνάρτηση `fraction::deliver`, η οποία είναι προσβάσιμη μόνο από τα modules του ίδιου πακέτου (package visibility). Έτσι, κάθε αλλαγή κατόχου υποχρεούται να περάσει από το module `market`.

Διαχείριση και ενοποίηση κλασμάτων.

- **Διάσπαση (`split`) και συγχώνευση (`merge`):** Ένας κάτοχος μπορεί να διασπάσει ένα κλάσμα σε δύο μικρότερα (`split`), παράγοντας ένα νέο αντικείμενο `Fraction` με νέο UID. Αντίστοιχα, δύο κλάσματα που ανήκουν στο ίδιο θησαυροφυλάκιο μπορούν να συγχωνευ-

τούν (`merge`), προσθέτοντας το ποσό στο πρώτο και διαγράφοντας το UID του δεύτερου (`id.delete()`).

- **Ενοποίηση (`consolidate`):** Εάν ένας επενδυτής συγκεντρώσει μέσω αγορών και συγχωνεύσεων το σύνολο της προσφοράς (`amount == total_supply`), διαλύει το θησαυροφυλάκιο (ΛΑ-14). Η συνάρτηση `consolidate` διαγράφει οριστικά τα UIDs του κλάσματος και του θησαυροφυλακίου, επαναφέρει το έργο τέχνης σε έναν μοναδικό ιδιοκτήτη (`owner: Some(by)`), και αποκαθιστά την κατάσταση σε `STATE_CERTIFIED`.
- **Διαγραφή σε απώλεια (`burn`):** Εάν η γκαλερί δηλώσει απώλεια του φυσικού έργου τέχνης, τα κλάσματα καθίστανται ανενεργά. Οι κάτοχοι μπορούν τότε να καλέσουν τη συνάρτηση `burn`, διαγράφοντας τα αντικείμενα από το πορτοφόλι τους.

```sui move
public struct FractionVault has key { id: UID, artwork: ID, total_supply: u64 }

public struct Fraction has key {
    id: UID, vault_id: ID, artwork_id: ID, amount: u64,
    title: String, image_blob_id: String,
}

public fun fractionalize(
    artwork: &mut Artwork, total_supply: u64, catalog: &mut Catalog,
    stats: &mut Stats, clock: &Clock, ctx: &mut TxContext,
) {
    let by = ctx.sender();
    artwork.assert_not_terminal();
    assert!(!artwork.is_fractionalised(), EAlreadyFractionalised);
    artwork.assert_owner(by);
    assert!(custody::is_open(artwork), ENoCustody);
    assert!(certificate::exists(artwork), ENoCertificate);
    assert!(!custody::redemption_pending(artwork), ERedemptionPending);
    assert!(total_supply > 0, EZeroSupply);

    let at = clock.timestamp_ms();
    let artwork_id = object::id(artwork);
    catalog.owner_remove(by, artwork_id);
    let vault = FractionVault { id: object::new(ctx), artwork: artwork_id, total_supply };
    let vault_id = object::id(&vault);
    let fraction = Fraction {
        id: object::new(ctx), vault_id, artwork_id, amount: total_supply,
        title: artwork.title(), image_blob_id: artwork.image_blob_id(),
    };

    artwork.set_owner(option::none(), option::some(vault_id));
    artwork.set_fractionalised();
    artwork.set_supply(total_supply);
    stats.vault_opened(total_supply);
    // ... καταγραφή στο provenance log και συγχρονισμός καταλόγου ...
    event::emit(Fractionalised { artwork: artwork_id, vault: vault_id, total_supply, by, at });
    transfer::share_object(vault);
    transfer::transfer(fraction, by);
}
```

Κώδικας 4.15. Κλασματοποίηση στο module `fraction`. Έλεγχοι προϋποθέσεων, δημιουργία του shared αντικειμένου `FractionVault` και παράδοση του πρώτου `Fraction` στον ιδιοκτήτη.

4.15 Αγορά και δικαίωμα παρακολούθησης

Η δευτερογενής διαπραγμάτευση κλασμάτων και η αυτόματη επιβολή του δικαιώματος παρακολούθησης (droit de suite) υλοποιούνται στο module `market` (**Κώδικας 4.16**).

Η δημιουργία αγγελίας και το escrow (`Shares`). Επειδή το `Fraction` στερείται της ιδιότητας `store`, δεν μπορεί να αποθηκευτεί άμεσα μέσα σε ένα shared αντικείμενο αγγελίας. Για να επιλυθεί αυτός ο περιορισμός στο επίπεδο τύπων, το πακέτο ορίζει τη βοηθητική δομή `Shares has store`. Στη συνάρτηση `list`, το κλάσμα του πωλητή μετατρέπεται σε `Shares` μέσω της `fraction::into_shares`, η οποία κάνει unpack το `Fraction` και διαγράφει το προσωρινό του UID. Αν ο πωλητής διαθέτει περισσότερα μερίδια από όσα καταχωρίζει στην αγγελία, τα υπόλοιπα μερίδια μετατρέπονται ξανά σε νέο `Fraction` και επιστρέφονται στο πορτοφόλι του.

Τα διατιθέμενα μερίδια δεσμεύονται σε escrow μέσα στη δομή `Listing`. Το αντικείμενο `Listing` δημοσιεύεται ως shared αντικείμενο, ώστε κάθε ενδιαφερόμενος αγοραστής να μπορεί να το εντοπίσει. Παράλληλα, το αναγνωριστικό της αγγελίας συνδέεται στο `FractionVault` ως δυναμικό πεδίο (`df::add`), διατηρώντας το ευρετήριο ενεργών αγγελιών του έργου τέχνης.

Η εκτέλεση της αγοράς (`buy`). Ο αγοραστής καλεί τη συνάρτηση `buy`, προσκομίζοντας το ακριβές αντίτιμο σε νόμισμα SUI (`Coin<SUI>`). Η συνάρτηση εκτελείται ατομικά και υλοποιεί τα ακόλουθα βήματα:

1. Επαληθεύει ότι η αγγελία είναι ενεργή, ότι ο αγοραστής δεν είναι ο ίδιος ο πωλητής, και ότι η ποσότητα είναι διαθέσιμη.
2. Υπολογίζει το δικαίωμα παρακολούθησης μέσω της συνάρτησης `royalty_of`. Ο υπολογισμός πραγματοποιείται σε αριθμητική 128 bit (`u128`), ώστε ο πολλαπλασιασμός του τιμήματος με τις μονάδες βάσης (`royalty_bps`) να μην προκαλέσει αριθμητική υπερχείλιση στα 64 bit.
3. Εάν ο πωλητής είναι ο αρχικός καλλιτέχνης, το ποσό δικαιώματος είναι μηδέν, καθώς το droit de suite αφορά αποκλειστικά τη μεταπώληση. Στη δευτερογενή πώληση, το ποσό του δικαιώματος αποκόπτεται άμεσα από το νόμισμα πληρωμής και μεταβιβάζεται στη διεύθυνση του καλλιτέχνη (`artwork.artist()`).
4. Το υπόλοιπο τίμημα μεταβιβάζεται στον πωλητή (`listing.seller`).
5. Τα αγορασθέντα μερίδια αποδεσμεύονται από το escrow και μετατρέπονται σε ένα νέο αντικείμενο `Fraction` με δικό του UID, το οποίο παραδίδεται στον αγοραστή (`fraction::deliver`).

Επειδή όλες οι παραπάνω μεταφορές εκτελούνται μέσα στην ίδια συναλλαγή, η αγορά καταβάλλει αυτόματα το ποσοστό που ορίζει η παράμετρος `royalty_bps` επί του δηλωμένου τιμήματος. Η εγγύηση αφορά τις μεταπωλήσεις κλασμάτων μέσω του συμβολαίου και δεν επαληθεύει τυχόν πρόσθετο τίμημα που συμφωνήθηκε εκτός της εφαρμογής.

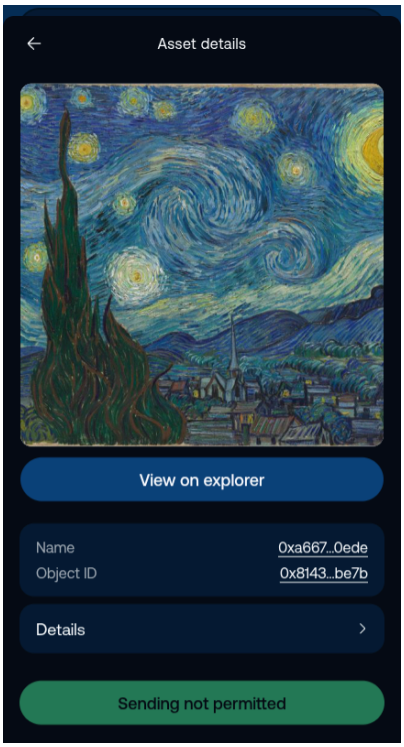
Ακύρωση αγγελίας (`cancel`). Ο πωλητής διατηρεί το δικαίωμα ακύρωσης μιας ενεργής αγγελίας. Η συνάρτηση `cancel` αποσυνδέει την αγγελία από το θησαυροφυλάκιο, εξάγει τα μερίδια `Shares` από το escrow, τα ανασυστήνει σε αντικείμενο `Fraction` και τα παραδίδει πίσω στον πωλητή.

```sui move
let artist = artwork.artist();
let royalty_amount =
    if (listing.seller == artist) 0 else royalty_of(total, params.royalty_bps());
let seller_amount = total - royalty_amount;
if (royalty_amount > 0) {
    transfer::public_transfer(payment.split(royalty_amount, ctx), artist);
};
if (payment.value() > 0) {
    transfer::public_transfer(payment, listing.seller);
} else {
    payment.destroy_zero();
};
// ...
stats.sale(artist, total, royalty_amount);
fraction::deliver(fraction::from_shares(bought, ctx), buyer);
```

Κώδικας 4.16. Ο διαμοιρασμός της πληρωμής σε μια πώληση κλασμάτων, από τη συνάρτηση `buy` του module `market`, χωρίς τους ελέγχους και το κλείσιμο της αγγελίας. Μηδενικό δικαίωμα στην πρωτογενή πώληση, δύο μεταφορές στην ίδια συναλλαγή.

Το πρότυπο εμφάνισης στο wallet. Ένα wallet δεν γνωρίζει εκ των προτέρων πώς να προβάλει ένα αντικείμενο τύπου `Fraction`. Το SUI το λύνει με το Object Display [135]. Ο εκδότης του πακέτου δηλώνει ένα πρότυπο εμφάνισης για τον τύπο `Fraction`. Το πρότυπο λέει ποιο πεδίο του αντικειμένου είναι το όνομα, ποιο η περιγραφή και ποιο η εικόνα, με θέσεις σε αγκύλες που κάθε wallet ή explorer αντικαθιστά με τα πεδία του συγκεκριμένου αντικειμένου. Ο **Κώδικας 4.17** δίνει το πρότυπο του κλάσματος. Η εικόνα είναι η διεύθυνση του aggregator του Walrus με το αναγνωριστικό του blob, οπότε το wallet κατεβάζει τη δημόσια εικόνα του έργου τέχνης απευθείας από το Walrus, χωρίς server της εφαρμογής. Ο σύνδεσμος οδηγεί στη σελίδα του έργου τέχνης στο Walrus Site.

Δύο ιδιότητες του προτύπου επηρέασαν το μοντέλο των αντικειμένων. Πρώτον, το πρότυπο αντικαθιστά μόνο πεδία του ίδιου του αντικειμένου, όχι αντικειμένων στα οποία αυτό δείχνει. Γι' αυτό το κλάσμα κρατά αντίγραφο του τίτλου και του αναγνωριστικού της εικόνας, όπως είδαμε στην ενότητα 4.8. Δεύτερον, το πρότυπο καταχωρίζεται ως shared αντικείμενο στο κεντρικό Display Registry του SUI (`0xd`), ενώ το δικαίωμα διαχείρισης (`DisplayCap`) ανήκει στην αρχή. Η αλλαγή του aggregator ή της διεύθυνσης του site είναι επομένως μία συναλλαγή της ρυθμιστικής αρχής με το capability της, χωρίς να επηρεαστεί κανένα εκδοθέν κλάσμα. Το αποτέλεσμα στο Slush φαίνεται στο **Σχήμα 4.12**, όπου το κουμπί αποστολής είναι απενεργοποιημένο λόγω έλλειψης του `store`.



Σχήμα 4.12. Το κλάσμα στο Slush. Η εικόνα κατεβαίνει από το Walrus χωρίς server της εφαρμογής. Το κουμπί αποστολής είναι απενεργοποιημένο, γιατί ο τύπος δεν έχει την ιδιότητα `store` και δεν μεταβιβάζεται έξω από το module `market`.

```
let image = concat(aggregator, b"/v1/blobs/{image_blob_id}");
// Fraction Display v2
let (mut fraction_display, fraction_cap) =
    display_registry::new_with_publisher<Fraction>(registry, publisher, ctx);
display_registry::set(&mut fraction_display, &fraction_cap, b"name".to_string(), b"{title}: {amount} fractions".to_string());
display_registry::set(&mut fraction_display, &fraction_cap, b"description".to_string(), b"Fractional ownership of the artwork {artwork_id}".to_string());
display_registry::set(&mut fraction_display, &fraction_cap, b"image_url".to_string(), image);
display_registry::set(&mut fraction_display, &fraction_cap, b"link".to_string(), concat(site, b"/artwork/{artwork_id}"));
display_registry::set(&mut fraction_display, &fraction_cap, b"project_url".to_string(), site);
display_registry::share(fraction_display);
```

Κώδικας 4.17. Το πρότυπο εμφάνισης του κλάσματος, από το module `display`. Η εικόνα έρχεται από τον aggregator του Walrus και ο σύνδεσμος από το Walrus Site, με αντικατάσταση πεδίων του κλάσματος.

4.16 Τα αρχεία στο Walrus και ο έλεγχος πρόσβασης με Seal

Τα ογκώδη δεδομένα ενός έργου τέχνης αποθηκεύονται ως blobs στο Walrus. Στην έκδοση, ο καλλιτέχνης προπληρώνει 30 epochs για τη δημόσια εικόνα και, αν την προσθέσει, την κρυπτογραφημένη εικόνα υψηλής ανάλυσης. Τα αναγνωριστικά τους αποθηκεύονται στο `Artwork`. Το έγγραφο τεκμηρίωσης προστίθεται χωριστά κατά την πιστοποίηση, με πληρωμή της γκαλερί για την ίδια διάρκεια. Το δικό του αναγνωριστικό βρίσκεται στο `Certificate`, στο πεδίο `documentation_blob_id`.

Η δημόσια εικόνα και η τεκμηρίωση ανακτώνται απευθείας από οποιονδήποτε aggregator του Walrus. Αντίθετα, η εικόνα υψηλής ανάλυσης κατεβαίνει ως κρυπτογραφημένο ciphertext και αποκρυπτογραφείται τοπικά στον browser μόνο εφόσον οι διακομιστές κλειδιών του Seal εγκρίνουν το αίτημα πρόσβασης.

Υλοποίηση. Ο adapter του Walrus τυλίγει τα τέσσερα βήματα του πρωτοκόλλου εγγραφής της ενότητας 3.2.3 σε ένα αντικείμενο ανά αρχείο. Η κωδικοποίηση γίνεται τοπικά με τον κώδικα WebAssembly του Walrus και δίνει το αναγνωριστικό του blob. Η καταχώριση προσθέτει σε ένα PTB την κλήση που πληρώνει την αποθήκευση για 30 epochs. Το upload στέλνει τα slivers στους nodes χωρίς υπογραφή, και η πιστοποίηση προσθέτει σε δεύτερο PTB το πιστοποιητικό διαθεσιμότητας που επέστρεψαν. Ο **Κώδικας 4.18** δείχνει πώς η έκδοση ενός έργου τέχνης ενώνει τα βήματα αυτά με το έξυπνο συμβόλαιο. Ο browser παράγει το τυχαίο salt 16 byte και υπολογίζει την ταυτότητα Seal, όπως είδαμε στην ενότητα 4.12. Η πρώτη υπογραφή καταχωρίζει τα δύο blobs, η δεύτερη τα πιστοποιεί και καλεί ταυτόχρονα τη συνάρτηση `artwork::mint` στο ίδιο PTB, ενώ η τρίτη υπογραφή συνδέει τα κοινόχρηστα αντικείμενα αποθήκευσης στο έργο τέχνης μέσω της `custody::attach_storage` (**Σχήμα 4.13**).

Η ατομικότητα αφορά κάθε συναλλαγή χωριστά. Αν αποτύχει το mint στη δεύτερη συναλλαγή, αναιρείται και η πιστοποίηση της αποθήκευσης που περιλαμβάνεται στο ίδιο PTB. Η καταχώριση και η πληρωμή της πρώτης συναλλαγής, όμως, παραμένουν, όπως και τα δεδομένα που έχουν ήδη σταλεί στο Walrus. Αν αποτύχει η τρίτη συναλλαγή, το έργο τέχνης έχει ήδη εκδοθεί και η διεπαφή εμφανίζει προειδοποίηση για τη σύνδεση αποθήκευσης. Δεν έχουμε υλοποιήσει αυτόματη συνέχιση της ροής από το τελευταίο επιτυχημένο βήμα.

```typescript
const image = storage.blob(args.preview);
const imageBlobId = await image.encode();
const sealSalt = newSealSalt();
const hires = storage.blob(
  await encryptPrivate(imageBlobId, sealSalt, hiresBytes),
);
const hiresBlobId = await hires.encode();

const reg = new Transaction();
image.register(reg, args.artist);
hires.register(reg, args.artist);
const registered = await execute(reg);
await image.upload(registered.digest, registered.created);
```

```
await hires.upload(registered.digest, registered.created);

const tx = new Transaction();
await image.certify(tx);
await hires.certify(tx);
tx.moveCall({
  target: target('artwork', 'mint'),
  arguments: [
    tx.object(config.objects.catalog),
    tx.object(config.objects.galleryRegistry),
    tx.pure.string(args.title), /* ... τα υπόλοιπα metadata ... */
    tx.pure.string(imageBlobId), tx.pure.string(hiresBlobId),
    tx.pure.vector('u8', sealSalt),
    tx.pure.address(args.intendedGallery.address),
    tx.pure.vector('string', keywords), tx.object(CLOCK),
  ],
});
await execute(tx);
```

Κώδικας 4.18. Η έκδοση ενός έργου τέχνης από τη διεπαφή χρήστη, απλοποιημένη (οι δύο πρώτες υπογραφές καταχώρισης και έκδοσης, πριν από την προσάρτηση της αποθήκευσης).

4.16.1 Ο έλεγχος πρόσβασης με Seal

Όπως είδαμε στην ενότητα 4.12, η ταυτότητα κρυπτογράφησης της εικόνας υψηλής ανάλυσης στο Seal προκύπτει από τη συνένωση του αναγνωριστικού της δημόσιας εικόνας (`image_blob_id`) με το τυχαίο salt των 16 byte (`seal_salt`). Η επιλογή αυτή επιβάλλεται επειδή το αναγνωριστικό του έργου τέχνης δεν υπάρχει ακόμη κατά την τοπική κρυπτογράφηση στον browser.

Ο **Κώδικας 4.19** δείχνει τον υπολογισμό της ταυτότητας στο module `artwork`. Η συνάρτηση `is_bound` επαληθεύει ότι η ταυτότητα παραμένει δεσμευμένη στο σωστό έργο τέχνης μέσα στον κατάλογο. Ένα νέο έργο τέχνης με την ίδια εικόνα αλλά διαφορετικό salt εκδίδεται κανονικά (όπως κατά την επανέκδοση μετά από ακύρωση), αλλά αποκτά ανεξάρτητη ταυτότητα Seal, ώστε το κλειδί του να μην αποκρυπτογραφεί το αρχείο του προηγούμενου.

```
public fun seal_identity(a: &Artwork): vector<u8> {                    Sui Move
    identity_bytes(&a.image_blob_id, &a.seal_salt)
}

fun identity_bytes(image_blob_id: &String, salt: &vector<u8>): vector<u8> {
    let mut out = *image_blob_id.as_bytes();
    out.append(*salt);
    out
}

public fun is_bound(c: &Catalog, a: &Artwork): bool {
    c.bound_identity(a.seal_identity()) == option::some(a.id.to_inner())
}
```

Κώδικας 4.19. Η ταυτότητα Seal ενός έργου τέχνης στο module `artwork`. Τα bytes είναι το αναγνωριστικό της δημόσιας εικόνας ακολουθούμενο από το salt.

Ποιος διαβάζει το ciphertext το ορίζουν δύο πολιτικές πρόσβασης, δηλαδή δύο συναρτήσεις `seal_approve` του module `policy`, που τις τρέχει κάθε διακομιστής κλειδιών για να αποφασίσει. Η πρώτη ανοίγει στον ιδιοκτήτη ενός μη κλασματοποιημένου έργου τέχνης και η δεύτερη σε όποιον κατέχει κλάσμα του, αφού και οι δύο ελέγξουν ότι η ταυτότητα είναι του έργου τέχνης και δεσμευμένη στον κατάλογο. Ο Seal server τρέχει την πολιτική σε dry-run και δίνει το μερίδιό του μόνο αν φτάσει στο τέλος χωρίς σφάλμα. Επειδή διαβάζει τη ζωντανή κατάσταση, η αλλαγή ιδιοκτησίας μεταβάλλει το δικαίωμα λήψης κλειδιού χωρίς επανακρυπτογράφηση. Όπως είδαμε στη θεωρία του Seal, η αλλαγή δεν αφαιρεί κλειδί ή αποκρυπτογραφημένο αρχείο που είχε ήδη αποθηκεύσει ο προηγούμενος κάτοχος.

Ο **Κώδικας 4.20** δίνει τις δύο πολιτικές. Είναι `entry` και δεν επιστρέφουν τίποτα, γιατί ο διακομιστής κλειδιών δεν διαβάζει τιμή επιστροφής, μόνο αν η εκτέλεση έφτασε στο τέλος. Στη δεύτερη το κλάσμα που περνά ως όρισμα είναι η απόδειξη κατοχής, επειδή ένα address-owned αντικείμενο μπαίνει σε συναλλαγή μόνο από τον κάτοχό του, και το dry-run κάνει τον ίδιο έλεγχο με μια κανονική συναλλαγή.

```sui move
entry fun seal_approve_full_res(
    id: vector<u8>, artwork: &Artwork, catalog: &Catalog, ctx: &TxContext,
) {
    assert!(is_for(id, artwork, catalog), EWrongIdentity);
    assert!(artwork.is_owner(ctx.sender()), ENoAccess);
}

entry fun seal_approve_full_res_holder(
    id: vector<u8>, artwork: &Artwork, catalog: &Catalog,
    fraction: &Fraction, _ctx: &TxContext,
) {
    assert!(is_for(id, artwork, catalog), EWrongIdentity);
    assert!(artwork.vault() == option::some(fraction.vault_id()), ENoAccess);
}

fun is_for(id: vector<u8>, artwork: &Artwork, catalog: &Catalog): bool {
    id == artwork.seal_identity() && catalog.is_bound(artwork)
}
```

Κώδικας 4.20. Οι δύο πολιτικές πρόσβασης του module `policy`. Η πρώτη ανοίγει το αρχείο υψηλής ανάλυσης στον ιδιοκτήτη και η δεύτερη στον κάτοχο κλάσματος.

Υλοποίηση. Ο client του Seal κρυπτογραφεί με threshold δύο προς τους τρεις διακομιστές κλειδιών, και με το πακέτο `art_tokenization` ως namespace των ταυτοτήτων. Για να ζητήσει μερίδια κλειδιού, ο χρήστης χρειάζεται ένα κλειδί συνεδρίας, που δημιουργείται στον browser, ισχύει για 30 λεπτά και εξουσιοδοτείται με ένα προσωπικό μήνυμα που υπογράφει το wallet χωρίς gas. Η αποκρυπτογράφηση του **Κώδικας 4.21** είναι τα βήματα της αποκρυπτογράφησης του **Σχήμα 3.4** από την πλευρά του browser, με τις δικές μας πολιτικές. Χτίζει το PTB με την πολιτική μόνο ως είδος συναλλαγής, χωρίς αποστολέα και gas, γιατί δεν θα υποβληθεί ποτέ. Ο client του Seal το στέλνει στους διακομιστές κλειδιών, μαζεύει δύο μερίδια και αποκρυπτο-

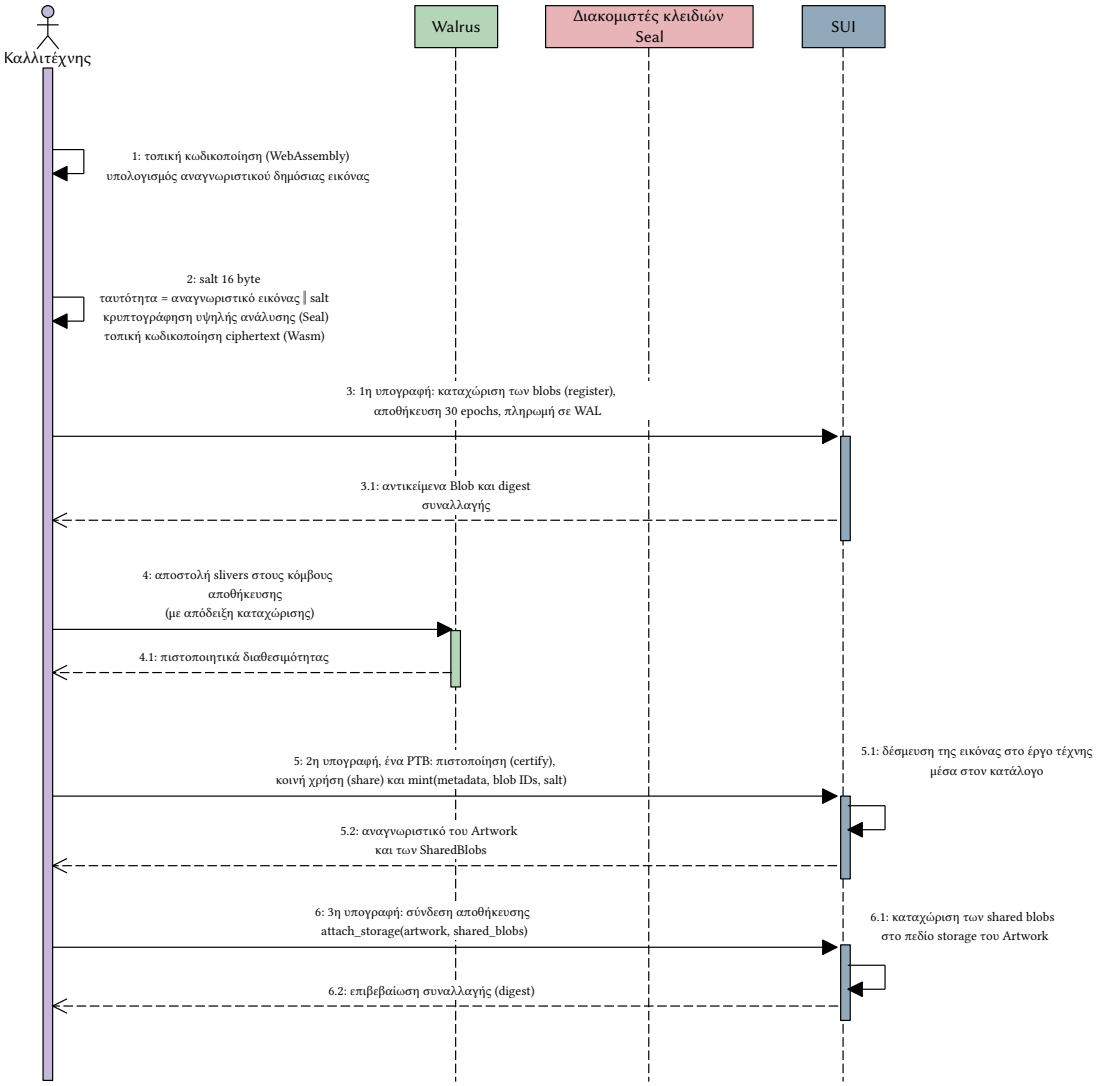
γραφεί τοπικά. Η εικόνα δεν γράφεται πουθενά, και η διεύθυνσή της στον browser ανακαλείται όταν ο χρήστης φύγει από τη σελίδα.

```typescript
const res = await fetch(config.walrus.url(blobId));
const data = new Uint8Array(await res.arrayBuffer());
const sessionKey = await sessionKeyFor(address);

const tx = new Transaction();
const id = tx.pure.vector('u8', identityBytes(imageBlobId, fromHex(sealSalt)));
tx.moveCall({
  target: `${config.latestPackageId}::policy::seal_approve_full_res_holder`,
  arguments: [
    id, tx.object(artworkId), tx.object(config.objects.catalog),
    tx.object(fractionId),
  ],
});
const txBytes = await tx.build({ client: suiClient, onlyTransactionKind: true });
return await sealClient.decrypt({ data, sessionKey, txBytes });
```

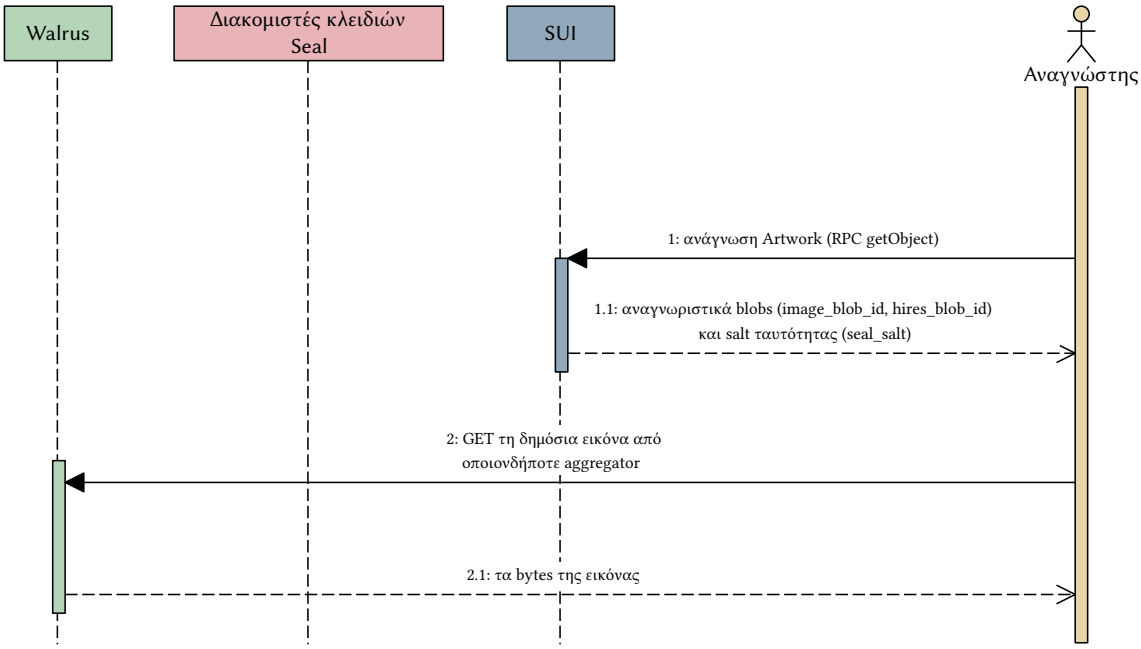
Κώδικας 4.21. Η αποκρυπτογράφηση της εικόνας υψηλής ανάλυσης από κάτοχο κλάσματος, απλοποιημένη. Το PTB με την πολιτική χτίζεται χωρίς αποστολέα και gas, και το στέλνει ο client του Seal στους διακομιστές κλειδιών.

Τα διαγράμματα ακολουθίας των **Σχήμα 4.13** και **Σχήμα 4.14** συνοψίζουν τη διαδρομή και των δύο μισών της ενότητας, δηλαδή τι γράφει ο καλλιτέχνης μία φορά στην έκδοση και τι διαβάζει ο καθένας μετά. Ανάμεσα στους δύο δεν μεσολαβεί κανένας δικός μας server, και την απόφαση της πρόσβασης την παίρνουν οι διακομιστές κλειδιών, τρέχοντας την πολιτική του συμβολαίου πάνω στη ζωντανή κατάσταση.

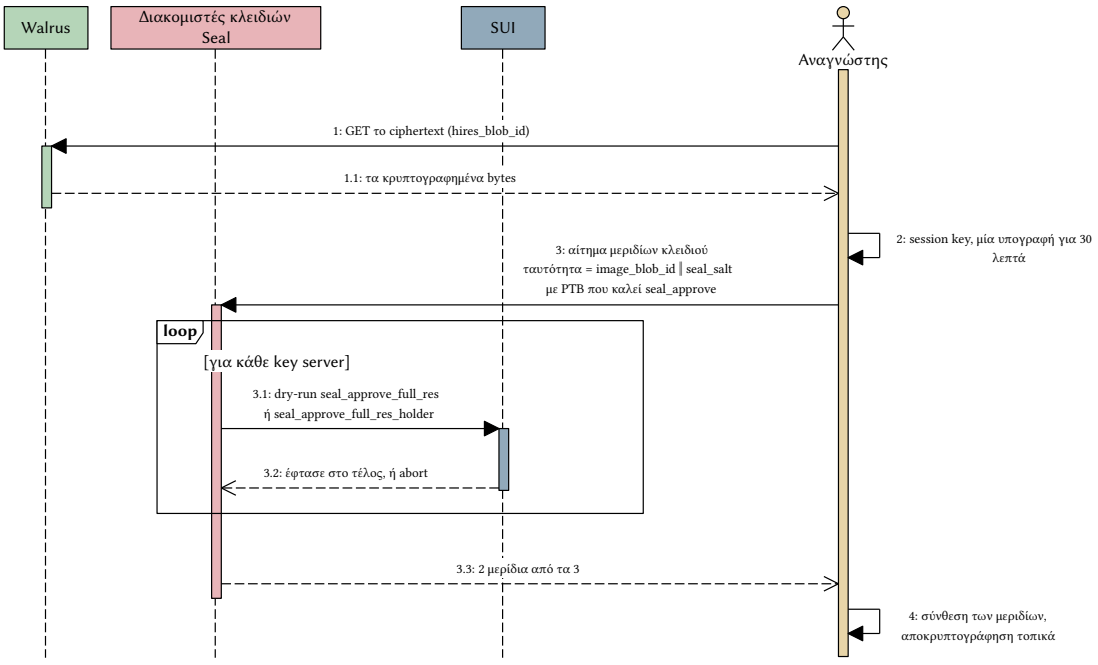


Σχήμα 4.13. Διάγραμμα ακολουθίας της έκδοσης έργου τέχνης και αποθήκευσης των αρχείων στο Walrus. Τοπική κωδικοποίηση και κρυπτογράφηση στον browser, καταχώριση των blobs, αποστολή slivers στους κόμβους αποθήκευσης και πιστοποίηση στο ίδιο PTB με την έκδοση του αντικειμένου `Artwork`.

α) Ανάγνωση δημόσιας εικόνας



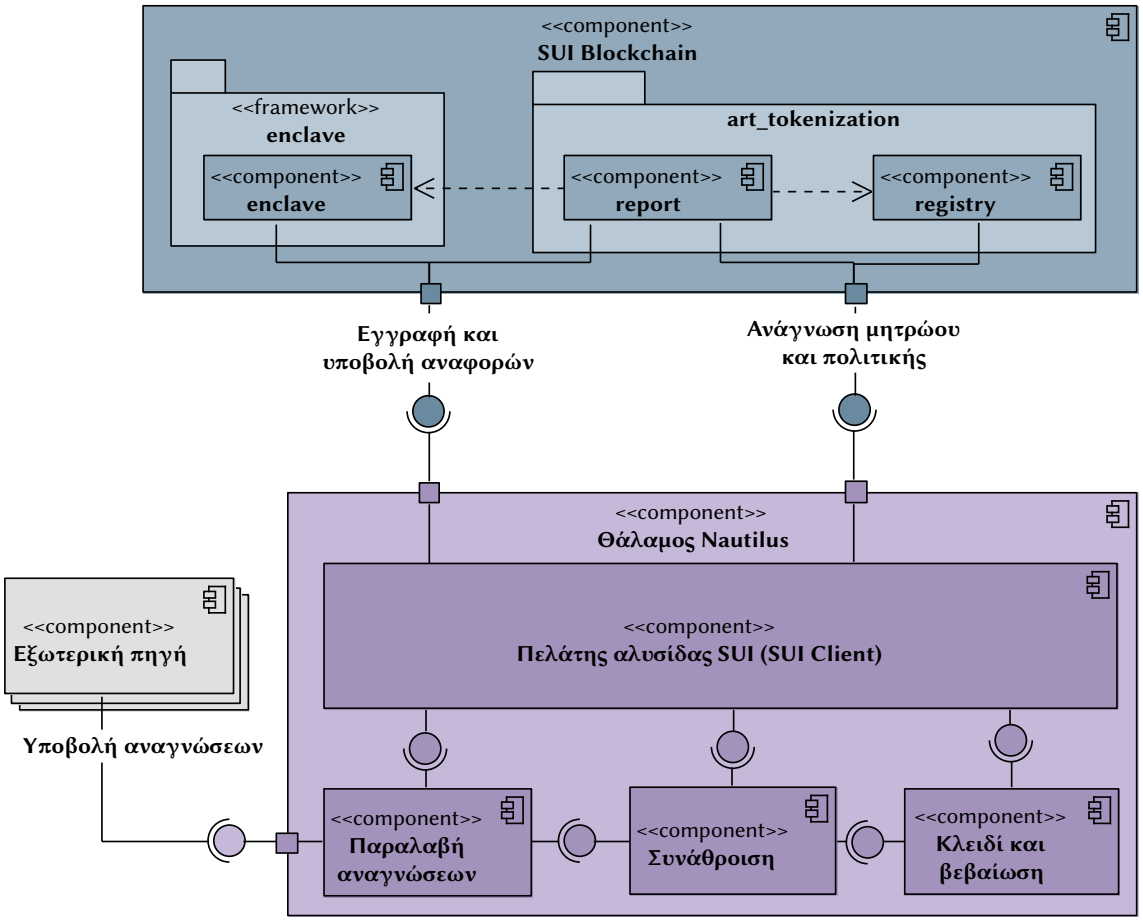
β) Ανάγνωση και αποκρυπτογράφηση υψηλής ανάλυσης με Seal



Σχήμα 4.14. Διαγράμματα ακολουθίας για την ανάγνωση αρχείων από το Walrus: (α) Ανάγνωση της δημόσιας εικόνας μέσω RPC ανάγνωσης του αντικειμένου `Artwork` και ανάκτησης του blob. (β) Ανάγνωση και τοπική αποκρυπτογράφηση της εικόνας υψηλής ανάλυσης από τον κάτοχο του κλάσματος μέσω λήψης μεριδίων από τους διακομιστές κλειδιών του Seal με dry-run εκτέλεση της πολιτικής στην αλυσίδα.

4.17 Σχεδιασμός του oracle με Nautilus

Οι αναφορές κατάστασης παράγονται από εξωτερικές πηγές (αισθητήρες IoT ή επιθεωρητές) και εισάγονται στο blockchain δίκτυο μέσω του oracle της εφαρμογής, υλοποιώντας την αρχιτεκτονική επαληθεύσιμου υπολογισμού της ενότητας 2.3.5. Το **Σχήμα 4.15** δείχνει τα συστατικά του συστήματος και τις εξωτερικές πηγές που το τροφοδοτούν.



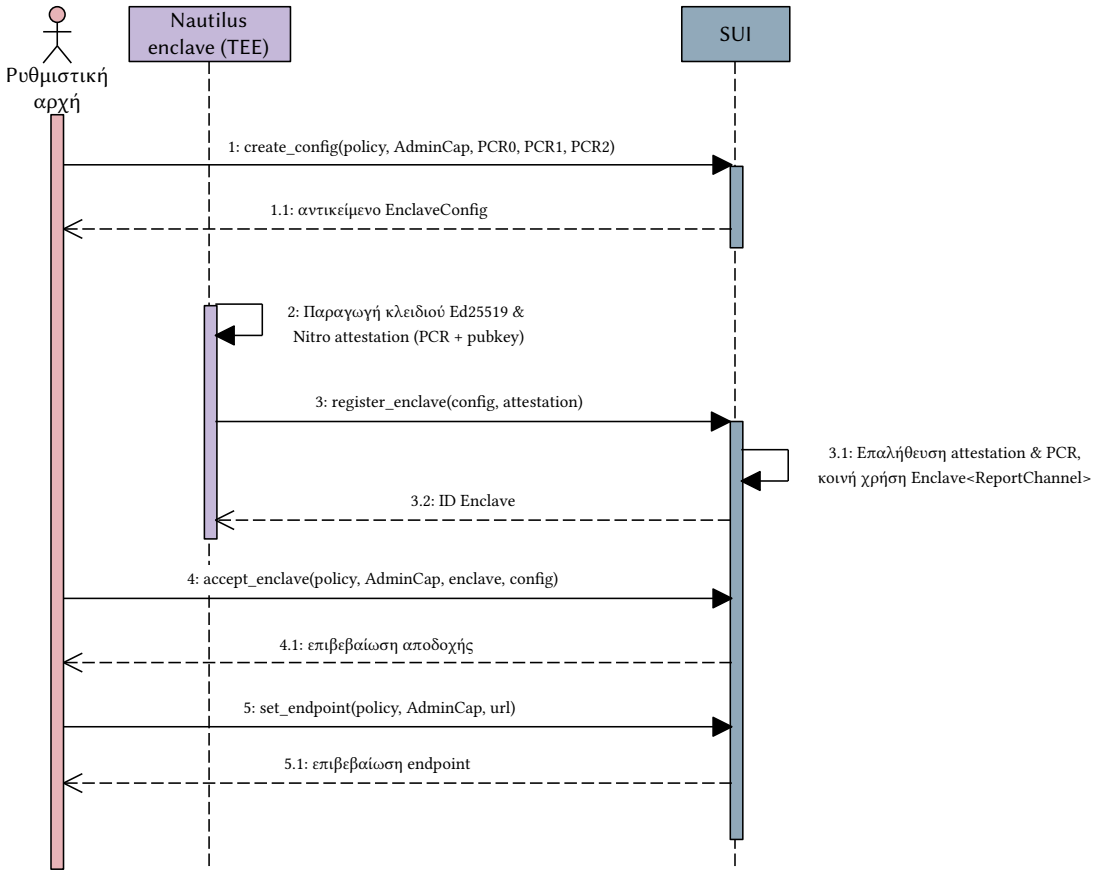
Σχήμα 4.15. Το διάγραμμα συστατικών του oracle.

Στο **Σχήμα 4.15** παρακολουθούμε τη ροή των αναφορών από τις εξωτερικές πηγές προς το enclave και από εκεί προς το SUI. Οι πηγές δεν υποβάλλουν συναλλαγές, αλλά μπορούν να διαβάζουν την αλυσίδα. Ο προσομοιωτής μας χρησιμοποιεί GraphQL για να ανακτήσει, μεταξύ άλλων, την πολιτική και το endpoint του oracle. Αυτή η ξεχωριστή σύνδεση ανάγνωσης δεν αποτυπώνεται στο διάγραμμα συστατικών, το οποίο εστιάζει στη διαδρομή των αναφορών.

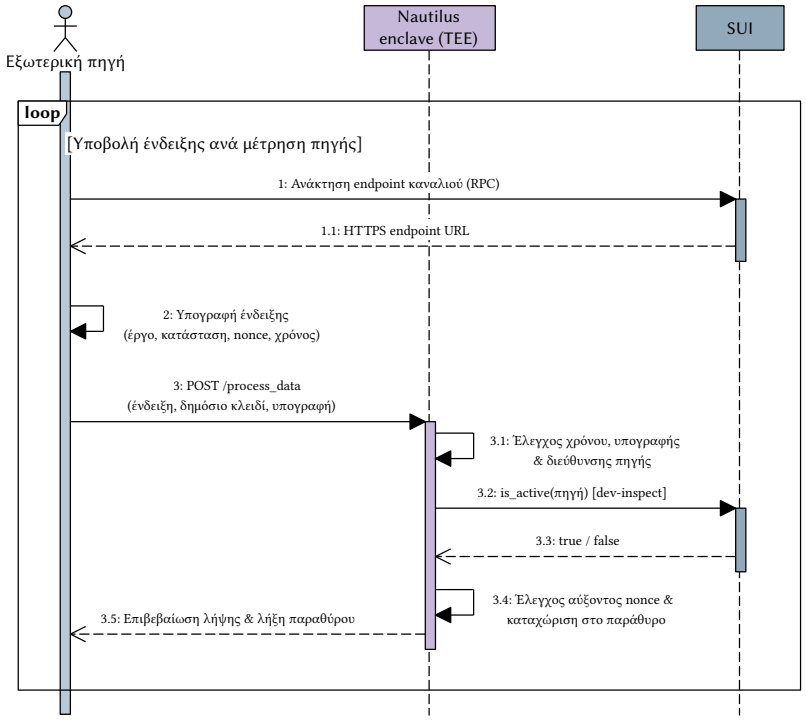
Στην πλευρά του SUI, το module `report` δέχεται την τελική αναφορά και το `registry` ελέγχει αν η πηγή είναι ενεργή. Το module `enclave` του Nautilus διατηρεί την εγγραφή και το δημόσιο κλειδί του enclave. Με αυτό το κλειδί ελέγχεται η υπογραφή της αναφοράς πριν από την καταγραφή.

Ο εσωτερικός σχεδιασμός του enclave περιλαμβάνει τέσσερα κύρια συστατικά. Η παραλαβή αναφορών επαληθεύει την υπογραφή κάθε ανάγνωσης και ελέγχει αν η πηγή είναι ενεργή στο μητρώο. Ο client αλυσίδας ανακτά το αποτέλεσμα του ελέγχου, το οποίο κρατάμε σε cache για 60 δευτερόλεπτα. Η συνάθροιση κρατά τα παράθυρα ανά ζεύγος πηγής και έργου τέχνης και στο κλείσιμο κάθε παραθύρου παράγει μία τελική αναφορά. Το κλειδί είναι η ταυτότητα του enclave. Υπογράφει κάθε τελική αναφορά, ώστε το συμβόλαιο να την αποδώσει σε αυτό, και δίνει το attestation document με το οποίο το enclave εγγράφεται μία φορά στο συμβόλαιο. Τέλος, ο client αλυσίδας είναι η μόνη έξοδος προς το blockchain δίκτυο, για την ανάγνωση του μητρώου και της πολιτικής, την εγγραφή και την υποβολή των αναφορών.

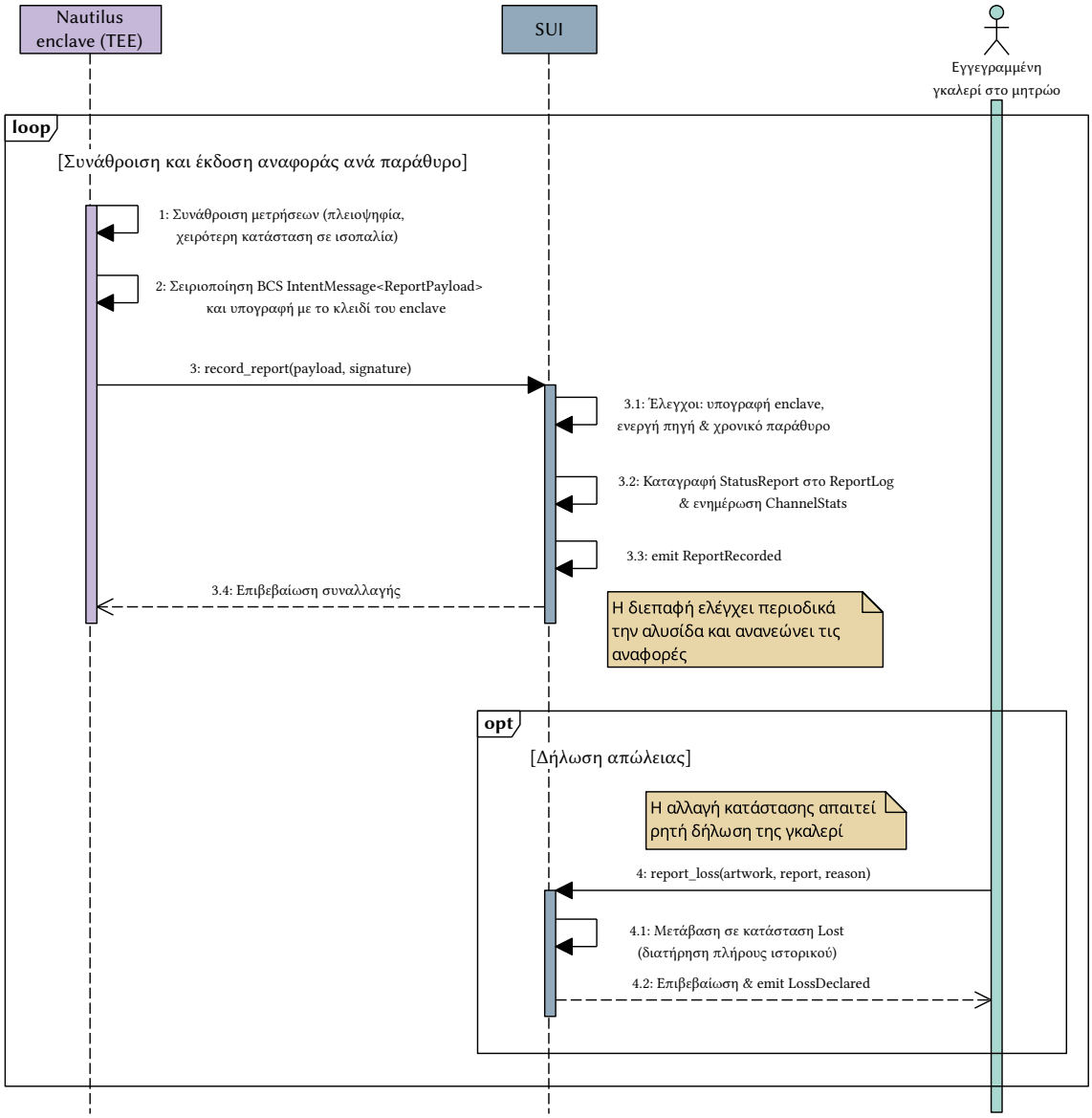
Τα διαγράμματα ακολουθίας των **Σχήμα 4.16**, **Σχήμα 4.17** και **Σχήμα 4.18** αποτυπώνουν τη διαδρομή μιας αναφοράς κατάστασης σε τρεις φάσεις. Αρχικά, γίνεται η εγγραφή και η ρύθμιση του enclave (**Σχήμα 4.16**). Μετά, οι πηγές υποβάλλουν τις αναφορές τους (**Σχήμα 4.17**). Τέλος, πραγματοποιείται η περιοδική συνάθροιση του παραθύρου και η καταγραφή της αναφοράς στην αλυσίδα (**Σχήμα 4.18**).



Σχήμα 4.16. Διάγραμμα ακολουθίας της αρχικής εγγραφής και ρύθμισης του enclave. Η ρυθμιστική αρχή ορίζει τις αποδεκτές τιμές PCR στο `EnclaveConfig`, το enclave παράγει το εφήμερο ζεύγος Ed25519 και το Nitro attestation document, και η αρχή ενεργοποιεί το καταχωρημένο αντικείμενο ορίζοντας το endpoint.



Σχήμα 4.17. Διάγραμμα ακολουθίας υποβολής αναφορών από εξωτερική πηγή. Η πηγή υπογράφει την αναφορά και το enclave ελέγχει την ενεργή κατάστασή της. Η ανάγνωση του μητρώου στα βήματα 3.2 και 3.3 απαιτείται όταν δεν υπάρχει πρόσφατο αποτέλεσμα στην cache των 60 δευτερολέπτων.



Σχήμα 4.18. Διάγραμμα ακολουθίας συνάθροισης και καταγραφής αναφοράς. Το enclave υπογράφει και καταγράφει την αναφορά στην αλυσίδα, ενώ σε ανεπανόρθωτη φθορά η γκαλερί δηλώνει απώλεια.

4.17.1 Προδιαγραφές πηγών και ο προσομοιωτής

Οι πηγές δεν είναι μέρος της σχεδίασης του oracle, τις θεωρούμε εξωτερικό σύστημα. Μπορεί να είναι συσκευές, επιθεωρητές με μια εφαρμογή ή τρίτα συστήματα, και το oracle δεν ενδιαφέρεται για το εσωτερικό τους. Ενδιαφέρεται μόνο για τους όρους με τους οποίους το enclave δέχεται τις αναφορές μιας πηγής. Οι απαιτήσεις αυτές είναι τρεις:

1. **Ένα ζεύγος κλειδιών Ed25519**, με το οποίο η πηγή υπογράφει κάθε αναφορά της. Ειδικότερα, η υπογραφή πιστοποιεί ότι την αναφορά την παρήγαγε ο κάτοχος του ιδιωτικού κλειδιού και ότι κανείς δεν την άλλαξε στη διαδρομή μέχρι το enclave. Ως εκ τούτου, κάθε αναφορά αποδίδεται σε συγκεκριμένη πηγή, και κατ' επέκταση η πηγή δεν μπορεί αργότερα

να την αρνηθεί. Η πηγή δεν στέλνει τίποτε στο blockchain δίκτυο η ίδια. Αντίθετα, παραδίδει μόνο την υπογεγραμμένη αναφορά στο enclave.

2. **Ένα όνομα στο μητρώο.** Συγκεκριμένα, είναι η διεύθυνση που αντιστοιχεί στο δημόσιο κλειδί της κατά τον κανόνα του SUI, `blake2b256(0x00 || public_key)`. Επειδή η πηγή δεν κατέχει τίποτε σε αυτή τη διεύθυνση ούτε συναλλάσσεται από αυτήν, η διεύθυνση αποτελεί αποκλειστικά ένα όνομα. Άλλωστε, το όνομα δεν αποδεικνύει τίποτε από μόνο του, γιατί το δημόσιο κλειδί είναι δημόσιο και κατά συνέπεια τη διεύθυνση μπορεί να την υπολογίσει οποιοσδήποτε. Αντίθετα, την πραγματική ταυτότητα την αποδεικνύει η υπογραφή της αναφοράς. Παρόλα αυτά, η χρήση αυτού του κανόνα παραγωγής ονόματος έχει δύο πλεονεκτήματα. Αφενός, δίνει στο μητρώο πηγών τον ίδιο τύπο κλειδιού με το μητρώο γκαλερί, και αφετέρου επιτρέπει σε οποιονδήποτε να επιβεβαιώσει τη διεύθυνση με τα εργαλεία του SUI. Με αυτό το όνομα η αρχή εγγράφει την πηγή όπως εγγράφει μια γκαλερί, και ομοίως την ανακαλεί, με ισχύ από την επόμενη συναλλαγή.

3. **Ένα σταθερό περιεχόμενο αναφοράς.** Το αίτημα περιέχει ακριβώς έξι πεδία, όπως φαίνεται στο **Κώδικας 4.22**. Συγκεκριμένα, μεταφέρει το έργο τέχνης, την κατάσταση, έναν αύξοντα nonce και τον χρόνο παρατήρησης. Παράλληλα, περιλαμβάνει το δημόσιο κλειδί της πηγής και την υπογραφή της πάνω σε αυτά τα τέσσερα δεδομένα.

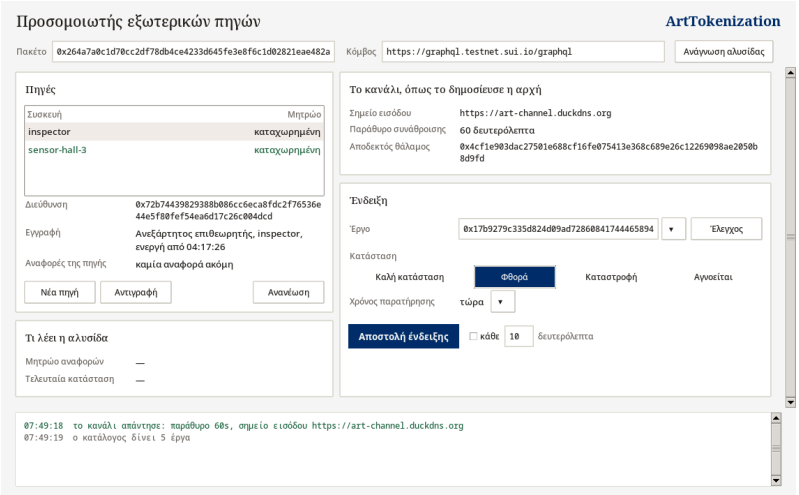
```json
{
  "payload": {
    "public_key":  "197f6b23e16c8532c6abc838facd5ea789be0c76b2920334039bfa8b3d368d61",
    "artwork":     "0x01c5e4703d8ab941c77c53ac590c7764b9e6f197cd1bb6692613a1d3df9a2564",
    "condition":   1,
    "nonce":       7,
    "observed_at": 1700000000000,
    "signature":   "1f30c03f5409235e52a31cd928069a77...232703f4db1b0b8e77d109bdecb2d90f"
  }
}
```

Κώδικας 4.22. Το αίτημα μιας αναφοράς κατάστασης, όπως το παράγει η συσκευή-πηγή. Η διεύθυνση της πηγής δεν είναι πεδίο του, την παράγει το enclave από το δημόσιο κλειδί.

Οι πηγές είναι ανεξάρτητες μεταξύ τους. Κάθε παράθυρο και κάθε αναφορά ανήκει σε μία πηγή. Αν δύο πηγές διαφωνούν για το ίδιο έργο τέχνης, το log δείχνει δύο αναφορές, και η κρίση μένει στη γκαλερί.

Στην πειραματική διάταξη προσομοιώνουμε τις εξωτερικές πηγές με μια εφαρμογή Python, τον προσομοιωτή πηγών (**Σχήμα 4.19**). Με μόνη είσοδο το αναγνωριστικό του πακέτου, η εφαρμογή ανακτά δυναμικά από το blockchain δίκτυο μέσω GraphQL το σημείο εισόδου του καναλιού, το παράθυρο συνάθροισης και τον αποδεκτό enclave. Μέσα από τη διεπαφή δημιουργούμε νέες συσκευές, παράγοντας τοπικά το ζεύγος κλειδιών Ed25519 στον φάκελο `~/.art-tokenization/devices/` και υπολογίζοντας την αντίστοιχη διεύθυνση SUI. Τη διεύθυνση αυτή η ρυθμιστική αρχή την εγγράφει από τη σελίδα διοίκησης στο μητρώο πηγών,

με επωνυμία και είδος, όπως ακριβώς εγγράφει και μια γκαλερί (ΛΑ-17). Για κάθε μέτρηση, ο προσομοιωτής υπογράφει την αναφορά και αποστέλλει στο σημείο εισόδου του enclave το αίτημα του **Κώδικας 4.22**.



Σχήμα 4.19. Ο προσομοιωτής εξωτερικών πηγών.

4.17.2 Επαλήθευση των αναφορών

Αφού η εξωτερική πηγή παράγει την αναφορά κατάστασης, το αίτημα φτάνει με POST στο σημείο εισόδου που διάβασε η πηγή από την πολιτική του καναλιού αναφορών. Ειδικότερα, στη γονική μηχανή ο reverse proxy εκθέτει μόνο αυτό το endpoint, με όριο 8 KB, και το προωθεί μέσω vsock στο enclave. Αντίθετα, τα υπόλοιπα endpoints του server (`/status` για τη ζωντανή κατάσταση και `/close_window` για χειροκίνητο κλείσιμο παραθύρου) μένουν προσβάσιμα μόνο τοπικά μέσω SSH στη γονική μηχανή για ανάγκες εποπτείας. Μέσα στο enclave, η παραλαβή ξεκινά από τη συνάρτηση του **Κώδικας 4.23**. Αρχικά, το enclave εκτελεί τους φθηνούς ελέγχους, δηλαδή της κατάστασης και του χρόνου παρατήρησης. Απορρίπτει αναφορές παλαιότερες των 10 λεπτών ή με χρόνο περισσότερο από 60 δευτερόλεπτα μπροστά από το ρολόι του enclave. Η ανοχή αυτή καλύπτει μικρές αποκλίσεις ρολογιών και ελέγχεται πριν από οποιαδήποτε κρυπτογραφική πράξη. Έπειτα ανασυνθέτει τα δεδομένα της αναφοράς, επαληθεύει την υπογραφή, και μόνο τότε παράγει τη διεύθυνση της πηγής από το δημόσιο κλειδί.

```rust
fn authenticate(
    request: ReportRequest, now: u64,
) -> Result<([u8; 32], [u8; 32], Reading), EnclaveError> {
    let ReportRequest {
        public_key, artwork, condition, nonce, observed_at, signature,
    } = request;
    if condition > CONDITION_MISSING { return Err(err("unknown condition")); }
    if observed_at > now + CLOCK_SKEW_MS { return Err(err("from the future")); }
    if observed_at + READING_MAX_AGE_MS < now { return Err(err("too old")); }
    let artwork = parse_address("artwork", &artwork)?;
    let public_key = Ed25519PublicKey::from_bytes(&parse_hex("public_key", &public_key, 32)?)
        .map_err(|e| err(format!("public_key: {e}")))?;
    let signature = Ed25519Signature::from_bytes(&parse_hex("signature", &signature, 64)?)
        .map_err(|e| err(format!("signature: {e}")))?;
    let message = bcs::to_bytes(&DeviceReading {
        domain: DEVICE_READING_DOMAIN, artwork, condition, nonce, observed_at,
    })
    .map_err(err)?;
    public_key
        .verify(&message, &signature)
        .map_err(|_| err("the reading is not signed by public_key"))?;
    Ok((sui_address(&public_key), artwork,
        Reading { condition, nonce, observed_at, received_at: now }))
}

fn sui_address(public_key: &Ed25519PublicKey) -> [u8; 32] {
    let mut flagged = vec![ED25519_FLAG];
    flagged.extend_from_slice(public_key.as_bytes());
    Blake2b256::digest(&flagged).digest
}
```

Κώδικας 4.23. Η επαλήθευση μιας αναφοράς στο enclave, με συντομευμένα μηνύματα σφάλματος. Οι έλεγχοι χρόνου προηγούνται της υπογραφής, και η διεύθυνση της πηγής παράγεται από το δημόσιο κλειδί με τον κανόνα του SUI.

4.17.3 Συνάθροιση παραθύρου και έκδοση αναφοράς

Μετά την επαλήθευση, το enclave συγκεντρώνει τις αναγνώσεις σε παράθυρα ανά ζεύγος πηγής και έργου τέχνης. Κάθε παράθυρο δίνει μία τελική αναφορά, αντί για ξεχωριστή συναλλαγή ανά ανάγνωση. Η ρυθμιστική αρχή ορίζει τη διάρκεια στην αλυσίδα, από ένα δευτερόλεπτο έως μία ημέρα. Το enclave ελέγχει την πολιτική όταν λήξει η cache των 60 δευτερολέπτων, χωρίς επανεκκίνηση. Η νέα διάρκεια εφαρμόζεται στα νέα παράθυρα, ενώ όσα έχουν ανοίξει κρατούν την αρχική λήξη τους.

Ο scheduler ελέγχει ποια παράθυρα έχουν λήξει και περιμένει 5 δευτερόλεπτα μετά από κάθε κύκλο εργασίας πριν ξεκινήσει τον επόμενο. Κλείνει επίσης όσα έχουν φτάσει το όριο των 1.000 αναγνώσεων, ακόμη και πριν από την προγραμματισμένη λήξη τους. Μέχρι να κλείσει ένα παράθυρο, η παραλαβή μπορεί να προσθέσει αναγνώσεις μετά τον χρόνο λήξης του, εφόσον δεν έχει φτάσει το όριο. Επομένως, η παρούσα υλοποίηση δεν εγγυάται αυστηρά χρονικά όρια συνάθροισης. Η καθυστέρηση εξαρτάται και από τον χρόνο εκτέλεσης του scheduler και των αιτημάτων δικτύου.

Στο κλείσιμο εφαρμόζουμε την ψηφοφορία του **Κώδικας 4.24**. Επικρατεί η κατάσταση με τις περισσότερες αναγνώσεις και, σε ισοπαλία, η χειρότερη. Το αποτέλεσμα περιέχει το έργο τέχνης, την πηγή, την κατάσταση και το πλήθος των αναγνώσεων. Η συνάρτηση του Nautilus το τυλίγει σε intent message με το πεδίο πρόθεσης και τον χρόνο του enclave. Έπειτα υπογράφει τα bytes BCS ολόκληρου του μηνύματος με το εφήμερο κλειδί.

```rust
// reading.rs
// Πλειοψηφική ψηφοφορία (στην ισοπαλία επικρατεί η χειρότερη κατάσταση)
fn prevailing_condition(readings: &[Reading]) -> u8 {
    let mut counts = [0usize; CONDITION_MISSING as usize + 1];
    for r in readings {
        counts[r.condition as usize] += 1;
    }
    (0..=CONDITION_MISSING)
        .filter(|&c| counts[c as usize] > 0)
        .max_by_key(|&c| counts[c as usize])
        .unwrap_or(0)
}

// mod.rs
let payload = ReportPayload {
    artwork, source,
    condition: prevailing_condition(&readings),
    readings: readings.len() as u64,
};
let report = to_signed_response(
    &state.eph_kp, payload.clone(), now, IntentScope::StatusReport as u8,
);
```

Κώδικας 4.24. Η ψηφοφορία και η υπογραφή ενός παραθύρου. Η επικρατέστερη κατάσταση ενσωματώνεται στην αναφορά, την οποία η συνάρτηση υπογραφής του framework Nautilus τυλίγει σε intent message.

Στη συνέχεια, το enclave υποβάλλει τη συναλλαγή μόνο του. Συνθέτει τη συναλλαγή της καταγραφής, την υπογράφει με το ίδιο κλειδί ή με ένα δανεικό wallet που μόνο πληρώνει gas, και τη στέλνει στην υπηρεσία GraphQL. Σε περίπτωση που η υποβολή αποτύχει, ξαναπροσπαθεί σε κάθε κύκλο μέχρι να περάσει το παράθυρο φρεσκάδας των 5 λεπτών, οπότε την εγκαταλείπει, γιατί το συμβόλαιο θα την απέρριπτε.

4.17.4 Επαλήθευση και καταγραφή στο έξυπνο συμβόλαιο

Στο τελικό στάδιο, η αναφορά φθάνει στο συμβόλαιο μέσω της συνάρτησης του **Κώδικας 4.25**. Αρχικά, το συμβόλαιο εκτελεί τους φθηνούς ελέγχους στο μητρώο και στο έργο τέχνης. Δηλαδή, βεβαιώνει ότι η πηγή είναι ενεργή, η κατάσταση έγκυρη και η φύλαξη του έργου τέχνης ανοιχτή. Έπειτα, επιβεβαιώνει ότι η ταυτότητα και η ρύθμιση του enclave συμβαδίζουν με την τρέχουσα πολιτική.

Έπειτα ακολουθεί ο έλεγχος του χρόνου. Η αναφορά απορρίπτεται αν είναι παλαιότερη από πέντε λεπτά ή αν η χρονική σήμανσή της βρίσκεται περισσότερο από 60 δευτερόλεπτα μπροστά από το on-chain ρολόι. Πρέπει επίσης να είναι νεότερη από την προηγούμενη καταγραφή

της ίδιας πηγής στο `ReportLog` του ίδιου έργου τέχνης, ώστε να αποτραπεί η επανάληψη. Οι έλεγχοι αυτοί περιορίζουν την απόκλιση από το on-chain ρολόι, επειδή το ρολόι του enclave προέρχεται από τη μη αξιόπιστη γονική μηχανή.

Στο τελευταίο βήμα, το συμβόλαιο εξετάζει την υπογραφή, που αποτελεί και την πιο ακριβή πράξη. Ειδικότερα, ανασυνθέτει το μήνυμα και το επαληθεύει κρυπτογραφικά με το κλειδί του enclave. Ο αποστολέας της συναλλαγής δεν εξετάζεται, καθώς η αυθεντικότητα των δεδομένων διασφαλίζεται αποκλειστικά από την υπογραφή.

```sui move
public fun record_report(
    artwork: &Artwork, log: &mut ReportLog, channel: &mut ChannelStats, sources: &Registry<Source>,
    policy: &ChannelPolicy, config: &EnclaveConfig<ReportChannel>, enclave: &Enclave<ReportChannel>,
    source: address, condition: u8, readings: u64, attested_at: u64, signature: vector<u8>,
    clock: &Clock, ctx: &mut TxContext,
) {
    assert!(sources.is_active(source), ENotActiveSource);
    assert!(condition ≤ CONDITION_MISSING, EUnknownCondition);
    assert!(readings > 0, EEmptyReport);
    let artwork_id = object::id(artwork);
    assert!(log.log_artwork() == artwork_id, EWrongLog);
    assert!(custody::is_open(artwork), ECustodyClosed);
    assert!(policy.accepted_enclave.contains(&object::id(enclave)), EWrongEnclave);
    assert!(policy.config == option::some(object::id(config)), EWrongConfig);
    assert!(enclave.config_version() == config.version(), EStaleEnclave);
    let at = clock.timestamp_ms();
    assert!(at ≤ attested_at + FRESHNESS_WINDOW_MS, EStaleReport);
    assert!(attested_at ≤ at + CLOCK_SKEW_MS, EFutureReport);
    assert!(attested_at > log.last_attested_of(source), EReplayedReport);
    let payload = ReportPayload { artwork: artwork_id, source, condition, readings };
    assert!(enclave.verify_signature(REPORT_INTENT, attested_at, payload, &signature), EInvalidSignature);
    let report = StatusReport {
        id: object::new(ctx), artwork: artwork_id, source, condition, readings, attested_at, at };
    let report_id = object::id(&report);
    custody::append_report(log, report_id, source, condition, at, attested_at);
    channel.reports = channel.reports + 1;
    transfer::freeze_object(report);
}
```

Κώδικας 4.25. Η καταγραφή μιας αναφοράς κατάστασης στο module `report`, χωρίς τους μετρητές ανά πηγή και το event. Οι φθηνοί έλεγχοι προηγούνται, η υπογραφή ελέγχεται τελευταία, και η αναφορά παγώνει ως αμετάβλητο αντικείμενο (immutable object).

Μετά την καταγραφή, το συμβόλαιο εκπέμπει το `ReportRecorded`. Το frontend δεν λαμβάνει απευθείας το event ως μήνυμα. Όσο υπάρχουν ενεργές συνδρομές της διεπαφής, ελέγχει ανά 30 δευτερόλεπτα τις εκδόσεις των αντικειμένων που παρακολουθεί, μεταξύ τους το `ChannelStats`. Όταν εντοπίσει αλλαγή, ανανεώνει τα δεδομένα και τις ειδοποιήσεις. Η ολοκλήρωση συναλλαγής από την ίδια τη διεπαφή προκαλεί επίσης ανανέωση, χωρίς αναμονή για τον επόμενο περιοδικό έλεγχο.

Καμία αναφορά δεν αλλάζει μόνη της την κατάσταση του έργου τέχνης ούτε αναστέλλει τις συναλλαγές του. Για παράδειγμα, αν η αναφορά δείχνει καταστροφή ή απώλεια, η γκαλερί δηλώνει την απώλεια με δική της πράξη, χρησιμοποιώντας την αναφορά ως τεκμήριο. Με άλλα λόγια, ο ισχυρισμός της πηγής δεν επιφέρει συνέπειες από μόνος του. Τις επιβάλλει ένα υπεύθυνο και εγγεγραμμένο μέρος. Ως αποτέλεσμα, μια χαλασμένη συσκευή, μια κακόβουλη

πηγή ή ένα κλεμμένο κλειδί συσκευής δεν μπορεί να καταστρέψει ένα tokenized έργο τέχνης. Το κλεμμένο κλειδί αντιμετωπίζεται θεσμικά μέσω ανάκλησης.

Κεφάλαιο 5

Επίδειξη βασικών περιπτώσεων χρήσης

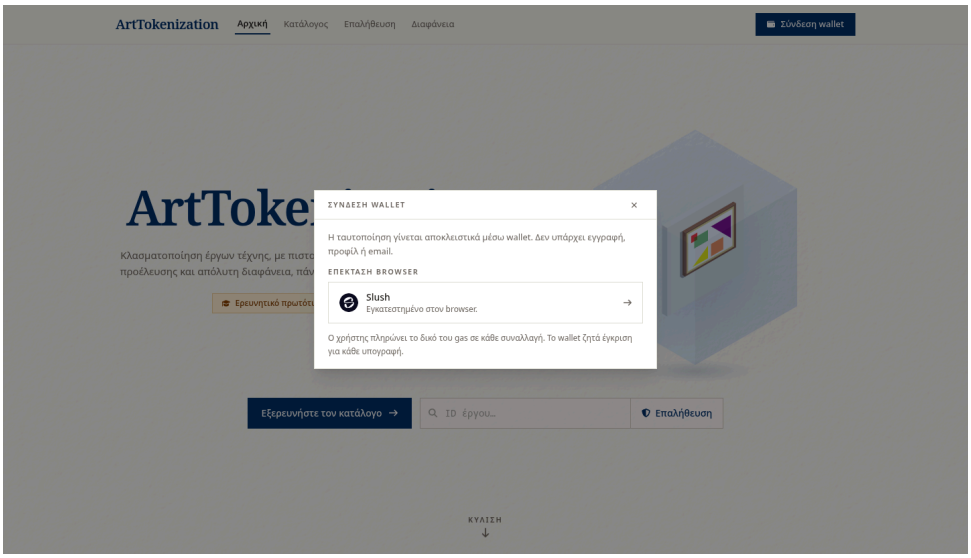
Στο κεφάλαιο αυτό παρουσιάζουμε τις βασικές περιπτώσεις χρήσης της πλατφόρμας. Αρχικά δείχνουμε τη σύνδεση του wallet και τις ενέργειες με τις οποίες η ρυθμιστική αρχή θέτει σε λειτουργία την αγορά. Στη συνέχεια παρακολουθούμε τον κύκλο ζωής ενός έργου τέχνης, ξεκινώντας από την έκδοση, τη φύλαξη και την πιστοποίησή του. Έπειτα παρουσιάζουμε την κλασματοποίηση, την αγορά κλασμάτων και την καταγραφή της φυσικής του κατάστασης. Τέλος εξετάζουμε τις ειδοποιήσεις, την αναζήτηση στον κατάλογο, τη δημόσια επαλήθευση και την πρόσβαση στο αρχείο υψηλής ανάλυσης.

Πίνακας 5.1. Οι βασικές περιπτώσεις χρήσης, οι λειτουργικές απαιτήσεις που καλύπτουν, και οι ρόλοι που συμμετέχουν.

Ενότητα	Περίπτωση χρήσης	Απαιτήσεις	Ρόλοι
5.1	Σύνδεση wallet	ΛΑ-01	Όλοι
5.2	Αρχική ρύθμιση της αγοράς	ΛΑ-17, ΛΑ-19	Ρυθμιστική αρχή
5.3	Έκδοση, φύλαξη και πιστοποίηση	ΛΑ-06, ΛΑ-07, ΛΑ-09, ΛΑ-10	Καλλιτέχνης, Gallery A, Αρχή
5.4	Κλασματοποίηση και ενοποίηση	ΛΑ-12, ΛΑ-14	Καλλιτέχνης, Επενδυτής
5.5	Αγορά κλασμάτων	ΛΑ-13	Καλλιτέχνης, Επενδυτές
5.6	Αναφορές κατάστασης και απώλεια	ΛΑ-11, ΛΑ-15	Oracle, Gallery A
5.7	Ειδοποιήσεις ανά ρόλο	ΛΑ-05	Gallery A, Επενδυτής
5.8	Πλοήγηση και αναζήτηση	ΛΑ-02	Επισκέπτης
5.9	Δημόσια επαλήθευση και προέλευση	ΛΑ-03, ΛΑ-04, ΛΑ-07	Επισκέπτης, Επενδυτής

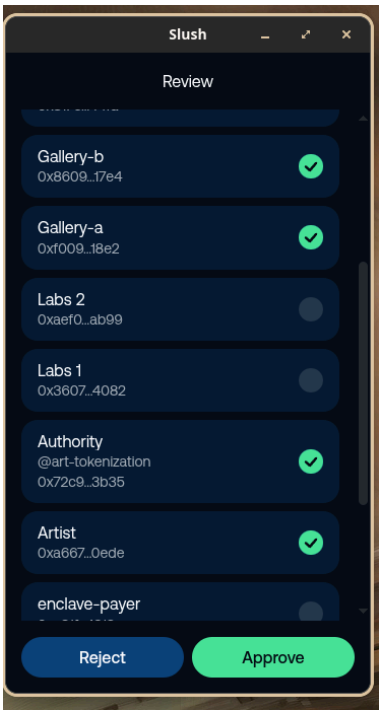
5.1 Σύνδεση wallet

Η αλληλεπίδραση ξεκινά με τη σύνδεση ενός πορτοφολιού συμβατού με το Wallet Standard (π.χ. Slush). Η εφαρμογή αναγνωρίζει τη δημόσια διεύθυνση και αντλεί αυτόματα τον ρόλο του χρήστη από τα μητρώα της αλυσίδας.

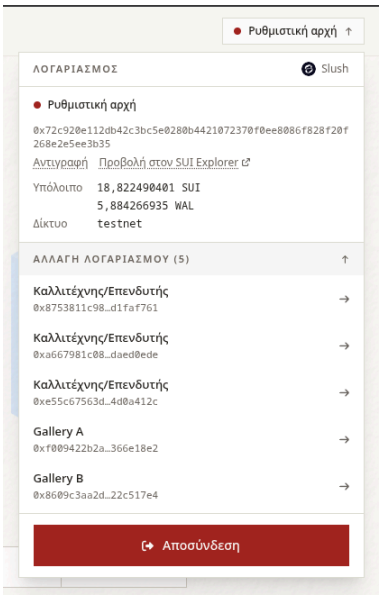


Σχήμα 5.1. Το παράθυρο σύνδεσης με τα εντοπισμένα wallets.

Με την ολοκλήρωση της σύνδεσης, το κεντρικό μενού προσαρμόζεται. Το αναπτυσσόμενο μενού παρέχει άμεση πληροφόρηση για τα διαθέσιμα νομίσματα και επιτρέπει την ταχεία εναλλαγή λογαριασμών.



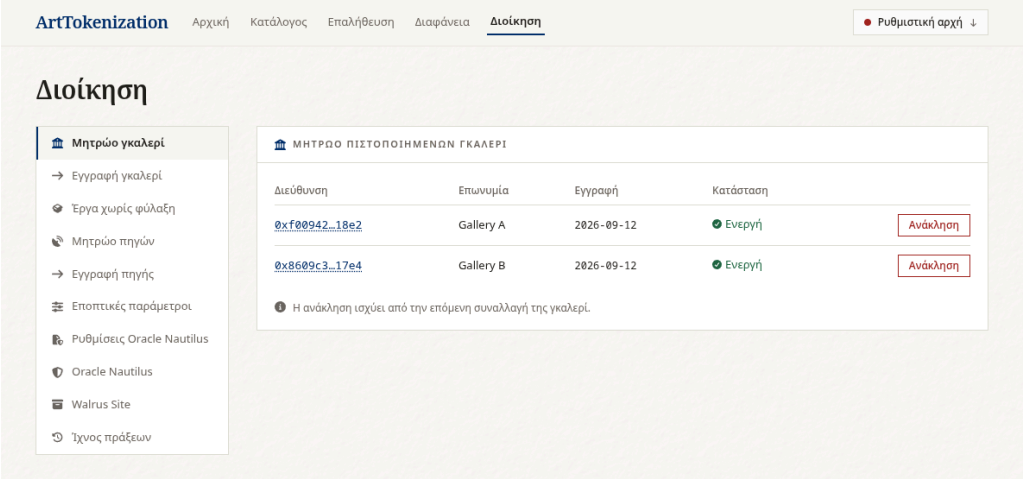
Σχήμα 5.2. Η οθόνη έγκρισης του Slush για σύνδεση.



Σχήμα 5.3. Το μενού λογαριασμού με τα υπόλοιπα.

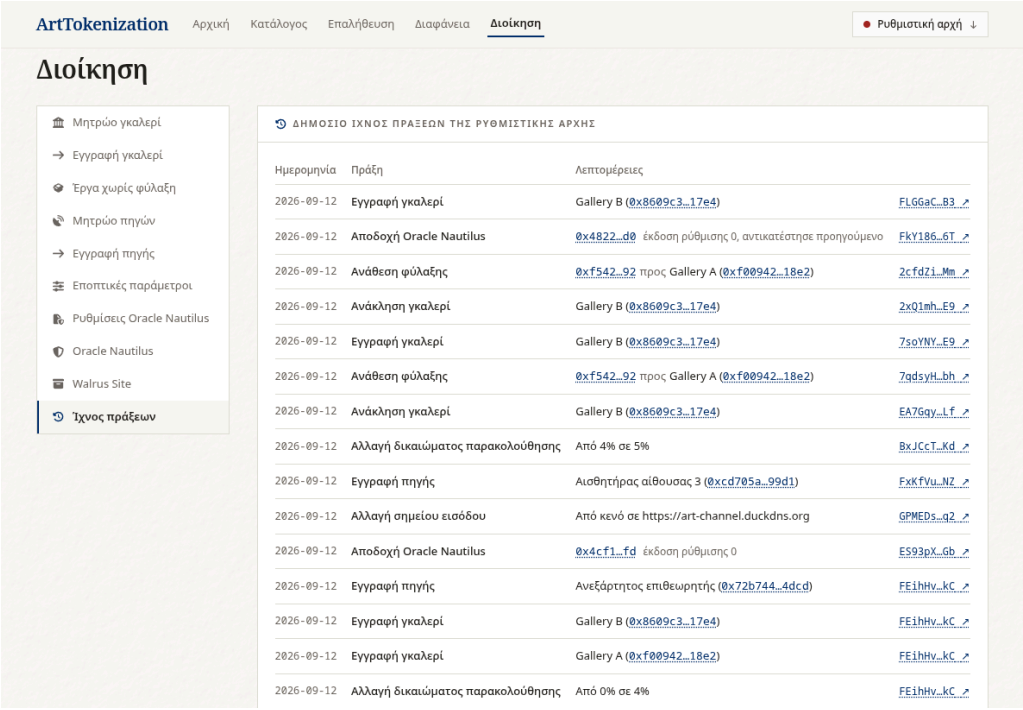
5.2 Αρχική ρύθμιση της αγοράς

Η εκκίνηση της αγοράς γίνεται από τη ρυθμιστική αρχή. Η αρχή καταχωρίζει τις διαπιστευμένες γκαλερί και τις εξωτερικές πηγές στα δημόσια μητρώα. Παράλληλα, ορίζει τις εποπτικές παραμέτρους, όπως το ποσοστό για το δικαίωμα παρακολούθησης του καλλιτέχνη.



Σχήμα 5.4. Το μητρώο διαπιστευμένων γκαλερί με την επιλογή ανάκλησης.

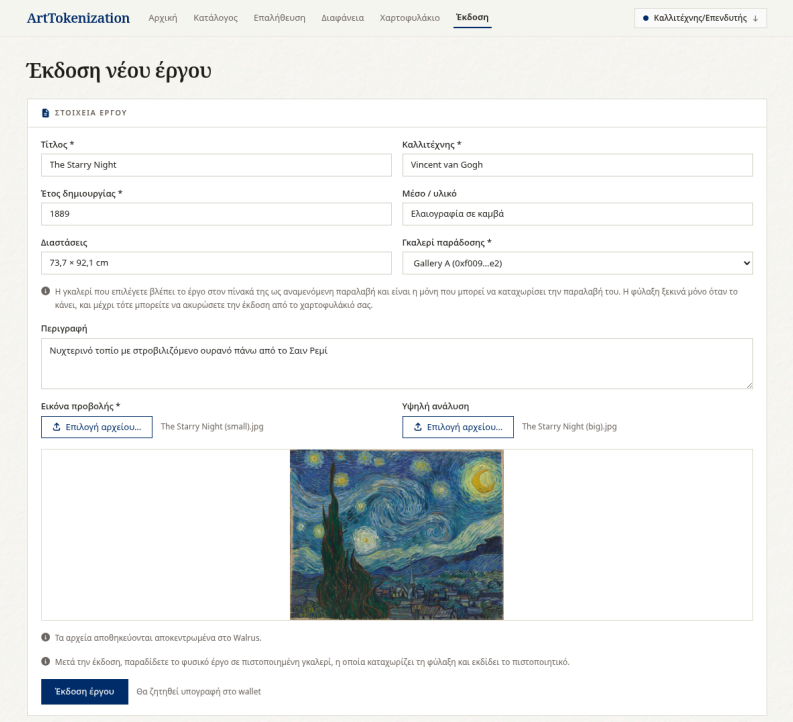
Σε περίπτωση παραβίασης όρων, η αρχή μπορεί να ανακαλέσει μια γκαλερί. Όλες οι πράξεις ελέγχου καταγράφονται μόνιμα στο ίχνος πράξεων (audit trail).



Σχήμα 5.5. Το ιστορικό εποπτικών ενεργειών με συνδέσμους συναλλαγών.

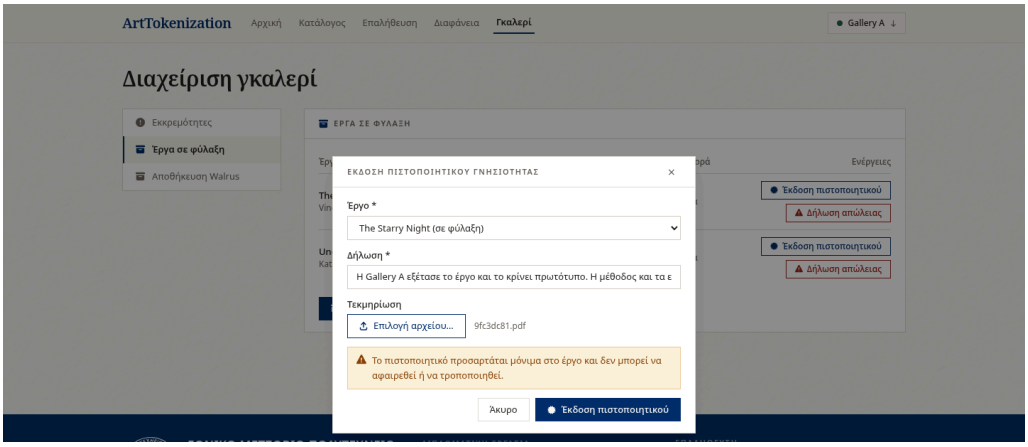
5.3 Έκδοση, φύλαξη και πιστοποίηση

Ο καλλιτέχνης εκδίδει το έργο τέχνης συμπληρώνοντας τα μεταδεδομένα του και ορίζοντας τη γκαλερί που θα το παραλάβει. Πριν από την αποθήκευση στο Walrus, η εικόνα υψηλής ανάλυσης κρυπτογραφείται τοπικά με την πολιτική πρόσβασης του Seal. Η δημόσια εικόνα και το κρυπτογραφημένο αρχείο αποθηκεύονται έπειτα στο Walrus.

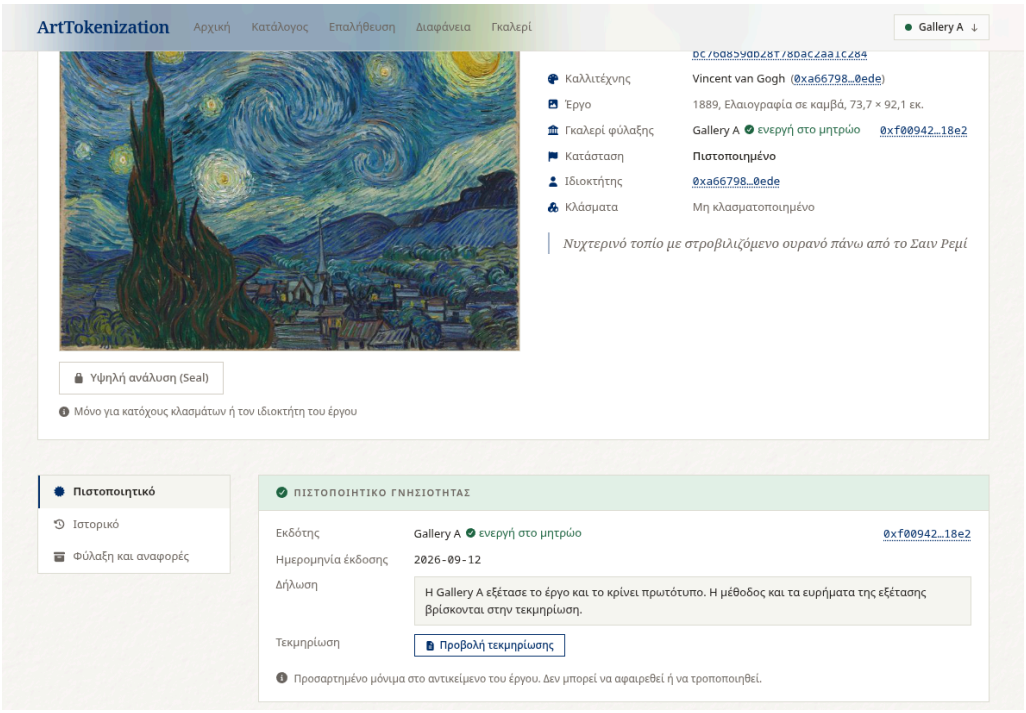


Σχήμα 5.6. Συμπλήρωση στοιχείων και επιλογή γκαλερί.

Η γκαλερί ειδοποιείται, παραλαμβάνει το φυσικό έργο τέχνης και αναρτά το ψηφιακό πιστοποιητικό γνησιότητας. Από εκείνη τη στιγμή, το έργο τέχνης εμφανίζεται ως «πιστοποιημένο» και μπορεί να κλασματοποιηθεί, ενώ η τεκμηρίωση είναι δημόσια προσβάσιμη.



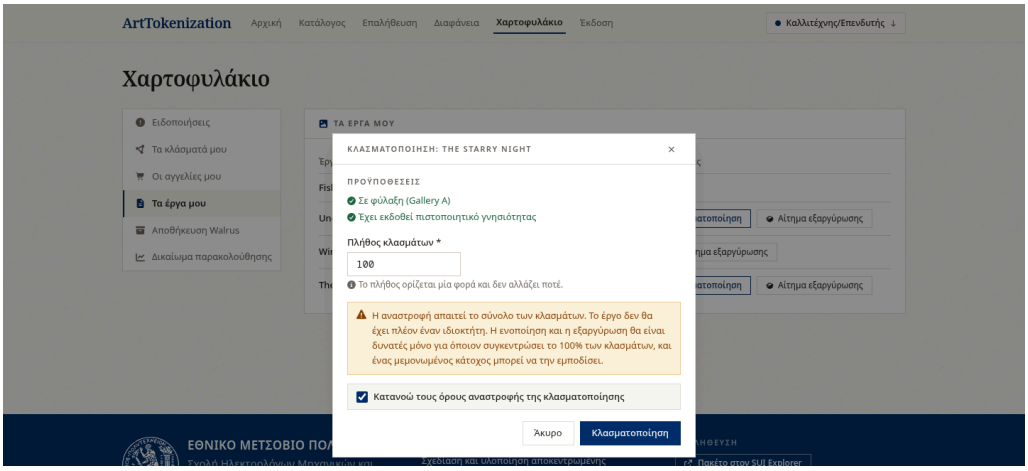
Σχήμα 5.7. Φόρμα δήλωσης γνησιότητας και ανάρτησης τεκμηρίωσης.



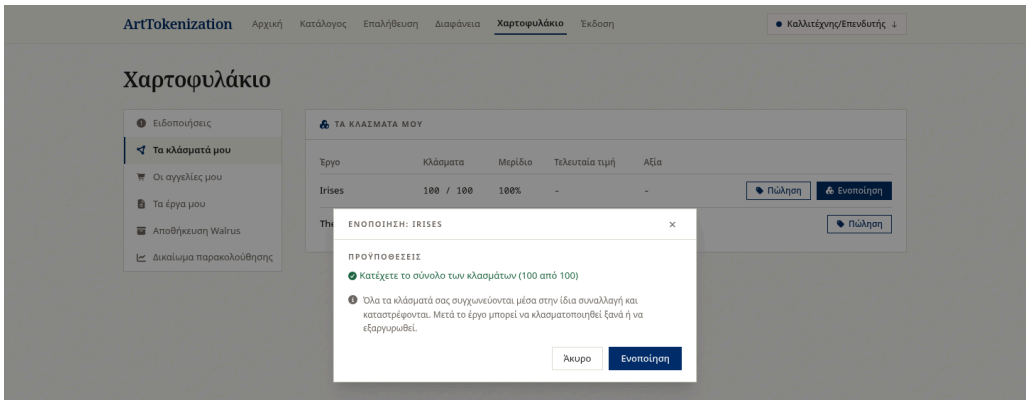
Σχήμα 5.8. Η σελίδα του έργου τέχνης μετά την πιστοποίηση.

5.4 Κλασματοποίηση και ενοποίηση

Ο ιδιοκτήτης μπορεί να διαιρέσει ένα πιστοποιημένο έργο τέχνης σε ίσα κλάσματα (π.χ. 100), τα οποία καταχωρίζονται στο χαρτοφυλάκιό του. Αντιστρόφως, εάν ένας επενδυτής συγκεντρώσει αργότερα όλα τα κλάσματα, μπορεί να τα ενοποιήσει, ανακτώντας την πλήρη κυριότητα.



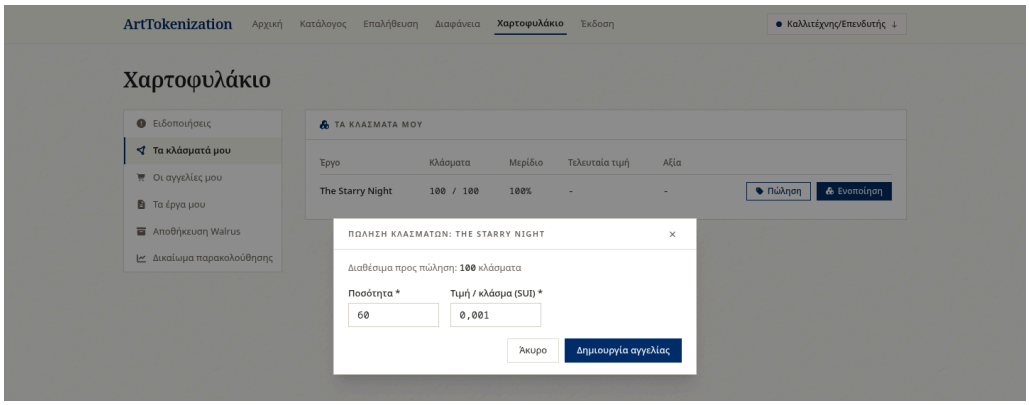
Σχήμα 5.9. Διαίρεση του έργου τέχνης σε πολλαπλά κλάσματα.



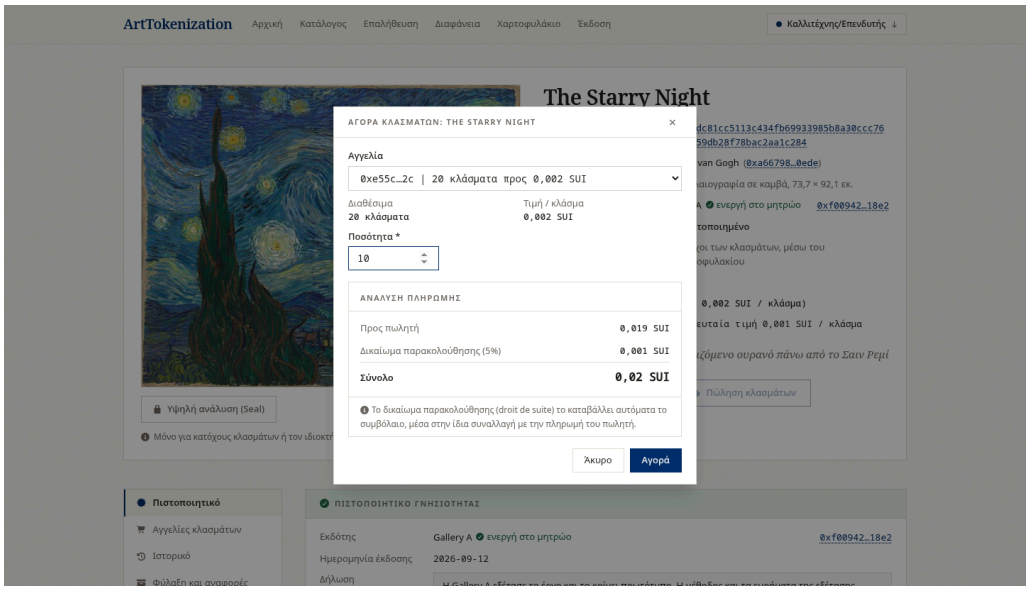
Σχήμα 5.10. Ενοποίηση των κλασμάτων και επαναφορά της ατομικής ιδιοκτησίας του έργου τέχνης.

5.5 Αγορά κλασμάτων

Η διαπραγμάτευση γίνεται μέσω αγγελιών. Ο πωλητής ορίζει ποσότητα και τιμή. Όταν εκτελείται η αγοραπωλησία, το συμβόλαιο διαιρεί αυτόματα την πληρωμή, φροντίζοντας να αποδοθεί το θεσμοθετημένο ποσοστό (δικαίωμα παρακολούθησης) στον καλλιτέχνη.



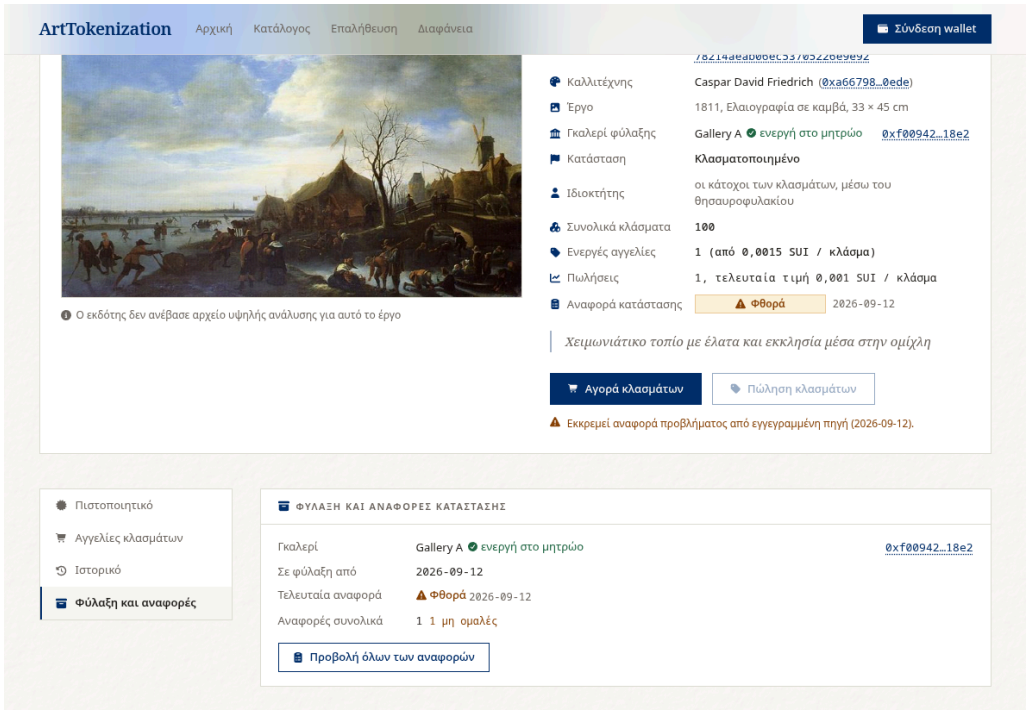
Σχήμα 5.11. Ορισμός τιμής πώλησης κλασμάτων.



Σχήμα 5.12. Μεταπώληση με αυτόματη παρακράτηση υπέρ του καλλιτέχνη.

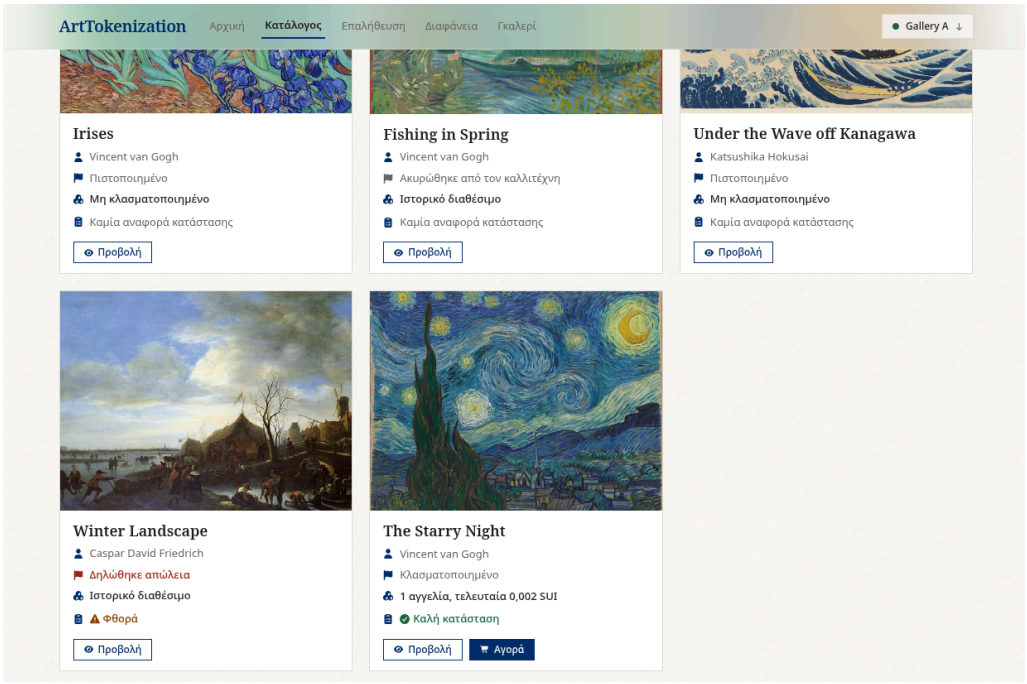
5.6 Αναφορές κατάστασης και απώλεια

Τη φυσική κατάσταση του έργου τέχνης αναφέρουν εξωτερικές πηγές, οι οποίες μπορεί να είναι συσκευές, άνθρωποι ή τρίτα συστήματα. Το oracle συγκεντρώνει τις αναγνώσεις και δημοσιεύει περιοδικές αναφορές στο blockchain δίκτυο. Οι αναφορές εμφανίζονται στη σελίδα του έργου τέχνης.



Σχήμα 5.13. Εμφάνιση της αναφοράς στη σελίδα του έργου τέχνης.

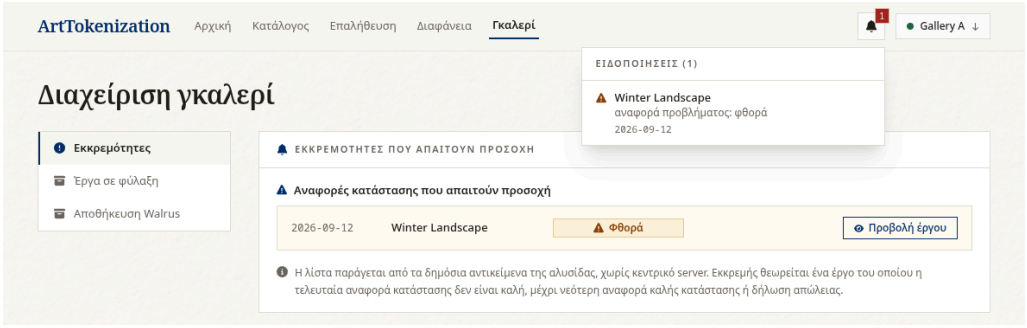
Εάν τεκμηριωθεί ανεπανόρθωτη φθορά ή κλοπή, η γκαλερί προβαίνει σε επίσημη δήλωση απώλειας. Το έργο τέχνης τίθεται σε κατάσταση «απολεσθέν» και η δευτερογενής αγορά του κλείνει, προστατεύοντας τους επενδυτές.



Σχήμα 5.14. Το απολεσθέν έργο τέχνης με απενεργοποιημένη αγορά.

5.7 Ειδοποιήσεις ανά ρόλο

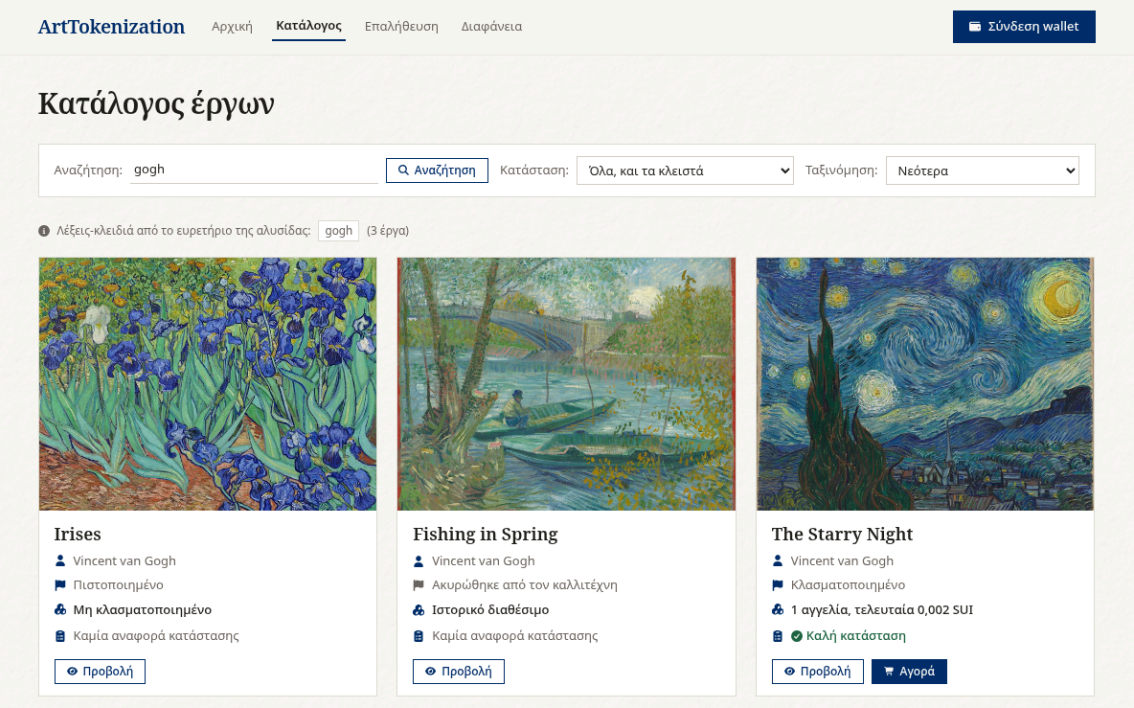
Οι χρήστες ενημερώνονται εγκαίρως για τις εξελίξεις που τους αφορούν. Χωρίς την ανάγκη κεντρικού εξυπηρετητή, η διεπαφή αντλεί τα γεγονότα απευθείας από την αλυσίδα. Για παράδειγμα, η γκαλερί ειδοποιείται όταν καταγράφεται φθορά σε έργο τέχνης που φυλάσσει.



Σχήμα 5.15. Ειδοποίηση της γκαλερί για φθορά σε φυλασσόμενο έργο τέχνης.

5.8 Πλοήγηση και αναζήτηση

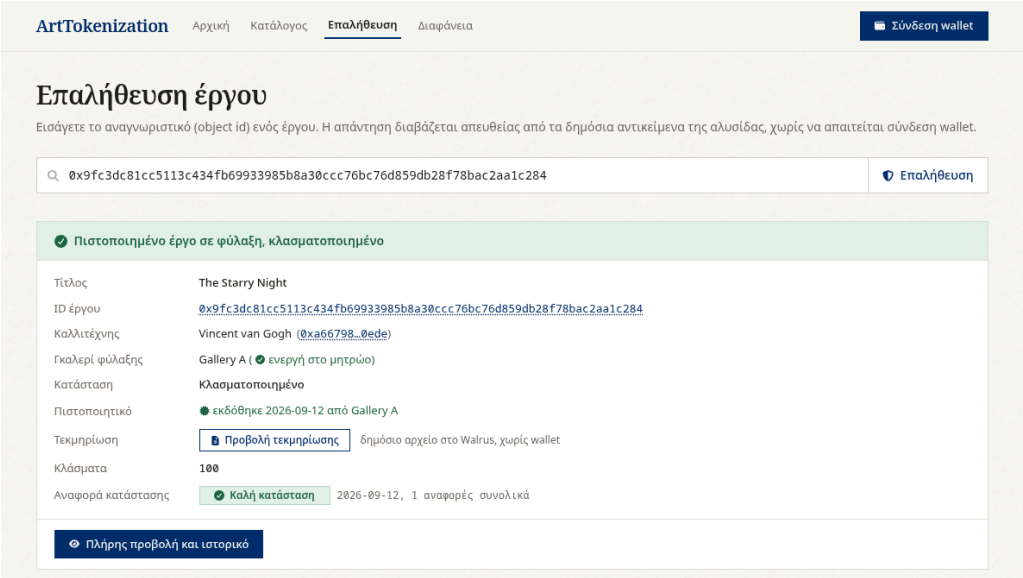
Ο δημόσιος κατάλογος επιτρέπει την άμεση αναζήτηση έργων τέχνης μέσω του index στο blockchain δίκτυο. Τα αποτελέσματα εμφανίζονται γρήγορα, και ο χρήστης μπορεί να αναζητήσει με λέξεις-κλειδιά ακόμα και αν δεν έχει συνδέσει το πορτοφόλι του.



Σχήμα 5.16. Αναζήτηση βάσει λέξης-κλειδιού μέσω του index στο blockchain δίκτυο.

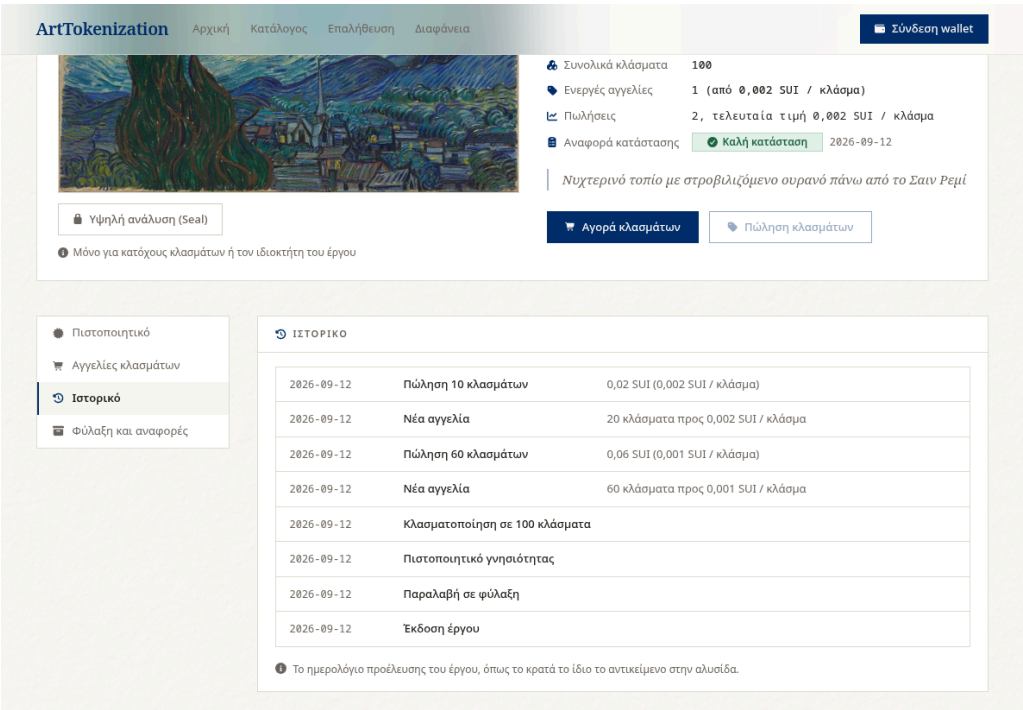
5.9 Δημόσια επαλήθευση και προέλευση

Οποιοσδήποτε μπορεί, μέσω του μοναδικού αναγνωριστικού ενός έργου τέχνης, να επαληθεύσει ποια γκαλερί εξέδωσε το πιστοποιητικό γνησιότητας και να ελέγξει το ιστορικό προέλευσης (provenance).



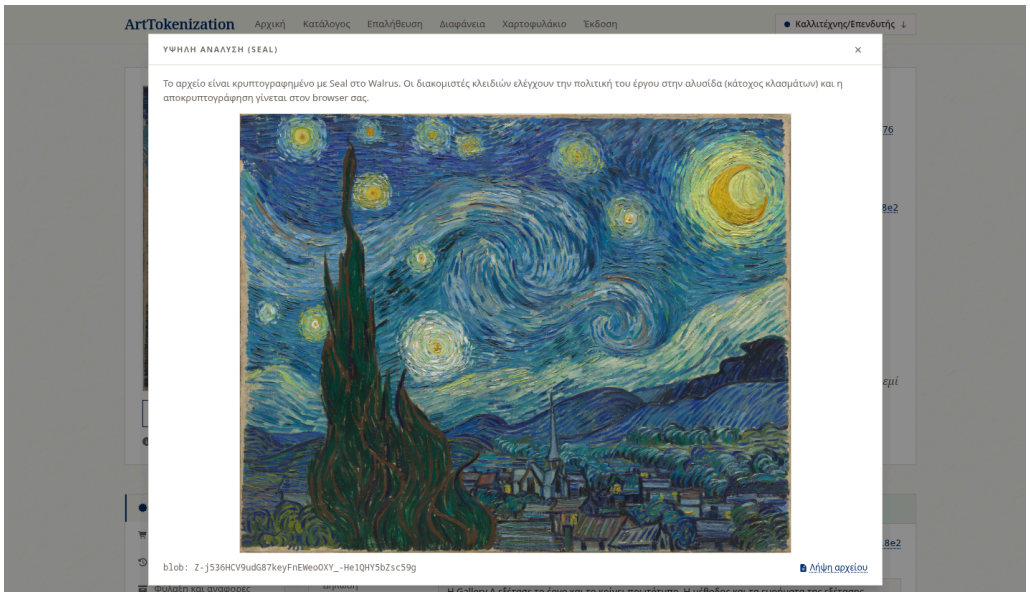
Σχήμα 5.17. Έλεγχος στοιχείων έργου τέχνης μέσω της ειδικής σελίδας.

Οι πράξεις του κύκλου ζωής από την έκδοση έως την πώληση καταγράφονται στο ιστορικό του έργου τέχνης. Οι παλιές εγγραφές παραμένουν διαθέσιμες όταν προστεθεί μια νέα πράξη.



Σχήμα 5.18. Το ιστορικό ενεργειών καταγεγραμμένο στο ίδιο το αντικείμενο.

Τέλος, το αρχείο υψηλής ανάλυσης παραμένει κρυπτογραφημένο. Ο ιδιοκτήτης ενός μη κλασματοποιημένου έργου τέχνης ή ο κάτοχος κλάσματος μπορεί να το αποκρυπτογραφήσει τοπικά στον browser, αφού οι διακομιστές κλειδιών του Seal εγκρίνουν το αίτημα.



Σχήμα 5.19. Το πρωτότυπο αρχείο αποκρυπτογραφημένο για τους κατόχους κλασμάτων.

Κεφάλαιο 6
Αξιολόγηση

Στο κεφάλαιο αυτό αξιολογούμε την εφαρμογή σε τρεις άξονες. Αρχικά, εξετάζουμε τους κανόνες ορθότητας και ασφάλειας των συμβολαίων και του oracle μέσω αυτοματοποιημένων δοκιμών. Στη συνέχεια, αναλύουμε το κόστος επιλεγμένων πράξεων στο δοκιμαστικό δίκτυο. Τέλος, αντιστοιχίζουμε τις λειτουργικές απαιτήσεις με την υλοποίηση, τις επιδείξεις και τους ελέγχους.

Η αξιολόγηση αφορά το λειτουργικό πρωτότυπο της πειραματικής διάταξης. Το δημόσιο testnet δεν μας παρέχει έλεγχο του φόρτου των υπηρεσιών ή σταθερές συνθήκες για επαναλήψιμη μέτρηση της μέγιστης κλιμάκωσης. Γι' αυτό δεν εξάγουμε όρια παραγωγικής απόδοσης από τη χρήση του.

6.1 Έλεγχος ορθότητας

Η επαλήθευση των επιχειρησιακών κανόνων και των ορίων ασφαλείας γίνεται με μια σουίτα αυτοματοποιημένων δοκιμών στο πακέτο Move. Οι αρνητικές δοκιμές επιχειρούν μη εξουσιοδοτημένες ή άκυρες μεταβάσεις καταστάσεων και επιβεβαιώνουν τον αναμενόμενο κωδικό σφάλματος (`abort code`), ενώ οι θετικές δοκιμές επικυρώνουν κρίσιμες αναλλοίωτες συνθήκες. Ακολουθούν οι έλεγχοι που επιβάλλονται εκτός συμβολαίου και οι δοκιμές του oracle σε Rust.

Πίνακας 6.1. Αυτοματοποιημένοι έλεγχοι Move για τη διαχείριση μητρώων και παραμέτρων (5.2).

Περίπτωση	Κωδικός	Δοκιμή	Είδος	Επιτυχία
Εγγραφή διεύθυνσης που είναι ήδη ενεργή	`EAlreadyActive`	`register_twice_aborts`	αρνητική	✓
Ανάκληση διεύθυνσης που δεν εγγράφηκε ποτέ	`ENotRegistered`	`revoke_unknown_aborts`	αρνητική	✓
Δεύτερη ανάκληση της ίδιας διεύθυνσης	`EAlreadyRevoked`	`revoke_twice_aborts`	αρνητική	✓
Εγγραφή χωρίς επωνυμία	`EEmptyName`	`register_without_name_aborts`	αρνητική	✓
Ποσοστό δικαιώματος πάνω από το 100%	`EInvalidBps`	`set_royalty_rejects_more_than_whole`	αρνητική	✓
Παράθυρο συνάθροισης εκτός ορίων	`EInvalidWindow`	`a_window_below_one_second_is_refused`	αρνητική	✓
Αποδοχή oracle με παλιότερες μετρήσεις	`EStaleEnclave`	`an_enclave_of_retired_measurements_aborts`	αρνητική	✓
Τα δύο μητρώα είναι ανεξάρτητα μεταξύ τους	–	`registries_are_independent`	θετική	✓

Πίνακας 6.2. Αυτοματοποιημένοι έλεγχοι Move για τον κύκλο έκδοσης, ανάθεσης φύλαξης και πιστοποίησης (5.3).

Περίπτωση	Κωδικός	Δοκιμή	Είδος	Επιτυχία
Ονομασία γκαλερί εκτός μητρώου	`EGalleryNotActive`	`mint_naming_unregistered_gallery_aborts`	αρνητική	✓
Ονομασία ανακληθείσας γκαλερί	`EGalleryNotActive`	`mint_naming_revoked_gallery_aborts`	αρνητική	✓
Έκδοση χωρίς τίτλο	`EEmptyTitle`	`mint_without_title_aborts`	αρνητική	✓
Δεύτερη έκδοση με την ίδια εικόνα και το ίδιο salt	`EIdentityAlreadyBound`	`copying_the_image_and_the_salt_cannot_be_minted`	αρνητική	✓
Παραλαβή από γκαλερί που δεν ορίστηκε	`ENotDesignated`	`intake_by_other_registered_gallery_aborts`	αρνητική	✓
Δεύτερη παραλαβή του ίδιου έργου τέχνης	`ENotDesignated`	`second_intake_while_held_aborts`	αρνητική	✓
Παραλαβή από διεύθυνση εκτός μητρώου	`ENotActive`	`intake_by_unregistered_address_aborts`	αρνητική	✓
Ανάθεση όσο η γκαλερί είναι ενεργή	`ECustodianActive`	`reassign_while_custodian_active_aborts`	αρνητική	✓
Ανάθεση σε ανακληθείσα γκαλερί	`EGalleryNotActive`	`reassign_to_revoked_gallery_aborts`	αρνητική	✓
Ανάθεση πριν από οποιαδήποτε παραλαβή	`ENoCustody`	`reassign_before_any_intake_aborts`	αρνητική	✓
Πιστοποίηση πριν από την παραλαβή	`ENoCustody`	`issue_before_intake_aborts`	αρνητική	✓
Πιστοποίηση από γκαλερί που δεν κρατά το έργο τέχνης	`ENotCustodian`	`issue_by_registered_non_holder_aborts`	αρνητική	✓
Πιστοποίηση από ανακληθείσα γκαλερί	`ENotActive`	`issue_by_revoked_holder_aborts`	αρνητική	✓
Δεύτερο πιστοποιητικό στο ίδιο έργο τέχνης	`EAlreadyCertified`	`issue_twice_aborts`	αρνητική	✓
Πιστοποίηση απολεσθέντος έργου τέχνης	`ETerminal`	`issue_after_loss_aborts`	αρνητική	✓

Πίνακας 6.3. Αυτοματοποιημένοι έλεγχοι Move για την κλασματοποίηση, διαίρεση, συγχώνευση και ενοποίηση (5.4).

Περίπτωση	Κωδικός	Δοκιμή	Είδος	Επιτυχία
Κλασματοποίηση χωρίς καταχωρημένη φύλαξη	`ENoCustody`	`fractionalize_without_custody_aborts`	αρνητική	✓
Κλασματοποίηση χωρίς πιστοποιητικό	`ENoCertificate`	`fractionalize_without_certificate_aborts`	αρνητική	✓
Κλασματοποίηση από μη ιδιοκτήτη	`ENotOwner`	`fractionalize_by_non_owner_aborts`	αρνητική	✓
Μηδενική συνολική προσφορά	`EZeroSupply`	`fractionalize_with_zero_supply_aborts`	αρνητική	✓
Δεύτερη κλασματοποίηση του ίδιου έργου τέχνης	`EAlreadyFractionalised`	`fractionalize_twice_aborts`	αρνητική	✓
Διαχωρισμός που αφήνει μηδέν σε μία πλευρά	`EInvalidSplit`	`split_zero_aborts`	αρνητική	✓
Συγχώνευση κλασμάτων διαφορετικών θησαυροφυλακίων	`EWrongVault`	`merge_across_vaults_aborts`	αρνητική	✓
Το άθροισμα των κλασμάτων μένει η αρχική προσφορά	–	`supply_is_conserved_across_many_operations`	θετική	✓

Πίνακας 6.4. Αυτοματοποιημένοι έλεγχοι Move για δημιουργία αγγελιών, αγορές και δικαιώματα μεταπώλησης (5.5).

Περίπτωση	Κωδικός	Δοκιμή	Είδος	Επιτυχία
Αγορά της δικής σου αγγελίας	`ESelfPurchase`	`buying_own_listing_aborts`	αρνητική	✓
Πληρωμή μικρότερη από το τίμημα	`EWrongPayment`	`underpaying_aborts`	αρνητική	✓
Πληρωμή μεγαλύτερη από το τίμημα	`EWrongPayment`	`overpaying_aborts`	αρνητική	✓
Ποσότητα μεγαλύτερη από τη διαθέσιμη	`EInsufficientQuantity`	`buying_more_than_listed_aborts`	αρνητική	✓
Αγορά από κλειστή αγγελία	`EListingClosed`	`buying_from_closed_listing_aborts`	αρνητική	✓
Αγγελία με μηδενική τιμή	`EZeroPrice`	`list_at_zero_price_aborts`	αρνητική	✓
Ακύρωση αγγελίας από άλλον	`ENotSeller`	`cancel_by_non_seller_aborts`	αρνητική	✓
Αγορά σε έργο τέχνης που δηλώθηκε απολεσθέν	`ETerminal`	`buying_on_lost_artwork_aborts`	αρνητική	✓
Σε μεταπώληση δικαίωμα και υπόλοιπο αθροίζουν στην τιμή	–	`resale_pays_royalty_to_artist_and_rest_to_seller`	θετική	✓
Στρογγυλοποίηση δικαιώματος (2,5% στα 333 MIST)	–	`royalty_rounds_down_and_remainder_goes_to_seller`	θετική	✓
Η ακύρωση αγγελίας πετυχαίνει και σε απολεσθέν έργο τέχνης	–	`cancel_on_lost_artwork_still_returns_fractions`	θετική	✓

Πίνακας 6.5. Αυτοματοποιημένοι έλεγχοι Move για αναφορές κατάστασης του oracle και δήλωση απώλειας (5.6).

Περίπτωση	Κωδικός	Δοκιμή	Είδος	Επιτυχία
Αλλαγή πεδίου μετά την υπογραφή	`EInvalidSignature`	`a_tampered_condition_aborts`	αρνητική	✓
Αναφορά μιας πηγής με το όνομα άλλης	`EInvalidSignature`	`a_report_moved_onto_another_source_aborts`	αρνητική	✓
Αναφορά από ανακληθείσα πηγή	`ENotActiveSource`	`a_revoked_source_aborts`	αρνητική	✓
Αναφορά από πηγή εκτός μητρώου	`ENotActiveSource`	`an_unregistered_source_aborts`	αρνητική	✓
Κατάσταση εκτός των τεσσάρων τιμών	`EUnknownCondition`	`an_unknown_condition_aborts`	αρνητική	✓
Αναφορά χωρίς καμία ανάγνωση	`EEmptyReport`	`a_report_of_no_readings_aborts`	αρνητική	✓
Log άλλου έργου τέχνης	`EWrongLog`	`a_report_into_another_artworks_log_aborts`	αρνητική	✓
Oracle που δεν έχει γίνει αποδεκτό	`EWrongEnclave`	`an_enclave_the_authority_has_not_accepted_aborts`	αρνητική	✓
Αναφορά παλιότερη από το παράθυρο φρεσκάδας	`EStaleReport`	`a_report_submitted_after_the_window_aborts`	αρνητική	✓
Αναφορά σφραγισμένη στο μέλλον	`EFutureReport`	`a_report_stamped_ahead_of_the_chain_aborts`	αρνητική	✓
Η ίδια αναφορά δεύτερη φορά	`EReplayedReport`	`the_same_report_cannot_be_recorded_twice`	αρνητική	✓
Δήλωση απώλειας από ανακληθείσα γκαλερί	`ENotActive`	`loss_by_revoked_custodian_aborts`	αρνητική	✓
Άσχετος λογαριασμός υποβάλλει, η πηγή	–	`anyone_may_submit_and_the_report_names_the_signing_source`	θετική	✓

Περίπτωση	Κωδικός	Δοκιμή	Είδος	Επιτυχία
μένει αυτή που υπέγραψε				
Αναφορά στο όριο του χρονικού παραθύρου	–	`a_report_at_the_edge_of_the_window_is_recorded`	θετική	✓
Αναφορά εντός ανεκτής απόκλισης ρολογιού	–	`a_report_within_the_tolerated_skew_is_recorded`	θετική	✓

Πίνακας 6.6. Αυτοματοποιημένοι έλεγχοι Move για την πολιτική πρόσβασης στο κρυπτογραφημένο πρωτότυπο (5.9).

Περίπτωση	Κωδικός	Δοκιμή	Είδος	Επιτυχία
Αίτημα από διεύθυνση χωρίς κλάσμα	`ENoAccess`	`stranger_may_not_decrypt_full_res`	αρνητική	✓
Ο καλλιτέχνης αφού πούλησε όλα τα κλάσματα	`ENoAccess`	`artist_loses_owner_access_once_fractionalised`	αρνητική	✓
Κλάσμα άλλου θησαυροφυλακίου ως απόδειξη	`ENoAccess`	`fraction_of_another_vault_is_rejected`	αρνητική	✓
Ταυτότητα με επίθεμα	`EWrongIdentity`	`identity_with_suffix_is_rejected`	αρνητική	✓
Ταυτότητα άλλου έργου τέχνης	`EWrongIdentity`	`identity_of_another_artwork_is_rejected`	αρνητική	✓
Δόλωμα με δικό του salt	`EWrongIdentity`	`a_decoy_with_its_own_salt_cannot_unlock_the_original`	αρνητική	✓
Ο ιδιοκτήτης αποκρυπτογραφεί την εικόνα υψηλής ανάλυσης	–	`owner_may_decrypt_full_res`	θετική	✓
Ο κάτοχος κλάσματος αποκρυπτογραφεί	–	`fraction_holder_may_decrypt_full_res`	θετική	✓
Η ίδια εικόνα εκδίδεται δεύτερη φορά με άλλο salt	–	`the_same_image_can_be_issued_twice_under_two_salts`	θετική	✓

Ο **Πίνακας 6.7** καταγράφει περιπτώσεις που απορρίπτονται σε προγενέστερα επίπεδα από τον μεταγλωττιστή Move, το runtime του SUI, το wallet ή τους διακομιστές του Seal.

Πίνακας 6.7. Έλεγχοι εγκυρότητας και ασφάλειας που επιβάλλονται πριν από την εκτέλεση στην αλυσίδα.

Περίπτωση	Ποιος απορρίπτει	Γιατί
Πράξη διαχείρισης από διεύθυνση χωρίς το `AdminCap`, από τη σελίδα `/admin` που ανοίγει για όλους	Το SUI, στην προσομοίωση	Αντικείμενο άλλου κατόχου δεν μπορεί να δοθεί ως είσοδος στη συναλλαγή [97]
Μεταβίβαση κλάσματος από το wallet	Ο μεταγλωττιστής και ο επαληθευτής bytecode	Το `Fraction` δεν έχει `store`, οπότε μόνο το module `fraction` μπορεί να το μεταφέρει
Μεταβίβαση του ίδιου του έργου τέχνης	Ο μεταγλωττιστής και ο επαληθευτής bytecode	Το `Artwork` δεν έχει `store`
Αναφορά για έργο τέχνης που δεν παραλήφθηκε ποτέ	Η σύνταξη της συναλλαγής	Δεν υπάρχει log αναφορών να δοθεί ως όρισμα
Ξένο κλάσμα ως απόδειξη κατοχής στο Seal	Ο διακομιστής κλειδιών, στο dry-run	Ελέγχει την κατοχή των owned εισόδων όπως μια κανονική συναλλαγή

Περίπτωση	Ποιος απορρίπτει	Γιατί
Ληγμένο κλειδί συνεδρίας του Seal	Ο διακομιστής κλειδιών, πριν από την εκτέλεση της πολιτικής	Το κλειδί ισχύει 30 λεπτά, η διεπαφή χρήστη ζητά νέα υπογραφή
Πλαστογράφηση της οθόνης έγκρισης από κακόβουλη σελίδα	Το extension του wallet	Η οθόνη και τα κλειδιά ανήκουν στο extension, ο χρήστης βλέπει τι υπογράφει

Ο **Πίνακας 6.8** παρουσιάζει τις δοκιμές μονάδας του oracle σε Rust. Οι δοκιμές επικυρώνουν την απόρριψη άκυρων ή εκπρόθεσμων αναγνώσεων και τη διαχείριση των παραθύρων συνάθροισης.

Πίνακας 6.8. Δοκιμές μονάδας του oracle σε Rust για την επικύρωση αναγνώσεων και παραθύρων, καθώς και για τη συμβατότητα των μηνυμάτων μεταξύ Python, Rust και Move.

Περίπτωση	Δοκιμή	Είδος	Επιτυχία
Πλαστή υπογραφή ή υπογραφή άλλου κλειδιού	`a_forged_signature_is_refused`	αρνητική	✓
Ανάγνωση που άλλαξε μετά την υπογραφή, σε οποιοδήποτε πεδίο	`a_tampered_reading_is_refused`	αρνητική	✓
Επανάληψη ή αναδιάταξη του nonce	`a_replayed_or_reordered_nonce_is_refused`	αρνητική	✓
Ανάγνωση παλαιότερη από 10 λεπτά ή μπροστά από το ρολόι	`a_stale_or_future_reading_is_refused`	αρνητική	✓
Κατάσταση εκτός των τεσσάρων τιμών	`an_unknown_condition_is_refused`	αρνητική	✓
Κακοσχηματισμένο κλειδί ή υπογραφή	`malformed_keys_and_signatures_are_refused`	αρνητική	✓
Αίτημα με δηλωμένη διεύθυνση δεν μπορεί να αποσειριοποιηθεί	`a_claimed_address_cannot_be_expressed`	αρνητική	✓
Κλείσιμο παραθύρου που δεν υπάρχει	`closing_a_window_that_does_not_exist_is_refused`	αρνητική	✓
Ένα παράθυρο κλείνει σε μία υπογεγραμμένη αναφορά, όχι πριν από την ώρα του	`a_window_closes_into_one_signed_report`	θετική	✓
Σε ισοπαλία επιλέγεται η χειρότερη κατάσταση και η μεμονωμένη απόκλιση μειοψηφεί	`ties_go_to_the_worse_condition_and_glitches_are_outvoted`	θετική	✓
Παράθυρα χωριστά ανά πηγή και ανά έργο τέχνης	`windows_are_per_source_and_per_artwork`	θετική	✓
Η διάρκεια ορίζεται από την πολιτική και μένει σταθερή μετά το άνοιγμα	`the_window_follows_the_policy_and_is_fixed_at_opening`	θετική	✓
Τα bytes της αναφοράς ταυτίζονται με το BCS της Move	`report_bytes_match_the_move_payload`	θετική	✓
Η Rust επαληθεύει υπογραφή που παράγει το script Python	`device_message_matches_device_py`	θετική	✓

6.2 Κόστος των πράξεων

Οι μετρήσεις κόστους (**Πίνακας 6.9**) προέρχονται από 18 επιλεγμένες συναλλαγές στο δοκιμαστικό δίκτυο, με τιμή αναφοράς gas 1.000 MIST. Κάθε ενέργεια περιλαμβάνει χρεώσεις υπολογισμού και αποθήκευσης στο SUI, καθώς και κόστος αποθήκευσης αρχείων στο Walrus σε WAL. Για την ενδεικτική αποτίμηση σε δολάρια χρησιμοποιούνται τιμές 0,70 USD ανά SUI και 0,03 USD ανά WAL.

Οι τιμές περιγράφουν μία καταγεγραμμένη εκτέλεση ανά πράξη και δεν είναι μέσοι όροι επαναλαμβανόμενων πειραμάτων. Το κόστος Walrus αφορά τα συγκεκριμένα αρχεία και τη διάρκεια αποθήκευσης των 30 epochs της εγκατάστασης.

Πίνακας 6.9. Το κόστος κάθε πράξης, σε SUI για το blockchain δίκτυο και σε WAL για τα αρχεία.

Πράξη	Συν.	Αλυσίδα SUI				Walrus	Δολάρια
		υπολογισμός	αποθήκευση	επιστροφή	**καθαρό**	WAL	
Έκδοση έργου	3	0,004830	0,220750	0,158057	**0,067523**	0,018294	0,0478
Έκδοση πιστοποιητικού	3	0,004600	0,157419	0,144890	**0,017129**	0,007633	0,0122
Καταχώριση παραλαβής	1	0,001000	0,037240	0,027244	**0,010996**	-	0,0077
Δημιουργία αγγελίας	1	0,001000	0,024685	0,016252	**0,009433**	-	0,0066
Κλασματοποίηση	1	0,001000	0,035553	0,028531	**0,008022**	-	0,0056
Αγορά, μεταπώληση	1	0,001000	0,024214	0,017727	**0,007487**	-	0,0052
Ανάθεση νέας γκαλερί	1	0,001000	0,030278	0,026868	**0,004410**	-	0,0031
Αγορά, πρωτογενής	1	0,001000	0,021576	0,015116	**0,007461**	-	0,0052
Δήλωση απώλειας	1	0,001000	0,033037	0,030502	**0,003535**	-	0,0025
Ακύρωση έκδοσης	1	0,001000	0,027375	0,024957	**0,003418**	-	0,0024
Ακύρωση αγγελίας	1	0,001000	0,021409	0,020149	**0,002260**	-	0,0016
Ανάκληση από το μητρώο	1	0,001000	0,007015	0,006884	**0,001130**	-	0,0008
Εγγραφή στο μητρώο	1	0,001000	0,006954	0,006945	**0,001009**	-	0,0007
Ενοποίηση κλασμάτων	1	0,001000	0,031996	0,032436	**0,000560**	-	0,0004

Τρία συμπεράσματα προκύπτουν από τον παραπάνω πίνακα:

1. **Ίδια χρέωση υπολογισμού στις περισσότερες μετρήσεις.** Δώδεκα από τις 18 συναλλαγές χρεώθηκαν ακριβώς 0,001 SUI, δηλαδή το ελάχιστο gas bucket [101]. Η ίδια χρέωση σημαίνει ότι οι συγκεκριμένες εκτελέσεις ανήκουν στην ίδια βαθμίδα κόστους.

2. **Κόστος έκδοσης.** Η έκδοση απαιτεί 0,068 SUI, περίπου το 56% του συνολικού κόστους της εξεταζόμενης ροής έως την πρώτη πώληση, καθώς ενημερώνει άμεσα τον κατάλογο, το index λέξεων και το ιστορικό προέλευσης στο blockchain δίκτυο (4.11).

3. **Επιστροφή αποθήκευσης κατά την ενοποίηση.** Η `consolidate` έχει καθαρό κόστος 0,000560 SUI. Η καταστροφή του θησαυροφυλακίου και των κλασμάτων αποδεσμεύει αποθηκευτικό χώρο, αποδίδοντας επιστροφή 0,032436 SUI [136].

Η έκδοση ενός έργου τέχνης απαιτεί τρεις διακριτές συναλλαγές για την καταχώριση στο Walrus, την κλήση του έξυπνου συμβολαίου και τη σύνδεση των αντικειμένων αποθήκευσης (4.8). Στην έκδοση, το 71,6% του κόστους αποθήκευσης επιστρέφεται άμεσα με την κατανάλωση των προσωρινών αντικειμένων [136]. Με τις τιμές της μέτρησης, το κόστος Walrus (0,0259 WAL) αντιστοιχεί σε λιγότερο από 0,001 USD.

Συνολικά, η εξεταζόμενη ροή από την έκδοση έως την πρώτη πώληση κοστίζει 0,121 SUI και 0,026 WAL, δηλαδή περίπου 0,085 USD με τις ενδεικτικές τιμές της μέτρησης.

6.3 Ιχνηλάτηση απαιτήσεων

Ο **Πίνακας 6.10** αντιστοιχίζει κάθε λειτουργική απαίτηση με την περίπτωση χρήσης που την επιδεικνύει, τη μονάδα Move που την υλοποιεί και τον σχετικό πίνακα ελέγχων ορθότητας.

Πίνακας 6.10. Κάθε λειτουργική απαίτηση, η αντίστοιχη περίπτωση χρήσης, η μονάδα Move που την υλοποιεί και ο πίνακας ελέγχων ορθότητας.

Κωδ.	Απαίτηση	Περίπτωση χρήσης	Μονάδα Move	Έλεγχος
ΛΑ-01	Σύνδεση wallet και ρόλος	5.1	-	-
ΛΑ-02	Πλοήγηση και αναζήτηση στον κατάλογο	5.8	artwork	-
ΛΑ-03	Δημόσια επαλήθευση προέλευσης και φύλαξης	5.9	-	-
ΛΑ-04	Δημόσια διαφάνεια	5.2, 5.6 και 5.9	admin, registry, report, stats	-
ΛΑ-05	Ειδοποιήσεις ανά ρόλο	5.7	-	-
ΛΑ-06	Έκδοση έργου τέχνης	5.3	artwork	**Πίν. 6.2**
ΛΑ-07	Κρυπτογράφηση της εικόνας υψηλής ανάλυσης	5.3 και 5.9	policy	**Πίν. 6.6**
ΛΑ-08	Ακύρωση της έκδοσης	-	artwork	**Πίν. 6.2**
ΛΑ-09	Καταχώριση παραλαβής σε φύλαξη	5.3	custody	**Πίν. 6.2**
ΛΑ-10	Έκδοση πιστοποιητικού γνησιότητας	5.3	certificate	**Πίν. 6.2**
ΛΑ-11	Δήλωση απώλειας	5.6	custody	**Πίν. 6.5**
ΛΑ-12	Κλασματοποίηση	5.4	fraction	**Πίν. 6.3**
ΛΑ-13	Αγορά και πώληση κλασμάτων	5.5	market	**Πίν. 6.4**
ΛΑ-14	Ενοποίηση όλων των κλασμάτων	5.4	fraction	**Πίν. 6.3**
ΛΑ-15	Αποστολή αναγνώσεων στο oracle	5.6	report	**Πίν. 6.5**
ΛΑ-16	Μετρήσεις, εγγραφή και αποδοχή του oracle	-	report	**Πίν. 6.1**
ΛΑ-17	Εγγραφή και ανάκληση στα μητρώα	5.2	admin, registry	**Πίν. 6.1**
ΛΑ-18	Ανάθεση νέας γκαλερί φύλαξης	-	custody	**Πίν. 6.2**
ΛΑ-19	Ορισμός εποπτικών παραμέτρων	5.2	admin, report	**Πίν. 6.1**

Η παύλα στη στήλη «Περίπτωση χρήσης» δηλώνει ότι η απαίτηση δεν περιλαμβάνεται στις επιλεγμένες ροές του κεφαλαίου 5, αλλά ελέγχεται από τον πίνακα δοκιμών της τελευταίας στήλης. Οι απαιτήσεις με παύλα στη στήλη της μονάδας Move αφορούν αναγνώσεις της διεπαφής μέσω RPC client, χωρίς μεταβολή της κατάστασης στην αλυσίδα, και γι’ αυτό δεν έχουν αντίστοιχες δοκιμές συμβολαίων.

Για τις λειτουργίες της διεπαφής, το κεφάλαιο 5 παρέχει επίδειξη επιλεγμένων ροών και στιγμιότυπα. Η αντιστοίχιση δεν ισοδυναμεί με αυτοματοποιημένη δοκιμή κάθε λειτουργικής απαίτησης από τον browser έως τις εξωτερικές υπηρεσίες.

Κεφάλαιο 7
Επίλογος

Στο κεφάλαιο αυτό κλείνουμε την εργασία. Ξεκινάμε με τα συμπεράσματα για το πώς ανταποκρίνεται η εφαρμογή στα πέντε προβλήματα της αγοράς τέχνης και για την ωριμότητα της στοίβας SUI. Συνεχίζουμε με τις τεχνικές προκλήσεις που συναντήσαμε κατά την υλοποίηση και με τους περιορισμούς και τις παραδοχές της σχεδίασης. Κλείνουμε με τις κατευθύνσεις στις οποίες μπορεί να επεκταθεί η εργασία.

7.1 Συμπεράσματα

Στην παρούσα εργασία υλοποιήσαμε μια αποκεντρωμένη εφαρμογή που αντιμετωπίζει με συγκεκριμένους μηχανισμούς τα πέντε προβλήματα της αγοράς τέχνης που ορίσαμε στην εισαγωγή. Για την αδιαφάνεια των τιμών, τα ποσά των αγορών και των μεταπωλήσεων καταγράφονται δημόσια στο blockchain και είναι προσβάσιμα σε κάθε ενδιαφερόμενο. Για την τεκμηρίωση της γνησιότητας και της προέλευσης, το πιστοποιητικό της γκαλερί συνδέεται με το ψηφιακό αντικείμενο του έργου τέχνης και συνοδεύεται από δημόσιο ιστορικό των πράξεων που το αφορούν. Έτσι, μπορεί να ελεγχθεί ποιος πιστοποίησε το έργο και πώς εξελίχθηκε η διαχείριση και η ιδιοκτησία του.

Στα υψηλά όρια εισόδου απαντά η κλασματική ιδιοκτησία, καθώς ο επενδυτής μπορεί να αγοράσει μερίδιο αντί ολόκληρου του έργου τέχνης, δεσμεύοντας μικρότερο κεφάλαιο. Για τη χαμηλή ρευστότητα, η εφαρμογή επιτρέπει τη χωριστή μεταπώληση των κλασμάτων, χωρίς να απαιτείται η πώληση ολόκληρου του έργου. Η δυνατότητα αυτή διευκολύνει τη διαπραγμάτευση. Τέλος, για την αμοιβή των καλλιτεχνών, το έξυπνο συμβόλαιο αποδίδει αυτόματα στον δημιουργό το προβλεπόμενο ποσοστό (royalty) επί του καταγεγραμμένου τιμήματος κάθε μεταπώλησης κλασμάτων.

Για να συνδέσουμε τις λειτουργίες αυτές, αξιοποιήσαμε το οικοσύστημα του SUI blockchain. Το Nautilus μεταφέρει στο blockchain δίκτυο τις αναφορές των εξωτερικών πηγών, ενώ το Walrus αποθηκεύει τα αρχεία και φιλοξενεί τη διεπαφή χρήστη. Το Seal προστατεύει τα αρχεία υψηλής ανάλυσης με κρυπτογράφηση και επιτρέπει την αποκρυπτογράφησή τους σύμφωνα με την κατοχή του έργου τέχνης ή κλασμάτων του, συνδέοντας την ψηφιακή ιδιοκτησία με το δικαίωμα πρόσβασης στο περιεχόμενο. Η εφαρμογή δεν χρειάζεται παραδοσιακό backend, επειδή η κρίσιμη κατάσταση και οι κανόνες της βρίσκονται στα έξυπνα συμβόλαια.

Η υλοποίηση έδειξε επίσης ότι τα εργαλεία του οικοσυστήματος SUI μπορούν να συνεργαστούν σε μία ενιαία εφαρμογή. Το μοντέλο αντικειμένων χρησιμοποιήθηκε για τα περιουσιακά στοιχεία και τα capabilities για τα δικαιώματα διαχείρισης. Στη ροή που μετρήσαμε, από την έκδοση

έως την πρώτη πώληση, οι συναλλαγές και η αποθήκευση στο Walrus κόστισαν περίπου 0,085 USD με τις ενδεικτικές τιμές του κεφαλαίου 6. Το χαμηλό αυτό κόστος αποτελεί ένδειξη ότι η λύση είναι οικονομικά βιώσιμη ως προς τις συναλλαγές και την αποθήκευση, επιτρέποντας τη διαχείριση και τη μεταβίβαση κλασμάτων χωρίς υψηλή επιβάρυνση των χρηστών.

Η αποκέντρωση των ψηφιακών λειτουργιών δεν καταργεί την εμπιστοσύνη στον φυσικό κόσμο. Η γκαλερί εξακολουθεί να πιστοποιεί και να φυλάσσει το έργο τέχνης, ενώ η ρυθμιστική αρχή εγκρίνει τις γκαλερί και τις εξωτερικές πηγές. Ωστόσο, οι πράξεις τους καταγράφονται δημόσια και ελέγχονται από τους κανόνες των έξυπνων συμβολαίων.

7.2 Τεχνικές προκλήσεις

Κατά την υλοποίηση αντιμετωπίσαμε τέσσερις τεχνικές προκλήσεις. Προέκυψαν τόσο από το εξελισσόμενο οικοσύστημα SUI όσο και από τις σχεδιαστικές επιλογές της εφαρμογής:

- **Εκμάθηση της Move και του οικοσυστήματος SUI.** Για την ανάπτυξη των έξυπνων συμβολαίων χρειάστηκε να μάθουμε τη Move και να κατανοήσουμε το μοντέλο αντικειμένων του SUI. Ιδιαίτερη προσοχή δώσαμε στους κανόνες ιδιοκτησίας και μεταβίβασης των αντικειμένων.
- **Η μετάβαση σε gRPC.** Το μεγαλύτερο μέρος της δημόσιας τεκμηρίωσης βασιζόταν στο JSON-RPC, το οποίο αποσύρθηκε από τους δημόσιους nodes το 2026 [137]. Αυτό απαίτησε εκτενή έρευνα για την υιοθέτηση του νέου gRPC προτύπου και την ενσωμάτωση πρόσφατων βιβλιοθηκών.
- **Ο κατάλογος και η αναζήτηση χωρίς εξωτερικό indexer.** Οι κλήσεις του full node που χρησιμοποιούσαμε δεν κάλυπταν την αναζήτηση λέξεων στα metadata των έργων τέχνης. Επιπλέον, τα `Artwork` είναι shared αντικείμενα, οπότε η ανάκτηση των αντικειμένων μιας διεύθυνσης δεν αρκούσε για να βρούμε ποια έργα τέχνης κατέχει ένας χρήστης ή φυλάσσει μια γκαλερί. Για να καλύψουμε αυτές τις ανάγκες χωρίς ξεχωριστή βάση δεδομένων και backend server, υλοποιήσαμε το index μέσα στα έξυπνα συμβόλαια, στο shared αντικείμενο `Catalog` της ενότητας 4.11. Ο κατάλογος κρατά συνοπτικές εγγραφές σε chunks, λίστες έργων τέχνης ανά διεύθυνση και ένα index που αντιστοιχίζει λέξεις σε αναγνωριστικά έργων τέχνης. Τα chunks ομαδοποιούν τις ανακτήσεις και περιορίζουν τις εγγραφές που ξαναγράφονται σε κάθε ενημέρωση. Κάθε πράξη ενημερώνει τις αντίστοιχες εγγραφές στην ίδια συναλλαγή, ώστε το index να παραμένει συνεπές με την κατάσταση των έργων τέχνης.
- **Η φιλοξενία Walrus Sites.** Η μοναδική δημόσια πύλη του δοκιμαστικού δικτύου έκλεισε κατά την υλοποίηση, οπότε εγκαταστήσαμε δική μας πύλη σε EC2. Η αλλαγή αφορούσε μόνο τον τρόπο πρόσβασης στη διεπαφή και δεν μετέβαλε τα δεδομένα στο blockchain δίκτυο.

7.3 Περιορισμοί και παραδοχές

Η αρχιτεκτονική βασίζεται σε συγκεκριμένες παραδοχές και έχει τα ακόλουθα όρια:

- **Εμπιστοσύνη στον φυσικό κόσμο:** Το σύστημα δεν μπορεί να επιβεβαιώσει ψηφιακά αν η γκαλερί κατέχει πράγματι το φυσικό έργο τέχνης ή αν μια εξωτερική πηγή αναφέρει αληθή δεδομένα. Καταγράφει όμως ποια οντότητα υπέγραψε κάθε δήλωση, ώστε η προέλευσή της να μπορεί να ελεγχθεί.
- **Το πρόβλημα του τελευταίου κατόχου:** Η ενοποίηση ενός έργου τέχνης προϋποθέτει την κατοχή του 100% των κλασμάτων. Έτσι, ο τελευταίος κάτοχος έχει δύναμη αρνησικυρίας (veto), αν αρνείται να πουλήσει ή έχει χάσει το κλειδί του.
- **Λήξη αποθήκευσης στο Walrus:** Τα αρχεία εικόνων και τεκμηρίωσης δεν αποθηκεύονται μόνιμα χωρίς κόστος. Η διατήρησή τους εξαρτάται από την τακτική πληρωμή τελών (ανά epoch) από τους ενδιαφερόμενους.
- **Απουσία αποζημιώσεων:** Σε περίπτωση καταστροφής ή κλοπής του έργου τέχνης, το σύστημα παγώνει τη δευτερογενή αγορά. Οι αποζημιώσεις παραμένουν ζήτημα του ασφαλιστικού και νομικού πλαισίου.
- **Αυτοδηλωμένη ταυτότητα:** Η πλατφόρμα γνωρίζει μόνο διευθύνσεις κρυπτονομισμάτων, όχι φυσικά πρόσωπα.

7.4 Μελλοντικές κατευθύνσεις

Η υλοποίηση καλύπτει τον κύκλο ζωής που ορίσαμε στις απαιτήσεις. Οι ακόλουθες κατευθύνσεις μπορούν να επεκτείνουν την οικονομική λειτουργία, την εμπειρία χρήστη, το oracle και τη διακυβέρνηση:

Εκθέσεις με εισιτήρια και κατανομή εσόδων. Η διοργάνωση φυσικών εκθέσεων επιτρέπει την έκδοση εισιτηρίων ως on-chain αντικειμένων. Τα έσοδα μοιράζονται αυτόματα ανάμεσα στη γκαλερί, τον καλλιτέχνη και τους κατόχους κλασμάτων μέσω του έξυπνου συμβολαίου, ενώ το oracle καταγράφει με επαληθεύσιμο τρόπο την ακύρωση των εισιτηρίων κατά την είσοδο.

Κάλυψη κόστους από τη ρυθμιστική αρχή. Μέσω των sponsored transactions του SUI [138], η ρυθμιστική αρχή μπορεί να επιδοτεί τα τέλη gas των χρηστών. Παράλληλα, μια αυτοματοποιημένη υπηρεσία της αρχής μπορεί να παρακολουθεί τα epochs λήξης και να ανανεώνει τα `SharedBlob` στο Walrus, ώστε η διατήρηση των αρχείων να μη χρειάζεται χειροκίνητη παρέμβαση.

Επαυξημένη πραγματικότητα και πρόσθετο ψηφιακό υλικό. Κάθε έργο τέχνης θα μπορούσε να συνοδεύεται από βίντεο της δημιουργίας του, ηχητικές αφηγήσεις του καλλιτέχνη ή τρισδιάστατες σαρώσεις. Οι σαρώσεις θα επέτρεπαν στους κατόχους κλασμάτων να προ-

βάλλουν ψηφιακά το έργο στον χώρο τους μέσω επαυξημένης πραγματικότητας (AR), όπως φαίνεται ενδεικτικά στο **Σχήμα 7.1**. Το πρόσθετο υλικό θα μπορούσε να αποθηκεύεται στο Walrus και να κρυπτογραφείται με το Seal, με πρόσβαση βάσει της ίδιας πολιτικής και τοπική αποκρυπτογράφηση στη συσκευή του κατόχου. Έτσι, η κατοχή κλασμάτων θα παρείχε και πρόσβαση σε υλικό που αναδεικνύει το έργο και τη δημιουργική διαδικασία.



Σχήμα 7.1. Προβολή τρισδιάστατου ψηφιακού διδύμου του έργου τέχνης σε περιβάλλον AR.

Τεχνητή νοημοσύνη εντός του oracle. Η αυτοματοποιημένη ανάλυση φωτογραφιών για εντοπισμό φθορών μπορεί να εκτελείται από ελαφρύ μοντέλο μηχανικής μάθησης ως μέρος του oracle του Nautilus (2.3.5). Το oracle υπογράφει το αποτέλεσμα μαζί με την ταυτότητα του κώδικα που το παρήγαγε. Η υπογραφή επιτρέπει να επαληθευτεί η προέλευση του αποτελέσματος, χωρίς να αποδεικνύει ότι η εκτίμηση του μοντέλου είναι σωστή.

Μεταφορά έργων τέχνης. Ένας νέος μηχανισμός θα μπορούσε να υποστηρίξει τη μετακίνηση ενός tokenized έργου τέχνης από μία γκαλερί σε άλλη. Η διαδικασία πρέπει να μεταφέρει τα σχετικά δικαιώματα διαχείρισης και την εξουσιοδότηση για την έκδοση αναφορών.

Διεύρυνση πρωτοκόλλων και υπογραφών IoT. Η υποστήριξη συσκευών IoT με περιορισμένους πόρους απαιτεί την αποδοχή πρόσθετων αλγορίθμων υπογραφής (ECDSA, RSA) και άλλων πρωτοκόλλων όπως MQTT ή CoAP. Η αναβάθμιση αυτή υλοποιείται αποκλειστικά στο oracle και σε πύλες διασύνδεσης (gateways), χωρίς τροποποίηση των έξυπνων συμβολαίων.

Υποχρεωτική εξαγορά και αποτίμηση. Για την επίλυση του προβλήματος του τελευταίου κατόχου κατά την ενοποίηση, μπορεί να εισαχθεί δικαίωμα υποχρεωτικής εξαγοράς (squeeze-out) από πλειοψηφικό επενδυτή ή δημόσια δημοπρασία.

Αποκεντρωμένη ταυτότητα συμμετεχόντων. Η σύνδεση των διευθύνσεων των χρηστών με επαληθεύσιμα διαπιστευτήρια (W3C Verifiable Credentials και Decentralized Identifiers [139], [140]) αντιμετωπίζει το ζήτημα της αυτοδηλωμένης ταυτότητας. Για τους δημιουργούς, βεβαιώνει το φυσικό πρόσωπο πίσω από την έκδοση. Για τις γκαλερί, πιστοποιεί τα επαγγελματικά και θεσμικά διαπιστευτήρια. Για τους επενδυτές, επιτρέπει ελέγχους συμμόρφωσης (όπως KYC και AML), διασφαλίζοντας την αξιοπιστία της αγοράς χωρίς καταγραφή ευαίσθητων προσωπικών δεδομένων στο blockchain δίκτυο.

Συλλογική διακυβέρνηση της ρυθμιστικής αρχής. Στη σημερινή σχεδίαση, η ρυθμιστική αρχή εκπροσωπείται από μία συγκεκριμένη διεύθυνση που κατέχει το `AdminCap`. Όπως αναλύθηκε στη θεωρητική θεμελίωση της ενότητας 2.2.2, η συγκέντρωση προνομίων σε ένα μόνο πρόσωπο αντιμετωπίζεται με τη μετάβαση σε αποκεντρωμένη αυτόνομη οργάνωση (DAO). Με τη μεταβίβαση του `AdminCap` σε ένα συμβόλαιο DAO, η έγκριση ή ανάκληση γκαλερί και εξωτερικών πηγών, καθώς και η ρύθμιση των εποπτικών παραμέτρων, πραγματοποιούνται μέσω συλλογικής ψηφοφορίας των μελών της ρυθμιστικής αρχής. Ένα μοντέλο δύο ταχυτήτων διασφαλίζει ευρεία απαρτία για κανονιστικές ρυθμίσεις και ταχεία διαδικασία έκτακτης ανάγκης για την ανάκληση παραβατικών οντοτήτων.

Συντομεύσεις και ακρωνύμια

API – **Application Programming Interface** 39

AWS – **Amazon Web Services** 12, 63, 63, 74

BCS – **Binary Canonical Serialization** 66, 115

BFT – **Byzantine Fault Tolerance** 25, 26, 26, 27, 27, 27, 27, 51

DAG – **Directed Acyclic Graph** 51

DAO – **Decentralized Autonomous Organization** 11, 31, 31, 31, 31, 80, 143, 143

DApp – **decentralized application** 30

DeFi – **Decentralized Finance** 31, 31, 37

EC2 – **Elastic Compute Cloud** 75, 75, 140

EIF – **Enclave Image File** 63

FT – **Fungible Token** 29

GST – **Global Stabilization Time** 26

HTTP – **Hypertext Transfer Protocol** 55, 58

IoT – **Internet of Things** 21, 39

NFT – **Non-Fungible Token** 11, 19, 29, 29, 31, 44, 44, 44

NSM – **Nitro Security Module** 64

PCR – **Platform Configuration Register** 64, 64, 65, 65, 65, 66, 76

PoA – **Point of Availability** 56

PTB – **Programmable Transaction Block** 47, 76, 82, 82, 102, 102, 102, 102, 104, 105

RWA – **Real World Asset** 11, 14, 14, 19, 32, 32, 32, 32, 33, 33, 33, 34, 34, 36, 43

SC – **smart contract** 28

SFT – **Semi-Fungible Token** 29

SMR – **State Machine Replication** 23, 24

***TEE* – Trusted Execution Environment** 42, 63, 63, 63, 63, 63, 64, 65, 67

***tx* – transaction** 23

***URL* – Uniform Resource Locator** 58, 74

Βιβλιογραφία

[1] C. McAndrew, 'The Art Basel and UBS Global Art Market Report 2026', 2026. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://www.ubs.com/global/en/our-firm/art/art-market-research.html

[2] F. Angelini και M. Castellani, 'Price and information disclosure in the private art market: A signalling game', *Research in Economics*, τ. 76, τχ. 1, σελ. 14–20, 2022, doi: 10.1016/j.rie.2022.01.002.

[3] G. David, K. Oosterlinck, και A. Szafarz, 'Art market inefficiency', *Economics Letters*, τ. 121, τχ. 1, σελ. 23–25, 2013, doi: 10.1016/j.econlet.2013.06.033.

[4] E. Coslor, ' Transparency in an opaque market: Evaluative frictions between "thick" valuation and "thin" price data in the art market ', *Accounting, Organizations and Society*, τ. 50, σελ. 13–26, 2016, doi: 10.1016/j.aos.2016.03.001.

[5] J. Prieto-Rodriguez και M. Vecco, ' Reading between the lines in the art market: Lack of transparency and price heterogeneity as an indicator of multiple equilibria ', *Economic Modelling*, τ. 102, σελ. 105587, 2021, doi: 10.1016/j.econmod.2021.105587.

[6] M. Gertsberg, 'Asymmetries in Art Markets', Doctoral dissertation, ProefschriftMaken Maastricht, Netherlands, 2019. doi: 10.26481/dis.20190906mg.

[7] Y. Li, X. Ma, και L. Renneboog, 'In Art We Trust', *Management Science*, τ. 70, τχ. 1, σελ. 98–127, 2024, doi: 10.1287/mnsc.2022.4633.

[8] Eurojust, 'International Operation Leads to Seizure of 2,000 Fake Works of Art with Potential Losses of EUR 200 Million'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://www.eurojust.europa.eu/news/international-operation-leads-seizure-2-000-fake-works-art-potential-losses-eur-200-million

[9] A. L. Bandle, 'Fake or Fortune? Art Authentication Rules in the Art Market and at Court', *International Journal of Cultural Property*, τ. 22, τχ. 2–3, σελ. 379–399, 2015, doi: 10.1017/S0940739115000107.

[10] V. Oliveri, G. Porter, C. Davies, και P. James, 'Art crime: the challenges of provenance, law and ethics', *Museum Management and Curatorship*, τ. 37, τχ. 2, σελ. 179–195, 2022, doi: 10.1080/09647775.2022.2052160.

[11] O. Ashenfelter και K. Graddy, 'Art Auctions', στο *Handbook of the Economics of Art and Culture*, τ. 1, V. A. Ginsburg και D. Throsby, Επιμ., Elsevier, 2006, σελ. 909–945. doi: 10.1016/S1574-0676(06)01026-X.

[12] A. Zhukova, V. Lakshina, και L. Leonova, 'Empirical Analysis of Investments on the Fine Art Market', στο *Proceedings of the Fourth Workshop on Computer Modelling in Decision Making (CMDM 2019)*, Atlantis Press, 2019, σελ. 67–72. doi: 10.2991/ahcs.k.191206.012.

[13] J. Mei και M. Moses, 'Art as an Investment and the Underperformance of Masterpieces', *The American Economic Review*, τ. 92, τχ. 5, σελ. 1656–1668, 2002, doi: 10.1257/000282802762024719.

[14] Y. Li, M. X. Ma, και L. Renneboog, 'Pricing art and the art of pricing: On returns and risk in art auction markets', *European Financial Management*, τ. 28, τχ. 5, σελ. 1139–1198, 2022, doi: 10.1111/eufm.12348.

[15] A. Whitaker και R. Kräussl, 'Fractional Equity, Blockchain, and the Future of Creative Work', *Management Science*, τ. 66, τχ. 10, σελ. 4594–4611, Οκτωβρίου 2020, doi: 10.1287/mnsc.2020.3633.

[16] S. P. Fraiberger, R. Sinatra, M. Resch, C. Riedl, και A.-L. Barabási, 'Quantifying reputation and success in art', *Science*, τ. 362, τχ. 6416, σελ. 825–829, 2018, doi: 10.1126/science.aau7224.

[17] J. L. Solow, 'An Economic Analysis of the Droit de Suite', *Journal of Cultural Economics*, τ. 22, τχ. 4, σελ. 209–226, Δεκεμβρίου 1998, doi: 10.1023/A:1007505016512.

[18] L. van Haaften-Schick και A. Whitaker, ' From the Artist's Contract to the blockchain ledger: new forms of artists' funding using equity and resale royalties ', *Journal of Cultural Economics*, τ. 46, τχ. 2, σελ. 287–315, Ιουνίου 2022, doi: 10.1007/s10824-022-09445-8.

[19] Z. Tian, ' Reshaping the art market: Blockchain technology, resale royalties, and the emerging Chinese paradigm ', *Arts & Communication*, τ. 3, τχ. 1, σελ. 2875, 2024, doi: 10.36922/ac.2875.

[20] Z. Wang, L. Yang, Q. Wang, D. Liu, Z. Xu, και S. Liu, 'ArtChain: Blockchain-enabled Platform for Art Marketplace', στο *2019 IEEE International Conference on Blockchain (Blockchain)*, 2019, σελ. 447–454. doi: 10.1109/Blockchain.2019.00068.

[21] A. Whitaker, ' Art and Blockchain: A Primer, History, and Taxonomy of Blockchain Use Cases in the Arts ', *Artivate*, τ. 8, τχ. 2, σελ. 21–46, 2019, doi: 10.34053/artivate.8.2.2.

[22] D. Fincham, 'Art, Antiquities, and Money Laundering', *Kentucky Law Journal*, τ. 111, σελ. 309–344, 2023, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://ssrn.com/abstract=4336668

[23] H. Gong, 'Non-Fungible Tokens (NFTs) and Digital Art', στο *The Web3 Revolution*, Springer, 2024, σελ. 199–226. doi: 10.1007/979-8-8688-0491-5_7.

[24] T. Bhandare και M. A. Kandi, 'Tokenization of Fine Arts: Revolutionizing the Fine Arts Industry with Blockchain', στο *Blockchain and Smart-Contract Technologies for Innovative Applications*, N. El Madhoun και others, Επιμ., Springer, 2024, σελ. 167–186. doi: 10.1007/978-3-031-50028-2_6.

[25] W. Choi, J. Woo, και J. W.-K. Hong, 'Fractional Non-Fungible Tokens: Overview, Evaluation, Marketplaces, and Challenges', *International Journal of Network Management*, τ. 34, τχ. 4, σελ. e2260, 2024, doi: 10.1002/nem.2260.

[26] H. Gong, 'Tokenised Real-World Assets (RWA) and Decentralised Physical Infrastructure Networks (DePIN)', στο *The Web3 Revolution*, Springer, 2024, σελ. 175–198. doi: 10.1007/979-8-8688-0491-5_6.

[27] N. Szabo, 'Formalizing and Securing Relationships on Public Networks', *First Monday*, τ. 2, τχ. 9, 1997, doi: 10.5210/fm.v2i9.548.

[28] J. O'Sullivan, 'Blockchain for the Arts and Humanities', *Future Humanities*, τ. 4, σελ. e70020, 2025, doi: 10.1002/fhu2.70020.

[29] The MystenLabs Team, 'The Sui Smart Contracts Platform'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://sui.io/whitepaper

[30] The MystenLabs Team, 'Walrus: An Efficient Decentralized Storage Network'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.wal.app/walrus.pdf

[31] Mysten Labs, 'Seal Design'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/seal/design

[32] Mysten Labs, 'Nautilus Design'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/nautilus/nautilus-design

[33] E. Sidorova, 'The Cyber Turn of the Contemporary Art Market', *Arts*, τ. 8, τχ. 3, σελ. 84, 2019, doi: 10.3390/arts8030084.

[34] S. Nakamoto, 'Bitcoin: A Peer-to-Peer Electronic Cash System'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://bitcoin.org/bitcoin.pdf

[35] T. Roughgarden, 'The Computer in the Sky (Keynote)', στο *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, στο STOC 2024. Vancouver, BC, Canada: Association for Computing Machinery, 2024, σελ. 1. doi: 10.1145/3618260.3664271.

[36] L. Lamport, 'Time, Clocks, and the Ordering of Events in a Distributed System', *Communications of the ACM*, τ. 21, τχ. 7, σελ. 558–565, 1978, doi: 10.1145/359545.359563.

[37] F. B. Schneider, 'Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial', *ACM Computing Surveys*, τ. 22, τχ. 4, σελ. 299–319, 1990, doi: 10.1145/98163.98167.

[38] T. Distler, 'Byzantine Fault-Tolerant State-Machine Replication from a Systems Perspective', *ACM Computing Surveys*, τ. 54, τχ. 1, σελ. 1–38, 2021, doi: 10.1145/3436728.

[39] C. Cachin, R. Guerraoui, και L. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*, 2ο έκδ. Berlin, Heidelberg: Springer, 2011. doi: 10.1007/978-3-642-15260-3.

[40] M. J. Fischer, N. A. Lynch, και M. S. Paterson, 'Impossibility of Distributed Consensus with One Faulty Process', *Journal of the ACM*, τ. 32, τχ. 2, σελ. 374–382, 1985, doi: 10.1145/3149.214121.

[41] L. Lamport, R. Shostak, και M. Pease, 'The Byzantine Generals Problem', *ACM Transactions on Programming Languages and Systems*, τ. 4, τχ. 3, σελ. 382–401, 1982, doi: 10.1145/357172.357176.

[42] A. Lewis-Pye και T. Roughgarden, 'Byzantine Generals in the Permissionless Setting'. 2023.

[43] C. Dwork, N. Lynch, και L. Stockmeyer, 'Consensus in the Presence of Partial Synchrony', *Journal of the ACM*, τ. 35, τχ. 2, σελ. 288–323, 1988, doi: 10.1145/42282.42283.

[44] J. R. Douceur, 'The Sybil Attack', στο *Peer-to-Peer Systems (IPTPS 2002)*, στο Lecture Notes in Computer Science, vol. 2429. Berlin, Heidelberg: Springer, 2002, σελ. 251–260. doi: 10.1007/3-540-45748-8_24.

[45] C. T. Nguyen, D. T. Hoang, D. N. Nguyen, D. Niyato, H. T. Nguyen, και E. Dutkiewicz, 'Proof-of-Stake Consensus Mechanisms for Future Blockchain Networks: Fundamentals, Applications and Opportunities', *IEEE Access*, τ. 7, σελ. 85727–85745, 2019, doi: 10.1109/ACCESS.2019.2925010.

[46] L. Ge, J. Wang, και G. Zhang, 'Survey of Consensus Algorithms for Proof of Stake in Blockchain', *Security and Communication Networks*, τ. 2022, σελ. 1–13, 2022, doi: 10.1155/2022/2812526.

[47] A. Kiayias, A. Russell, B. David, και R. Oliynykov, 'Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol', στο *Advances in Cryptology – CRYPTO 2017*, στο Lecture Notes in Computer Science, vol. 10401. Cham: Springer, 2017, σελ. 357–388. doi: 10.1007/978-3-319-63688-7_12.

[48] The Mysten Labs Team, 'The Sui Smart Contracts Platform: Economics and Incentives'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://sui.io/whitepaper

[49] V. Buterin *κ.ά.*, 'Combining GHOST and Casper'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2003.03052

[50] M. Bartoletti και L. Pompianu, ' An Empirical Analysis of Smart Contracts: Platforms, Applications, and Design Patterns ', στο *Financial Cryptography and Data Security 2017 International Workshops*, στο Lecture Notes in Computer Science, vol. 10323. Springer, 2017, σελ. 494–509. doi: 10.1007/978-3-319-70278-0_31.

[51] Q. Wang, R. Li, Q. Wang, και S. Chen, ' Non-Fungible Token (NFT): Overview, Evaluation, Opportunities and Challenges ', *arXiv preprint arXiv:2105.07447*, 2021, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2105.07447

[52] W. Wang, M. Meng, Y. Cai, R. Chow, Z. Wu, και A. Du, 'ERC-3525: Semi-Fungible Token'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://eips.ethereum.org/EIPS/eip-3525

[53] P. Ohlhaver, E. G. Weyl, και V. Buterin, 'Decentralized Society: Finding Web3's Soul'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://ssrn.com/abstract=4105763

[54] G. Wood, 'ĐApps: What Web 3.0 Looks Like'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://gavwood.com/dappsweb3.html

[55] Q. Yang, H. Huang, K. Lin, και J. Wu, 'An Introduction to Web3 and Metaverse', στο *From Blockchain to Web3 & Metaverse*, H. Huang, J. Wu, και Z. Zheng, Επιμ., Singapore: Springer Nature Singapore, 2023, σελ. 1–26. doi: 10.1007/978-981-99-3648-9_1.

[56] T. Berners-Lee, J. Hendler, και O. Lassila, 'The Semantic Web', *Scientific American*, τ. 284, τχ. 5, σελ. 34–43, 2001, doi: 10.1038/scientificamerican0501-34.

[57] K. Wu, Y. Ma, G. Huang, και X. Liu, 'A First Look at Blockchain-based Decentralized Applications', *arXiv preprint arXiv:1909.00939*, 2019, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/1909.00939

[58] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, και W. J. Knottenbelt, 'SoK: Decentralized Finance (DeFi)', στο *Proceedings of the 4th ACM Conference on Advances in Financial Technologies (AFT)*, ACM, 2022, σελ. 30–46. doi: 10.1145/3558535.3559780.

[59] Uniswap Labs, 'Uniswap Protocol'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://uniswap.org/

[60] MakerDAO, 'MakerDAO. An Unbiased Global Financial System'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://makerdao.com/

[61] OpenSea, 'OpenSea. Token Trading and NFT Marketplace'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://opensea.io/

[62] D. Das, P. Bose, N. Ruaro, C. Kruegel, και G. Vigna, 'Understanding Security Issues in the NFT Ecosystem', στο *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2022, σελ. 667–681. doi: 10.1145/3548606.3559342.

[63] Farcaster, 'Farcaster Docs'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.farcaster.xyz/

[64] S. Hassan και P. De Filippi, 'Decentralized Autonomous Organization', *Internet Policy Review*, τ. 10, τχ. 2, 2021, doi: 10.14763/2021.2.1556.

[65] J. Luo *κ.ά.*, ' SoK of RWA Tokenization: A Systematization of Concepts, Architectures, and Legal Interoperability ', *arXiv preprint arXiv:2604.06608*, 2026, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2604.06608

[66] N. Xia, X. Zhao, Y. Yang, Y. Li, και Y. Li, 'Exploration on Real World Assets and Tokenization', *arXiv preprint arXiv:2503.01111*, 2025, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2503.01111

[67] G. Sazandrishvili, 'Asset Tokenization in Plain English', *Journal of Corporate Accounting & Finance*, τ. 31, τχ. 2, σελ. 68–73, 2020, doi: 10.1002/jcaf.22432.

[68] Bank for International Settlements, ' Blueprint for the Future Monetary System: Improving the Old, Enabling the New ', Annual Economic Report, Chapter III, 2023. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://www.bis.org/publ/arpdf/ar2023e3.htm

[69] T. Kranz, V. Schaaf, T. Guggenberger, και J. Strüker, ' From Bricks to Blocks: Designing a Framework for the Tokenization of Real Estate for DeFi ', *IEEE Access*, τ. 14, 2026, doi: 10.1109/ACCESS.2025.3648172.

[70] L. Zhang, ' SoK: Stablecoins for Digital Transformation — Design, Metrics, and Application with Real World Asset Tokenization as a Case Study ', *arXiv preprint arXiv:2508.02403*, 2025, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2508.02403

[71] R. Mirdala, 'Tokenization of Real-World Assets: Legal Frameworks, Market Dynamics, and Policy Pathways for a Decentralized Financial Future', *Journal of Applied Economic Sciences*, τ. 20, τχ. 2, σελ. 285–312, 2025, doi: 10.57017/jaes.v20.2(88).09.

[72] R. Mühlberger *κ.ά.*, 'Foundational Oracle Patterns: Connecting Blockchain to the Off-Chain World', στο *Business Process Management: Blockchain and Robotic Process Automation Forum (BPM 2020)*, στο Lecture Notes in Business Information Processing, vol. 393. Springer, 2020, σελ. 35–51. doi: 10.1007/978-3-030-58779-6_3.

[73] H. Al-Breiki, M. H. U. Rehman, K. Salah, και D. Svetinovic, ' Trustworthy Blockchain Oracles: Review, Comparison, and Open Research Challenges ', *IEEE Access*, τ. 8, σελ. 85675–85685, 2020, doi: 10.1109/ACCESS.2020.2992698.

[74] G. Caldarelli, 'Understanding the Blockchain Oracle Problem: A Call for Action', *Information*, τ. 11, τχ. 11, σελ. 509, 2020, doi: 10.3390/info11110509.

[75] S. Eskandari, M. Salehi, W. C. Gu, και J. Clark, 'SoK: Oracles from the Ground Truth to Market Manipulation', στο *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT)*, ACM, 2021, σελ. 127–141. doi: 10.1145/3479722.3480994.

[76] A. Beniiche, 'A Study of Blockchain Oracles', *arXiv preprint arXiv:2004.07140*, 2020, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2004.07140

[77] A. Pasdar, Y. C. Lee, και Z. Dong, 'Connect API with Blockchain: A Survey on Blockchain Oracle Implementation', *ACM Computing Surveys*, τ. 55, τχ. 10, σελ. 1–39, 2023, doi: 10.1145/3567582.

[78] M. Bartholic, A. Laszka, G. Yamamoto, και E. W. Burger, 'A Taxonomy of Blockchain Oracles: The Truth Depends on the Question', στο *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, IEEE, 2022, σελ. 1–15. doi: 10.1109/ICBC54727.2022.9805555.

[79] C. Finkbeiner και G. Almashaqbeh, 'SoK: Blockchain Oracles Between Theory and Practice'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://eprint.iacr.org/2025/2106

[80] R. Gennaro, C. Gentry, και B. Parno, 'Non-interactive Verifiable Computing: Outsourcing Computation to Untrusted Workers', στο *Advances in Cryptology – CRYPTO 2010*, στο Lecture Notes in Computer Science, vol. 6223. Springer, 2010, σελ. 465–482. doi: 10.1007/978-3-642-14623-7_25.

[81] M. Walfish και A. J. Blumberg, 'Verifying Computations Without Reexecuting Them', *Communications of the ACM*, τ. 58, τχ. 2, σελ. 74–84, 2015, doi: 10.1145/2641562.

[82] F. Zhang, E. Cecchetti, K. Croman, A. Juels, και E. Shi, 'Town Crier: An Authenticated Data Feed for Smart Contracts', στο *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2016, σελ. 270–282. doi: 10.1145/2976749.2978326.

[83] S. K. Ezzat, Y. N. M. Saleh, και A. A. Abdel-Hamid, 'Blockchain Oracles: State-of-the-Art and Research Directions', *IEEE Access*, τ. 10, σελ. 67551–67572, 2022, doi: 10.1109/ACCESS.2022.3184726.

[84] S. Woo, J. Song, και S. Park, 'A Distributed Oracle Using Intel SGX for Blockchain-Based IoT Applications', *Sensors*, τ. 20, τχ. 9, σελ. 2725, 2020, doi: 10.3390/s20092725.

[85] R. Cheng *κ.ά.*, 'Ekiden: A Platform for Confidentiality-Preserving, Trustworthy, and Performant Smart Contracts', στο *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, IEEE, 2019, σελ. 185–200. doi: 10.1109/EuroSP.2019.00023.

[86] J. Eberhardt και S. Tai, 'ZoKrates – Scalable Privacy-Preserving Off-Chain Computations', στο *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, IEEE, 2018, σελ. 1084–1091. doi: 10.1109/Cybermatics_2018.2018.00199.

[87] RWA.xyz, 'Analytics on Tokenized Real-World Assets'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://app.rwa.xyz/

[88] S. Kalbermatten, 'The `Assetization' of Art on an Institutional Level—Fractional Ownership Implemented in the Royal Museum of Fine Arts Antwerp', *Arts*, τ. 13, τχ. 1, σελ. 16, 2024, doi: 10.3390/arts13010016.

[89] D. Avrilionis και T. Hardjono, ' Towards Blockchain-enabled Open Architectures for Scalable Digital Asset Platforms ', *arXiv preprint arXiv:2110.12553*, 2021, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2110.12553

[90] W. Haouari και M. Fokaefs, 'Towards a Standardization of Fractionalization Smart Contracts for Non-Fungible Tokens', στο *Proceedings of the 33rd Annual International Conference on Computer Science and Software Engineering (CASCON)*, ACM, 2023, σελ. 132–141. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://dl.acm.org/doi/10.5555/3615924.3615938

[91] Βουλή των Ελλήνων, 'Νόμος 2121/1993: Πνευματική ιδιοκτησία, συγγενικά δικαιώματα και πολιτιστικά θέματα'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: http://elib.aade.gr/elib/view?d=/gr/act/1993/2121

[92] Βουλή των Ελλήνων, 'Νόμος 3524/2007: Εναρμόνιση της ελληνικής νομοθεσίας προς τις Οδηγίες 2001/84/ΕΚ και 2004/48/ΕΚ'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: http://elib.aade.gr/elib/view?d=/gr/act/2007/3524

[93] S. Blackshear *κ.ά.*, 'Move: A Language With Programmable Resources'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://developers.diem.com/papers/diem-move-a-language-with-programmable-resources/2019-06-18.pdf

[94] A. Giatzis, S. Papangelou, και C. K. Georgiadis, ' A Comparative Study of Solidity and Sui Move: Advancing Smart Contract Development ', *IEEE Access*, τ. 13, 2025, doi: 10.1109/ACCESS.2025.3615072.

[95] Mysten Labs, 'Object Model'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/sui-architecture/object-model

[96] Mysten Labs, 'Object Versioning'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/objects/versioning

[97] Mysten Labs, 'Types of Object Ownership'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/objects/object-ownership

[98] Mysten Labs, 'Dynamic Fields'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/concepts/dynamic-fields

[99] Mysten Labs, 'Sui Architecture: Consensus'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/sui-architecture/consensus

[100] Mysten Labs, 'Programmable Transaction Blocks'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/transactions/ptbs/prog-txn-blocks

[101] Mysten Labs, 'Gas Fees'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/transaction-payment/gas-in-sui

[102] Mysten Labs, 'Life of a Transaction'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/transactions/transaction-lifecycle

[103] S. Blackshear *κ.ά.*, 'Resources: A Safe Language Abstraction for Money', *arXiv preprint arXiv:2004.05106*, 2020, [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://arxiv.org/abs/2004.05106

[104] A. Welc και S. Blackshear, 'Sui Move: Modern Blockchain Programming with Objects', στο *Companion Proceedings of the 2023 ACM SIGPLAN International Conference on*

Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '23), ACM, 2023, σελ. ref–ref. doi: 10.1145/3618305.3623605.

[105] The Move Team, 'The Move Book'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://move-book.com/

[106] The Move Team, 'The Move Reference: Abilities'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://move-book.com/reference/abilities/

[107] Mysten Labs, 'Move Syntax Fundamentals: Abilities'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/write-move/move-fundamentals#abilities

[108] T. Kichidis, L. Kokoris-Kogias, A. Koshy, και M. Tian, 'Mysticeti v2: Faster and Lighter Sui Transaction Processing'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://www.sui.io/blog/mysticeti-v2-sui-consensus

[109] Mysten Labs, 'Walrus Fundamentals'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.wal.app/docs/system-overview/core-concepts

[110] Mysten Labs, 'Walrus Operations'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.wal.app/docs/system-overview/operations

[111] I. S. Reed και G. Solomon, 'Polynomial Codes Over Certain Finite Fields', *Journal of the Society for Industrial and Applied Mathematics*, τ. 8, τχ. 2, σελ. 300–304, 1960, doi: 10.1137/0108018.

[112] Mysten Labs, 'Walrus Sites Components'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.wal.app/docs/sites/introduction/components

[113] Mysten Labs, 'Storage Costs'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.wal.app/docs/system-overview/storage-costs

[114] Mysten Labs, 'Batch Storage with Quilt'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.wal.app/docs/system-overview/quilt

[115] Mysten Labs, 'Walrus Sites: Technical Overview'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.wal.app/docs/sites/introduction/technical-overview

[116] Mysten Labs, 'Domain Names with SuiNS'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/suins/sui-stack-suins

[117] Mysten Labs, 'Onchain Websites with Walrus Sites'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/walrus/sui-stack-walrus-sites

[118] Mysten Labs, 'What is Seal?'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/seal

[119] Mysten Labs, 'Seal: Decentralized Secrets Management'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/assets/files/Seal_White_Paper_v2-c4d2d851256e4e60fb23eeac1043484d.pdf

[120] Mysten Labs, ' Seal Mainnet Launch: Bringing Data Privacy and Access Control to Web3 '. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://www.mystenlabs.com/blog/seal-mainnet-launch-privacy-access-control

[121] A. Shamir, 'Identity-Based Cryptosystems and Signature Schemes', στο *Advances in Cryptology — CRYPTO 1984*, στο Lecture Notes in Computer Science, vol. 196. Springer, 1984, σελ. 47–53. doi: 10.1007/3-540-39568-7_5.

[122] D. Boneh και M. Franklin, 'Identity-Based Encryption from the Weil Pairing', *SIAM Journal on Computing*, τ. 32, τχ. 3, σελ. 586–615, 2003, doi: 10.1137/S0097539701398521.

[123] M. Dworkin, ' Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC ', technical report NIST Special Publication 800–38D, 2007. doi: 10.6028/NIST.SP.800-38D.

[124] A. Shamir, 'How to Share a Secret', *Communications of the ACM*, τ. 22, τχ. 11, σελ. 612–613, 1979, doi: 10.1145/359168.359176.

[125] Mysten Labs, 'Using Seal'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/seal/using-seal

[126] Mysten Labs, 'Encryption with Seal'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/seal/sui-stack-seal

[127] Mysten Labs, 'Security Best Practices and Risk Mitigations'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/seal/security-best-practices

[128] Mysten Labs, 'What is Nautilus?'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/sui-stack/nautilus/nautilus-overview

[129] Mysten Labs, 'Nautilus: Verifiable Offchain Computation on Sui'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://github.com/MystenLabs/nautilus

[130] M. Sabt, M. Achemlal, και A. Bouabdallah, 'Trusted Execution Environment: What It Is, and What It Is Not', στο *Proceedings of the 2015 IEEE Trustcom/BigDataSE/ISPA*, IEEE, 2015, σελ. 57–64. doi: 10.1109/Trustcom.2015.357.

[131] V. Costan και S. Devadas, 'Intel SGX Explained'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://eprint.iacr.org/2016/086

[132] Amazon Web Services, 'AWS Nitro Enclaves User Guide'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.aws.amazon.com/enclaves/latest/user/

[133] Mysten Labs, 'Networks'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/sui-architecture/networks

[134] Mysten Labs, '2025 in Review: How The Sui Stack Came Together'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://blog.sui.io/2025-sui-stack-developments/

[135] Mysten Labs, 'Object Display'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/standards/display

[136] Mysten Labs, 'Sui Architecture: Storage'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/sui-architecture/sui-storage

[137] Mysten Labs, 'JSON-RPC to gRPC and GraphQL Migration'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/accessing-data/json-rpc-migration

[138] Mysten Labs, 'Sponsored Transactions'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://docs.sui.io/develop/transaction-payment/sponsor-txn

[139] World Wide Web Consortium, 'Verifiable Credentials Data Model v2.0'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://www.w3.org/TR/vc-data-model-2.0/

[140] World Wide Web Consortium, 'Decentralized Identifiers (DIDs) v1.0'. [Έκδοση σε ψηφιακή μορφή]. Διαθέσιμο στο: https://www.w3.org/TR/did-core/